```
// RC3 SCOPE FIX - based directly on RC3 commit 62b2b0c; NOT APPROVED.
// COMPILACAO NAO VERIFICADA - CANDIDATA NAO HOMOLOGADA.
// See experimental/docs/scope-fix-report.md for this correction and evidence.
// RC2 EXPERIMENTAL: PDF-derived reference; NOT COMPILED IN METAEDITOR.
// No production approval. Local freshness budgets remain uncalibrated.
// Remora_Scalper_Book_v1_124_DOM_Confluence.mq5
// EXPERIMENTAL. Reconstructed exclusively from the supplied BOOK 1.123 PDF.
// NOT COMPILED IN METAEDITOR. EMPIRICAL DOM VALIDATION PENDING.
// Source equivalence to an original .mq5 is not demonstrated.
// Provenance, transcription repairs and behavioral diff: experimental/docs/.
// The inherited historical header below belongs to the supplied PDF reference.

// RC12_SURGICAL_CANDIDATE_01: authorized RC1.2 PDF baseline; not production
approved.
// RC1.2 derived directly and exclusively from RC1.1: /workspace/R-mora-/scalper 2.pdf.
// Canonical read-only reference: /workspace/R-mora-/5.0 Rêmora .pdf.
// PDF transcription repairs and operational changes have separate full diffs.
// CÓDIGO ESTATICAMENTE AUDITADO, MAS NÃO COMPILADO.
// REQUER COMPILAÇÃO NO METAEDITOR.
//+------------------------------------------------------------------+
//| Remora_Scalper_v1_123_DOM_Shadow.mq5                             |
//| Descendente conceitual da Rêmora 2.44                            |
//| Arquitetura: M3 (Contexto) / M1 (Sinal) / Tick (Execução)        |
//|                                                                  |
//| CÓDIGO NÃO COMPILADO — REQUER COMPILAÇÃO NO METAEDITOR.
// |
//| Edge empírico não demonstrado. Não declarar rentabilidade.       |
//+------------------------------------------------------------------+
#property copyright "Remora Scalper Research"
#property version "1.125" // EXPERIMENTAL: PDF reconstruction + directional DOM
confluence.
#property description "Experimental PDF-derived 1.123; optional directional DOM
gate;compilation and empirical validation pending"

#include <Trade\Trade.mqh>
#include <Trade\PositionInfo.mqh>

//+------------------------------------------------------------------+
// ENUMS
//+------------------------------------------------------------------+
enum SIZING_MODE
{
  SIZING_FIXED = 0, // Lote fixo
  SIZING_RISK_PERCENT = 1 // % risco por trade
};

enum ENUM_MACRO_STATUS
{
```

```
  MACRO_NOT_COMPUTED = 0,
  MACRO_OK,
  MACRO_VWAP_INVALID,
  MACRO_SIGMA_INVALID,
  MACRO_STRUCT_INVALID,
   MACRO_NO_TRIGGER
};

enum ENUM_MSS
{
  MSS_NOT_COMPUTED = 0,
  MSS_OK,
  MSS_MACRO_INVALID,
  MSS_VWAP_D1_INVALID,
  MSS_SIGMA_INVALID,
  MSS_STRUCT_INVALID,
  MSS_STOP_KAMIKAZE_CONT_ONLY,
   MSS_STOP_NO_C5_TRIGGER
};

enum ENUM_APS
{
   APS_STILL_ARMED = 0,
   APS_ENDED_REEVAL_ALLOWED,
   APS_ENDED_FAIL_CLOSED,
   APS_TRADE_OR_POSITION,
   APS_ENTRY_READY // Internal result of the unchanged Stage2 checks.
};

enum ENUM_POC_STATE
{
   POCS_INVALID = 0,
   POCS_NO_DISPLACEMENT,
   POCS_FOLLOWING,
   POCS_DIVERGENT
};

//+------------------------------------------------------------------+
// CONSTANTS
//+------------------------------------------------------------------+
const int D1_SESSION_START = 900;
const int D1_SESSION_END_INDEX = 1825;
const int D1_SESSION_END_DOLLAR = 1830;
const int D1_MIN_M1_BARS = 480;
const int MAX_SEARCH_DAYS = 40;
#define MAX_TRADE_RECORDS 200

//+------------------------------------------------------------------+
```

```
// INPUTS
//==========================================================================
//--- Identidade
input int Inp_Magic_WIN = 60001;
input int Inp_Magic_WDO = 60002;
input string CSVFileName = "Scalper01_RC1"; // RC1 is also enforced in filenames
input datetime BacktestStart = D'2025.01.01';

//--- Timeframes
input ENUM_TIMEFRAMES Inp_ContextTF = PERIOD_M3; // TF Contexto (M3)
input ENUM_TIMEFRAMES Inp_SignalTF = PERIOD_M1; // TF Sinal (M1)

//--- Sessão
input int Inp_StartHHMM = 900; // Início entradas
input int Inp_StopEntryHHMM = 1530; // Fim entradas
input int Inp_EOD_CloseHHMM = 1750; // EOD (RESEARCH)
//--- Sizing
input SIZING_MODE Inp_SizingMode                = SIZING_FIXED; // Modo sizing
input double Inp_FixedLots = 1.0; // Lote fixo
input double Inp_RiskPerTradePct = 0.15;       // % risco (RISK_PERCENT)
input double Inp_MaxLots = 10.0; // Lote máximo

//--- SL/TP fixos (pts) — hipóteses V0.1, não declarar ótimos
input double Inp_SL_Pts_WIN = 150.0; // SL WIN (pts)
input double Inp_TP_Pts_WIN = 300.0; // TP WIN (pts)
input double Inp_SL_Pts_WDO = 5.0; // SL WDO (pts)
input double Inp_TP_Pts_WDO = 12.5; // TP WDO (pts)

//--- Spread
input double Inp_SpreadMaxPts_WIN = 5.0; // Spread máx WIN (pts)
input double Inp_SpreadMaxPts_WDO = 0.5; // Spread máx WDO (pts)

//--- Gestão RESEARCH (padrão OFF — medir entrada/saída fixa)
input bool Inp_EnableBreakEven = true; // +1R real confirmado -> SL no preço de entrada

//--- Proteção diária RESEARCH (padrão OFF)
input bool Inp_EnableDailyRiskStop = false; // Stop perda diária
input double Inp_MaxDailyLossR = 3.0; // Máx perda (em R)
input bool Inp_EnableConsecutiveLossStop = false; // Stop perdas seguidas
input int Inp_MaxConsecutiveLosses = 3; // Nº perdas seguidas

//--- Limites (LIMITE, não meta)
input int Inp_MaxTradesDay = 10; // Máx trades/dia

//--- Cooldown (segundos desde fechamento confirmado)
input int Inp_CooldownSec = 0; // Cooldown geral (s)
input int Inp_LossCooldownSec = 0; // Cooldown após perda (s)
```

```
//--- Armed
input int Inp_ArmedTimeoutSec = 90; // Timeout armed (s)
//--- Região / Rompimento
input int Inp_Region_Candles = 2; // Candles região (M1 fechadas)
input double Inp_BreakoutMargin_WIN = 5.0; // Margem rompimento WIN (pts)
input double Inp_BreakoutMargin_WDO = 0.5; // Margem rompimento WDO (pts)
input double Inp_MaxOvershoot_WIN = 25.0; // Overshoot máx WIN (pts)
input double Inp_MaxOvershoot_WDO          = 1.0; // Overshoot máx WDO (pts)

//--- Weis Wave M1
input double Inp_ABS_Effort = 1.50; // WW esforço ABS (C1)
input double Inp_CONT_Effort = 1.25; // WW esforço CONT (C1)
input double Inp_ABS_BreakoutEffort = 1.50; // WW onda viva ABS (Stage2)
input double Inp_CONT_BreakoutEffort = 1.50; // WW onda viva CONT (Stage2)
//--- POC M1
input int Inp_POC_WindowSec = 60; // Janela POC (s) — baseline = M1
input double Inp_POC_FollowRatio = 0.50; // Razão POC segue preço
input bool Inp_RejectPOCTies = false; // Ablation only; baseline keeps canonical POC

//--- ABS thresholds (pts — hipóteses, não declarar ótimos)
input double Inp_ABS_MinPriceMovePts_WIN = 60.0; // ABS displ min WIN (pts)
input double Inp_ABS_MaxBodyPts_WIN              = 60.0; // ABS corpo max WIN (pts)
input double Inp_ABS_MinPriceMovePts_WDO = 2.0; // ABS displ min WDO (pts)
input double Inp_ABS_MaxBodyPts_WDO                  = 2.0; // ABS corpo max WDO (pts)

//--- CONT thresholds (pts — hipóteses)
input double Inp_CONT_MinPriceMovePts_WIN = 50.0; // CONT displ min WIN (pts)
input double Inp_CONT_MinPriceMovePts_WDO = 1.5; // CONT displ min WDO (pts)
input bool Inp_Cont_RequireCandleDirection = true; // CONT: vela na direção

//--- Geometria VWAP (herdada da 2.44)
input double BandMultiplier = 1.0;
input double Inp_VWAP_Inner_Pct = 0.60;
input double Inp_VWAP_Outer_Pct = 1.50;

//--- C2/C3/C4 (herdados da 2.44 — hipóteses para scalp, não validados OOS)
input double Inp_Rule_40_60 = 0.40;
input double Inp_C2_WIN = 700.0;
input double Inp_C3_Min_WIN = 125.0;
input double Inp_C3_Max_WIN = 400.0;
input double Inp_C2_WDO = 25.0;
input double Inp_C3_Min_WDO = 5.0;
input double Inp_C3_Max_WDO = 15.0;

//--- Sigma histórico (ainda necessário para bandas/C2/C3/C4)
input int SigmaLookbackDays = 5;
input double Inp_SigmaFloor_WIN = 200.0;
input double Inp_SigmaFloor_WDO = 7.0;
```

```
//--- Execução
input int Inp_MaxDeviationPts_WIN = 10;
input int Inp_MaxDeviationPts_WDO = 2;
//--- Rotas habilitadas
input bool Inp_EnableMacroDirectionGate = true;
input bool Inp_EnableLateDirectionBlock = false;
input bool Inp_EnableAbsorption = true;
input bool Inp_EnableContinuation = true;

//--- Optional descriptive book telemetry; original 66 inputs above unchanged.
input bool Inp_DOM_Enable = false;
input string Inp_DOM_InstanceTag = "shadow";

// Experimental gate: the preceding 68 declarations are unchanged.
input bool Inp_DOM_TradeGateEnable = false;
input double Inp_DOM_MinDominancePct = 55.0;

//+------------------------------------------------------------------+
// STRUCTS
//+------------------------------------------------------------------+
struct MacroContext
{
   ENUM_MACRO_STATUS status;
   datetime barTime;
   double closedPrice;
   double vwap;
   double SigmaD1; // value actually used by the closed M3 snapshot
   double distVWAP_D1;
   double posRel;
   bool glass;
   int macroAbsDir; // dir C5 ABS (M3)
   int macroContDir; // dir C5 CONT (M3)
};

struct POCResult
{
   bool ok;
   double poc;
   double totalVolume;
   double lastValidPrice;
   int tradeCount;
   int levels;
   bool hasTie;
   double top1Price, top1Vol;
   double top2Price, top2Vol;
   double dominance;
   double buyVol, sellVol, unkVol;
```

```
};
struct MatrixSnapshot
{
   ENUM_MSS status;
   // Bandas
   double vwapVig, upper, lower, bandRange;
   // Candle M1 fechada
   double price, open1, candleBody;
   double posRel;
   bool glass;
   // C5
   int dir, dirCont;
   bool c5AbsOk, c5ContOk;
   // Thresholds
   double distAbs;
   double minDispl, maxBody, contMinDispl;
   // C2/C3/C4
   bool c2, c3, c23, c4;
   bool c2Cont, c3Cont, c23Cont, c4Cont;
   double posRange;
   // POC
   POCResult pocNow, pocPrev;
      bool windowsValid;
      double priceMove, pocMove;
      bool displacementOk;
      int moveDir;
      ENUM_POC_STATE pocState;
      bool contDisplacementOk;
      int contMoveDir;
      ENUM_POC_STATE contPocState;
      // WW
      bool wwok;
      double effort;
      int waveDir;
      int wwWaveBars;
      long wwWaveAgeSec;
      bool wwWaveAgeAvailable;
      double wwAccumVolume, wwDistancePts;
      // C1 ABS sub
      bool c1_effort_ok, c1_body_ok, c1_displacement_ok;
      bool c1_movedir_ok, c1_wavedir_ok;
      // ABS contexts
      bool absDivergence, absStallStrict, absFollowingReaction, absFollowingExtension;
      bool absPocContextOk, c1;
      // C1 CONT sub
      bool cont_effort_ok, cont_displacement_ok, cont_poc_ok;
      bool cont_movedir_ok, cont_wavedir_ok, cont_candle_dir_ok;
      bool c1Cont;
```

```
      // Mode
       bool turboAbs, baseAbs, turboCont, baseCont;
     int scoreEff, scoreContEff;
     // Final validity
     bool      absValid, contValid;
     int candleDir;
     // MacroContext filter results (telemetria)
     bool      macroAbsCompatible, macroContCompatible;
};

struct ArmedSignal
{
  bool active;
  int dir;
  double regionHigh, regionLow;
  double activeSL_Pts, activeTP_Pts;
  datetime regionEndTime;
  string mode;
  double lots;
  string route;
  double breakoutEffortMin;
  datetime armedAt;
   bool sameSideGlassLogged;
   // Snapshot para auditoria
   double posRel, distD1, effort, vwapVig;
   int c1, c2, c3, c4, c5, scoreEff;
};

// RC1.2 execution and accounting state.
// RC1.2: exact order identity survives disappearance from OrdersTotal.
enum PendingOrderNature
{
   PENDING_ORDER_UNKNOWN=0,
   PENDING_ORDER_ENTRY,
   PENDING_ORDER_EXIT
};

struct UnresolvedOrderContext
{
  bool active, classificationPending;
  bool persistencePending;
  ulong orderTicket, positionID, positionByID;
  datetime firstSeen, setupTime, doneTime;
  PendingOrderNature nature;
  ENUM_ORDER_STATE lastState;
  long orderType, orderReason;
  double initialVolume, remainingVolume;
  string comment, route, mode;
```

```
  string persistenceSnapshot;
};
UnresolvedOrderContext gUnresolvedOrders[];
bool gMultipleOwnOrdersActive=false;
bool gOrderRegistryUnavailable=false;
ulong gOrderRegistryFallback[MAX_TRADE_RECORDS];
int gOrderRegistryFallbackCount=0;
bool gOrderRegistryOverflow=false;
bool gOrderObligationScanUnresolved=false;
bool gOrderObligationRestoring=false;
datetime gNextOrderReconcile=0;


// Separate BE evidence; no field below replaces the original SL/risk.
enum BE_STAGE
{
   BE_NONE=0, BE_TRIGGERED, BE_REQUEST_PENDING, BE_ACKNOWLEDGED,
   BE_CONFIRMED, BE_UNRESOLVED
};
struct BreakEvenState
{
   int stage, direction;
   ulong ticket;
   double entryPrice, initialSL, price, tp;
   long triggerMSC, confirmedMSC;
   uint requestID, retcode;
   bool durable, successLogged, exitLogged;
   datetime nextTry, lastFailureLog;
};

struct TradeRecord
{
  ulong positionID;
  int direction;
  string route, mode;
  bool quotaCounted;
  double requestPrice, breakoutPrice;
  bool requestPriceAvailable, breakoutPriceAvailable;
    ulong entryOrderTicket;
    double expectedEntryVolume;
    datetime entryTime;
    double entryPrice, weightedEntryPrice, totalVolumeIn, entrySlippage;
    double slPrice, tpPrice;
    datetime exitTime;
    long exitTimeMSC;
    ulong lastExitDealTicket;
    double exitPrice, weightedExitPrice, totalVolumeOut, volume;
    double grossProfit, commission, fee, swap, netProfit;
```

```
   string exitReason;
    double MAE, MFE;
    long holdSeconds;
    double initialRiskMoney, PnL_R;
    bool closed, maeMfePartialAfterRestart, volumeClosedComplete;
    bool finalizationWritten, dailyApplied;
    bool accountingResolved, riskValid, pnlRValid;
    bool initialStopsProven, economicMarkerPending;
    bool economicMarkerCleanupPending; // Maintenance only after fresh durable proof.
    int economicMarkerDeleteError;
    bool historyRefreshPending;
    // Rebuilt from authoritative deals; never ownership of the live residual.
    bool residualForeignDetected, ownCycleEconomicallyClosed;
    bool volumeOwnershipResolved, inoutCostsUnresolved;
     BreakEvenState be;
     bool breakEvenExit; // Derived only from all confirmed own exit components.
     bool breakEvenClassificationPending; // Do not publish a loss before BE proof resolves.
};

// Zero/default and legacy journals must always retain dispatch uncertainty.
enum ENUM_REQUEST_DISPATCH_PHASE
{
 REQUEST_MAY_HAVE_DISPATCHED=0,
 REQUEST_PREPARED=1
};

struct PendingExecContext
{
 bool active;
 ENUM_REQUEST_DISPATCH_PHASE dispatchPhase;
 int direction;
 string route, mode;
 double requestPrice, requestedLots, slPrice, tpPrice;
 datetime sentAt;
 ulong orderTicket, resultDeal;
 uint requestID;
 bool accepted, requestPriceAvailable;
 double breakoutPrice;
 bool breakoutPriceAvailable;
};

struct PendingExitIntent
{
 bool active;
 ENUM_REQUEST_DISPATCH_PHASE dispatchPhase;
 ulong positionID, positionTicket;
 string reason;
 datetime sentAt;
```

```
  ulong orderTicket, resultDeal;
  uint requestID;
  bool accepted, uncertain;
  MqlTradeRequest requestSnapshot;
};



struct OpenPositionInfo
{
  bool active;
  ulong positionID;
  int direction;
  double entryPrice;
  double MAE, MFE;
  datetime entryTime;
  string route, mode;
  double lots;
  double initialRiskMoney;
    int tradeRecordIdx;
};

struct PerfStat
{
   long count;
   double sumUs, maxUs;
};

//+------------------------------------------------------------------+
// GLOBAL VARIABLES
//+------------------------------------------------------------------+
CTrade trade;
CPositionInfo posInfo;

// Família e conta
bool gIsIndexFamily = false;
bool gIsDollarFamily = false;
bool gNetting = false;
double gTickSize = 1.0;
int gDayKey      = -1;

// Parâmetros resolvidos no OnInit (família-específicos)
int gMagicCore = 0;
double gC2 = 0, gC3Min = 0, gC3Max = 0;
int gMaxDeviationPts = 0;
double gSpreadMaxPts = 5.0;
double gBreakoutMargin = 5.0;
```

```
double gMaxOvershoot = 25.0;
double gSL_Pts = 150.0;
double gTP_Pts = 300.0;
double gABS_MinDisplPts = 60.0;
double gABS_MaxBodyPts = 60.0;
double gCONT_MinDisplPts= 50.0;

// D-1
double H1 = 0.0, L1 = 0.0, VWAP_D1 = 0.0, SigmaD1 = 0.0;
bool VWAP_D1_Official = false;
bool d1Ready = false;
datetime nextD1Retry = 0;

// VWAP vigente (incremental acumulador diário)
double vwapSumPV = 0.0, vwapSumVol = 0.0;
ulong vwapNextFromMsc = 0;
bool vwapReady = false;
int vwapDayKey = -1;
double gVwapValue = 0.0;
bool gVwapOk = false;
// Weis Wave cache M1
bool wwValid = false;
bool wwUseReal       = false;
int wwDirClosed = 0;
double wwAccumClosed = 0.0;
double wwAvgClosed         = 0.0;
double wwAvgIfFlip = 0.0;
double wwCloseRef       = 0.0;
double wwEffortClosed = 0.0;
int wwWaveDirClosed = 0;
int wwCurrentWaveBars= 0; // telemetria WW_WAVE_BARS
double wwWaveStartPrice = 0.0; // para WW_DISTANCE_PTS
// Temporal
datetime lastContextBarTime = 0; // M3
datetime lastSignalBarTime = 0; // M1
datetime gLastMatrixEvalBar = 0; // anti-duplo
datetime gLastReadOnlyMatrixBar = 0; // Diagnostic attempts never consume the entry
clock.

// MacroContext (M3)
MacroContext gMacro;

// Armed
ArmedSignal armed;
// Open position tracker (MAE/MFE)
OpenPositionInfo gOpenPos;

// Trade records
```

```
TradeRecord gTradeRecords[];
int gTradeRecordCount = 0;

// Contadores diários
// Accounting globals. Existing daily counters and gTradeRecords remain.
PendingExecContext gPendingExec;
PendingExitIntent gPendingExit;
int requestsAcceptedToday = 0;
double dailyPnLR = 0.0;
bool gDailyPnLRValid = true;
bool gDailyEconomicsValid = true;
bool gQuotaHistoryValid = true;
bool gAccountingUnresolved = false;
enum DailyEconomicsRecoveryState
{
   DAILY_ECONOMICS_VALID=0,
   TEMPORARY_HISTORY_UNAVAILABLE,
   ECONOMIC_ALLOCATION_IMPOSSIBLE,
   DAILY_ECONOMICS_UNRESOLVED
};
DailyEconomicsRecoveryState gDailyEconomicsState=DAILY_ECONOMICS_VALID;
bool gLiveOwnershipUnresolved=false;
bool gAccountingCapacityUnresolved=false;
bool gDailyRiskLimitLatched=false;
bool gDailyHistoryRetryRequired=false;
datetime gLastAccountingRecoveryTime=0;
ulong gDeferredAccountingPIDs[];
bool gDeferredRegistryUnavailable=false;
ulong gDeferredFallback[MAX_TRADE_RECORDS];
int gDeferredFallbackCount=0;
bool gDeferredRegistryOverflow=false;
ulong gDeferredDeals[];
ulong gDeferredDealFallback[MAX_TRADE_RECORDS];
int gDeferredDealFallbackCount=0;
bool gDeferredDealOverflow=false;
bool gEconomicMarkerScanUnresolved=false;
bool gEMRMaintenanceNeeded=false;
string gEMRMaintenanceCursor="";
bool gIdentityLossUnresolved=false;
bool gIdentityLossPersistPending=false;
bool gRequestJournalUnresolved=false;
bool gRequestJournalWritePending=false;
bool gEntryJournalCleanupPending=false;
bool gExitJournalCleanupPending=false;
bool gEntryJournalInvalid=false, gExitJournalInvalid=false;
bool gEntryJournalWritePending=false, gExitJournalWritePending=false;
bool gRequestJournalRestoring=false;
datetime gNextRequestJournalRecovery=0;
```

```
// Independent guard: a history retry cannot erase an unclassified own order.
bool gPendingOrderClassificationUnresolved = false;
bool gNeedsAccountingSync = false;
datetime gLastPendingReconcileTime = 0;
ulong gConfirmedPositionsToday[];

int ordersEnteredToday = 0;
int consecutiveLosses = 0;
double dailyNetPnL = 0.0;
datetime lastCloseConfirmedTime = 0;
bool dailyRiskBlocked = false;
bool lastTradeWasLoss      = false;

// EOD
bool eodDone = false;
bool eodStuckLogged = false;
datetime eodNextTry = 0;

// Bandas cache
double UpperBand1 = 0.0, LowerBand1 = 0.0;
// CSV handles
int csvAuditHandle = INVALID_HANDLE;
int csvTradeHandle = INVALID_HANDLE;
int csvShadowHandle = INVALID_HANDLE;

// Performance telemetry (contagem de execução)
PerfStat perfMacro, perfMicro, perfStage2;
long gLastPerfLoggedMicroCount = 0;
int hardFlipsPerDay = 0, armedCancelHard = 0, oppositeRearmAfterHard = 0;
double gLastBreakoutPrice = 0.0;
bool gLateMismatchLogged = false, gSpreadHoldLogged = false;
MatrixSnapshot gCachedMicroSnapshot;
datetime gCachedMicroBar = 0;
bool gCachedMicroReady = false;

// Flush periódico (evitar por tick)
datetime gLastFlushTime = 0;

// Descriptive DOM cache. No trade, account, persistence, network or book API calls.
// Collection timestamps are local monotonic times; they are NOT exchange times.
#define DOM_RAW_RECORD_LIMIT 4096
enum ENUM_DOM_QUALITY
{
  DOM_QUALITY_NONE=0,
  DOM_QUALITY_OK=1,
  DOM_QUALITY_EMPTY=2,
  DOM_QUALITY_TOO_LARGE=3,
  DOM_QUALITY_INVALID=4,
```

```
   DOM_QUALITY_ONE_SIDED=5,
    DOM_QUALITY_CROSSED=6,
    DOM_QUALITY_SHALLOW=7,
    DOM_QUALITY_API_FAILURE=8,
    DOM_QUALITY_NO_SUBSCRIPTION=9,
    DOM_QUALITY_DISCONNECTED=10,
    DOM_QUALITY_SYMBOL_MISMATCH=11
};

struct DOMSnapshot
{
  ulong collectedMS;
  ulong sequence;
  ENUM_DOM_QUALITY quality;
   bool bookValid;
   bool i5Available;
   bool micropriceAvailable;
   int rawRecords;
   int bidLevels;
   int askLevels;
   int bidPriceRecords;
   int askPriceRecords;
   int marketBuyRecords;
   int marketSellRecords;
   int invalidRecords;
   int zeroVolumeRecords;
   int volumeFallbackRecords;
   // At most five distinct prices on each side, best price first.
   // Missing slots are unavailable; consult min(5, bidLevels/askLevels).
   double bidDepthPrice[5];
   double bidDepthVolume[5];
   double askDepthPrice[5];
   double askDepthVolume[5];
   double bestBid;
   double bestAsk;
   double spread;
   double bidVolume1;
   double askVolume1;
   double bidVolume5;
   double askVolume5;
   double marketBuyVolume;
   double marketSellVolume;
   double i5;
   double midpoint;
   double microprice;
   double micropriceMinusMid;
};
```

```
// Fixed workspaces: 131072 bytes of price/volume arrays, reused at 1 Hz.
// More than 4096 raw records is unavailable, never a truncated valid snapshot.
double gDOMBidPrices[DOM_RAW_RECORD_LIMIT];
double gDOMBidVolumes[DOM_RAW_RECORD_LIMIT];
double gDOMAskPrices[DOM_RAW_RECORD_LIMIT];
double gDOMAskVolumes[DOM_RAW_RECORD_LIMIT];
DOMSnapshot gDOMCurrent;
ulong gDOMSequence=0;

void DOMClearSnapshot(DOMSnapshot &out)
{
  ZeroMemory(out);
  out.quality=DOM_QUALITY_NONE;
}
void DOMResetCache()
{
  DOMClearSnapshot(gDOMCurrent);
  gDOMSequence=0;
}

bool DOMFinite(const double value)
{
  return MathIsValidNumber(value) && value!=EMPTY_VALUE;
}

bool DOMAddVolume(double &total,const double volume)
{
  double sum=total+volume;
  if(!DOMFinite(sum) || sum<0.0) return false;
  total=sum;
  return true;
}

// volume_real==0 is treated as unavailable; only then is integer volume used.
// NaN, infinity or negative volume_real is corrupt data, never a fallback case.
// A positive volume_real takes precedence; integer volume may lose precision.
bool DOMRecordVolume(const MqlBookInfo &record,double &volume,bool &fallback)
{
   volume=0.0;
   fallback=false;
   if(!DOMFinite(record.volume_real) || record.volume_real<0.0 || record.volume<0)
      return false;
   if(record.volume_real>0.0)
      volume=record.volume_real;
   else
   {
      // A larger integer cannot always be represented exactly by a double.
      // Prefer unavailable evidence over silently rounding the fallback size.
```

```
      if(record.volume>9007199254740992) return false;
     volume=(double)record.volume;
     fallback=(record.volume>0);
    }
    return DOMFinite(volume) && volume>=0.0;
}

void DOMSwapLevel(double &prices[],double &volumes[],const int a,const int b)
{
  double price=prices[a];
  double volume=volumes[a];
  prices[a]=prices[b]; volumes[a]=volumes[b];
  prices[b]=price; volumes[b]=volume;
}

// Heapsort gives a deterministic O(n log n) bound with fixed auxiliary storage.
void DOMSiftLevels(double &prices[],double &volumes[],int root,const int count)
{
   while(root<count/2)
   {
     int child=2*root+1;
     if(child+1<count && prices[child]<prices[child+1]) child++;
     if(prices[root]>=prices[child]) return;
     DOMSwapLevel(prices,volumes,root,child);
     root=child;
   }
}

void DOMSortLevels(double &prices[],double &volumes[],const int count)
{
  for(int root=count/2-1;root>=0;root--) DOMSiftLevels(prices,volumes,root,count);
  for(int end=count-1;end>0;end--)
  {
    DOMSwapLevel(prices,volumes,0,end);
    DOMSiftLevels(prices,volumes,0,end);
  }
}

bool DOMSummarizeSide(double &prices[],double &volumes[],const int count,
          const bool buy,int &levels,double &best,
          double &volume1,double &volume5,
          double &depthPrice[],double &depthVolume[])
{
  levels=0; best=0.0; volume1=0.0; volume5=0.0;
  for(int i=0;i<5;i++) { depthPrice[i]=0.0; depthVolume[i]=0.0; }
  if(count<=0) return true;
  DOMSortLevels(prices,volumes,count);
  int step=buy ? -1 : 1;
```

```
  int first=buy ? count-1 : 0;
    double groupPrice=prices[first];
    double groupVolume=0.0;
    for(int offset=0;offset<count;offset++)
    {
       int idx=first+offset*step;
       if(prices[idx]!=groupPrice)
       {
          levels++;
          if(levels==1) { best=groupPrice; volume1=groupVolume; }
          if(levels<=5)
          {
             depthPrice[levels-1]=groupPrice; depthVolume[levels-1]=groupVolume;
           if(!DOMAddVolume(volume5,groupVolume)) return false;
         }
         groupPrice=prices[idx]; groupVolume=0.0;
       }
       if(!DOMAddVolume(groupVolume,volumes[idx])) return false;
      }
      levels++;
      if(levels==1) { best=groupPrice; volume1=groupVolume; }
      if(levels<=5)
      {
         depthPrice[levels-1]=groupPrice; depthVolume[levels-1]=groupVolume;
         if(!DOMAddVolume(volume5,groupVolume)) return false;
      }
      return true;
}

// The caller validates connection, exact chart symbol and subscription first.
// Equal double prices on the SAME side are aggregated without rounding.
bool DOMBuildSnapshot(const MqlBookInfo &book[],const int count,
         const ulong collectedMS,DOMSnapshot &out)
{
   DOMClearSnapshot(out);
   out.collectedMS=collectedMS;
   out.rawRecords=count;
   if(count<=0) { out.quality=DOM_QUALITY_EMPTY; return false; }
   if(count>DOM_RAW_RECORD_LIMIT)
   { out.quality=DOM_QUALITY_TOO_LARGE; return false; }
   if(count>ArraySize(book))
   { out.quality=DOM_QUALITY_INVALID; out.invalidRecords=1; return false; }

int bids=0,asks=0;
for(int i=0;i<count;i++)
{
  double volume=0.0;
  bool fallback=false;
```

```
  if(!DOMRecordVolume(book[i],volume,fallback))
   { out.invalidRecords++; continue; }
   if(fallback) out.volumeFallbackRecords++;
   if(volume==0.0) out.zeroVolumeRecords++;
   if(book[i].type==BOOK_TYPE_BUY_MARKET ||
book[i].type==BOOK_TYPE_SELL_MARKET)
   {
      // Market records have no usable price and never contribute to depth.
      if(book[i].type==BOOK_TYPE_BUY_MARKET)
      {
         out.marketBuyRecords++;
         if(!DOMAddVolume(out.marketBuyVolume,volume)) out.invalidRecords++;
      }
   else
   {
     out.marketSellRecords++;
     if(!DOMAddVolume(out.marketSellVolume,volume)) out.invalidRecords++;
   }
   continue;
 }
 if((book[i].type!=BOOK_TYPE_BUY && book[i].type!=BOOK_TYPE_SELL) ||
    !DOMFinite(book[i].price) || book[i].price<=0.0)
 { out.invalidRecords++; continue; }
 if(volume==0.0) continue; // Zero quantity is not an available price level.
 if(book[i].type==BOOK_TYPE_BUY)
 {
    gDOMBidPrices[bids]=book[i].price;
    gDOMBidVolumes[bids]=volume;
    bids++;
 }
 else
 {
    gDOMAskPrices[asks]=book[i].price;
    gDOMAskVolumes[asks]=volume;
    asks++;
 }
}
out.bidPriceRecords=bids;
out.askPriceRecords=asks;
if(out.invalidRecords>0) { out.quality=DOM_QUALITY_INVALID; return false; }
if(!DOMSummarizeSide(gDOMBidPrices,gDOMBidVolumes,bids,true,out.bidLevels,
            out.bestBid,out.bidVolume1,out.bidVolume5,
            out.bidDepthPrice,out.bidDepthVolume) ||
   !DOMSummarizeSide(gDOMAskPrices,gDOMAskVolumes,asks,false,out.askLevels,
            out.bestAsk,out.askVolume1,out.askVolume5,
            out.askDepthPrice,out.askDepthVolume))
{ out.quality=DOM_QUALITY_INVALID; out.invalidRecords++; return false; }
if(out.bidLevels==0 || out.askLevels==0)
```

```
{ out.quality=DOM_QUALITY_ONE_SIDED; return false; }
if(out.bestBid>=out.bestAsk)
{ out.quality=DOM_QUALITY_CROSSED; return false; }

out.spread=out.bestAsk-out.bestBid;
out.midpoint=out.bestBid+out.spread*0.5;
double depthSum=out.bidVolume5+out.askVolume5;
double bestSum=out.bidVolume1+out.askVolume1;
if(!DOMFinite(out.spread) || !DOMFinite(out.midpoint) ||
   !DOMFinite(depthSum) || depthSum<=0.0 ||
   !DOMFinite(bestSum) || bestSum<=0.0)
{ out.quality=DOM_QUALITY_INVALID; return false; }
out.i5=(out.bidVolume5-out.askVolume5)/depthSum;
    // Algebraically (Ask*BidVol1 + Bid*AskVol1)/(BidVol1+AskVol1).
    // This equivalent form avoids multiplying a price by a potentially huge size.
    out.microprice=out.bestBid+out.spread*(out.bidVolume1/bestSum);
    out.micropriceMinusMid=out.microprice-out.midpoint;
    if(!DOMFinite(out.i5) || out.i5 < -1.0 || out.i5 > 1.0 ||
       !DOMFinite(out.microprice) || !DOMFinite(out.micropriceMinusMid))
    { out.quality=DOM_QUALITY_INVALID; return false; }
    out.bookValid=true;
    out.i5Available=(out.bidLevels>=5 && out.askLevels>=5);
    if(!out.i5Available) out.i5=0.0; // No five-price imbalance on shallow depth.
    out.micropriceAvailable=true;
    out.quality=(out.bidLevels>=5 && out.askLevels>=5) ?
          DOM_QUALITY_OK : DOM_QUALITY_SHALLOW;
    return true;
}

// Copying this observational snapshot has no operational return value.
// It does not classify a signal, authorize an order or change strategy state.
void DOMStoreSnapshot(const DOMSnapshot &snapshot)
{
   DOMSnapshot next=snapshot;
   next.sequence=++gDOMSequence;
   gDOMCurrent=next;
}

string DOMQualityName(const ENUM_DOM_QUALITY quality)
{
  switch(quality)
  {
    case DOM_QUALITY_NONE: return "NONE";
    case DOM_QUALITY_OK: return "OK";
    case DOM_QUALITY_EMPTY: return "EMPTY";
    case DOM_QUALITY_TOO_LARGE: return "TOO_LARGE";
    case DOM_QUALITY_INVALID: return "INVALID";
    case DOM_QUALITY_ONE_SIDED: return "ONE_SIDED";
```

```
    case DOM_QUALITY_CROSSED: return "CROSSED_OR_LOCKED";
     case DOM_QUALITY_SHALLOW: return "SHALLOW";
     case DOM_QUALITY_API_FAILURE: return "API_FAILURE";
     case DOM_QUALITY_NO_SUBSCRIPTION: return "NO_SUBSCRIPTION";
     case DOM_QUALITY_DISCONNECTED: return "DISCONNECTED";
     case DOM_QUALITY_SYMBOL_MISMATCH: return "SYMBOL_MISMATCH";
    }
    return "NONE";
}


// Optional descriptive DOM telemetry. No trading/accounting state is changed.
// This module is called after the original timer work; no OnBookEvent handler.
#define DOM_EVENT_CAPACITY 256
#define DOM_TELEMETRY_DAY_BYTES 10485760
#define DOM_LATENCY_WINDOW 128
enum ENUM_DOM_LATENCY_KIND
{
   DOM_LAT_API=0, DOM_LAT_AGG=1, DOM_LAT_WRITE=2, DOM_LAT_RUNTIME=3
};
struct DOMTelemetryEvent
{
   int kind, status, direction, route;
   ulong sequence, localMS;
   datetime signalBar, contextBar;
   ulong positionID, entryOrder, entryDeal, exitPositionID, exitOrder, exitDeal;
   uint entryRequestID, exitRequestID;
   bool entryActive, exitActive;
   double price, trigger, lots, sl, tp, netProfit;
   bool closed, volumeComplete, accountingResolved, finalizationWritten;
   bool apiCalled, aggregationDone;
   ulong apiDurationUS, aggregationUS;
   int apiError;
   int latencyKind;
   ulong samples, p50, p95, p99, maximum;
   bool percentilesAvailable;
   DOMSnapshot snapshot;
};
DOMTelemetryEvent gDOMEvents[DOM_EVENT_CAPACITY];
int gDOMEventHead=0, gDOMEventCount=0;
ulong gDOMEventSequence=0, gDOMDroppedEvents=0;
ulong gDOMLatencySamples[4][DOM_LATENCY_WINDOW];
int gDOMLatencyNext[4];
ulong gDOMLatencySeen[4], gDOMLatencyMaximum[4];
bool gDOMSubscriptionOwned=false, gDOMRuntimeReady=false;
bool gDOMCollectionEnabled=false, gDOMTelemetryEnabled=false;
string gDOMBookSymbol="", gDOMTelemetryPrefix="";
string gDOMTelemetryDay="", gDOMTelemetryPath="";
```

```
ulong gDOMTelemetryBytes=0, gDOMRunStartOffset=0;
ulong gDOMLastCollectMS=0, gDOMLastFlushMS=0, gDOMLastSummaryMS=0;
ulong gDOMSuspendUntilMS=0, gDOMSamplingGaps=0;
ulong gDOMTelemetryErrors=0, gDOMApiFailures=0, gDOMOversizedBooks=0;
int gDOMApiFailureRun=0, gDOMSlowRun=0, gDOMSlowWriteRun=0;
int gDOMContainmentEvents=0, gDOMLastApiError=0;

void DOMLatency(const int kind,const ulong elapsedUS)
{
  if(!Inp_DOM_Enable || kind<0 || kind>=4) return;
  gDOMLatencySamples[kind][gDOMLatencyNext[kind]]=elapsedUS;
  gDOMLatencyNext[kind]=(gDOMLatencyNext[kind]+1)%DOM_LATENCY_WINDOW;
  gDOMLatencySeen[kind]++;
    if(elapsedUS>gDOMLatencyMaximum[kind]) gDOMLatencyMaximum[kind]=elapsedUS;
}

bool DOMPushEvent(const DOMTelemetryEvent &event)
{
  if(!Inp_DOM_Enable || !gDOMRuntimeReady) return false;
  if(!gDOMTelemetryEnabled || gDOMEventCount>=DOM_EVENT_CAPACITY)
  { gDOMDroppedEvents++; return false; }
  int index=(gDOMEventHead+gDOMEventCount)%DOM_EVENT_CAPACITY;
  gDOMEvents[index]=event;
  gDOMEvents[index].sequence=++gDOMEventSequence;
  gDOMEventCount++;
  return true;
}

int DOMAuditStatusCode(const string &status)
{
  // Existing audit events only. No inference of execution or profitability.
  if(status=="ARMED") return 1;
  if(status=="REQUEST_ACCEPTED") return 2;
  if(status=="ORDER_FAILED") return 3;
  if(status=="TIMEOUT_ARMED") return 4;
  if(status=="ABORT_SESSION_END") return 5;
  if(status=="ABORT_EOD") return 6;
  if(status=="ARMED_CANCEL_HARD") return 7;
  if(status=="OVERSHOOT_BLOCK") return 8;
  if(status=="LATE_BREAKOUT_LOST") return 9;
  if(status=="LATE_OVERSHOOT_BLOCK") return 10;
  if(status=="HOLD_SPREAD") return 11;
  if(status=="LATE_PENDING_BLOCK") return 12;
  if(status=="LATE_POSITION_BLOCK") return 13;
  if(status=="DAILY_RISK_BLOCK" || status=="LATE_DAILY_RISK_BLOCK") return 14;
  if(status=="LATE_QUOTA_BLOCK") return 15;
  if(status=="LATE_DIRECTION_BLOCK") return 16;
  if(status=="LATE_GLASS_BLOCK") return 17;
```

```
  if(status=="LATE_VWAP_INVALID") return 18;
   if(status=="STOPS_LEVEL_BLOCK") return 19;
   if(status=="NO_MARGIN") return 20;
   return 0;
}

int DOMRouteCode(const string &mode)
{
  if(mode=="BASE") return 1;
  if(mode=="TURBO") return 2;
  if(mode=="BASE_CONT") return 3;
  if(mode=="TURBO_CONT") return 4;
  return 0;
}

void DOMCopyCorrelation(DOMTelemetryEvent &event)
{
  event.signalBar=gLastMatrixEvalBar;
  event.contextBar=gMacro.barTime;
  event.positionID=gOpenPos.active ? gOpenPos.positionID : 0;
  event.entryActive=gPendingExec.active;
  if(event.entryActive)
  {
    event.entryOrder=gPendingExec.orderTicket;
    event.entryDeal=gPendingExec.resultDeal;
    event.entryRequestID=gPendingExec.requestID;
  }
  event.exitActive=gPendingExit.active;
  if(event.exitActive)
  {
    event.exitPositionID=gPendingExit.positionID;
    event.exitOrder=gPendingExit.orderTicket;
    event.exitDeal=gPendingExit.resultDeal;
    event.exitRequestID=gPendingExit.requestID;
  }
  event.snapshot=gDOMCurrent;
}

void DOMObserveAudit(const string &status,const string &direction,
       const string &mode,const double price,const double trigger)
{
  if(!Inp_DOM_Enable || !gDOMRuntimeReady) return;
  int code=DOMAuditStatusCode(status);
  if(code==0) return;
  if(!gDOMTelemetryEnabled) { gDOMDroppedEvents++; return; }
  DOMTelemetryEvent event;
  ZeroMemory(event);
  event.kind=2; event.status=code; event.localMS=GetTickCount64();
```

```
  event.direction=(direction=="BUY" ? 1 : (direction=="SELL" ? -1 : 0));
   event.route=DOMRouteCode(mode); event.price=price; event.trigger=trigger;
   DOMCopyCorrelation(event);
   DOMPushEvent(event); // Fixed numeric copy only: no file/book/history/allocation.
}

void DOMObserveTrade(const TradeRecord &record)
{
  if(!Inp_DOM_Enable || !gDOMRuntimeReady) return;
  if(!gDOMTelemetryEnabled) { gDOMDroppedEvents++; return; }
  DOMTelemetryEvent event;
  ZeroMemory(event);
  event.kind=3; event.localMS=GetTickCount64();
   event.direction=record.direction; event.route=DOMRouteCode(record.mode);
   // A recovered row can refer to a different cycle from the current pending
   // request. Copy only identifiers present in this exact record; unknown
   // request/deal identities remain zero, never borrowed from another cycle.
   event.signalBar=gLastMatrixEvalBar; event.contextBar=gMacro.barTime;
   event.snapshot=gDOMCurrent;
   event.positionID=record.positionID; event.entryOrder=record.entryOrderTicket;
   event.exitDeal=record.lastExitDealTicket;
   event.price=record.entryPrice; event.lots=record.volume;
   event.sl=record.slPrice; event.tp=record.tpPrice; event.netProfit=record.netProfit;
   event.closed=record.closed; event.volumeComplete=record.volumeClosedComplete;
   event.accountingResolved=record.accountingResolved;
   event.finalizationWritten=record.finalizationWritten;
   // The caller's durable CSV write is the event being observed. These copied
   // flags are telemetry only and cannot establish economic resolution.
   DOMPushEvent(event);
}

bool DOMTelemetryNamespace()
{
  if(StringLen(Inp_DOM_InstanceTag)<1 || StringLen(Inp_DOM_InstanceTag)>32)
     return false;
  for(int i=0;i<StringLen(Inp_DOM_InstanceTag);i++)
  {
     ushort c=StringGetCharacter(Inp_DOM_InstanceTag,i);
     if(!((c>='a' && c<='z') || (c>='A' && c<='Z') ||
          (c>='0' && c<='9') || c=='_' || c=='-')) return false;
  }
  string server=AccountInfoString(ACCOUNT_SERVER);
  string symbol=Symbol();
  long account=AccountInfoInteger(ACCOUNT_LOGIN);
  if(account<=0 || StringLen(server)<1 || StringLen(server)>512 ||
     StringLen(symbol)<1 || StringLen(symbol)>512 || gMagicCore<=0) return false;
  // Length-delimited UTF-8 inputs avoid delimiter ambiguity. Full SHA-256
  // scopes only these new telemetry files, never financial persistence.
```

```
  string canonical=(string)StringLen(server)+":"+server+"|"+(string)account+
      "|"+(string)StringLen(symbol)+":"+symbol+"|"+(string)gMagicCore+
      "|"+(string)StringLen(Inp_DOM_InstanceTag)+":"+Inp_DOM_InstanceTag;
    uchar data[], key[], digest[];
    int size=StringToCharArray(canonical,data,0,WHOLE_ARRAY,CP_UTF8);
    if(size<=1 || size>8192 || ArraySize(data)!=size || data[size-1]!=0 ||
       ArrayResize(data,size-1)!=size-1) return false;
    ResetLastError();
    int written=CryptEncode(CRYPT_HASH_SHA256,data,key,digest);
    if(written!=32 || ArraySize(digest)!=32 || GetLastError()!=0) return false;
    string hash="";
    for(int i=0;i<32;i++) hash+=StringFormat("%02x",(uint)digest[i]);
    gDOMTelemetryPrefix="DOMOBS_1123_"+hash+"_";
      return true;
}

string DOMCsvField(string value)
{
  StringReplace(value,"\"","\"\"");
  StringReplace(value,"\r"," "); StringReplace(value,"\n"," ");
  return "\""+value+"\"";
}

string DOMDayString()
{
  MqlDateTime parts;
  TimeToStruct(TimeCurrent(),parts);
  return StringFormat("%04d%02d%02d",parts.year,parts.mon,parts.day);
}

void DOMDisableTelemetry(const string reason)
{
  if(gDOMTelemetryEnabled)
     Print("[SCALPER_DOM][TELEMETRY_SUSPENDED] ",reason,
      " queued=",gDOMEventCount," dropped=",gDOMDroppedEvents);
  gDOMTelemetryEnabled=false;
  gDOMDroppedEvents+=(ulong)gDOMEventCount;
  gDOMEventHead=0; gDOMEventCount=0;
}

bool DOMOpenTelemetryDay()
{
  // One stable file per account/server/symbol/Magic/tag/day. No directory
  // scans, chart-ID filenames, file deletion or financial-file inspection.
  gDOMTelemetryDay=DOMDayString();
  gDOMTelemetryPath=gDOMTelemetryPrefix+gDOMTelemetryDay+".csv";
  gDOMTelemetryEnabled=true;
  ResetLastError();
```

```
  int handle=FileOpen(gDOMTelemetryPath,
     FILE_READ|FILE_WRITE|FILE_BIN|FILE_SHARE_READ);
  if(handle==INVALID_HANDLE)
  { gDOMTelemetryErrors++; DOMDisableTelemetry("cannot open daily file"); return false; }
  ResetLastError();
  ulong bytes=FileSize(handle);
  bool valid=(GetLastError()==0 && bytes<=(ulong)DOM_TELEMETRY_DAY_BYTES);
  if(valid && bytes>0)
  {
     // Never append behind an interrupted partial row. A failed query is
     // uncertainty, not proof that the file is absent or empty.
     if(bytes<2 || !FileSeek(handle,-2,SEEK_END)) valid=false;
     else
   {
        uchar tail[2];
        ResetLastError();
        uint read=FileReadArray(handle,tail,0,2);
        if(read!=2 || GetLastError()!=0 || tail[0]!=13 || tail[1]!=10) valid=false;
   }
  }
  ResetLastError(); FileClose(handle);
  if(GetLastError()!=0) valid=false;
  if(!valid)
  { gDOMTelemetryErrors++; DOMDisableTelemetry("unverified size or incomplete row");
return false; }
  gDOMTelemetryBytes=bytes; gDOMRunStartOffset=bytes;
  if(bytes>=(ulong)DOM_TELEMETRY_DAY_BYTES)
  { DOMDisableTelemetry("daily 10 MiB cap reached"); return false; }
  return true;
}

string DOMTelemetryHeader()
{
   string
row="account;server;symbol;magic;chart;instance;day;run_start_offset;event_seq;kind;status
_code;local_ms;signal_bar;context_bar;direction;route;position_id;entry_active;entry_order;e
ntry_deal;entry_request_id;exit_active;exit_position_id;exit_order;exit_deal;exit_request_id;p
rice;trigger;lots;sl;tp;net_profit;closed;volume_complete;accounting_resolved;finalization_writ
ten;quality;sample_ms;sample_seq;book_valid;bid_levels;ask_levels;bid;ask;spread;bid_vol
1;ask_vol1;bid_vol5;ask_vol5;i5;microprice;micro_minus_mid;market_buy_vol;market_sell_v
ol;invalid_records;zero_volume_records;fallback_records";
   for(int i=1;i<=5;i++)
    row+=";bid_price_"+(string)i+";bid_volume_"+(string)i+
       ";ask_price_"+(string)i+";ask_volume_"+(string)i;
   return
row+";api_duration_us;aggregation_us;sample_api_error;latency_kind;samples_since_attac
h;latency_window_count;p50_us;p95_us;p99_us;max_since_attach_us;terminal_memory_m
b;dropped;sampling_gaps;api_failures;io_errors;oversized_books;api_error\r\n";
```

```
}
string DOMNumber(const bool available,const double value)
{
  return available ? StringFormat("%.17g",value) : "NA";
}

string DOMTelemetryRow(const DOMTelemetryEvent &event)
{
  DOMSnapshot snapshot=event.snapshot;
  string kind=(event.kind==1 ? "SNAPSHOT" : (event.kind==2 ? "BASE_AUDIT" :
       (event.kind==3 ? "BASE_TRADE_CSV_WRITTEN" : "LATENCY")));
  string row=(string)AccountInfoInteger(ACCOUNT_LOGIN)+";"+
DOMCsvField(AccountInfoString(ACCOUNT_SERVER))+";"+DOMCsvField(Symbol())+";"+
  (string)gMagicCore+";"+(string)ChartID()+";"+DOMCsvField(Inp_DOM_InstanceTag)+";"+
  gDOMTelemetryDay+";"+(string)gDOMRunStartOffset+";"+(string)event.sequence+";"+
  kind+";"+(string)event.status+";"+(string)event.localMS+";"+
  (string)(long)event.signalBar+";"+(string)(long)event.contextBar+";"+
  (string)event.direction+";"+(string)event.route+";"+(string)event.positionID+";"+

(string)(int)event.entryActive+";"+(string)event.entryOrder+";"+(string)event.entryDeal+";"+
   (string)event.entryRequestID+";"+(string)(int)event.exitActive+";"+
   (string)event.exitPositionID+";"+(string)event.exitOrder+";"+(string)event.exitDeal+";"+
   (string)event.exitRequestID+";"+StringFormat("%.17g",event.price)+";"+
   StringFormat("%.17g",event.trigger)+";"+StringFormat("%.17g",event.lots)+";"+
   StringFormat("%.17g",event.sl)+";"+StringFormat("%.17g",event.tp)+";"+
   (event.kind==3 ? StringFormat("%.17g",event.netProfit) : "NA")+";"+
   (string)(int)event.closed+";"+(string)(int)event.volumeComplete+";"+
   (string)(int)event.accountingResolved+";"+(string)(int)event.finalizationWritten+";"+
   DOMQualityName(snapshot.quality)+";"+(string)snapshot.collectedMS+";"+
   (string)snapshot.sequence+";"+(string)(int)snapshot.bookValid+";"+
   (string)snapshot.bidLevels+";"+(string)snapshot.askLevels+";"+
   DOMNumber(snapshot.bookValid,snapshot.bestBid)+";"+
   DOMNumber(snapshot.bookValid,snapshot.bestAsk)+";"+
   DOMNumber(snapshot.bookValid,snapshot.spread)+";"+
   DOMNumber(snapshot.bookValid,snapshot.bidVolume1)+";"+
   DOMNumber(snapshot.bookValid,snapshot.askVolume1)+";"+
   DOMNumber(snapshot.i5Available,snapshot.bidVolume5)+";"+
   DOMNumber(snapshot.i5Available,snapshot.askVolume5)+";"+
   DOMNumber(snapshot.i5Available,snapshot.i5)+";"+
   DOMNumber(snapshot.micropriceAvailable,snapshot.microprice)+";"+
   DOMNumber(snapshot.micropriceAvailable,snapshot.micropriceMinusMid)+";"+
   DOMNumber(snapshot.bookValid,snapshot.marketBuyVolume)+";"+
   DOMNumber(snapshot.bookValid,snapshot.marketSellVolume)+";"+
   (string)snapshot.invalidRecords+";"+(string)snapshot.zeroVolumeRecords+";"+
   (string)snapshot.volumeFallbackRecords;
 for(int i=0;i<5;i++)
 {
   bool bid=snapshot.bookValid && i<snapshot.bidLevels;
```

```
  bool ask=snapshot.bookValid && i<snapshot.askLevels;
 row+=";"+DOMNumber(bid,snapshot.bidDepthPrice[i])+";"+
  DOMNumber(bid,snapshot.bidDepthVolume[i])+";"+
  DOMNumber(ask,snapshot.askDepthPrice[i])+";"+
  DOMNumber(ask,snapshot.askDepthVolume[i]);
}
return row+";"+(event.apiCalled ? (string)event.apiDurationUS : "NA")+";"+
  (event.aggregationDone ? (string)event.aggregationUS : "NA")+";"+
  (event.apiCalled ? (string)event.apiError : "NA")+";"+
  (string)event.latencyKind+";"+(string)event.samples+";"+
  (string)(int)MathMin((ulong)DOM_LATENCY_WINDOW,event.samples)+";"+
 (event.percentilesAvailable ? (string)event.p50 : "NA")+";"+
 (event.percentilesAvailable ? (string)event.p95 : "NA")+";"+
 (event.percentilesAvailable ? (string)event.p99 : "NA")+";"+

(string)event.maximum+";"+(string)TerminalInfoInteger(TERMINAL_MEMORY_USED)+";"+
   (string)gDOMDroppedEvents+";"+(string)gDOMSamplingGaps+";"+
   (string)gDOMApiFailures+";"+(string)gDOMTelemetryErrors+";"+
   (string)gDOMOversizedBooks+";"+(string)gDOMLastApiError+"\r\n";
}

void DOMContainment(const bool slow,const ulong nowMS,const bool disk=false)
{
  if(!slow)
  {
     if(disk) gDOMSlowWriteRun=0; else gDOMSlowRun=0;
     return;
  }
  if(disk) gDOMSlowWriteRun++; else gDOMSlowRun++;
  int count=disk ? gDOMSlowWriteRun : gDOMSlowRun;
  if(count<3) return;
  if(disk)
  {
     gDOMSlowWriteRun=0;
     DOMDisableTelemetry("three slow writes; only telemetry stopped");
     return;
  }
  gDOMSlowRun=0; gDOMContainmentEvents++; gDOMSamplingGaps++;
  if(gDOMContainmentEvents>=3) gDOMCollectionEnabled=false;
  else gDOMSuspendUntilMS=nowMS+(gDOMContainmentEvents==1 ? 5000 : 30000);
  Print("[SCALPER_DOM][COLLECTION_CONTAINMENT]
event=",gDOMContainmentEvents,
        " collection=",gDOMCollectionEnabled,
        "; synchronous calls are measured after return, not preempted.");
}

void DOMFlushTelemetry(const ulong nowMS)
{
```

```
if(!Inp_DOM_Enable || !gDOMTelemetryEnabled || gDOMEventCount==0) return;
if(gDOMEventCount<8 && nowMS-gDOMLastFlushMS<5000) return;
ulong begin=GetMicrosecondCount();
ResetLastError();
int handle=FileOpen(gDOMTelemetryPath,
   FILE_READ|FILE_WRITE|FILE_BIN|FILE_SHARE_READ);
if(handle==INVALID_HANDLE)
{ gDOMTelemetryErrors++; DOMDisableTelemetry("open failure"); return; }
ResetLastError();
ulong existing=FileSize(handle);
if(GetLastError()!=0 || existing!=gDOMTelemetryBytes || !FileSeek(handle,0,SEEK_END))
 {
    FileClose(handle); gDOMTelemetryErrors++;
    DOMDisableTelemetry("unverified or externally changed size"); return;
 }
 int batch=0; bool failed=false;
 while(batch<16 && gDOMEventCount>0)
 {
   string line=(existing==0 ? DOMTelemetryHeader() : "");
   line+=DOMTelemetryRow(gDOMEvents[gDOMEventHead]);
   if(StringLen(line)>8192)
   { DOMDisableTelemetry("bounded line limit"); break; }
   uchar encoded[];
   int count=StringToCharArray(line,encoded,0,WHOLE_ARRAY,CP_UTF8);
   int bytes=count-1;
   if(count<=1 || ArraySize(encoded)!=count || encoded[count-1]!=0 ||
      (ulong)bytes>(ulong)DOM_TELEMETRY_DAY_BYTES-gDOMTelemetryBytes)
   { ArrayFree(encoded); DOMDisableTelemetry("daily cap or encoding failure"); break; }
   ResetLastError();
   uint written=FileWriteArray(handle,encoded,0,bytes);
   int writeError=GetLastError();
   ArrayFree(encoded);
   ResetLastError();
   ulong after=FileSize(handle);
   if(written!=(uint)bytes || writeError!=0 || GetLastError()!=0 ||
      after<existing || after-existing!=(ulong)bytes)
   { failed=true; break; }
   gDOMTelemetryBytes=after; existing=after;
   gDOMEventHead=(gDOMEventHead+1)%DOM_EVENT_CAPACITY;
   gDOMEventCount--; batch++;
 }
 ResetLastError(); FileFlush(handle);
 if(GetLastError()!=0) failed=true;
 ResetLastError(); FileClose(handle);
 if(GetLastError()!=0) failed=true;
 gDOMLastFlushMS=nowMS;
 ulong elapsed=GetMicrosecondCount()-begin;
 DOMLatency(DOM_LAT_WRITE,elapsed);
```

```
   if(failed)
   { gDOMTelemetryErrors++; DOMDisableTelemetry("partial or failed write; inspect CSV"); }
   DOMContainment(elapsed>5000,GetTickCount64(),true);
}

void DOMQueueLatencySummary(const ulong nowMS)
{
 for(int kind=0;kind<4;kind++)
 {
   DOMTelemetryEvent event;
   ZeroMemory(event);
   event.kind=4; event.localMS=nowMS; event.latencyKind=kind;
   event.samples=gDOMLatencySeen[kind];
event.maximum=gDOMLatencyMaximum[kind];
   int count=(int)MathMin((ulong)DOM_LATENCY_WINDOW,event.samples);
   if(count>=100)
   {
     ulong sorted[DOM_LATENCY_WINDOW];
     for(int i=0;i<count;i++) sorted[i]=gDOMLatencySamples[kind][i];
     for(int i=1;i<count;i++)
     {
       ulong value=sorted[i]; int j=i-1;
       while(j>=0 && sorted[j]>value) { sorted[j+1]=sorted[j]; j--; }
       sorted[j+1]=value;
     }
     event.p50=sorted[(int)MathCeil(count*0.50)-1];
     event.p95=sorted[(int)MathCeil(count*0.95)-1];
     event.p99=sorted[(int)MathCeil(count*0.99)-1];
     event.percentilesAvailable=true;
   }
   DOMPushEvent(event);
 }
}

void DOMRuntimeInit()
{
 if(!Inp_DOM_Enable) return;
 DOMResetCache();
 // Conservative allowance includes bounded line/UTF-8/hash temporaries and
 // fixed scalar/string metadata. MarketBookGet's API-returned array cannot
 // be hard-capped by its public signature; it is separately released below.
 long ownBytes=(long)sizeof(gDOMEvents)+(long)sizeof(gDOMCurrent)+
   (long)sizeof(gDOMBidPrices)+(long)sizeof(gDOMBidVolumes)+
   (long)sizeof(gDOMAskPrices)+(long)sizeof(gDOMAskVolumes)+
   (long)sizeof(gDOMLatencySamples)+65536;
 if(ownBytes>1048576 || !DOMTelemetryNamespace())
 {
```

```
    Print("[SCALPER_DOM][UNAVAILABLE] invalid telemetry identity or buffer bound; base
unchanged.");
    return; // Never fail the EA initialization because of DOM.
   }
   gDOMBookSymbol=Symbol(); gDOMRuntimeReady=true;
   gDOMLastFlushMS=GetTickCount64(); gDOMLastSummaryMS=gDOMLastFlushMS;
   DOMOpenTelemetryDay();
   ResetLastError();
   gDOMSubscriptionOwned=MarketBookAdd(gDOMBookSymbol);
   gDOMLastApiError=GetLastError();
   gDOMCollectionEnabled=gDOMSubscriptionOwned;
   if(!gDOMSubscriptionOwned)
     Print("[SCALPER_DOM][SUBSCRIPTION_FAILED] error=",gDOMLastApiError,
       "; no repeated subscription attempts; base unchanged.");
    Print("[SCALPER_DOM][OBSERVATION_ONLY] own_buffer_bound=",ownBytes,
      " max_hz=1; local capture is not exchange freshness.");
}

void DOMRuntimeTimer()
{
 if(!Inp_DOM_Enable || !gDOMRuntimeReady) return;
 ulong beginRuntime=GetMicrosecondCount();
 ulong nowMS=GetTickCount64();
 if(DOMDayString()!=gDOMTelemetryDay)
 {
   gDOMDroppedEvents+=(ulong)gDOMEventCount;
   gDOMEventHead=0; gDOMEventCount=0;
   DOMOpenTelemetryDay(); // At most one storage re-open attempt per new day.
 }
 if(gDOMLastCollectMS==0 || nowMS-gDOMLastCollectMS>=1000)
 {
   if(gDOMLastCollectMS>0 && nowMS-gDOMLastCollectMS>1500)
gDOMSamplingGaps++;
   gDOMLastCollectMS=nowMS;
   DOMSnapshot snapshot;
   DOMClearSnapshot(snapshot); snapshot.collectedMS=nowMS;
   bool apiCalled=false, aggregationDone=false;
   ulong apiDurationUS=0, aggregationUS=0;
   int capturedApiError=0;
   if(!gDOMSubscriptionOwned) snapshot.quality=DOM_QUALITY_NO_SUBSCRIPTION;
   else if(!gDOMCollectionEnabled || nowMS<gDOMSuspendUntilMS)
     snapshot.quality=DOM_QUALITY_NONE;
   else if(Symbol()!=gDOMBookSymbol)
snapshot.quality=DOM_QUALITY_SYMBOL_MISMATCH;
   else if(!TerminalInfoInteger(TERMINAL_CONNECTED))
snapshot.quality=DOM_QUALITY_DISCONNECTED;
   else
   {
```

```
      MqlBookInfo raw[];
      ResetLastError();
      ulong begin=GetMicrosecondCount();
      bool received=MarketBookGet(gDOMBookSymbol,raw);
      ulong apiElapsed=GetMicrosecondCount()-begin;
      int apiError=GetLastError();
      apiCalled=true; apiDurationUS=apiElapsed; capturedApiError=apiError;
      DOMLatency(DOM_LAT_API,apiElapsed); gDOMLastApiError=apiError;
      snapshot.collectedMS=GetTickCount64(); // Local receipt, NOT B3 time.
      ulong aggregateElapsed=0;
      if(!received || apiError!=0)
      {
       snapshot.quality=DOM_QUALITY_API_FAILURE;
       gDOMApiFailures++; gDOMApiFailureRun++;
       if(gDOMApiFailureRun>=3)
       {
         gDOMCollectionEnabled=false;
         Print("[SCALPER_DOM][COLLECTION_SUSPENDED] three API failures; base
unchanged.");
       }
      }
      else
      {
       gDOMApiFailureRun=0; begin=GetMicrosecondCount();
       DOMBuildSnapshot(raw,ArraySize(raw),snapshot.collectedMS,snapshot);
       aggregateElapsed=GetMicrosecondCount()-begin;
       aggregationDone=true; aggregationUS=aggregateElapsed;
       DOMLatency(DOM_LAT_AGG,aggregateElapsed);
       if(snapshot.quality==DOM_QUALITY_TOO_LARGE) gDOMOversizedBooks++;
      }
      ArrayFree(raw);
      DOMContainment(apiElapsed>2000 || aggregateElapsed>2000,GetTickCount64());
    }
    DOMStoreSnapshot(snapshot);
    DOMTelemetryEvent event;
    ZeroMemory(event);
    event.kind=1; event.localMS=snapshot.collectedMS;
    DOMCopyCorrelation(event);
    event.apiCalled=apiCalled; event.apiDurationUS=apiDurationUS;
    event.aggregationDone=aggregationDone; event.aggregationUS=aggregationUS;
    event.apiError=capturedApiError;
    DOMPushEvent(event);
  }
  if(nowMS-gDOMLastSummaryMS>=60000)
  { DOMQueueLatencySummary(nowMS); gDOMLastSummaryMS=nowMS; }
  DOMFlushTelemetry(nowMS);
  DOMLatency(DOM_LAT_RUNTIME,GetMicrosecondCount()-beginRuntime);
}
```

```
void DOMRuntimeDeinit()
{
   if(!Inp_DOM_Enable) return;
   if(gDOMRuntimeReady)
   {
     gDOMLastFlushMS=0;
     DOMFlushTelemetry(GetTickCount64()); // One bounded batch, never a wait loop.
   }
   if(gDOMSubscriptionOwned)
   {
     ResetLastError();
     if(!MarketBookRelease(gDOMBookSymbol))
       Print("[SCALPER_DOM][RELEASE_FAILED] error=",GetLastError());
     gDOMSubscriptionOwned=false;
    }
    if(gDOMEventCount>0) gDOMDroppedEvents+=(ulong)gDOMEventCount;
    Print("[SCALPER_DOM][SUMMARY] dropped=",gDOMDroppedEvents,
       " gaps=",gDOMSamplingGaps," api_failures=",gDOMApiFailures,
       " io_errors=",gDOMTelemetryErrors," oversized=",gDOMOversizedBooks);
    gDOMEventHead=0; gDOMEventCount=0;
    gDOMRuntimeReady=false; gDOMCollectionEnabled=false;
}

// Optional observer runs only AFTER the original financial timer body.
void OnTimer()
{
  DOMBaseOnTimer();
  DOMGateSubscriptionTimer(); // Only after the original financial timer body.
  if(Inp_DOM_Enable) DOMRuntimeTimer();
}

// BEGIN EXPERIMENTAL DOM CONFLUENCE MODULE
// Experimental 1.124 addition. Embedded by build_candidate.py into the full EA.
// No cached DOM approval, no alternate order path, no financial state persistence.
enum ENUM_BOOK_CONFLUENCE
{
  BOOK_CONFIRM_BUY=0, BOOK_CONFIRM_SELL, BOOK_NEUTRAL_WAIT,
  BOOK_OPPOSITE_BUY, BOOK_OPPOSITE_SELL, BOOK_UNAVAILABLE,
  BOOK_REVERSAL_WATCH, BOOK_ENTRY_RELEASED, BOOK_ENTRY_BLOCKED
};
enum ENUM_SEND_OUTCOME
{
  SEND_FINAL_FAILURE=0, SEND_ACCEPTED, SEND_WAIT_ARMED,
  SEND_CANCEL_REEVAL, SEND_TRADE_OR_POSITION,
  SEND_FINANCIAL_UNRESOLVED, SEND_ORIGINAL_CANCEL
};

// Experimental local budgets, not exchange freshness guarantees or strategy inputs.
```

```
// Reject rather than trade after an unbounded read/revalidation/persistence delay.
#define BOOK_GATE_API_BUDGET_US 10000
#define BOOK_GATE_AGG_BUDGET_US 5000
#define BOOK_GATE_DECISION_BUDGET_US 50000
#define BOOK_GATE_LOGS_PER_DAY 512

struct BookGateDecision
{
  DOMSnapshot snapshot;
  ENUM_BOOK_CONFLUENCE state;
  double pctBid, pctAsk;
  ulong beginUS, apiUS, aggregationUS;
  datetime signalClock, contextClock;
  int apiError;
  string reason;
};

bool gBookGateSubscriptionOwned=false;
string gBookGateSymbol="";
bool gBookGateWaiting=false, gBookGateOppositeWatch=false;
bool gBookGateHadOpposite=false;
ulong gBookGateObservationMS=0;
ulong gBookGateNextSubscriptionMS=0, gBookGateRetryDelayMS=5000;
ulong gBookGateSubscriptionAttempts=0, gBookGateSubscriptionRecoveries=0;
ulong gBookGateApiBudgetBlocks=0, gBookGateAggregationBudgetBlocks=0;
ulong gBookGateDecisionBudgetBlocks=0, gBookGateQuoteDeferrals=0;
ulong gBookGatePrepareMaxUS=0, gBookGateJournalMaxUS=0;
double gBookGatePrepareSumUS=0, gBookGateJournalSumUS=0;
ulong gBookGatePrepareCalls=0, gBookGateJournalCalls=0;
datetime gBookGateArmedAt=0, gBookGateRegionEnd=0;
int gBookGateDirection=0, gBookGateLastState=-1;
double gBookGateRegionHigh=0, gBookGateRegionLow=0;
ulong gBookGateLastLogMS=0;
int gBookGateLogDay=0, gBookGateDayLogs=0;
ulong gBookGateReads=0, gBookGateBlocks=0, gBookGateReleased=0;
ulong gBookGateUnavailable=0, gBookGateLogsDropped=0;
ulong gBookGateApiMaxUS=0, gBookGateAggregationMaxUS=0;
ulong gBookGateDecisionMaxUS=0, gBookGateDispatchMaxUS=0;
ulong gBookGateRevalidationMaxUS=0, gBookGateRevalidations=0;
double gBookGateRevalidationSumUS=0;
ulong gBookGateLateDeferrals=0;
double gBookGateApiSumUS=0, gBookGateAggregationSumUS=0;
ulong gBookGateApiCalls=0, gBookGateAggregations=0;

bool DOMGateValidateInputs()
{
 if(!DOMFinite(Inp_DOM_MinDominancePct) ||
    Inp_DOM_MinDominancePct<55.0 || Inp_DOM_MinDominancePct>100.0)
```

```
  {
    Print("[BOOK_GATE][INVALID_INPUT] Inp_DOM_MinDominancePct must be finite and
in [55,100]; no correction.");
   return false;
  }
  if(Inp_DOM_MinDominancePct==100.0)
    Print("[BOOK_GATE][INPUT_WARNING] 100% is mathematically unattainable with
positive volume on both sides; floating point percentages do not prove zero opposing
volume.");
  return true;
}

// Pure classification: I5 and microprice are descriptive, never extra trade filters.
ENUM_BOOK_CONFLUENCE DOMGateClassify(const DOMSnapshot &s,const int
direction,
                const double threshold,double &pctBid,double &pctAsk)
{
   pctBid=0.0; pctAsk=0.0;
   if(!DOMFinite(threshold) || threshold<55.0 || threshold>100.0 ||
      (direction!=1 && direction!=-1) || !s.bookValid ||
      s.quality!=DOM_QUALITY_OK || s.bidLevels<5 || s.askLevels<5 ||
      !DOMFinite(s.bidVolume5) || !DOMFinite(s.askVolume5) ||
      s.bidVolume5<=0.0 || s.askVolume5<=0.0)
      return BOOK_UNAVAILABLE;
   double total=s.bidVolume5+s.askVolume5;
   if(!DOMFinite(total) || total<=0.0) return BOOK_UNAVAILABLE;
   pctBid=100.0*(s.bidVolume5/total);
   pctAsk=100.0*(s.askVolume5/total);
   if(!DOMFinite(pctBid) || !DOMFinite(pctAsk)) return BOOK_UNAVAILABLE;
   // Positive sizes on both sides cannot yield a mathematical 100% dominance.
   // Avoid accidental release from floating point rounding at the endpoint.
   if(threshold==100.0) return BOOK_NEUTRAL_WAIT;
   if(direction==1)
   {
      if(pctBid>=threshold) return BOOK_CONFIRM_BUY;
      if(pctAsk>=threshold) return BOOK_OPPOSITE_SELL;
   }
   else
   {
      if(pctAsk>=threshold) return BOOK_CONFIRM_SELL;
      if(pctBid>=threshold) return BOOK_OPPOSITE_BUY;
   }
   return BOOK_NEUTRAL_WAIT;
}

string DOMGateStateName(const ENUM_BOOK_CONFLUENCE state)
{
  switch(state)
```

```
  {
    case BOOK_CONFIRM_BUY: return "BOOK_CONFIRM_BUY";
    case BOOK_CONFIRM_SELL: return "BOOK_CONFIRM_SELL";
     case BOOK_NEUTRAL_WAIT: return "BOOK_NEUTRAL_WAIT";
     case BOOK_OPPOSITE_BUY: return "BOOK_OPPOSITE_BUY";
     case BOOK_OPPOSITE_SELL: return "BOOK_OPPOSITE_SELL";
     case BOOK_UNAVAILABLE: return "BOOK_UNAVAILABLE";
     case BOOK_REVERSAL_WATCH: return "BOOK_REVERSAL_WATCH";
     case BOOK_ENTRY_RELEASED: return "BOOK_ENTRY_RELEASED";
     case BOOK_ENTRY_BLOCKED: return "BOOK_ENTRY_BLOCKED";
    }
    return "BOOK_UNAVAILABLE";
}

void DOMGateSyncIdentity()
{
  if(gBookGateArmedAt==armed.armedAt && gBookGateDirection==armed.dir &&
     gBookGateRegionEnd==armed.regionEndTime &&
     gBookGateRegionHigh==armed.regionHigh &&
gBookGateRegionLow==armed.regionLow)
     return;
  gBookGateArmedAt=armed.armedAt; gBookGateDirection=armed.dir;
  gBookGateRegionEnd=armed.regionEndTime;
  gBookGateRegionHigh=armed.regionHigh; gBookGateRegionLow=armed.regionLow;
  gBookGateWaiting=false; gBookGateOppositeWatch=false;
  gBookGateHadOpposite=false; gBookGateObservationMS=0;
  gBookGateLastState=-1; gBookGateLastLogMS=0;
}

bool DOMGateLogAllowed()
{
  MqlDateTime dt; TimeToStruct(TimeTradeServer(),dt);
  int day=dt.year*1000+dt.day_of_year;
  if(day!=gBookGateLogDay) { gBookGateLogDay=day; gBookGateDayLogs=0; }
  if(gBookGateDayLogs>=BOOK_GATE_LOGS_PER_DAY)
  { gBookGateLogsDropped++; return false; }
  gBookGateDayLogs++;
  return true;
}

void DOMGateReport(const BookGateDecision &decision,const bool confirmed)
{
  DOMGateSyncIdentity();
  bool wasWaiting=gBookGateWaiting;
  if(!confirmed)
  {
    gBookGateBlocks++; gBookGateWaiting=true;
    if(decision.state==BOOK_UNAVAILABLE) gBookGateUnavailable++;
```

```
  }
  bool opposite=(decision.state==BOOK_OPPOSITE_BUY ||
decision.state==BOOK_OPPOSITE_SELL);
 // This flag describes this read only; historical opposition is separate.
 gBookGateOppositeWatch=opposite;
 // A local intent/price rejection is not a new depth observation.

gBookGateObservationMS=(decision.state==BOOK_ENTRY_BLOCKED)?0:GetTickCount6
4();
 if(opposite) gBookGateHadOpposite=true;
 ulong nowMS=GetTickCount64();
 // First confirmation/block plus at most one changed state per second per signal.
 bool emit=(gBookGateLastState<0 || (!confirmed && !wasWaiting) ||
       (gBookGateLastState!=(int)decision.state &&
nowMS-gBookGateLastLogMS>=1000));
 if(emit && DOMGateLogAllowed())
 {
  PrintFormat("[BOOK_GATE][%s] state=%s attention=%s symbol=%s dir=%d
armedAt=%I64d bidPct=%.8f askPct=%.8f min=%.2f quality=%s reason=%s apiError=%d
apiUS=%I64u aggregationUS=%I64u; local receipt is not exchange age",

confirmed?"BOOK_CONFIRM":"BOOK_ENTRY_BLOCKED",DOMGateStateName(decision.
state),
     opposite?"BOOK_REVERSAL_WATCH":"NONE",gBookGateSymbol,armed.dir,
     (long)armed.armedAt,decision.pctBid,decision.pctAsk,Inp_DOM_MinDominancePct,
     DOMQualityName(decision.snapshot.quality),decision.reason,decision.apiError,
     decision.apiUS,decision.aggregationUS);
    gBookGateLastState=(int)decision.state; gBookGateLastLogMS=nowMS;
  }
  else if(!emit) gBookGateLogsDropped++;
}

ENUM_SEND_OUTCOME DOMGateDefer(const BookGateDecision &decision,const string
reason)
{
  if(HasExecutionInFlight() || gEntryJournalInvalid || gExitJournalInvalid)
     return SEND_FINANCIAL_UNRESOLVED;
  if(!armed.active) return SEND_ORIGINAL_CANCEL;
  ArmedSignal original=armed;
  double effort,trigger,last;
  MqlTick current;
  ENUM_APS readiness=CheckArmedEntryReady(effort,trigger,last,current);
  if(readiness==APS_ENDED_REEVAL_ALLOWED) return SEND_CANCEL_REEVAL;
  if(readiness==APS_TRADE_OR_POSITION) return SEND_TRADE_OR_POSITION;
  if(readiness==APS_ENDED_FAIL_CLOSED || !armed.active) return
SEND_ORIGINAL_CANCEL;
  if(readiness==APS_ENTRY_READY)
  {
```

```
    // A sampling/DOM veto must not hide a late execution failure on this
    // same sample. Stage2's own STILL result retains its original meaning.
    double price=(original.dir==1)?current.ask:current.bid;
    if(!DOMGateFinalTickReady(current,original,trigger,price,readiness) &&
       readiness==APS_ENDED_FAIL_CLOSED) return SEND_ORIGINAL_CANCEL;
    double sl=NormEntrySL(original.dir,price-original.dir*original.activeSL_Pts);
    double tp=NormEntryTP(original.dir,price+original.dir*original.activeTP_Pts);
    double lots=ComputeLotsForEntry(original.dir,price,sl);
    if(!DOMGateTermsCurrent(current,original,price,lots,sl,tp,readiness) &&
       readiness==APS_ENDED_FAIL_CLOSED) return SEND_ORIGINAL_CANCEL;
  }
  BookGateDecision rejected=decision;
  rejected.state=BOOK_ENTRY_BLOCKED; rejected.reason=reason+";
source="+decision.reason;
  gBookGateLateDeferrals++;
  ulong elapsed=GetMicrosecondCount()-decision.beginUS;
  if(elapsed>gBookGateDecisionMaxUS) gBookGateDecisionMaxUS=elapsed;
  DOMGateReport(rejected,false);
  return SEND_WAIT_ARMED;
}

void DOMGateSubscriptionTimer()
{
  if(!Inp_DOM_TradeGateEnable || gBookGateSubscriptionOwned) return;
  ulong nowMS=GetTickCount64();
  if(nowMS<gBookGateNextSubscriptionMS) return;
  bool retry=(gBookGateSubscriptionAttempts>0);
  // Frequency control is operational, never an entry or signal parameter.
  gBookGateNextSubscriptionMS=nowMS+gBookGateRetryDelayMS;
  if(!TerminalInfoInteger(TERMINAL_CONNECTED))
  {
    gBookGateOppositeWatch=false; gBookGateObservationMS=0;
    Print("[BOOK_GATE][BOOK_SUBSCRIPTION_UNAVAILABLE] terminal disconnected;
no approval retained");
    return;
  }
  if(retry) PrintFormat("[BOOK_GATE][BOOK_SUBSCRIPTION_RETRY] attempt=%I64u
symbol=%s",
              gBookGateSubscriptionAttempts+1,gBookGateSymbol);
  ResetLastError();
  bool added=MarketBookAdd(gBookGateSymbol);
  int error=GetLastError();
  gBookGateSubscriptionAttempts++;
  if(added)
  {
    // Own exactly one successful reference; never re-add an owned reference.
    gBookGateSubscriptionOwned=true;
    gBookGateRetryDelayMS=5000;
```

```
    if(retry)
    {
      gBookGateSubscriptionRecoveries++;
      PrintFormat("[BOOK_GATE][BOOK_SUBSCRIPTION_RECOVERED] symbol=%s;
fresh read and original ARMED required",
            gBookGateSymbol);
    }
    return;
  }
  gBookGateOppositeWatch=false; gBookGateObservationMS=0;
  PrintFormat("[BOOK_GATE][BOOK_SUBSCRIPTION_FAILED] symbol=%s error=%d
attempt=%I64u",
         gBookGateSymbol,error,gBookGateSubscriptionAttempts);
  PrintFormat("[BOOK_GATE][BOOK_SUBSCRIPTION_UNAVAILABLE]
advertised_depth=%I64d; provider support must be verified; entries stay blocked",
         SymbolInfoInteger(gBookGateSymbol,SYMBOL_TICKS_BOOKDEPTH));
  // Backoff 5,10,20,40,60 seconds; no Sleep, no tick-driven subscription calls.

gBookGateRetryDelayMS=(ulong)MathMin((double)60000,(double)gBookGateRetryDelayM
S*2.0);
}

void DOMGateInit()
{
  gBookGateSubscriptionOwned=false; gBookGateSymbol="";
  gBookGateWaiting=false; gBookGateOppositeWatch=false; gBookGateHadOpposite=false;
  gBookGateObservationMS=0; gBookGateArmedAt=0; gBookGateDirection=0;
  gBookGateRegionEnd=0; gBookGateRegionHigh=0; gBookGateRegionLow=0;
  gBookGateLastState=-1; gBookGateNextSubscriptionMS=0;
gBookGateRetryDelayMS=5000;
  gBookGateSubscriptionAttempts=0; gBookGateSubscriptionRecoveries=0;
  if(!Inp_DOM_TradeGateEnable) return;
  gBookGateSymbol=Symbol();
  DOMGateSubscriptionTimer(); // One initial attempt; retries exclusively on timer.
  Print("[BOOK_GATE][INIT] RC2 experimental; 10/5/50ms local budgets
UNCALIBRATED;no exchange-age guarantee");
}

bool DOMGateRead(const int direction,BookGateDecision &decision)
{
  ZeroMemory(decision);
  decision.beginUS=GetMicrosecondCount();
  decision.signalClock=iTime(Symbol(),Inp_SignalTF,0);
  decision.contextClock=iTime(Symbol(),Inp_ContextTF,0);
  decision.state=BOOK_UNAVAILABLE;
  DOMClearSnapshot(decision.snapshot);
  gBookGateReads++;
  if(!gBookGateSubscriptionOwned)
```

```
    decision.snapshot.quality=DOM_QUALITY_NO_SUBSCRIPTION;
  else if(Symbol()!=gBookGateSymbol ||
      SymbolInfoInteger(gBookGateSymbol,SYMBOL_CUSTOM)!=0 ||

SymbolInfoInteger(gBookGateSymbol,SYMBOL_EXPIRATION_TIME)<=TimeTradeServer())
  decision.snapshot.quality=DOM_QUALITY_SYMBOL_MISMATCH;
else if(!TerminalInfoInteger(TERMINAL_CONNECTED))
  decision.snapshot.quality=DOM_QUALITY_DISCONNECTED;
else
{
  MqlBookInfo raw[];
  ResetLastError();
  ulong begin=GetMicrosecondCount();
  bool received=MarketBookGet(gBookGateSymbol,raw);
  decision.apiUS=GetMicrosecondCount()-begin;
  decision.apiError=GetLastError();
  gBookGateApiCalls++; gBookGateApiSumUS+=(double)decision.apiUS;
  if(decision.apiUS>gBookGateApiMaxUS) gBookGateApiMaxUS=decision.apiUS;
  if(!received || decision.apiError!=0)
    decision.snapshot.quality=DOM_QUALITY_API_FAILURE;
  else
  {
    begin=GetMicrosecondCount();
    DOMBuildSnapshot(raw,ArraySize(raw),GetTickCount64(),decision.snapshot);
    // Structural price-grid check, not an additional strategic indicator.
    if(decision.snapshot.quality==DOM_QUALITY_OK)
    {
    for(int n=0;n<ArraySize(raw);n++)
      if(raw[n].type==BOOK_TYPE_BUY || raw[n].type==BOOK_TYPE_SELL)
      {
         if(!DOMFinite(gTickSize) || gTickSize<=0.0)
         { decision.snapshot.quality=DOM_QUALITY_INVALID; break; }
         double tickUnits=raw[n].price/gTickSize;
         // An infinite ratio would otherwise make Abs(inf-inf) NaN,
         // whose comparison is false and would fail open on the grid.
         if(!DOMFinite(tickUnits) ||
          MathAbs(tickUnits-MathRound(tickUnits))>0.0000001)
        { decision.snapshot.quality=DOM_QUALITY_INVALID; break; }
    }
     }
     decision.aggregationUS=GetMicrosecondCount()-begin;
     gBookGateAggregations++;
gBookGateAggregationSumUS+=(double)decision.aggregationUS;
     if(decision.aggregationUS>gBookGateAggregationMaxUS)
       gBookGateAggregationMaxUS=decision.aggregationUS;
  }
   ArrayFree(raw);
   if(decision.apiUS>BOOK_GATE_API_BUDGET_US) gBookGateApiBudgetBlocks++;
```

```
   if(decision.aggregationUS>BOOK_GATE_AGG_BUDGET_US)
gBookGateAggregationBudgetBlocks++;
   if(decision.apiUS>BOOK_GATE_API_BUDGET_US ||
      decision.aggregationUS>BOOK_GATE_AGG_BUDGET_US)
   {
      decision.state=BOOK_UNAVAILABLE;
      decision.reason="LOCAL_READ_OR_AGGREGATION_BUDGET_EXCEEDED";
      DOMGateReport(decision,false);
      return false;
   }
}
if(decision.reason=="") decision.reason=DOMQualityName(decision.snapshot.quality);

decision.state=DOMGateClassify(decision.snapshot,direction,Inp_DOM_MinDominancePct,
            decision.pctBid,decision.pctAsk);
  bool confirmed=(direction==1 && decision.state==BOOK_CONFIRM_BUY) ||
       (direction==-1 && decision.state==BOOK_CONFIRM_SELL);
  DOMGateReport(decision,confirmed);
  return confirmed;
}


bool DOMGateIdentityCurrent(const ArmedSignal &original)
{
  return armed.active && original.active && armed.dir==original.dir &&
    armed.armedAt==original.armedAt && armed.regionEndTime==original.regionEndTime
&&
    armed.regionHigh==original.regionHigh && armed.regionLow==original.regionLow &&
    armed.route==original.route && armed.mode==original.mode &&
   armed.activeSL_Pts==original.activeSL_Pts &&
armed.activeTP_Pts==original.activeTP_Pts &&
   armed.breakoutEffortMin==original.breakoutEffortMin;
}

// These helpers are used only in the current synchronous SendOrder call.
// No local no-dispatch witness is restored from a journal or reused after return.
bool DOMGateIntentMatches(const PendingExecContext &expected)
{
  if(!gPendingExec.active || !expected.active || gPendingExec.accepted ||
     gPendingExec.orderTicket!=0 || gPendingExec.resultDeal!=0 ||
gPendingExec.requestID!=0 ||
     gPendingExec.dispatchPhase!=expected.dispatchPhase ||
     gPendingExec.direction!=expected.direction || gPendingExec.route!=expected.route ||
     gPendingExec.mode!=expected.mode ||
gPendingExec.requestPrice!=expected.requestPrice ||
     gPendingExec.requestedLots!=expected.requestedLots ||
gPendingExec.slPrice!=expected.slPrice ||
     gPendingExec.tpPrice!=expected.tpPrice || gPendingExec.sentAt!=expected.sentAt ||
```

```
    gPendingExec.requestPriceAvailable!=expected.requestPriceAvailable ||
    gPendingExec.breakoutPrice!=expected.breakoutPrice ||
    gPendingExec.breakoutPriceAvailable!=expected.breakoutPriceAvailable) return false;
  return true;
}

bool DOMGateOwnUndispatchedIntent(const PendingExecContext &expected)
{
  if(!DOMGateIntentMatches(expected)) return false;
  // All original HasExecutionInFlight reasons, excluding only this proven intent.
  if(gOrderObligationScanUnresolved) return false;
  for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
    if(gUnresolvedOrders[i].active && (gUnresolvedOrders[i].classificationPending ||
       gUnresolvedOrders[i].persistencePending)) return false;
  if(gRequestJournalUnresolved || gEntryJournalCleanupPending ||
    gExitJournalCleanupPending || gRequestJournalWritePending ||
    gEntryJournalInvalid || gExitJournalInvalid ||
    gPendingExit.active || gPendingOrderClassificationUnresolved ||
HasOwnPendingOrder()) return false;
  for(int i=0;i<gTradeRecordCount;i++)
    if(gTradeRecords[i].positionID>0 && !gTradeRecords[i].closed &&
       gTradeRecords[i].totalVolumeIn>0) return false;
  return true;
}

// Validate predicates, never equality of event timestamps or traded volume.
// Full inherited Stage2 runs on every decision before this final sample.
bool DOMGateFinalTickReady(const MqlTick &current,const ArmedSignal &original,
                           const double trigger,const double price,ENUM_APS &readiness)
{
  double exec=(original.dir==1)?current.ask:current.bid;
  // These are terminal failures in the original SendOrder, even if DOM vetoes.
  if(!DOMFinite(current.bid) || !DOMFinite(current.ask) || current.bid<=0 ||
    current.ask<=0 || current.ask<current.bid || current.ask-current.bid>gSpreadMaxPts)
  { ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED; return false; }
  if(original.dir*(exec-trigger)<0)
  {
    WriteAuditRow("LATE_BREAKOUT_LOST",DirStr(original.dir),original.mode,exec,
                  original.posRel,original.distD1,original.effort,trigger);
    ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED; return false;
  }
  if(original.dir*(exec-trigger)>gMaxOvershoot)
  {
    WriteAuditRow("LATE_OVERSHOOT_BLOCK",DirStr(original.dir),original.mode,exec,
                  original.posRel,original.distD1,original.effort,trigger);
    ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED; return false;
  }
  // Original Stage2 can still wait for a real trade / last-price breakout.
```

```
  bool realTrade=((current.flags&TICK_FLAG_LAST)!=0) ||
                 ((current.flags&TICK_FLAG_VOLUME)!=0) ||
                 ((current.flags&TICK_FLAG_BUY)!=0) ||
                 ((current.flags&TICK_FLAG_SELL)!=0);
  if(current.time_msc<=0 || !DOMFinite(current.last) || current.last<=0 ||
     !realTrade || original.dir*(current.last-trigger)<0) return false;
  return true;
}

// Preserve the inherited terminal overshoot decision on the observed sample;
// a later retracement must not turn that original cancellation into WAIT.
bool DOMGateOriginalOvershoot(const MqlTick &current,const ArmedSignal &original,
                              const double trigger)
{
  bool realTrade=((current.flags&TICK_FLAG_LAST)!=0) ||
                 ((current.flags&TICK_FLAG_VOLUME)!=0) ||
                 ((current.flags&TICK_FLAG_BUY)!=0) ||
                 ((current.flags&TICK_FLAG_SELL)!=0);
  double exec=(original.dir==1)?current.ask:current.bid;
  if(!realTrade || current.last<=0 || exec<=0 ||
     original.dir*(current.last-trigger)<0) return false;
  return MathMax(original.dir*(current.last-trigger),
                 original.dir*(exec-trigger))>gMaxOvershoot;
}

// Invalid original sizing/stops/margin cancel. Only still-valid changed terms
// may defer an immutable intent after proven safe cleanup.
bool DOMGateTermsCurrent(const MqlTick &pricedTick,const ArmedSignal &original,
                         const double price,const double lots,const double sl,const double tp,
                         ENUM_APS &readiness)
{
  int direction=original.dir;
  double currentPrice=(direction==1)?pricedTick.ask:pricedTick.bid;
  double currentSL=NormEntrySL(direction,currentPrice-direction*original.activeSL_Pts);
  double currentTP=NormEntryTP(direction,currentPrice+direction*original.activeTP_Pts);
  double currentLots=ComputeLotsForEntry(direction,currentPrice,currentSL);
  double minDist=(double)SymbolInfoInteger(Symbol(),SYMBOL_TRADE_STOPS_LEVEL)*
                 SymbolInfoDouble(Symbol(),SYMBOL_POINT);
  double closingQuote=(direction==1)?pricedTick.bid:pricedTick.ask;
  bool valid=currentSL>0 && currentTP>0 && currentLots>0 &&
       direction*(currentPrice-currentSL)>=minDist &&
direction*(currentTP-currentPrice)>=minDist &&
       direction*(closingQuote-currentSL)>=minDist &&
direction*(currentTP-closingQuote)>=minDist;
  ENUM_ORDER_TYPE type=(direction==1)?ORDER_TYPE_BUY:ORDER_TYPE_SELL;
  double margin=0;
  if(valid) valid=OrderCalcMargin(type,Symbol(),currentLots,currentPrice,margin) &&
```

```
      MathIsValidNumber(margin) && margin>=0 &&
AccountInfoDouble(ACCOUNT_MARGIN_FREE)>=margin;
  if(!valid)
  { ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED; return false; }
  // A still-valid changed quote requires fresh terms after safe cleanup; it
  // cannot be used to conceal a terminal margin/stops/sizing rejection above.
  return price==currentPrice && sl==currentSL && tp==currentTP && lots==currentLots;
}

// Finite sampling: full inherited Stage2 -> current terms -> NEW DOM -> witness.
// Ticks need not stop arriving. There is no atomic tick/history/DOM API: VWAP and
// live-wave validity apply to the Stage2 sample, with a bounded residual window.
bool DOMGateDecisionCurrent(BookGateDecision &decision,const MqlTick &pricedTick,
                const ArmedSignal &original,const bool prepared,
                const PendingExecContext &expected,ENUM_APS &readiness,
                double &effort,double &trigger,double &last,
                const double price,const double lots,const double sl,const double tp)
{
  readiness=APS_STILL_ARMED;
  ulong windowBegin=GetMicrosecondCount();
  datetime signalClock=decision.signalClock, contextClock=decision.contextClock;
  if(!DOMGateIdentityCurrent(original))
  { decision.reason="ARMED_IDENTITY_CHANGED"; return false; }
  if(prepared && !DOMGateOwnUndispatchedIntent(expected))
  { decision.reason="FINANCIAL_GUARD_NOT_READY"; return false; }
  bool clocksCurrent=(signalClock>0 && contextClock>0 &&
    iTime(Symbol(),Inp_SignalTF,0)==signalClock &&
    iTime(Symbol(),Inp_ContextTF,0)==contextClock);
  MqlTick stageTick;
  ulong stageBegin=GetMicrosecondCount();
  if(prepared) readiness=CheckArmedMarketReady(effort,trigger,last,stageTick);
  else readiness=CheckArmedEntryReady(effort,trigger,last,stageTick);
  ulong stageUS=GetMicrosecondCount()-stageBegin;
  gBookGateRevalidations++; gBookGateRevalidationSumUS+=(double)stageUS;
  if(stageUS>gBookGateRevalidationMaxUS) gBookGateRevalidationMaxUS=stageUS;
  if(readiness!=APS_ENTRY_READY || !DOMGateIdentityCurrent(original))
  { decision.reason="ORIGINAL_STAGE2_NOT_READY"; return false; }
  if(!DOMGateFinalTickReady(stageTick,original,trigger,price,readiness))
  { gBookGateQuoteDeferrals++;
decision.reason="CURRENT_EXECUTION_TERMS_CHANGED"; return false; }
  if(!SpreadOk() || HasOpenPosition() || ordersEnteredToday>=Inp_MaxTradesDay ||
     dailyRiskBlocked || gAccountingUnresolved)
  { ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED;
    decision.reason="ORIGINAL_OPERATIONAL_GUARD"; return false; }
  if(prepared ? !DOMGateOwnUndispatchedIntent(expected) : HasExecutionInFlight())
  { decision.reason="FINANCIAL_GUARD_NOT_READY"; return false; }
  if(!DOMGateTermsCurrent(stageTick,original,price,lots,sl,tp,readiness))
  { decision.reason="SIZING_STOPS_OR_MARGIN_CHANGED"; return false; }
```

```
  if(GetMicrosecondCount()-windowBegin>BOOK_GATE_DECISION_BUDGET_US)
  { gBookGateDecisionBudgetBlocks++;
decision.reason="LOCAL_REVALIDATION_BUDGET_EXCEEDED"; return false; }
  // No cached DOM authorization survives either journal or slow calculation.
  bool bookConfirmed=DOMGateRead(original.dir,decision);
  clocksCurrent=clocksCurrent && decision.signalClock==signalClock &&
                decision.contextClock==contextClock;
  MqlTick current;
  if(!SymbolInfoTick(Symbol(),current))
  { ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED;
    decision.reason="FINAL_TICK_UNAVAILABLE"; return false; }
  datetime now=TimeTradeServer();
  MqlDateTime dt; TimeToStruct(now,dt);
  if(!DOMGateIdentityCurrent(original))
  { decision.reason="ARMED_IDENTITY_CHANGED"; return false; }
  if((armed.armedAt>0 && now-armed.armedAt>=(datetime)Inp_ArmedTimeoutSec) ||
     dt.hour*100+dt.min>=Inp_StopEntryHHMM)
  {
     // Preserve original timeout/session precedence over tick predicates.
     readiness=CheckArmedMarketReady(effort,trigger,last,current);
     decision.reason="ORIGINAL_TIMEOUT_OR_SESSION_END"; return false;
  }
  if(DOMGateIdentityCurrent(original) &&
DOMGateOriginalOvershoot(current,original,trigger))
  {
     WriteAuditRow("OVERSHOOT_BLOCK",DirStr(armed.dir),armed.mode,current.last,
                   armed.posRel,armed.distD1,effort,trigger);
     ResetArmedSignal(); readiness=APS_ENDED_FAIL_CLOSED;
     decision.reason="ORIGINAL_OVERSHOOT_BLOCK"; return false;
  }
  if(!DOMGateFinalTickReady(current,original,trigger,price,readiness))
  { gBookGateQuoteDeferrals++;
decision.reason="FINAL_MARKET_PREDICATE_NOT_READY"; return false; }
  if(!DOMGateTermsCurrent(current,original,price,lots,sl,tp,readiness))
  { decision.reason="FINAL_SIZING_STOPS_OR_MARGIN_CHANGED"; return false; }
  // Unavailable DOM cannot suppress an original terminal execution decision.
  if(!gBookGateSubscriptionOwned || Symbol()!=gBookGateSymbol ||
     !TerminalInfoInteger(TERMINAL_CONNECTED))
  { decision.reason="SUBSCRIPTION_UNAVAILABLE"; return false; }
  if(!clocksCurrent || iTime(Symbol(),Inp_SignalTF,0)!=signalClock ||
     iTime(Symbol(),Inp_ContextTF,0)!=contextClock)
  { decision.reason="BAR_CLOCK_CHANGED"; return false; }
  ulong elapsed=GetMicrosecondCount()-windowBegin;
  if(elapsed>gBookGateDecisionMaxUS) gBookGateDecisionMaxUS=elapsed;
  if(elapsed>BOOK_GATE_DECISION_BUDGET_US)
  { gBookGateDecisionBudgetBlocks++;
decision.reason="LOCAL_REVALIDATION_BUDGET_EXCEEDED"; return false; }
  return bookConfirmed;
```

```
}

ENUM_SEND_OUTCOME DOMGateRejectPrepared(const BookGateDecision &decision,
                const string reason,const ArmedSignal &original,
                const PendingExecContext &expected,const bool apiCallStarted,
                const ENUM_APS readiness)
{
  BookGateDecision rejected=decision;
  rejected.state=BOOK_ENTRY_BLOCKED; rejected.reason=reason+";
source="+decision.reason;
  DOMGateReport(rejected,false);
  // Tickets alone are NOT negative proof. This witness belongs to the same
  // SendOrder invocation that prepared expected, before its only API call.
  // Never call this helper from recovery, a timer, or after PositionOpen.
  if(apiCallStarted || !DOMGateIntentMatches(expected) || HasOpenPosition() ||
     HasOwnPendingOrder() || gPendingExit.active ||
     gOrderObligationScanUnresolved || gPendingOrderClassificationUnresolved)
  {
     Print("[BOOK_GATE][FINANCIAL_UNRESOLVED] no exclusive local no-dispatch proof;
reconciliation required");
     return SEND_FINANCIAL_UNRESOLVED;
  }
  for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
     if(gUnresolvedOrders[i].active && (gUnresolvedOrders[i].classificationPending ||
        gUnresolvedOrders[i].persistencePending)) return SEND_FINANCIAL_UNRESOLVED;
  for(int i=0;i<gTradeRecordCount;i++)
     if(gTradeRecords[i].positionID>0 && !gTradeRecords[i].closed &&
        gTradeRecords[i].totalVolumeIn>0) return SEND_FINANCIAL_UNRESOLVED;
  // MAY is not downgraded. An explicit durable result precedes memory cleanup.
  if(!ResolveRequestJournal(false))
  {
     Print("[BOOK_GATE][JOURNAL_RESOLUTION_FAILED] pending intent retained; no
retry");
     return SEND_FINANCIAL_UNRESOLVED;
  }
  ResetLastError();
  bool journalExists=FileIsExist(RequestJournalPath(false));
  int journalProbeError=GetLastError();
  if(journalExists || (journalProbeError!=0 && journalProbeError!=ERR_FILE_NOT_EXIST) ||
     gEntryJournalCleanupPending ||
     gEntryJournalWritePending || gEntryJournalInvalid)
  {
     PrintFormat("[BOOK_GATE][JOURNAL_PROOF_FAILED] probe_error=%d; pending
intent retained",journalProbeError);
     return SEND_FINANCIAL_UNRESOLVED;
  }
  ResetPendingExec(false);
  if(HasExecutionInFlight() || gEntryJournalInvalid || gExitJournalInvalid)
```

```
    return SEND_FINANCIAL_UNRESOLVED;
  PrintFormat("[BOOK_GATE][NO_DISPATCH_RESOLVED] phase=%d reason=%s; durable
cleanup confirmed",
            (int)expected.dispatchPhase,reason);
  if(readiness==APS_ENDED_REEVAL_ALLOWED) return SEND_CANCEL_REEVAL;
  if(readiness==APS_ENDED_FAIL_CLOSED || !DOMGateIdentityCurrent(original))
    return SEND_ORIGINAL_CANCEL;
  if(readiness==APS_TRADE_OR_POSITION) return SEND_TRADE_OR_POSITION;
  // Cleanup itself can take time. Reuse original cancellation/operational rules
  // on the current sample; do not return WAIT with an expired signal.
  double effort,trigger,last;
  MqlTick current;
  ENUM_APS currentReadiness=CheckArmedEntryReady(effort,trigger,last,current);
  if(currentReadiness==APS_ENDED_REEVAL_ALLOWED) return
SEND_CANCEL_REEVAL;
  if(currentReadiness==APS_ENDED_FAIL_CLOSED || !DOMGateIdentityCurrent(original))
    return SEND_ORIGINAL_CANCEL;
  if(currentReadiness==APS_TRADE_OR_POSITION) return
SEND_TRADE_OR_POSITION;
  if(currentReadiness==APS_ENTRY_READY)
  {
    // Use the same Stage2 sample; never mix its LAST with a newer quote.
    if(DOMGateOriginalOvershoot(current,original,trigger))
    {
      WriteAuditRow("OVERSHOOT_BLOCK",DirStr(original.dir),original.mode,current.last,
                original.posRel,original.distD1,effort,trigger);
      ResetArmedSignal(); return SEND_ORIGINAL_CANCEL;
    }
    double exec=(original.dir==1)?current.ask:current.bid;
    if(!DOMGateFinalTickReady(current,original,trigger,exec,currentReadiness) &&
      (currentReadiness==APS_ENDED_FAIL_CLOSED ||
!DOMGateIdentityCurrent(original)))
      return SEND_ORIGINAL_CANCEL;
  }
  if(HasExecutionInFlight() || gEntryJournalInvalid || gExitJournalInvalid)
    return SEND_FINANCIAL_UNRESOLVED;
  return SEND_WAIT_ARMED;
}

void DOMGateObserveMatrix(const MatrixSnapshot &s)
{
  if(!Inp_DOM_TradeGateEnable || !armed.active) return;
  DOMGateSyncIdentity();
  if(!gBookGateHadOpposite || !DOMGateLogAllowed()) return;
  PrintFormat("[BOOK_GATE][BOOK_LAST_OBSERVATION] opposite=%s ageMS=%I64u
available=%s; not a fresh book read",
    BStr(gBookGateOppositeWatch),
    gBookGateObservationMS>0?GetTickCount64()-gBookGateObservationMS:0,
```

```
    BStr(gBookGateObservationMS>0));
  bool soleRoute=(s.absValid!=s.contValid);
  int matrixDir=s.absValid?s.dir:s.dirCont;
  bool oppositeComplete=(s.status==MSS_OK && soleRoute && matrixDir==-armed.dir);
  PrintFormat("[BOOK_GATE][BOOK_OPPOSITION_HISTORY] bar=%I64d armedAt=%I64d
armedDir=%d status=%s absValid=%s contValid=%s absDir=%d contDir=%d
completeOpposite=%s c1abs=%s c1cont=%s c2abs=%s c3abs=%s c4abs=%s c2cont=%s
c3cont=%s c4cont=%s c5abs=%s c5cont=%s; OBSERVATION ONLY, original
hard-contradiction/rearm controls execution",
    (long)gCachedMicroBar,(long)armed.armedAt,armed.dir,MssStr(s.status),
    BStr(s.absValid),BStr(s.contValid),s.dir,s.dirCont,BStr(oppositeComplete),
    BStr(s.c1),BStr(s.c1Cont),BStr(s.c2),BStr(s.c3),BStr(s.c4),
    BStr(s.c2Cont),BStr(s.c3Cont),BStr(s.c4Cont),BStr(s.c5AbsOk),BStr(s.c5ContOk));
}

void DOMGateDeinit()
{
  if(gBookGateSubscriptionOwned)
  {
    ResetLastError();
    if(!MarketBookRelease(gBookGateSymbol))
      Print("[BOOK_GATE][RELEASE_FAILED] own reference error=",GetLastError());
    gBookGateSubscriptionOwned=false;
  }
  if(Inp_DOM_TradeGateEnable)
    PrintFormat("[BOOK_GATE][RC2_METRICS] symbol=%s api_budget_blocks=%I64u
agg_budget_blocks=%I64u decision_budget_blocks=%I64u quote_deferrals=%I64u
prepare_calls=%I64u prepare_sum_us=%.0f prepare_max_us=%I64u journal_calls=%I64u
journal_sum_us=%.0f journal_max_us=%I64u subscription_attempts=%I64u
subscription_recoveries=%I64u terminal_memory_mb=%I64d; budgets uncalibrated;
CPUrequires terminal/OS measurement",
      Symbol(),gBookGateApiBudgetBlocks,gBookGateAggregationBudgetBlocks,
      gBookGateDecisionBudgetBlocks,gBookGateQuoteDeferrals,gBookGatePrepareCalls,
      gBookGatePrepareSumUS,gBookGatePrepareMaxUS,gBookGateJournalCalls,
      gBookGateJournalSumUS,gBookGateJournalMaxUS,gBookGateSubscriptionAttempts,

gBookGateSubscriptionRecoveries,TerminalInfoInteger(TERMINAL_MEMORY_USED));
  if(Inp_DOM_TradeGateEnable)
    PrintFormat("[BOOK_GATE][SUMMARY] reads=%I64u blocked=%I64u
unavailable=%I64u accepted=%I64u api_calls=%I64u api_sum_us=%.0f
api_max_us=%I64u aggregation_calls=%I64u aggregation_sum_us=%.0f
aggregation_max_us=%I64u decision_max_us=%I64u dispatch_gap_max_us=%I64u
revalidation_calls=%I64u revalidation_sum_us=%.0f revalidation_max_us=%I64u
late_deferrals=%I64u logs_dropped=%I64u; no financial performance claim",
     gBookGateReads,gBookGateBlocks,gBookGateUnavailable,gBookGateReleased,

gBookGateApiCalls,gBookGateApiSumUS,gBookGateApiMaxUS,gBookGateAggregations,
```

```
gBookGateAggregationSumUS,gBookGateAggregationMaxUS,gBookGateDecisionMaxUS,
  gBookGateDispatchMaxUS,gBookGateRevalidations,gBookGateRevalidationSumUS,
  gBookGateRevalidationMaxUS,gBookGateLateDeferrals,gBookGateLogsDropped);
}

// END EXPERIMENTAL DOM CONFLUENCE MODULE

//+------------------------------------------------------------------+
// RC1 telemetry. No decision gates, CopyTicks, Print or tick flushing here.
MatrixSnapshot gLastTelemetryMatrix;
bool gLastTelemetryMatrixValid = false;
datetime gLastTelemetrySignalBar = 0;
bool gLiveDirectionTelemetryValid = false;
int gLiveDirAbs = 0;
int gLiveDirCont = 0;
bool gLateDirectionMismatch = false;
ulong gTradeCsvWrittenPIDs[];
bool gTradeCsvIndexLoaded = false;
bool gTradeCsvAppendSafe = false;
string gTradeCsvFileName="";
datetime gNextTradeCsvRecovery=0;
// HELPER FUNCTIONS
//+------------------------------------------------------------------+
string DirStr(int d) { return d==1 ? "BUY" : (d==-1 ? "SELL" : "NONE"); }
string BStr(bool b) { return b ? "1" : "0"; }

string PocStateStr(ENUM_POC_STATE st)
{
  switch(st)
  {
    case POCS_NO_DISPLACEMENT: return "POC_NO_DISPLACEMENT";
    case POCS_FOLLOWING:       return "POC_FOLLOWING";
    case POCS_DIVERGENT:       return "POC_DIVERGENT";
    default: return "POC_INVALID";
  }
}
string MacroStatusStr(ENUM_MACRO_STATUS st)
{
   switch(st)
   {
      case MACRO_OK:        return "MACRO_OK";
      case MACRO_VWAP_INVALID: return "MACRO_VWAP_INVALID";
      case MACRO_SIGMA_INVALID: return "MACRO_SIGMA_INVALID";
      case MACRO_STRUCT_INVALID: return "MACRO_STRUCT_INVALID";
      case MACRO_NO_TRIGGER: return "MACRO_NO_TRIGGER";
      default: return "MACRO_NOT_COMPUTED";
     }
}
```

```
string MssStr(ENUM_MSS st)
{
  switch(st)
  {
    case MSS_OK:        return "OK";
    case MSS_MACRO_INVALID:             return "MACRO_INVALID";
    case MSS_VWAP_D1_INVALID:               return "VWAP_D1_INVALID";
    case MSS_SIGMA_INVALID:          return "SIGMA_INVALID";
    case MSS_STRUCT_INVALID:            return "STRUCT_INVALID";
    case MSS_STOP_KAMIKAZE_CONT_ONLY: return "KAMIKAZE_CONT_ONLY";
    case MSS_STOP_NO_C5_TRIGGER:                    return "NO_C5_TRIGGER";
    default: return "NOT_COMPUTED";
  }
}

int GetD1SessionEndHHMM()
{
  return gIsDollarFamily ? D1_SESSION_END_DOLLAR : D1_SESSION_END_INDEX;
}
double NormPriceDigits(double p)
{
  return NormalizeDouble(p, (int)SymbolInfoInteger(Symbol(), SYMBOL_DIGITS));
}

double RoundUpTick(double p)
{
  double r = p / gTickSize;
  double n = MathRound(r);
  if(MathAbs(r - n) < 1e-6) return NormPriceDigits(n * gTickSize);
  return NormPriceDigits(MathCeil(r) * gTickSize);
}

double RoundDownTick(double p)
{
  double r = p / gTickSize;
  double n = MathRound(r);
   if(MathAbs(r - n) < 1e-6) return NormPriceDigits(n * gTickSize);
   return NormPriceDigits(MathFloor(r) * gTickSize);
}

double NormEntrySL(int dir, double sl)
{ return (dir == 1) ? RoundDownTick(sl) : RoundUpTick(sl); }

double NormEntryTP(int dir, double tp)
{ return (dir == 1) ? RoundUpTick(tp) : RoundDownTick(tp); }

long TickIndex(double price) { return (long)MathRound(price / gTickSize); }
```

```
double ComputeTrigger(int dir, double regHigh, double regLow)
{
  if(dir == 1) return RoundUpTick(regHigh + gBreakoutMargin);
  return RoundDownTick(regLow - gBreakoutMargin);
}

bool IsFokSupported()
{
  ENUM_SYMBOL_TRADE_EXECUTION exec =
      (ENUM_SYMBOL_TRADE_EXECUTION)SymbolInfoInteger(Symbol(),
SYMBOL_TRADE_EXEMODE);
  if(exec == SYMBOL_TRADE_EXECUTION_INSTANT ||
     exec == SYMBOL_TRADE_EXECUTION_REQUEST)
      return true;
  long filling = SymbolInfoInteger(Symbol(), SYMBOL_FILLING_MODE);
  return ((filling & SYMBOL_FILLING_FOK) == SYMBOL_FILLING_FOK);
}
void ResetPerfStat(PerfStat &p) { p.count = 0; p.sumUs = 0.0; p.maxUs = 0.0; }
void RecordPerf(PerfStat &p, double us)
{
  p.count++;
  p.sumUs += us;
  if(us > p.maxUs) p.maxUs = us;
}

void ResetMacroContext(MacroContext &m)
{
  m.status = MACRO_NOT_COMPUTED;
  m.barTime = 0;
  m.closedPrice = 0.0;
  m.vwap = 0.0;
  m.SigmaD1 = 0.0;
  m.distVWAP_D1 = 0.0;
  m.posRel = 0.0;
  m.glass = false;
  m.macroAbsDir = 0;
   m.macroContDir = 0;
}

void ResetPOCResult(POCResult &r)
{
  r.ok = false; r.poc = 0; r.totalVolume = 0; r.lastValidPrice = 0;
  r.tradeCount = 0; r.levels = 0; r.hasTie = false;
    r.top1Price = 0; r.top1Vol = 0; r.top2Price = 0; r.top2Vol = 0;
    r.dominance = 0; r.buyVol = 0; r.sellVol = 0; r.unkVol = 0;
}
```

```
void ResetMatrixSnapshot(MatrixSnapshot &s)
{
  s.status = MSS_NOT_COMPUTED;
  s.vwapVig=0;s.upper=0;s.lower=0;s.bandRange=0;
  s.price=0;s.open1=0;s.candleBody=0;s.posRel=0;
  s.glass=false;s.dir=0;s.dirCont=0;
  s.c5AbsOk=false;s.c5ContOk=false;
  s.distAbs=0;s.minDispl=0;s.maxBody=0;s.contMinDispl=0;
  s.c2=false;s.c3=false;s.c23=false;s.c4=false;
  s.c2Cont=false;s.c3Cont=false;s.c23Cont=false;s.c4Cont=false;
  s.posRange=0;
  ResetPOCResult(s.pocNow);
  ResetPOCResult(s.pocPrev);
  s.windowsValid=false;
  s.priceMove=0;s.pocMove=0;s.displacementOk=false;s.moveDir=0;
  s.pocState=POCS_INVALID;
  s.contDisplacementOk=false;s.contMoveDir=0;s.contPocState=POCS_INVALID;
    s.wwok=false;s.effort=0;s.waveDir=0;
     s.wwWaveBars=0; s.wwWaveAgeSec=0; s.wwWaveAgeAvailable=false;
     s.wwAccumVolume=0; s.wwDistancePts=0;
     s.c1_effort_ok=false;s.c1_body_ok=false;s.c1_displacement_ok=false;
     s.c1_movedir_ok=false;s.c1_wavedir_ok=false;
     s.absDivergence=false;s.absStallStrict=false;
     s.absFollowingReaction=false;s.absFollowingExtension=false;
     s.absPocContextOk=false;s.c1=false;
     s.cont_effort_ok=false;s.cont_displacement_ok=false;s.cont_poc_ok=false;
     s.cont_movedir_ok=false;s.cont_wavedir_ok=false;s.cont_candle_dir_ok=false;
     s.c1Cont=false;
     s.turboAbs=false;s.baseAbs=false;s.turboCont=false;s.baseCont=false;
     s.scoreEff=0;s.scoreContEff=0;
     s.absValid=false;s.contValid=false;
     s.candleDir=0;
     s.macroAbsCompatible=false;s.macroContCompatible=false;
}

void ResetArmedSignal()
{
  armed.active=false;armed.dir=0;
  armed.regionHigh=0;armed.regionLow=0;
  armed.activeSL_Pts=0;armed.activeTP_Pts=0;
  armed.regionEndTime=0;
  armed.mode="NONE";armed.lots=0;armed.route="NONE";
  armed.breakoutEffortMin=0;armed.armedAt=0;
  armed.sameSideGlassLogged=false;
  armed.posRel=0;armed.distD1=0;armed.effort=0;armed.vwapVig=0;
  armed.c1=0;armed.c2=0;armed.c3=0;armed.c4=0;armed.c5=0;armed.scoreEff=0;
}
```

```
void ResetOpenPos()
{
  gOpenPos.active=false;gOpenPos.positionID=0;gOpenPos.direction=0;
  gOpenPos.entryPrice=0;gOpenPos.MAE=0;gOpenPos.MFE=0;
  gOpenPos.entryTime=0;gOpenPos.route="";gOpenPos.mode="";
  gOpenPos.lots=0;gOpenPos.initialRiskMoney=0;gOpenPos.tradeRecordIdx=-1;
}

//+------------------------------------------------------------------+
// TICK COPY — FAIL CLOSED (from v2.44, unchanged)
//+------------------------------------------------------------------+
int CopyTradeTicksStrict(ulong from_msc, ulong to_msc, MqlTick &ticksOut[])
{
   if(to_msc < from_msc) return -1;
   ResetLastError();
   int copied = CopyTicksRange(Symbol(), ticksOut, COPY_TICKS_TRADE, from_msc,
to_msc);
    if(copied < 0 || GetLastError() != 0) return -1;
    return copied;
}

//+------------------------------------------------------------------+
// VWAP INCREMENTAL (from v2.44 — lógica idêntica, prefixo SCALPER)
//+------------------------------------------------------------------+
void ResetVwapCache()
{
   vwapSumPV=0;vwapSumVol=0;vwapNextFromMsc=0;
   vwapReady=false;vwapDayKey=-1;gVwapValue=0;gVwapOk=false;
}

bool UpdateVwapIncremental(datetime t0)
{
  MqlDateTime d; TimeToStruct(t0, d);
  int key = d.year*1000 + d.day_of_year;
  if(key != vwapDayKey)
  {
vwapSumPV=0;vwapSumVol=0;vwapNextFromMsc=0;vwapReady=false;vwapDayKey=key;
 }
 ulong t0_msc = (ulong)t0 * 1000;
 if(!vwapReady)
 {
     d.hour=0;d.min=0;d.sec=0;
     datetime dayStart = StructToTime(d);
      ulong from = (ulong)dayStart * 1000;
      if(t0_msc <= from) {
vwapSumPV=0;vwapSumVol=0;vwapReady=false;vwapNextFromMsc=0; return false; }
      ulong to = t0_msc - 1;
      MqlTick tk[];
```

```
      int copied = CopyTradeTicksStrict(from, to, tk);
      if(copied <= 0) {
vwapSumPV=0;vwapSumVol=0;vwapReady=false;vwapNextFromMsc=0; return false; }
      double pv=0,vv=0; bool any=false;
      for(int i=0;i<copied;i++)
         if(tk[i].last>0&&tk[i].volume>0){ pv+=tk[i].last*(double)tk[i].volume;
vv+=(double)tk[i].volume; any=true; }
      if(!any||vv<=0) {
vwapSumPV=0;vwapSumVol=0;vwapReady=false;vwapNextFromMsc=0; return false; }
      vwapSumPV=pv;vwapSumVol=vv;vwapReady=true;vwapNextFromMsc=t0_msc;
      return true;
   }
   if(vwapNextFromMsc >= t0_msc) return (vwapSumVol > 0.0);
   ulong from = vwapNextFromMsc;
   ulong to = t0_msc - 1;
   MqlTick tk[];
   int copied = CopyTradeTicksStrict(from, to, tk);
   if(copied < 0) return false;
   if(copied == 0) { vwapNextFromMsc=t0_msc; return (vwapSumVol>0.0); }
   double pv=0,vv=0;
   for(int i=0;i<copied;i++)
      if(tk[i].last>0&&tk[i].volume>0){ pv+=tk[i].last*(double)tk[i].volume;
vv+=(double)tk[i].volume; }
   vwapSumPV+=pv;vwapSumVol+=vv;vwapNextFromMsc=t0_msc;
   return (vwapSumVol > 0.0);
}

bool GetLiveVwapSnapshot(ulong to_msc, double &liveVwap)
{
   liveVwap = 0.0;
   if(!vwapReady || vwapSumVol <= 0.0 || vwapNextFromMsc == 0) return false;
   double localPV = vwapSumPV, localVol = vwapSumVol;
   if(to_msc >= vwapNextFromMsc)
   {
      MqlTick ticks[];
      int copied = CopyTradeTicksStrict(vwapNextFromMsc, to_msc, ticks);
     if(copied <= 0) return false;
     for(int i=0;i<copied;i++)
        if(ticks[i].last>0&&ticks[i].volume>0)
        { localPV+=ticks[i].last*(double)ticks[i].volume; localVol+=(double)ticks[i].volume; }
    }
    if(localVol <= 0.0) return false;
    liveVwap = localPV / localVol;
      return liveVwap > 0.0;
}

//+------------------------------------------------------------------+
// POC M1 COM DETECÇÃO DE EMPATE
```

```
// Preserva o princípio do ComputePOCWindow da 2.44.
// Tie is telemetry; canonical ascending scan keeps first maximum without a new tie rule.
//=====================================================================
bool ComputePOCWithTie(ulong from_msc, ulong to_msc, POCResult &res)
{
   ResetPOCResult(res);
   if(to_msc < from_msc) return false;

  MqlTick t[];
  int copied = CopyTradeTicksStrict(from_msc, to_msc, t);
  if(copied < 0) return false;
  if(copied == 0) return false;
double ts = SymbolInfoDouble(Symbol(), SYMBOL_TRADE_TICK_SIZE);
if(ts <= 0.0) ts = SymbolInfoDouble(Symbol(), SYMBOL_POINT);
if(ts <= 0.0) return false;

double pmin=DBL_MAX, pmax=-DBL_MAX; bool any=false;
for(int i=0;i<copied;i++)
{
    if(t[i].last>0.0 && t[i].volume>0)
    {
        any=true;
        if(t[i].last < pmin) pmin=t[i].last;
        if(t[i].last > pmax) pmax=t[i].last;
        res.lastValidPrice=t[i].last;
        res.tradeCount++;
        double v=(double)t[i].volume;
        bool fb=((t[i].flags & TICK_FLAG_BUY) != 0);
        bool fs=((t[i].flags & TICK_FLAG_SELL) != 0);
        if(fb && !fs) res.buyVol += v;
        else if(fs && !fb) res.sellVol += v;
        else           res.unkVol += v;
    }
}
if(!any || pmax < pmin) return false;
long levelsL = (long)MathRound((pmax-pmin)/ts) + 1;
if(levelsL < 1 || levelsL > 100000) return false;
int levCount = (int)levelsL;

double vol[];
ArrayResize(vol, levCount);
ArrayInitialize(vol, 0.0);
for(int i=0;i<copied;i++)
{
    if(t[i].last>0.0 && t[i].volume>0)
    {
        int idx=(int)MathRound((t[i].last-pmin)/ts);
        if(idx<0||idx>=levCount) continue;
```

```
        vol[idx]+=(double)t[i].volume;
        res.totalVolume+=(double)t[i].volume;
    }
}
if(res.totalVolume <= 0.0) return false;

// Encontrar top1 e top2 separadamente (sem colapso silencioso de empate)
int best=-1, second=-1; double bestV=-1.0, secondV=-1.0; int nz=0;
for(int i=0;i<levCount;i++)
{
    if(vol[i] > 0.0) nz++;
    if(vol[i] > bestV) { bestV=vol[i]; best=i; }
  }
  for(int i=0;i<levCount;i++)
  {
      if(i==best) continue;
   if(vol[i] > 0.0 && vol[i] > secondV) { secondV=vol[i]; second=i; }
}
if(best < 0) return false;

res.levels = nz;
res.poc = pmin + best*ts;
res.top1Price = res.poc;
res.top1Vol = bestV;
if(second >= 0)
{
    res.top2Price = pmin + second*ts;
    res.top2Vol = secondV;
    // Canonical diagnostic: exactly equal executed volumes
    res.hasTie = (secondV == bestV);
}
else { res.top2Price=0; res.top2Vol=0; res.hasTie=false; }

 res.dominance = (res.totalVolume>0 && second>=0) ?
(res.top1Vol-res.top2Vol)/res.totalVolume : 0.0;
  res.ok = true;
   return true;
}

//+------------------------------------------------------------------+
// WEIS WAVE M1
// Lógica operacional idêntica à v2.44.
// Adaptado: usa Inp_SignalTF (PERIOD_M1) em vez de Inp_Timeframe.
// Adicionado: wwCurrentWaveBars e wwWaveStartPrice (telemetria apenas).
//+------------------------------------------------------------------+
void RebuildWeisCache()
{
   wwValid=false;
```

```
   int need=150;
   double close[]; long rvol[],tvol[];
   ArraySetAsSeries(close,true);
   ArraySetAsSeries(rvol,true);
   ArraySetAsSeries(tvol,true);
   int n=CopyClose(Symbol(),Inp_SignalTF,0,need,close);
   if(n<12) return;
   bool useReal=(CopyRealVolume(Symbol(),Inp_SignalTF,0,need,rvol)==n);
    if(!useReal && CopyTickVolume(Symbol(),Inp_SignalTF,0,need,tvol)!=n) return;

double waves[]; int wc=0;
int curDir=0; double accum=0.0;
int curWaveBars=0; double curWaveStart=0.0;

for(int i=n-2;i>=1;i--)
{
    int bd=(close[i]>close[i+1])?1:(close[i]<close[i+1]?-1:curDir);
       if(bd==0) bd=(curDir==0?1:curDir);
       double v=useReal?(double)rvol[i]:(double)tvol[i];
       if(curDir==0)
       {
          curDir=bd; accum=v; curWaveBars=1; curWaveStart=close[i+1];
       }
       else if(bd==curDir)
       {
          accum+=v; curWaveBars++;
       }
       else
       {
          ArrayResize(waves,wc+1); waves[wc++]=accum;
          curDir=bd; accum=v; curWaveBars=1; curWaveStart=close[i+1];
       }
   }
       int cnt=MathMin(10,wc); double s=0.0;
       for(int k=wc-1;k>=wc-cnt&&k>=0;k--) s+=waves[k];
       double avgClosed=(cnt>0)?s/cnt:0.0;
       int cntF=MathMin(10,wc+1); double sF=accum;
       for(int k=wc-1;k>=wc-(cntF-1)&&k>=0;k--) sF+=waves[k];
       double avgFlip=(cntF>0)?sF/cntF:0.0;
        wwUseReal        = useReal;
        wwDirClosed = curDir;
        wwAccumClosed = accum;
        wwAvgClosed = avgClosed;
        wwAvgIfFlip = avgFlip;
        wwCloseRef = close[1];
        wwWaveDirClosed = curDir;
        wwEffortClosed = (avgClosed>0.0)?accum/avgClosed:0.0;
        wwCurrentWaveBars = curWaveBars; // telemetria
```

```
        wwWaveStartPrice = curWaveStart; // telemetria
        wwValid = true;
}

// ComputeLiveWaveO1 adaptado para M1 — idêntico à v2.44 mas usa Inp_SignalTF
void ComputeLiveWaveO1(double &effort, double &curVol, double &avgVol, int &waveDir)
{
  double v0 = wwUseReal ?
      (double)iRealVolume(Symbol(),Inp_SignalTF,0) :
      (double)iVolume(Symbol(),Inp_SignalTF,0);
  double c0 = iClose(Symbol(),Inp_SignalTF,0);
  int bd0=(c0>wwCloseRef)?1:((c0<wwCloseRef)?-1:wwDirClosed);
  if(bd0==0) bd0=wwDirClosed;
  if(bd0==wwDirClosed)
    { curVol=wwAccumClosed+v0; waveDir=wwDirClosed; avgVol=wwAvgClosed; }
    else
    { curVol=v0; waveDir=bd0; avgVol=wwAvgIfFlip; }
    effort=(avgVol>0.0)?curVol/avgVol:0.0;
}

//+------------------------------------------------------------------+
// D-1 LITERAL (from v2.44 — idêntico, prefixo SCALPER nos prints)
//+------------------------------------------------------------------+
void CalcD1Structure()
{
   H1=0;L1=0;VWAP_D1=0;VWAP_D1_Official=false;
   MqlDateTime td; TimeToStruct(TimeTradeServer(),td);
   td.hour=0;td.min=0;td.sec=0;
   datetime todayStart=StructToTime(td);
   datetime d1_0=iTime(Symbol(),PERIOD_D1,0);
   if(d1_0<=0) { Print("[SCALPER][D1] iTime D1 shift 0 falhou."); return; }
   datetime expected=0;
   if(d1_0>=todayStart)
   {
      expected=iTime(Symbol(),PERIOD_D1,1);
   if(expected<=0) { Print("[SCALPER][D1] iTime D1 shift 1 falhou."); return; }
   }
   else expected=d1_0;
   MqlDateTime ed; TimeToStruct(expected,ed);
  ed.hour=0;ed.min=0;ed.sec=0;
  datetime dayStart=StructToTime(ed);
  datetime dayEnd=dayStart+86400-1;
  datetime timeBuf[];
  int nBars=CopyTime(Symbol(),PERIOD_M1,dayStart,dayEnd,timeBuf);
  if(nBars<D1_MIN_M1_BARS)
  { PrintFormat("[SCALPER][D1] M1 insuficiente (%d<%d).",nBars,D1_MIN_M1_BARS);
return; }
  MqlDateTime b0; TimeToStruct(timeBuf[0],b0);
```

```
  if(b0.year!=ed.year||b0.mon!=ed.mon||b0.day!=ed.day)
  { Print("[SCALPER][D1] Data carregada difere."); return; }
  double highBuf[],lowBuf[],closeBuf[];
 if(CopyHigh (Symbol(),PERIOD_M1,dayStart,dayEnd,highBuf )!=nBars) return;
 if(CopyLow (Symbol(),PERIOD_M1,dayStart,dayEnd,lowBuf )!=nBars) return;
 if(CopyClose(Symbol(),PERIOD_M1,dayStart,dayEnd,closeBuf)!=nBars) return;
 long realVol[],tickVol[];
 bool
useRealVol=(CopyRealVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,realVol)==nBars);

if(!useRealVol&&CopyTickVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,tickVol)!=nBars)
return;
    int sessEnd=GetD1SessionEndHHMM();
    double dayHigh=0,dayLow=DBL_MAX;
    double sumTPVol=0,sumVol=0; int sessBar=0;
    for(int i=0;i<nBars;i++)
    {
    MqlDateTime b; TimeToStruct(timeBuf[i],b);
    int h=b.hour*100+b.min;
    if(h>=D1_SESSION_START&&h<sessEnd)
    {
        if(highBuf[i]>dayHigh) dayHigh=highBuf[i];
        if(lowBuf[i]<dayLow) dayLow=lowBuf[i];
        double tp=(highBuf[i]+lowBuf[i]+closeBuf[i])/3.0;
        double vol=useRealVol?(double)realVol[i]:(double)tickVol[i];
        if(vol>0){sumTPVol+=tp*vol;sumVol+=vol;}
        sessBar++;
    }
  }
  if(dayHigh<=0||dayLow>=DBL_MAX||dayHigh<=dayLow||sessBar==0||sumVol<=0) return;
  double dayVWAP=sumTPVol/sumVol;
  if(dayVWAP<=0) return;
  H1=dayHigh;L1=dayLow;VWAP_D1=dayVWAP;VWAP_D1_Official=true;
  PrintFormat("[SCALPER][D1] %s: VWAP=%.4f H1=%.4f L1=%.4f Barras=%d",
     TimeToString(dayStart,TIME_DATE),VWAP_D1,H1,L1,nBars);
}
//+------------------------------------------------------------------+
// SIGMA HISTÓRICO (from v2.44 — idêntico na lógica)
//+------------------------------------------------------------------+
void CalcSigmaHistory()
{
  SigmaD1=0;
  MqlDateTime todayDT; TimeToStruct(TimeTradeServer(),todayDT);
  todayDT.hour=0;todayDT.min=0;todayDT.sec=0;
  datetime searchDay=StructToTime(todayDT);
  int targetDays=SigmaLookbackDays; // validated, no silent input clamp
  double sigmaArray[20]; int validDaysFound=0;
  int sessEnd=GetD1SessionEndHHMM();
```

```
  for(int
attempt=0;attempt<MAX_SEARCH_DAYS&&validDaysFound<targetDays;attempt++)
  {
    searchDay-=86400;
    MqlDateTime sdt; TimeToStruct(searchDay,sdt);
    if(sdt.day_of_week==0||sdt.day_of_week==6) continue;
    sdt.hour=0;sdt.min=0;sdt.sec=0;
    datetime dayStart=StructToTime(sdt);
    datetime dayEnd=dayStart+86400-1;
    datetime timeBuf[];
    int nBars=CopyTime(Symbol(),PERIOD_M1,dayStart,dayEnd,timeBuf);
    if(nBars<D1_MIN_M1_BARS) continue;
    double highBuf[],lowBuf[],closeBuf[];
   if(CopyHigh(Symbol(),PERIOD_M1,dayStart,dayEnd,highBuf)!=nBars) continue;
   if(CopyLow (Symbol(),PERIOD_M1,dayStart,dayEnd,lowBuf )!=nBars) continue;
   double dayHigh=0,dayLow=DBL_MAX; int sessBar=0;
   for(int i=0;i<nBars;i++)
 {
      MqlDateTime b; TimeToStruct(timeBuf[i],b);
      int h=b.hour*100+b.min;
      if(h>=D1_SESSION_START&&h<sessEnd)
      { if(highBuf[i]>dayHigh)dayHigh=highBuf[i]; if(lowBuf[i]<dayLow)dayLow=lowBuf[i];
sessBar++; }
  }
  if(dayHigh<=0||dayLow>=DBL_MAX||dayHigh<=dayLow||sessBar==0) continue;
  if(CopyClose(Symbol(),PERIOD_M1,dayStart,dayEnd,closeBuf)!=nBars) continue;
  long realVol[],tickVol[];
  bool
useRealVol=(CopyRealVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,realVol)==nBars);
  int copiedTV=0;
  if(!useRealVol)
copiedTV=CopyTickVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,tickVol);
  if(!useRealVol&&copiedTV!=nBars) continue;
  double sumTPVol=0,sumVol=0;
  for(int i=0;i<nBars;i++)
  {
     MqlDateTime b; TimeToStruct(timeBuf[i],b);
     int h=b.hour*100+b.min;
   if(h>=D1_SESSION_START&&h<sessEnd)
   {
      double tp=(highBuf[i]+lowBuf[i]+closeBuf[i])/3.0;
      double vol=useRealVol?(double)realVol[i]:(double)tickVol[i];
      if(vol>0){sumTPVol+=tp*vol;sumVol+=vol;}
   }
   }
   if(sumVol<=0) continue;
   double dayVWAP=sumTPVol/sumVol;
   double sumWVar=0,sumVolB=0;
```

```
 for(int j=0;j<nBars;j++)
 {
    MqlDateTime b2; TimeToStruct(timeBuf[j],b2);
    int h2=b2.hour*100+b2.min;
    if(h2>=D1_SESSION_START&&h2<sessEnd)
    {
       double tp=(highBuf[j]+lowBuf[j]+closeBuf[j])/3.0;
       double vol=useRealVol?(double)realVol[j]:(double)tickVol[j];
       if(vol>0){double dd=tp-dayVWAP;sumWVar+=vol*dd*dd;sumVolB+=vol;}
    }
 }
 double daySigma=(sumVolB>0)?MathSqrt(sumWVar/sumVolB):0.0;
 sigmaArray[validDaysFound++]=daySigma;
 }
 if(validDaysFound==0) { Print("[SCALPER][SIGMA] CRÍTICO: nenhum dia válido.");
return; }
 double sumS=0;
  for(int k=0;k<validDaysFound;k++) sumS+=sigmaArray[k];
  SigmaD1=sumS/(double)validDaysFound;
  double pisoSigma=gIsIndexFamily?Inp_SigmaFloor_WIN:Inp_SigmaFloor_WDO;
  if(SigmaD1<pisoSigma)
  { PrintFormat("[SCALPER] Sigma=%.4f<piso=%.4f. Aplicando piso.",SigmaD1,pisoSigma);
SigmaD1=pisoSigma; }
  PrintFormat("[SCALPER][SIGMA] SigmaD1=%.4f
DiasValidos=%d/%d",SigmaD1,validDaysFound,targetDays);
}

void TryCalcD1()
{
  if(!VWAP_D1_Official) CalcD1Structure();
  if(SigmaD1<=0.0) CalcSigmaHistory();
    d1Ready=(VWAP_D1_Official&&VWAP_D1>0&&SigmaD1>0&&H1>0&&L1>0&&H1>L1);
    if(!d1Ready) nextD1Retry=TimeTradeServer()+60;
}
//+------------------------------------------------------------------+
// MACRO CONTEXT (M3) — NOVO
// Determina regime estrutural. NÃO calcula ABS/CONT completo.
// Usa mesma lógica C5 da v2.44 para determinar macroAbsDir/macroContDir.
//+------------------------------------------------------------------+
void ComputeMacroContext()
{
   datetime contextClock=iTime(Symbol(),Inp_ContextTF,0);
   ResetMacroContext(gMacro);
   if(VWAP_D1<=0||!VWAP_D1_Official) { gMacro.status=MACRO_VWAP_INVALID; return;
}
   if(SigmaD1<=0)       { gMacro.status=MACRO_SIGMA_INVALID; return; }
    if(!gVwapOk||gVwapValue<=0)        { gMacro.status=MACRO_STRUCT_INVALID; return;
}
```

```
   double closedM3=iClose(Symbol(),Inp_ContextTF,1);
   if(closedM3<=0) { gMacro.status=MACRO_STRUCT_INVALID; return; }
   gMacro.barTime =iTime(Symbol(),Inp_ContextTF,1);
   gMacro.closedPrice=closedM3;
   gMacro.vwap      =gVwapValue;
   gMacro.SigmaD1 =SigmaD1;
   gMacro.distVWAP_D1=MathAbs(closedM3-VWAP_D1);
   double upper=gMacro.vwap+SigmaD1*BandMultiplier;
   double lower=gMacro.vwap-SigmaD1*BandMultiplier;
   double range=upper-lower;
   if(range<=0) { gMacro.status=MACRO_STRUCT_INVALID; return; }
   gMacro.posRel=(closedM3-lower)/range;
gMacro.glass=(gMacro.posRel>Inp_VWAP_Outer_Pct)||(gMacro.posRel<(1.0-Inp_VWAP_O
uter_Pct));
  // C5 idêntico ao da v2.44
  gMacro.macroAbsDir=0;
  if(gMacro.posRel>=Inp_VWAP_Inner_Pct) gMacro.macroAbsDir=-1;
   else if(gMacro.posRel<=(1.0-Inp_VWAP_Inner_Pct)) gMacro.macroAbsDir=1;
   gMacro.macroContDir=0;
   if(gMacro.posRel>=Inp_VWAP_Inner_Pct) gMacro.macroContDir=1;
   else if(gMacro.posRel<=(1.0-Inp_VWAP_Inner_Pct)) gMacro.macroContDir=-1;
   if(gMacro.macroAbsDir==0&&gMacro.macroContDir==0)
   { gMacro.status=MACRO_NO_TRIGGER; return; }
   if(contextClock<=0 || iTime(Symbol(),Inp_ContextTF,0)!=contextClock)
   { gMacro.status=MACRO_STRUCT_INVALID; return; }
   gMacro.status=MACRO_OK;
}

//==========================================================================
// MATRIX SNAPSHOT M1
// Preserva todas as fórmulas da v2.44 (C2/C3/C4/C5/WW/POC/ABS/CONT).
// Diferenças chave vs. v2.44:
// - usa Inp_SignalTF (M1) para close/open/volume
// - displacement e corpo em pts fixos (não sigma-fração)
// - DIVERGENCE requires displacement, exactly as canonical v2.44
// - POC: janela 60s, com detecção de empate
// - filtro MacroContext ao final
//==========================================================================
ENUM_MSS ComputeMatrixSnapshot(MatrixSnapshot &s)
{
   ResetMatrixSnapshot(s);

  // MacroContext deve ser válido (fail closed se não)
  if(Inp_EnableMacroDirectionGate && gMacro.status!=MACRO_OK)
  { s.status=MSS_MACRO_INVALID; return s.status; }
 if(VWAP_D1<=0||!VWAP_D1_Official) { s.status=MSS_VWAP_D1_INVALID; return
s.status; }
 if(SigmaD1<=0) { s.status=MSS_SIGMA_INVALID; return s.status; }
```

```
 if(H1<=0||L1<=0||H1<=L1)      { s.status=MSS_STRUCT_INVALID; return s.status; }

s.vwapVig=gVwapValue;
if(s.vwapVig<=0) { s.status=MSS_STRUCT_INVALID; return s.status; }

s.upper =s.vwapVig+SigmaD1*BandMultiplier;
s.lower =s.vwapVig-SigmaD1*BandMultiplier;
s.bandRange=s.upper-s.lower;
if(s.bandRange<=0) { s.status=MSS_STRUCT_INVALID; return s.status; }

// Candle M1 fechada (shift 1 = última fechada)
s.price =iClose(Symbol(),Inp_SignalTF,1);
s.open1 =iOpen (Symbol(),Inp_SignalTF,1);
s.candleBody=MathAbs(s.price-s.open1);
if(s.price<=0) { s.status=MSS_STRUCT_INVALID; return s.status; }

// C5 (idêntico à v2.44)
s.posRel=(s.price-s.lower)/s.bandRange;
s.glass=(s.posRel>Inp_VWAP_Outer_Pct)||(s.posRel<(1.0-Inp_VWAP_Outer_Pct));
s.dir=0;
if(s.posRel>=Inp_VWAP_Inner_Pct)      s.dir=-1;
else if(s.posRel<=(1.0-Inp_VWAP_Inner_Pct)) s.dir=1;
s.dirCont=0;
if(s.posRel>=Inp_VWAP_Inner_Pct)      s.dirCont=1;
else if(s.posRel<=(1.0-Inp_VWAP_Inner_Pct)) s.dirCont=-1;

 if(s.glass&&!Inp_EnableAbsorption) { s.status=MSS_STOP_KAMIKAZE_CONT_ONLY;
return s.status; }
 if(s.dir==0&&s.dirCont==0) { s.status=MSS_STOP_NO_C5_TRIGGER; return
s.status; }
s.c5AbsOk =(s.dir!=0);
s.c5ContOk=(s.dirCont!=0)&&!s.glass;
// Distância D-1
s.distAbs =MathAbs(s.price-VWAP_D1);
bool priceAbove=(s.price>VWAP_D1);
// Thresholds (pts fixos — hipóteses)
s.minDispl =gABS_MinDisplPts;
s.maxBody =gABS_MaxBodyPts;
s.contMinDispl=gCONT_MinDisplPts;

// C2/C3 ABS (fórmulas idênticas à v2.44)
bool sideC2Ok=((s.dir==-1&&priceAbove)||(s.dir==1&&!priceAbove));
s.c2=(sideC2Ok&&s.distAbs>=gC2);
bool sideC3Ok=((s.dir==-1&&!priceAbove)||(s.dir==1&&priceAbove));
s.c3=(sideC3Ok&&s.distAbs>=gC3Min&&s.distAbs<=gC3Max);
s.c23=(s.c2||s.c3);

 // C2/C3 CONT
```

```
 bool
sideContOk=((s.dirCont==1&&s.price>VWAP_D1)||(s.dirCont==-1&&s.price<VWAP_D1));
 s.c2Cont=(sideContOk&&s.distAbs>=gC2);
 s.c3Cont=(sideContOk&&s.distAbs>=gC3Min&&s.distAbs<=gC3Max);
 s.c23Cont=(s.c2Cont||s.c3Cont);

// C4 (idêntico à v2.44)
s.posRange=(s.price-L1)/(H1-L1);
s.c4
=(s.dir==-1)?(s.posRange>=1.0-Inp_Rule_40_60):(s.posRange<=Inp_Rule_40_60);

s.c4Cont=(s.dirCont==1)?(s.posRange>=1.0-Inp_Rule_40_60):(s.posRange<=Inp_Rule_40
_60);

// POC M1 — janelas 60s referenciadas ao início da M1 atual (causal)
// Nunca inclui ticks da barra viva (to = t0 - 1ms)
datetime t0_bar=iTime(Symbol(),Inp_SignalTF,0); // início da barra viva atual
ulong t0_msc=(ulong)t0_bar*1000;
ulong win_msc=(ulong)Inp_POC_WindowSec*1000;

if(t0_msc>2*win_msc)
{
   ulong fromNow =t0_msc-win_msc;
   ulong toNow =t0_msc-1;
   ulong fromPrev=t0_msc-2*win_msc;
   ulong toPrev =t0_msc-win_msc-1;
ComputePOCWithTie(fromNow, toNow, s.pocNow);
ComputePOCWithTie(fromPrev, toPrev, s.pocPrev);

// Optional tie rejection; baseline false preserves canonical deterministic POC
// Comportamento como input: frequência real de ties ainda desconhecida
 if(Inp_RejectPOCTies)
{
   if(s.pocNow.ok && s.pocNow.hasTie) s.pocNow.ok = false;
   if(s.pocPrev.ok && s.pocPrev.hasTie) s.pocPrev.ok = false;
 }
}
s.windowsValid=(s.pocNow.ok&&s.pocPrev.ok);

// Movimento de preço e POC
if(s.windowsValid)
{
   s.priceMove=s.pocNow.lastValidPrice-s.pocPrev.lastValidPrice;
   s.pocMove =s.pocNow.poc-s.pocPrev.poc;
}

// Canonical v2.44 classification; only scale is fixed points.
s.displacementOk=s.windowsValid && MathAbs(s.priceMove)>=s.minDispl;
```

```
s.moveDir=0;
s.pocState=POCS_INVALID;
if(s.windowsValid)
{
   if(s.displacementOk)
   {
      s.moveDir=(s.priceMove>0)?1:-1;
   bool follows=(s.pocMove*s.priceMove>0) &&
     MathAbs(s.pocMove)>=MathAbs(s.priceMove)*Inp_POC_FollowRatio;
   s.pocState=follows?POCS_FOLLOWING:POCS_DIVERGENT;
  }
  else s.pocState=POCS_NO_DISPLACEMENT;
}
s.contDisplacementOk=s.windowsValid && MathAbs(s.priceMove)>=s.contMinDispl;
s.contMoveDir=0;
s.contPocState=POCS_INVALID;
if(s.windowsValid)
{
   if(s.contDisplacementOk)
   {
      s.contMoveDir=(s.priceMove>0)?1:-1;
      bool follows=(s.pocMove*s.priceMove>0) &&
       MathAbs(s.pocMove)>=MathAbs(s.priceMove)*Inp_POC_FollowRatio;
      s.contPocState=follows?POCS_FOLLOWING:POCS_DIVERGENT;
   }
   else s.contPocState=POCS_NO_DISPLACEMENT;
}

// WW M1 (do cache reconstruído)
s.wwok =wwValid;
s.effort =wwEffortClosed;
s.waveDir=wwWaveDirClosed;
 if(s.wwok)
 {
    s.wwWaveBars=wwCurrentWaveBars;
   s.wwAccumVolume=wwAccumClosed;
   s.wwDistancePts=(wwWaveStartPrice>0)?MathAbs(s.price-wwWaveStartPrice):0;
   datetime waveStart=iTime(Symbol(),Inp_SignalTF,wwCurrentWaveBars);
   s.wwWaveAgeAvailable=wwCurrentWaveBars>0 && waveStart>0 &&
t0_bar>=waveStart;
   if(s.wwWaveAgeAvailable) s.wwWaveAgeSec=(long)(t0_bar-waveStart);
 }

// C1 ABS — fórmula idêntica à v2.44
s.c1_effort_ok =s.wwok&&(s.effort>=Inp_ABS_Effort);
s.c1_body_ok =(s.candleBody<=s.maxBody);
s.c1_displacement_ok=s.displacementOk;
s.c1_movedir_ok =(s.moveDir==-s.dir); // telemetria
```

```
s.c1_wavedir_ok =(s.waveDir==s.moveDir); // telemetria

// Contextos POC ABS
// DIVERGENCE: possible only after displacementOk in canonical classification
s.absDivergence =s.windowsValid&&(s.pocState==POCS_DIVERGENT);
// STALL_STRICT: below minimum displacement + small candle body
s.absStallStrict
=s.windowsValid&&(s.pocState==POCS_NO_DISPLACEMENT)&&s.c1_body_ok;
// STALL_RAW: sem movimento (sem exigência de corpo — telemetria)
// FOLLOWING_REACTION: POC segue, preço na direção ABS, com displacement
s.absFollowingReaction =s.windowsValid&&(s.pocState==POCS_FOLLOWING)&&
  s.displacementOk&&(s.moveDir==s.dir);
// FOLLOWING_EXTENSION: POC segue, preço contra ABS — NÃO valida ABS
s.absFollowingExtension =s.windowsValid&&(s.pocState==POCS_FOLLOWING)&&
  s.displacementOk&&(s.moveDir==-s.dir);
s.absPocContextOk=s.absDivergence||s.absStallStrict||s.absFollowingReaction;
// C1 ABS (mestre)
s.c1=s.c1_effort_ok&&s.absPocContextOk;

// C1 CONT — fórmula idêntica à v2.44
s.cont_effort_ok =s.wwok&&(s.effort>=Inp_CONT_Effort);
s.cont_displacement_ok=s.contDisplacementOk;
s.cont_poc_ok
=s.windowsValid&&s.contDisplacementOk&&(s.contPocState==POCS_FOLLOWING);
s.cont_movedir_ok =(s.contMoveDir==s.dirCont);
s.cont_wavedir_ok =(s.waveDir==s.dirCont);
s.candleDir =(s.price>s.open1)?1:((s.price<s.open1)?-1:0);
s.cont_candle_dir_ok=!Inp_Cont_RequireCandleDirection||
  (s.dirCont==1&&s.price>s.open1)||
  (s.dirCont==-1&&s.price<s.open1);
s.c1Cont=s.cont_effort_ok&&s.cont_displacement_ok&&s.cont_poc_ok&&
   s.cont_movedir_ok&&s.cont_wavedir_ok&&s.cont_candle_dir_ok;
  // BASE / TURBO (idêntico à v2.44)
  s.turboAbs =(s.c1&&s.c23&&s.c4);
  s.baseAbs =(s.c1&&(s.c23||s.c4)&&!s.turboAbs);
   s.scoreEff =(s.c1?1:0)+(s.c23?1:0)+(s.c4?1:0);
   s.turboCont=(s.c1Cont&&s.c23Cont&&s.c4Cont);
   s.baseCont =(s.c1Cont&&(s.c23Cont||s.c4Cont)&&!s.turboCont);
   s.scoreContEff=(s.c1Cont?1:0)+(s.c23Cont?1:0)+(s.c4Cont?1:0);

   // Validade antes do filtro macro
   s.absValid =Inp_EnableAbsorption &&s.c5AbsOk &&(s.turboAbs||s.baseAbs);
   s.contValid=Inp_EnableContinuation&&s.c5ContOk&&(s.turboCont||s.baseCont);

// FILTRO MACROCONTEXT
// ABS M1 precisa concordar com macroAbsDir
// CONT M1 precisa concordar com macroContDir
s.macroAbsCompatible =(s.dir ==gMacro.macroAbsDir) &&(s.dir
```

```
!=0)&&(gMacro.status==MACRO_OK);

s.macroContCompatible=(s.dirCont==gMacro.macroContDir)&&(s.dirCont!=0)&&(gMacro.sta
tus==MACRO_OK);
 if(Inp_EnableMacroDirectionGate)
 {
    if(s.absValid && !s.macroAbsCompatible) s.absValid=false;
       if(s.contValid && !s.macroContCompatible) s.contValid=false;
    }

    s.status=MSS_OK;
    return s.status;
}

//==========================================================================
// HARD DIRECTIONAL CONTRADICTION (idêntico à v2.44)
//==========================================================================
bool HasHardDirectionalContradiction(const MatrixSnapshot &s, int armedDir)
{
  if(s.status!=MSS_OK) return false;
  if(armedDir!=1&&armedDir!=-1) return false;
  return (s.absValid&&s.dir==-armedDir)||(s.contValid&&s.dirCont==-armedDir);
}

//==========================================================================
// SIZING
//==========================================================================
double ComputeLotsForEntry(int direction, double entryPrice, double slPrice)
{
   if(Inp_SizingMode==SIZING_FIXED) return Inp_FixedLots; // already validated
   double profit=0.0;
 ENUM_ORDER_TYPE type=(direction==1)?ORDER_TYPE_BUY:ORDER_TYPE_SELL;
 if(!OrderCalcProfit(type,Symbol(),1.0,entryPrice,slPrice,profit) ||
    !MathIsValidNumber(profit) || profit>=0.0)
 {
  WriteAuditRow("RISK_CALC_FAIL",DirStr(direction),armed.mode,entryPrice,0,0,0,0);
  return 0.0;
 }
 double riskCash=AccountInfoDouble(ACCOUNT_EQUITY)*Inp_RiskPerTradePct/100.0;
 if(!MathIsValidNumber(riskCash) || riskCash<=0) return 0.0;
 double step=SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_STEP);
 double
cap=MathMin(Inp_MaxLots,SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_MAX));
 double raw=MathMin(riskCash/MathAbs(profit),cap);
 double lots=NormalizeDouble(MathFloor(raw/step)*step,8);
 // Never allow a rounded floating representation to exceed raw risk/cap.
 if(lots>raw+1e-10) lots=NormalizeDouble(lots-step,8);
 if(lots<SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_MIN)-1e-10)
```

```
 {

WriteAuditRow("RISK_BELOW_MIN_LOT",DirStr(direction),armed.mode,entryPrice,0,0,0,0);
    return 0.0;
  }
  return lots;
}
bool MarginOk(ENUM_ORDER_TYPE otype, double lots)
{
  double margin=0;
  double price=(otype==ORDER_TYPE_BUY)?
      SymbolInfoDouble(Symbol(),SYMBOL_ASK):
      SymbolInfoDouble(Symbol(),SYMBOL_BID);
  if(!OrderCalcMargin(otype,Symbol(),lots,price,margin)) return false;
  return (AccountInfoDouble(ACCOUNT_MARGIN_FREE)>=margin);
}
bool SpreadOk()
{
  MqlTick tick;
  if(!SymbolInfoTick(Symbol(),tick) || tick.ask<=0 || tick.bid<=0 || tick.ask<tick.bid)
     return false;
  return tick.ask-tick.bid<=gSpreadMaxPts;
}
//+------------------------------------------------------------------+
// POSITION HELPERS (adaptados da v2.44)
//+------------------------------------------------------------------+
bool HasOpenPosition()
{
    for(int i=PositionsTotal()-1;i>=0;i--)
    {
      if(!posInfo.SelectByIndex(i)) continue;
     if(posInfo.Symbol()!=Symbol()) continue;
     if(gNetting) return true;
     if(posInfo.Magic()==(long)gMagicCore) return true;
    }
    return false;
}

bool HasOwnPositionStrict()
{
  for(int i=PositionsTotal()-1;i>=0;i--)
  {
    if(!posInfo.SelectByIndex(i)) continue;
    if(posInfo.Symbol()!=Symbol()) continue;
    if(posInfo.Magic()==(long)gMagicCore) return true;
  }
  return false;
}
```

```
// Conta trades por POSITION_ID (não por deals) — idêntico à v2.44
//+------------------------------------------------------------------+
// MAE/MFE TRACKING (por tick, sem escrita por tick)
//+------------------------------------------------------------------+
void UpdateMAEMFE()
{
   if(!gOpenPos.active) return;
   MqlTick tk;
   if(!SymbolInfoTick(Symbol(),tk)) return;
   // Usar bid/ask para cálculo conservador
   double current=(gOpenPos.direction==1)?tk.bid:tk.ask;
   double move=(double)gOpenPos.direction*(current-gOpenPos.entryPrice);
    double adverse=-move;
    if(adverse>gOpenPos.MAE) gOpenPos.MAE=adverse;
    if(move>gOpenPos.MFE) gOpenPos.MFE=move;
}

//+------------------------------------------------------------------+
// TELEMETRIA CSV
// Não escreve por tick. FileFlush apenas em eventos importantes ou no Timer.
//+------------------------------------------------------------------+
string CsvCell(string value)
{
  if(StringLen(value) == 0) return "\"\"";
   bool quoted = StringFind(value,",") >= 0 || StringFind(value,"\"") >= 0 ||
       StringFind(value,"\r") >= 0 || StringFind(value,"\n") >= 0;
    if(!quoted) return value;
    StringReplace(value,"\"","\"\"");
    return "\"" + value + "\"";
}
void CsvAdd(string &line,string value)
{
  if(StringLen(line) > 0) line += ",";
  line += CsvCell(value);
}

string CsvD(double value)
{
  if(!MathIsValidNumber(value) || value == EMPTY_VALUE) return "";
  return DoubleToString(value,8);
}

string CsvTime(datetime value)
{
  return value > 0 ? TimeToString(value,TIME_DATE|TIME_SECONDS) : "";
}
bool TradeCsvContainsPID(ulong positionID)
```

```
{
  for(int i=0;i<ArraySize(gTradeCsvWrittenPIDs);i++)
   if(gTradeCsvWrittenPIDs[i] == positionID) return true;
  return false;
}

bool TradeCsvAddPID(ulong positionID)
{
  if(TradeCsvContainsPID(positionID)) return true;
  int n=ArraySize(gTradeCsvWrittenPIDs);
  if(ArrayResize(gTradeCsvWrittenPIDs,n+1) != n+1) return false;
  gTradeCsvWrittenPIDs[n]=positionID;
  return true;
}

bool CsvParsePID(string value,ulong &positionID)
{
  positionID=0;
  if(StringLen(value)==0) return false;
  for(int i=0;i<StringLen(value);i++)
  {
     ushort c=StringGetCharacter(value,i);
     if(c<48 || c>57) return false;
     ulong digit=(ulong)(c-48);
     ulong next=positionID*10+digit;
     if(next<digit || (next-digit)/10 != positionID) return false;
     positionID=next;
    }
    return positionID>0;
}
// Strict quote-aware parsing of one complete logical CSV record.
// CR/LF inside quoted fields are preserved; malformed quoting fails closed.
bool CsvParseRecord(string record,string &fields[])
{
   ArrayResize(fields,0);
   string value="";
   bool quoted=false,afterQuote=false;
   int len=StringLen(record);
   for(int i=0;i<len;i++)
   {
     ushort c=StringGetCharacter(record,i);
     if(quoted)
     {
        if(c==34)
        {
         if(i+1<len && StringGetCharacter(record,i+1)==34)
         { value+="\""; i++; }
         else { quoted=false; afterQuote=true; }
```

```
      }
      else value+=StringSubstr(record,i,1);
    }
    else if(c==44)
    {
      int n=ArraySize(fields);
      if(ArrayResize(fields,n+1)!=n+1) return false;
      fields[n]=value; value=""; afterQuote=false;
    }
    else if(c==34)
    {
      if(StringLen(value)>0 || afterQuote) return false;
      quoted=true;
    }
    else
    {
      if(afterQuote || c==10 || c==13) return false;
      value+=StringSubstr(record,i,1);
    }
   }
   if(quoted) return false;
   int n=ArraySize(fields);
    if(ArrayResize(fields,n+1)!=n+1) return false;
    fields[n]=value;
    return true;
}

int CsvColumnIndex(string &headers[],string name)
{
  for(int i=0;i<ArraySize(headers);i++) if(headers[i]==name) return i;
  return -1;
}

// One attach-time scan. Never truncate, repair, or overwrite an existing CSV.
// A nonterminated tail is conservatively incomplete, even if fields look valid.
bool InitializeTradeCsvIndex(string fileName)
{
  ArrayResize(gTradeCsvWrittenPIDs,0);
  gTradeCsvIndexLoaded=false; gTradeCsvAppendSafe=false;
  int
h=FileOpen(fileName,FILE_READ|FILE_BIN|FILE_ANSI|FILE_SHARE_READ|FILE_SHAR
E_WRITE);
  if(h==INVALID_HANDLE)
  { Print("[RC1][TRADE_CSV_INDEX_UNAVAILABLE] ",fileName); return false; }
  ulong size=FileSize(h);
   if(size==0 || size>2147483647)
   { FileClose(h); Print("[RC1][TRADE_CSV_INDEX_INVALID_SIZE] ",fileName); return
false;
```

```
}
  uchar bytes[];
  if(ArrayResize(bytes,(int)size)!=(int)size)
  { FileClose(h); Print("[RC1][TRADE_CSV_INDEX_MEMORY_FAIL] ",fileName); return
false; }
  uint read=FileReadArray(h,bytes,0,(int)size);
  FileClose(h);
  if(read!=(uint)size)
  { Print("[RC1][TRADE_CSV_INDEX_READ_FAIL] ",fileName); return false; }
  for(int i=0;i<(int)read;i++)
     if(bytes[i]==0)
     { Print("[RC1][TRADE_CSV_INDEX_INVALID_NUL] ",fileName); return false; }
  string content=CharArrayToString(bytes,0,(int)read,CP_ACP);
  string expectedLine="";
  CsvConfigHeader(expectedLine); CsvTradeHeader(expectedLine);
  string expected[],headers[];
  if(!CsvParseRecord(expectedLine,expected)) return false;
  int schema=-1,symbol=-1,magic=-1,pid=-1,closed=-1,accounting=-1;
  bool haveHeader=false,inQuote=false;
  int recordStart=0,len=StringLen(content);
 for(int i=0;i<len;i++)
 {
    ushort c=StringGetCharacter(content,i);
  if(c==34)
  {
  if(inQuote && i+1<len && StringGetCharacter(content,i+1)==34) i++;
  else inQuote=!inQuote;
  }
  if((c!=10 && c!=13) || inQuote) continue;
  string record=StringSubstr(content,recordStart,i-recordStart);
  if(c==13 && i+1<len && StringGetCharacter(content,i+1)==10) i++;
  recordStart=i+1;
  if(!haveHeader)
  {
     if(!CsvParseRecord(record,headers) || ArraySize(headers)!=ArraySize(expected))
     { Print("[RC1][TRADE_CSV_SCHEMA_MISMATCH] ",fileName); return false; }
     for(int k=0;k<ArraySize(expected);k++)
        if(headers[k]!=expected[k])
        { Print("[RC1][TRADE_CSV_SCHEMA_MISMATCH] ",fileName); return false; }
     schema=CsvColumnIndex(headers,"SCHEMA_VERSION");
     symbol=CsvColumnIndex(headers,"SYMBOL");
     magic=CsvColumnIndex(headers,"MAGIC");
     pid=CsvColumnIndex(headers,"POSITION_ID");
     closed=CsvColumnIndex(headers,"CLOSED");
     accounting=CsvColumnIndex(headers,"ACCOUNTING_RESOLVED");
     if(schema<0 || symbol<0 || magic<0 || pid<0 || closed<0) return false;
   haveHeader=true; gTradeCsvIndexLoaded=true;
    }
```

```
    else if(StringLen(record)>0)
    {
      string fields[];
      if(!CsvParseRecord(record,fields) || ArraySize(fields)!=ArraySize(headers))
      { Print("[RC1][TRADE_CSV_INCOMPLETE_OR_INVALID_ROW] ",fileName); return
false; }
      if(fields[schema]=="RC1" && fields[symbol]==Symbol() &&
        fields[magic]==IntegerToString(gMagicCore) && fields[closed]=="1")
      {
        ulong positionID=0;
        if(!CsvParsePID(fields[pid],positionID) || !TradeCsvAddPID(positionID))
        { Print("[RC1][TRADE_CSV_INDEX_INVALID_PID] ",fileName); return false; }
        if(accounting>=0 && fields[accounting]=="0")
         {
            if(!PersistEconomicMarker(positionID)) gEconomicMarkerScanUnresolved=true;
            RememberDeferredAccountingPID(positionID);
         }
      }
   }
  }
   if(!haveHeader || inQuote || recordStart!=len)
   { Print("[RC1][TRADE_CSV_INCOMPLETE_TAIL] append disabled: ",fileName); return
false; }
   gTradeCsvAppendSafe=true;
   return true;
}

void CsvConfigHeader(string &line)
{
   CsvAdd(line,"SCHEMA_VERSION");
   CsvAdd(line,"MACRO_GATE_ENABLED");
   CsvAdd(line,"LATE_DIRECTION_BLOCK_ENABLED");
   CsvAdd(line,"REJECT_POC_TIES");
   CsvAdd(line,"SYMBOL");
   CsvAdd(line,"MAGIC");
}

void CsvConfigRow(string &line)
{
   CsvAdd(line,"RC1"); // SCHEMA_VERSION
   CsvAdd(line,BStr(Inp_EnableMacroDirectionGate)); // MACRO_GATE_ENABLED
   CsvAdd(line,BStr(Inp_EnableLateDirectionBlock)); //
// LATE_DIRECTION_BLOCK_ENABLED
   CsvAdd(line,BStr(Inp_RejectPOCTies)); // REJECT_POC_TIES
   CsvAdd(line,Symbol()); // SYMBOL
   CsvAdd(line,IntegerToString(gMagicCore)); // MAGIC
}
```

```
void CsvMacroHeader(string &line)
{
  CsvAdd(line,"MACRO_STATUS");
  CsvAdd(line,"CONTEXT_BAR");
  CsvAdd(line,"MACRO_CLOSED_PRICE");
  CsvAdd(line,"MACRO_VWAP");
  CsvAdd(line,"MACRO_SIGMA_D1");
  CsvAdd(line,"MACRO_DIST_VWAP_D1");
  CsvAdd(line,"MACRO_POSREL");
  CsvAdd(line,"MACRO_GLASS");
  CsvAdd(line,"MACRO_ABS_DIR");
  CsvAdd(line,"MACRO_CONT_DIR");
}
void CsvMacroRow(string &line)
{
  CsvAdd(line,MacroStatusStr(gMacro.status)); // MACRO_STATUS
  CsvAdd(line,CsvTime(gMacro.barTime)); // CONTEXT_BAR
  CsvAdd(line,CsvD(gMacro.closedPrice)); // MACRO_CLOSED_PRICE
  CsvAdd(line,CsvD(gMacro.vwap)); // MACRO_VWAP
  CsvAdd(line,CsvD(gMacro.SigmaD1)); // MACRO_SIGMA_D1
  CsvAdd(line,CsvD(gMacro.distVWAP_D1)); // MACRO_DIST_VWAP_D1
  CsvAdd(line,CsvD(gMacro.posRel)); // MACRO_POSREL
  CsvAdd(line,BStr(gMacro.glass)); // MACRO_GLASS
  CsvAdd(line,DirStr(gMacro.macroAbsDir)); // MACRO_ABS_DIR
  CsvAdd(line,DirStr(gMacro.macroContDir)); // MACRO_CONT_DIR
}

void CsvMatrixHeader(string &line)
{
 CsvAdd(line,"MATRIX_SNAPSHOT_AVAILABLE");
 CsvAdd(line,"SIGNAL_BAR");
 CsvAdd(line,"MATRIX_STATUS");
 CsvAdd(line,"ROUTE");
 CsvAdd(line,"MODE");
 CsvAdd(line,"DIR");
 CsvAdd(line,"PRICE");
 CsvAdd(line,"OPEN");
 CsvAdd(line,"CANDLE_BODY");
 CsvAdd(line,"CANDLE_DIR");
 CsvAdd(line,"POSREL");
 CsvAdd(line,"POS_RANGE");
 CsvAdd(line,"GLASS");
 CsvAdd(line,"DIST_VWAP_D1");
 CsvAdd(line,"VWAP_VIG");
 CsvAdd(line,"UPPER_BAND");
 CsvAdd(line,"LOWER_BAND");
 CsvAdd(line,"BAND_RANGE");
 CsvAdd(line,"SIGMA_D1");
```

```
 CsvAdd(line,"VWAP_D1");
 CsvAdd(line,"H1");
 CsvAdd(line,"L1");
 CsvAdd(line,"ABS_DIR");
 CsvAdd(line,"CONT_DIR");
 CsvAdd(line,"C1");
 CsvAdd(line,"C2");
 CsvAdd(line,"C3");
 CsvAdd(line,"C4");
 CsvAdd(line,"C5");
 CsvAdd(line,"C23");
 CsvAdd(line,"BASE");
CsvAdd(line,"TURBO");
CsvAdd(line,"CONT_C1");
CsvAdd(line,"CONT_C2");
CsvAdd(line,"CONT_C3");
CsvAdd(line,"CONT_C4");
CsvAdd(line,"CONT_C5");
CsvAdd(line,"CONT_C23");
CsvAdd(line,"CONT_BASE");
CsvAdd(line,"CONT_TURBO");
CsvAdd(line,"ABS_SCORE");
CsvAdd(line,"CONT_SCORE");
CsvAdd(line,"ABS_VALID");
CsvAdd(line,"CONT_VALID");
CsvAdd(line,"MACRO_ABS_COMPATIBLE");
CsvAdd(line,"MACRO_CONT_COMPATIBLE");
CsvAdd(line,"WW_VALID");
CsvAdd(line,"WW_EFFORT");
CsvAdd(line,"WW_DIR");
CsvAdd(line,"WW_WAVE_BARS");
CsvAdd(line,"WW_WAVE_AGE_SEC");
CsvAdd(line,"WW_ACCUM_VOLUME");
CsvAdd(line,"WW_DISTANCE_PTS");
CsvAdd(line,"WINDOWS_VALID");
CsvAdd(line,"ABS_POC_STATE");
CsvAdd(line,"CONT_POC_STATE");
CsvAdd(line,"ABS_MIN_MOVE");
CsvAdd(line,"ABS_MAX_BODY");
CsvAdd(line,"CONT_MIN_MOVE");
CsvAdd(line,"ABS_DISPLACEMENT_OK");
CsvAdd(line,"CONT_DISPLACEMENT_OK");
CsvAdd(line,"ABS_DIVERGENCE");
CsvAdd(line,"ABS_STALL_STRICT");
CsvAdd(line,"ABS_BODY_OK");
CsvAdd(line,"ABS_FOLLOWING_REACTION");
CsvAdd(line,"ABS_FOLLOWING_EXTENSION");
CsvAdd(line,"ABS_EFFORT_OK");
```

```
CsvAdd(line,"CONT_EFFORT_OK");
CsvAdd(line,"CONT_POC_OK");
CsvAdd(line,"CONT_MOVE_DIR_OK");
CsvAdd(line,"CONT_WAVE_DIR_OK");
CsvAdd(line,"CONT_CANDLE_DIR_OK");
CsvAdd(line,"PRICE_MOVE");
CsvAdd(line,"POC_MOVE");
CsvAdd(line,"MOVE_DIR");
CsvAdd(line,"CONT_MOVE_DIR");
CsvAdd(line,"POC_NOW");
CsvAdd(line,"POC_NOW_VOLUME");
CsvAdd(line,"POC_NOW_WINDOW_STATE");
CsvAdd(line,"POC_NOW_TIE");
CsvAdd(line,"POC_NOW_TRADE_COUNT");
CsvAdd(line,"POC_NOW_LEVELS");
CsvAdd(line,"POC_NOW_LAST_PRICE");
CsvAdd(line,"POC_NOW_TOP1_PRICE");
CsvAdd(line,"POC_NOW_TOP1_VOL");
CsvAdd(line,"POC_NOW_TOP2_PRICE");
CsvAdd(line,"POC_NOW_TOP2_VOL");
CsvAdd(line,"POC_NOW_DOMINANCE");
CsvAdd(line,"AGG_BUY_NOW");
CsvAdd(line,"AGG_SELL_NOW");
CsvAdd(line,"AGG_UNKNOWN_NOW");
CsvAdd(line,"AGG_DELTA_NOW");
CsvAdd(line,"AGG_IMBALANCE_NOW");
CsvAdd(line,"AGG_KNOWN_PCT_NOW");
CsvAdd(line,"POC_PREV");
  CsvAdd(line,"POC_PREV_VOLUME");
  CsvAdd(line,"POC_PREV_WINDOW_STATE");
  CsvAdd(line,"POC_PREV_TIE");
  CsvAdd(line,"POC_PREV_TRADE_COUNT");
  CsvAdd(line,"POC_PREV_LEVELS");
  CsvAdd(line,"POC_PREV_LAST_PRICE");
  CsvAdd(line,"POC_PREV_TOP1_PRICE");
  CsvAdd(line,"POC_PREV_TOP1_VOL");
  CsvAdd(line,"POC_PREV_TOP2_PRICE");
  CsvAdd(line,"POC_PREV_TOP2_VOL");
  CsvAdd(line,"POC_PREV_DOMINANCE");
  CsvAdd(line,"AGG_BUY_PREV");
  CsvAdd(line,"AGG_SELL_PREV");
  CsvAdd(line,"AGG_UNKNOWN_PREV");
  CsvAdd(line,"AGG_DELTA_PREV");
  CsvAdd(line,"AGG_IMBALANCE_PREV");
  CsvAdd(line,"AGG_KNOWN_PCT_PREV");
  CsvAdd(line,"FLOW_REGIME");
  CsvAdd(line,"VOL_RATIO");
  CsvAdd(line,"PRICE_EFFICIENCY");
```

```
  CsvAdd(line,"POC_RESPONSE");
}

void CsvMatrixRow(string &line,const MatrixSnapshot &s,bool available,datetime signalBar)
{
  string route = s.absValid && s.contValid ? "CONFLICT" :
      (s.absValid ? "ABS" : (s.contValid ? "CONT" : "NONE"));
  string mode = s.absValid && s.contValid ? "CONFLICT" :
     (s.absValid ? (s.turboAbs ? "TURBO" : "BASE") :
      (s.contValid ? (s.turboCont ? "TURBO" : "BASE") : "NONE"));
  int direction = s.absValid && !s.contValid ? s.dir :
       (s.contValid && !s.absValid ? s.dirCont : 0);
// These four formulas are executable diagnostics in canonical v2.44.
// They remain telemetry and cannot change ABS/CONT or execution.
string flowRegime = !s.c1_effort_ok ? "FLOW_WEAK" :
     (!s.displacementOk ? "FLOW_STALL" :
      (s.moveDir == s.dir ? "FLOW_REACTION" :
       (s.moveDir == -s.dir ? "FLOW_EXTENSION" : "FLOW_NA")));
CsvAdd(line,BStr(available)); // MATRIX_SNAPSHOT_AVAILABLE
CsvAdd(line,CsvTime(signalBar)); // SIGNAL_BAR
CsvAdd(line,available ? (EnumToString(s.status)) : ""); // MATRIX_STATUS
CsvAdd(line,available ? (route) : ""); // ROUTE
CsvAdd(line,available ? (mode) : ""); // MODE
CsvAdd(line,available ? (DirStr(direction)) : ""); // DIR
CsvAdd(line,available ? (CsvD(s.price)) : ""); // PRICE
CsvAdd(line,available ? (CsvD(s.open1)) : ""); // OPEN
CsvAdd(line,available ? (CsvD(s.candleBody)) : ""); // CANDLE_BODY
CsvAdd(line,available ? (IntegerToString(s.candleDir)) : ""); // CANDLE_DIR
CsvAdd(line,available ? (CsvD(s.posRel)) : ""); // POSREL
CsvAdd(line,available ? (CsvD(s.posRange)) : ""); // POS_RANGE
CsvAdd(line,available ? (BStr(s.glass)) : ""); // GLASS
CsvAdd(line,available ? (CsvD(s.distAbs)) : ""); // DIST_VWAP_D1
CsvAdd(line,available ? (CsvD(s.vwapVig)) : ""); // VWAP_VIG
CsvAdd(line,available ? (CsvD(s.upper)) : ""); // UPPER_BAND
CsvAdd(line,available ? (CsvD(s.lower)) : ""); // LOWER_BAND
CsvAdd(line,available ? (CsvD(s.bandRange)) : ""); // BAND_RANGE
CsvAdd(line,available ? (CsvD(SigmaD1)) : ""); // SIGMA_D1
CsvAdd(line,available ? (CsvD(VWAP_D1)) : ""); // VWAP_D1
CsvAdd(line,available ? (CsvD(H1)) : ""); // H1
CsvAdd(line,available ? (CsvD(L1)) : ""); // L1
CsvAdd(line,available ? (DirStr(s.dir)) : ""); // ABS_DIR
CsvAdd(line,available ? (DirStr(s.dirCont)) : ""); // CONT_DIR
CsvAdd(line,available ? (BStr(s.c1)) : ""); // C1
CsvAdd(line,available ? (BStr(s.c2)) : ""); // C2
CsvAdd(line,available ? (BStr(s.c3)) : ""); // C3
CsvAdd(line,available ? (BStr(s.c4)) : ""); // C4
CsvAdd(line,available ? (BStr(s.c5AbsOk)) : ""); // C5
CsvAdd(line,available ? (BStr(s.c23)) : ""); // C23
```

```
CsvAdd(line,available ? (BStr(s.baseAbs)) : ""); // BASE
CsvAdd(line,available ? (BStr(s.turboAbs)) : ""); // TURBO
CsvAdd(line,available ? (BStr(s.c1Cont)) : ""); // CONT_C1
CsvAdd(line,available ? (BStr(s.c2Cont)) : ""); // CONT_C2
CsvAdd(line,available ? (BStr(s.c3Cont)) : ""); // CONT_C3
CsvAdd(line,available ? (BStr(s.c4Cont)) : ""); // CONT_C4
CsvAdd(line,available ? (BStr(s.c5ContOk)) : ""); // CONT_C5
CsvAdd(line,available ? (BStr(s.c23Cont)) : ""); // CONT_C23
CsvAdd(line,available ? (BStr(s.baseCont)) : ""); // CONT_BASE
CsvAdd(line,available ? (BStr(s.turboCont)) : ""); // CONT_TURBO
CsvAdd(line,available ? (IntegerToString(s.scoreEff)) : ""); // ABS_SCORE
CsvAdd(line,available ? (IntegerToString(s.scoreContEff)) : ""); // CONT_SCORE
  CsvAdd(line,available ? (BStr(s.absValid)) : ""); // ABS_VALID
  CsvAdd(line,available ? (BStr(s.contValid)) : ""); // CONT_VALID
  CsvAdd(line,available ? (BStr(s.macroAbsCompatible)) : ""); //
// MACRO_ABS_COMPATIBLE
  CsvAdd(line,available ? (BStr(s.macroContCompatible)) : ""); //
// MACRO_CONT_COMPATIBLE
  CsvAdd(line,available ? (BStr(s.wwok)) : ""); // WW_VALID
  CsvAdd(line,available ? (CsvD(s.effort)) : ""); // WW_EFFORT
  CsvAdd(line,available ? (IntegerToString(s.waveDir)) : ""); // WW_DIR
  CsvAdd(line,available ? (s.wwok ? IntegerToString(s.wwWaveBars) : "") : ""); //
// WW_WAVE_BARS
  CsvAdd(line,available ? (s.wwok && s.wwWaveAgeAvailable ?
IntegerToString(s.wwWaveAgeSec) : "") : ""); // WW_WAVE_AGE_SEC
  CsvAdd(line,available ? (s.wwok ? CsvD(s.wwAccumVolume) : "") : ""); //
// WW_ACCUM_VOLUME
 CsvAdd(line,available ? (s.wwok ? CsvD(s.wwDistancePts) : "") : ""); //
// WW_DISTANCE_PTS
 CsvAdd(line,available ? (BStr(s.windowsValid)) : ""); // WINDOWS_VALID
 CsvAdd(line,available ? (PocStateStr(s.pocState)) : ""); // ABS_POC_STATE
 CsvAdd(line,available ? (PocStateStr(s.contPocState)) : ""); // CONT_POC_STATE
 CsvAdd(line,available ? (CsvD(s.minDispl)) : ""); // ABS_MIN_MOVE
 CsvAdd(line,available ? (CsvD(s.maxBody)) : ""); // ABS_MAX_BODY
 CsvAdd(line,available ? (CsvD(s.contMinDispl)) : ""); // CONT_MIN_MOVE
 CsvAdd(line,available ? (BStr(s.displacementOk)) : ""); // ABS_DISPLACEMENT_OK
 CsvAdd(line,available ? (BStr(s.contDisplacementOk)) : ""); //
// CONT_DISPLACEMENT_OK
 CsvAdd(line,available ? (BStr(s.absDivergence)) : ""); // ABS_DIVERGENCE
 CsvAdd(line,available ? (BStr(s.absStallStrict)) : ""); // ABS_STALL_STRICT
 CsvAdd(line,available ? (BStr(s.c1_body_ok)) : ""); // ABS_BODY_OK
 CsvAdd(line,available ? (BStr(s.absFollowingReaction)) : ""); //
// ABS_FOLLOWING_REACTION
 CsvAdd(line,available ? (BStr(s.absFollowingExtension)) : ""); //
// ABS_FOLLOWING_EXTENSION
 CsvAdd(line,available ? (BStr(s.c1_effort_ok)) : ""); // ABS_EFFORT_OK
 CsvAdd(line,available ? (BStr(s.cont_effort_ok)) : ""); // CONT_EFFORT_OK
 CsvAdd(line,available ? (BStr(s.cont_poc_ok)) : ""); // CONT_POC_OK
```

```
 CsvAdd(line,available ? (BStr(s.cont_movedir_ok)) : ""); // CONT_MOVE_DIR_OK
 CsvAdd(line,available ? (BStr(s.cont_wavedir_ok)) : ""); // CONT_WAVE_DIR_OK
 CsvAdd(line,available ? (BStr(s.cont_candle_dir_ok)) : ""); // CONT_CANDLE_DIR_OK
 CsvAdd(line,available ? (CsvD(s.priceMove)) : ""); // PRICE_MOVE
 CsvAdd(line,available ? (CsvD(s.pocMove)) : ""); // POC_MOVE
 CsvAdd(line,available ? (IntegerToString(s.moveDir)) : ""); // MOVE_DIR
 CsvAdd(line,available ? (IntegerToString(s.contMoveDir)) : ""); // CONT_MOVE_DIR
 CsvAdd(line,available ? (CsvD(s.pocNow.poc)) : ""); // POC_NOW
 CsvAdd(line,available ? (CsvD(s.pocNow.totalVolume)) : ""); // POC_NOW_VOLUME
 CsvAdd(line,available ? (s.pocNow.ok ? "VALID" : "INVALID") : ""); //
// POC_NOW_WINDOW_STATE
 CsvAdd(line,available ? (BStr(s.pocNow.hasTie)) : ""); // POC_NOW_TIE
 CsvAdd(line,available ? (IntegerToString(s.pocNow.tradeCount)) : ""); //
// POC_NOW_TRADE_COUNT
 CsvAdd(line,available ? (IntegerToString(s.pocNow.levels)) : ""); // POC_NOW_LEVELS
 CsvAdd(line,available ? (CsvD(s.pocNow.lastValidPrice)) : ""); // POC_NOW_LAST_PRICE
 CsvAdd(line,available ? (CsvD(s.pocNow.top1Price)) : ""); // POC_NOW_TOP1_PRICE
 CsvAdd(line,available ? (CsvD(s.pocNow.top1Vol)) : ""); // POC_NOW_TOP1_VOL
 CsvAdd(line,available ? (s.pocNow.levels >= 2 ? CsvD(s.pocNow.top2Price) : "") : ""); //
// POC_NOW_TOP2_PRICE
 CsvAdd(line,available ? (s.pocNow.levels >= 2 ? CsvD(s.pocNow.top2Vol) : "") : ""); //
// POC_NOW_TOP2_VOL
 CsvAdd(line,available ? (CsvD(s.pocNow.dominance)) : ""); // POC_NOW_DOMINANCE
 CsvAdd(line,available ? (CsvD(s.pocNow.buyVol)) : ""); // AGG_BUY_NOW
 CsvAdd(line,available ? (CsvD(s.pocNow.sellVol)) : ""); // AGG_SELL_NOW
 CsvAdd(line,available ? (CsvD(s.pocNow.unkVol)) : ""); // AGG_UNKNOWN_NOW
   CsvAdd(line,available ? (CsvD(s.pocNow.buyVol - s.pocNow.sellVol)) : ""); //
// AGG_DELTA_NOW
   CsvAdd(line,available ? ((s.pocNow.buyVol + s.pocNow.sellVol > 0.0) ?
CsvD((s.pocNow.buyVol - s.pocNow.sellVol)/(s.pocNow.buyVol + s.pocNow.sellVol)) : "") : "");
// AGG_IMBALANCE_NOW
   CsvAdd(line,available ? ((s.pocNow.buyVol + s.pocNow.sellVol + s.pocNow.unkVol > 0.0) ?
CsvD((s.pocNow.buyVol + s.pocNow.sellVol)/(s.pocNow.buyVol + s.pocNow.sellVol +
s.pocNow.unkVol)) : "") : ""); // AGG_KNOWN_PCT_NOW
   CsvAdd(line,available ? (CsvD(s.pocPrev.poc)) : ""); // POC_PREV
   CsvAdd(line,available ? (CsvD(s.pocPrev.totalVolume)) : ""); // POC_PREV_VOLUME
   CsvAdd(line,available ? (s.pocPrev.ok ? "VALID" : "INVALID") : ""); //
// POC_PREV_WINDOW_STATE
   CsvAdd(line,available ? (BStr(s.pocPrev.hasTie)) : ""); // POC_PREV_TIE
   CsvAdd(line,available ? (IntegerToString(s.pocPrev.tradeCount)) : ""); //
// POC_PREV_TRADE_COUNT
   CsvAdd(line,available ? (IntegerToString(s.pocPrev.levels)) : ""); // POC_PREV_LEVELS
   CsvAdd(line,available ? (CsvD(s.pocPrev.lastValidPrice)) : ""); //
// POC_PREV_LAST_PRICE
   CsvAdd(line,available ? (CsvD(s.pocPrev.top1Price)) : ""); // POC_PREV_TOP1_PRICE
   CsvAdd(line,available ? (CsvD(s.pocPrev.top1Vol)) : ""); // POC_PREV_TOP1_VOL
   CsvAdd(line,available ? (s.pocPrev.levels >= 2 ? CsvD(s.pocPrev.top2Price) : "") : ""); //
// POC_PREV_TOP2_PRICE
```

```
   CsvAdd(line,available ? (s.pocPrev.levels >= 2 ? CsvD(s.pocPrev.top2Vol) : "") : ""); //
// POC_PREV_TOP2_VOL
   CsvAdd(line,available ? (CsvD(s.pocPrev.dominance)) : ""); // POC_PREV_DOMINANCE
   CsvAdd(line,available ? (CsvD(s.pocPrev.buyVol)) : ""); // AGG_BUY_PREV
   CsvAdd(line,available ? (CsvD(s.pocPrev.sellVol)) : ""); // AGG_SELL_PREV
   CsvAdd(line,available ? (CsvD(s.pocPrev.unkVol)) : ""); // AGG_UNKNOWN_PREV
   CsvAdd(line,available ? (CsvD(s.pocPrev.buyVol - s.pocPrev.sellVol)) : ""); //
// AGG_DELTA_PREV
   CsvAdd(line,available ? ((s.pocPrev.buyVol + s.pocPrev.sellVol > 0.0) ?
CsvD((s.pocPrev.buyVol - s.pocPrev.sellVol)/(s.pocPrev.buyVol + s.pocPrev.sellVol)) : "") :
""); // AGG_IMBALANCE_PREV
   CsvAdd(line,available ? ((s.pocPrev.buyVol + s.pocPrev.sellVol + s.pocPrev.unkVol > 0.0) ?
CsvD((s.pocPrev.buyVol + s.pocPrev.sellVol)/(s.pocPrev.buyVol + s.pocPrev.sellVol +
s.pocPrev.unkVol)) : "") : ""); // AGG_KNOWN_PCT_PREV
   CsvAdd(line,available ? (flowRegime) : ""); // FLOW_REGIME
   CsvAdd(line,available ? (CsvD(s.pocPrev.totalVolume > 0.0 ?
s.pocNow.totalVolume/s.pocPrev.totalVolume : 0.0)) : ""); // VOL_RATIO
   CsvAdd(line,available ? (CsvD(s.minDispl > 0.0 ? MathAbs(s.priceMove)/s.minDispl : 0.0))
: ""); // PRICE_EFFICIENCY
  CsvAdd(line,available ? (CsvD(MathAbs(s.priceMove) > 0.0 ?
MathAbs(s.pocMove)/MathAbs(s.priceMove) : 0.0)) : ""); // POC_RESPONSE
}

void CsvArmedHeader(string &line)
{
  CsvAdd(line,"ARMED_ACTIVE");
  CsvAdd(line,"ARMED_DIR");
  CsvAdd(line,"ARMED_ROUTE");
   CsvAdd(line,"ARMED_MODE");
   CsvAdd(line,"REGION_HIGH");
   CsvAdd(line,"REGION_LOW");
   CsvAdd(line,"TRIGGER");
   CsvAdd(line,"ARMED_AT");
   CsvAdd(line,"ARMED_AGE_SEC");
   CsvAdd(line,"REQUESTS_ACCEPTED_TODAY");
   CsvAdd(line,"ORDERS_ENTERED_TODAY");
   CsvAdd(line,"HARD_FLIPS_PER_DAY");
   CsvAdd(line,"ARMED_CANCEL_HARD");
   CsvAdd(line,"OPPOSITE_REARM_AFTER_HARD");
}

void CsvArmedRow(string &line)
{
  datetime now = TimeTradeServer();
  bool armedAgeAvailable = armed.active && armed.armedAt > 0 && now >=
armed.armedAt;
  long armedAge = armedAgeAvailable ? (long)(now - armed.armedAt) : 0;
  CsvAdd(line,BStr(armed.active)); // ARMED_ACTIVE
```

```
  CsvAdd(line,armed.active ? DirStr(armed.dir) : ""); // ARMED_DIR
  CsvAdd(line,armed.active ? armed.route : ""); // ARMED_ROUTE
  CsvAdd(line,armed.active ? armed.mode : ""); // ARMED_MODE
  CsvAdd(line,armed.active ? CsvD(armed.regionHigh) : ""); // REGION_HIGH
  CsvAdd(line,armed.active ? CsvD(armed.regionLow) : ""); // REGION_LOW
  CsvAdd(line,armed.active ?
CsvD(ComputeTrigger(armed.dir,armed.regionHigh,armed.regionLow)) : ""); // TRIGGER
  CsvAdd(line,armed.active ? CsvTime(armed.armedAt) : ""); // ARMED_AT
  CsvAdd(line,armedAgeAvailable ? IntegerToString(armedAge) : ""); // ARMED_AGE_SEC
  CsvAdd(line,IntegerToString(requestsAcceptedToday)); //
// REQUESTS_ACCEPTED_TODAY
  CsvAdd(line,IntegerToString(ordersEnteredToday)); // ORDERS_ENTERED_TODAY
  CsvAdd(line,IntegerToString(hardFlipsPerDay)); // HARD_FLIPS_PER_DAY
  CsvAdd(line,IntegerToString(armedCancelHard)); // ARMED_CANCEL_HARD
  CsvAdd(line,IntegerToString(oppositeRearmAfterHard)); //
// OPPOSITE_REARM_AFTER_HARD
}

void CsvExecutionHeader(string &line)
{
  CsvAdd(line,"EVENT_DIR");
  CsvAdd(line,"EVENT_MODE");
  CsvAdd(line,"EXEC_ROUTE");
  CsvAdd(line,"EVENT_PRICE");
  CsvAdd(line,"EVENT_POSREL");
  CsvAdd(line,"EVENT_DIST_D1");
  CsvAdd(line,"EVENT_EFFORT");
  CsvAdd(line,"EVENT_TRIGGER");
   CsvAdd(line,"BREAKOUT_PRICE");
   CsvAdd(line,"REQUEST_PRICE");
   CsvAdd(line,"WEIGHTED_ENTRY_PRICE");
   CsvAdd(line,"ENTRY_SLIPPAGE");
   CsvAdd(line,"SL_PRICE");
   CsvAdd(line,"TP_PRICE");
   CsvAdd(line,"LOTS");
   CsvAdd(line,"LIVE_DIRECTION_AVAILABLE");
   CsvAdd(line,"LIVE_DIR_ABS");
   CsvAdd(line,"LIVE_DIR_CONT");
   CsvAdd(line,"LATE_DIRECTION_MISMATCH");
}

void CsvExecutionRow(string &line,string dirStr,string mode,double price,double
posRel,double distD1,double effort,double trigger)
{
  int idx = gOpenPos.tradeRecordIdx;
  bool tracked = gOpenPos.active && idx >= 0 && idx < gTradeRecordCount &&
     gTradeRecords[idx].positionID == gOpenPos.positionID;
  CsvAdd(line,dirStr); // EVENT_DIR
```

```
  CsvAdd(line,mode); // EVENT_MODE
  CsvAdd(line,gPendingExec.active ? gPendingExec.route : (armed.active ? armed.route :
"")); // EXEC_ROUTE
  CsvAdd(line,CsvD(price)); // EVENT_PRICE
  CsvAdd(line,CsvD(posRel)); // EVENT_POSREL
  CsvAdd(line,CsvD(distD1)); // EVENT_DIST_D1
  CsvAdd(line,CsvD(effort)); // EVENT_EFFORT
  CsvAdd(line,CsvD(trigger)); // EVENT_TRIGGER
  CsvAdd(line,gLastBreakoutPrice > 0.0 ? CsvD(gLastBreakoutPrice) : ""); //
// BREAKOUT_PRICE
  CsvAdd(line,gPendingExec.active && gPendingExec.requestPriceAvailable ?
CsvD(gPendingExec.requestPrice) : (tracked && gTradeRecords[idx].requestPriceAvailable
? CsvD(gTradeRecords[idx].requestPrice) : "")); // REQUEST_PRICE
  CsvAdd(line,tracked ? CsvD(gTradeRecords[idx].weightedEntryPrice) : ""); //
// WEIGHTED_ENTRY_PRICE
  CsvAdd(line,tracked && gTradeRecords[idx].requestPriceAvailable ?
CsvD(gTradeRecords[idx].entrySlippage) : ""); // ENTRY_SLIPPAGE
  CsvAdd(line,tracked ? CsvD(gTradeRecords[idx].slPrice) : (gPendingExec.active ?
CsvD(gPendingExec.slPrice) : "")); // SL_PRICE
  CsvAdd(line,tracked ? CsvD(gTradeRecords[idx].tpPrice) : (gPendingExec.active ?
CsvD(gPendingExec.tpPrice) : "")); // TP_PRICE
  CsvAdd(line,tracked ? CsvD(gTradeRecords[idx].totalVolumeIn) : (gPendingExec.active ?
CsvD(gPendingExec.requestedLots) : "")); // LOTS
  CsvAdd(line,BStr(gLiveDirectionTelemetryValid)); // LIVE_DIRECTION_AVAILABLE
  CsvAdd(line,gLiveDirectionTelemetryValid ? IntegerToString(gLiveDirAbs) : ""); //
// LIVE_DIR_ABS
  CsvAdd(line,gLiveDirectionTelemetryValid ? IntegerToString(gLiveDirCont) : ""); //
// LIVE_DIR_CONT
  CsvAdd(line,gLiveDirectionTelemetryValid ? BStr(gLateDirectionMismatch) : ""); //
// LATE_DIRECTION_MISMATCH
}

void CsvTradeHeader(string &line)
{
  CsvAdd(line,"TIMESTAMP");
  CsvAdd(line,"POSITION_ID");
  CsvAdd(line,"DIR");
  CsvAdd(line,"ROUTE");
  CsvAdd(line,"MODE");
  CsvAdd(line,"ENTRY_TIME");
  CsvAdd(line,"EXIT_TIME");
  CsvAdd(line,"TOTAL_VOLUME_IN");
  CsvAdd(line,"TOTAL_VOLUME_OUT");
  CsvAdd(line,"WEIGHTED_ENTRY_PRICE");
  CsvAdd(line,"WEIGHTED_EXIT_PRICE");
  CsvAdd(line,"BREAKOUT_PRICE");
  CsvAdd(line,"BREAKOUT_PRICE_AVAILABLE");
  CsvAdd(line,"REQUEST_PRICE");
```

```
  CsvAdd(line,"ENTRY_SLIPPAGE");
  CsvAdd(line,"REQUEST_PRICE_STATUS");
  CsvAdd(line,"SL_PRICE");
  CsvAdd(line,"TP_PRICE");
  CsvAdd(line,"MAE");
  CsvAdd(line,"MFE");
  CsvAdd(line,"HOLD_SECONDS");
  CsvAdd(line,"GROSS_PNL");
  CsvAdd(line,"COMMISSION");
  CsvAdd(line,"FEE");
  CsvAdd(line,"SWAP");
  CsvAdd(line,"NET_PNL");
    CsvAdd(line,"INITIAL_RISK_MONEY");
    CsvAdd(line,"PNL_R");
    CsvAdd(line,"EXIT_REASON");
    CsvAdd(line,"QUOTA_COUNTED");
    CsvAdd(line,"MAE_MFE_PARTIAL_AFTER_RESTART");
    CsvAdd(line,"ACCOUNTING_RESOLVED");
    CsvAdd(line,"ACCOUNTING_STATUS");
    CsvAdd(line,"RISK_VALID");
    CsvAdd(line,"PNL_R_VALID");
    CsvAdd(line,"VOLUME_CLOSED_COMPLETE");
    CsvAdd(line,"CLOSED");
    CsvAdd(line,"DAILY_NET_PNL");
    CsvAdd(line,"DAILY_PNL_R");
    CsvAdd(line,"CONSECUTIVE_LOSSES");
    CsvAdd(line,"DAILY_RISK_BLOCKED");
    CsvAdd(line,"REQUESTS_ACCEPTED_TODAY");
     CsvAdd(line,"ORDERS_ENTERED_TODAY");
     CsvAdd(line,"HARD_FLIPS_PER_DAY");
     CsvAdd(line,"ARMED_CANCEL_HARD");
     CsvAdd(line,"OPPOSITE_REARM_AFTER_HARD");
}

void CsvTradeRow(string &line,const TradeRecord &r)
{
  CsvAdd(line,CsvTime(TimeTradeServer())); // TIMESTAMP
  CsvAdd(line,StringFormat("%I64u",r.positionID)); // POSITION_ID
  CsvAdd(line,DirStr(r.direction)); // DIR
  CsvAdd(line,r.route); // ROUTE
  CsvAdd(line,r.mode); // MODE
  CsvAdd(line,CsvTime(r.entryTime)); // ENTRY_TIME
  CsvAdd(line,CsvTime(r.exitTime)); // EXIT_TIME
  CsvAdd(line,CsvD(r.totalVolumeIn)); // TOTAL_VOLUME_IN
  CsvAdd(line,CsvD(r.totalVolumeOut)); // TOTAL_VOLUME_OUT
  CsvAdd(line,CsvD(r.weightedEntryPrice)); // WEIGHTED_ENTRY_PRICE
  CsvAdd(line,CsvD(r.weightedExitPrice)); // WEIGHTED_EXIT_PRICE
  CsvAdd(line,r.breakoutPriceAvailable ? CsvD(r.breakoutPrice) : ""); // BREAKOUT_PRICE
```

```
  CsvAdd(line,BStr(r.breakoutPriceAvailable)); // BREAKOUT_PRICE_AVAILABLE
  CsvAdd(line,r.requestPriceAvailable ? CsvD(r.requestPrice) : ""); // REQUEST_PRICE
  CsvAdd(line,r.requestPriceAvailable ? CsvD(r.entrySlippage) : ""); // ENTRY_SLIPPAGE
  CsvAdd(line,r.requestPriceAvailable ? "AVAILABLE" : (r.maeMfePartialAfterRestart ?
"REQUEST_PRICE_UNAVAILABLE_AFTER_RESTART" :
"REQUEST_PRICE_UNAVAILABLE")); // REQUEST_PRICE_STATUS
  CsvAdd(line,CsvD(r.slPrice)); // SL_PRICE
  CsvAdd(line,CsvD(r.tpPrice)); // TP_PRICE
  CsvAdd(line,CsvD(r.MAE)); // MAE
  CsvAdd(line,CsvD(r.MFE)); // MFE
  CsvAdd(line,IntegerToString(r.holdSeconds)); // HOLD_SECONDS
  CsvAdd(line,CsvD(r.grossProfit)); // GROSS_PNL
  CsvAdd(line,r.inoutCostsUnresolved ? "" : CsvD(r.commission)); // COMMISSION
  CsvAdd(line,r.inoutCostsUnresolved ? "" : CsvD(r.fee)); // FEE
  CsvAdd(line,CsvD(r.swap)); // SWAP
  CsvAdd(line,r.accountingResolved ? CsvD(r.netProfit) : ""); // NET_PNL
  CsvAdd(line,r.riskValid ? CsvD(r.initialRiskMoney) : ""); // INITIAL_RISK_MONEY
  CsvAdd(line,r.pnlRValid ? CsvD(r.PnL_R) : ""); // PNL_R
  CsvAdd(line,r.exitReason); // EXIT_REASON
  CsvAdd(line,BStr(r.quotaCounted)); // QUOTA_COUNTED
  CsvAdd(line,BStr(r.maeMfePartialAfterRestart)); //
// MAE_MFE_PARTIAL_AFTER_RESTART
  CsvAdd(line,BStr(r.accountingResolved)); // ACCOUNTING_RESOLVED
  CsvAdd(line,r.accountingResolved ? "RESOLVED" : (r.inoutCostsUnresolved ?
    "INOUT_COST_ALLOCATION_UNAVAILABLE" : (r.maeMfePartialAfterRestart ?
   "RESTART_ACCOUNTING_UNRESOLVED" : "ACCOUNTING_UNRESOLVED"))); //
// ACCOUNTING_STATUS
  CsvAdd(line,BStr(r.riskValid)); // RISK_VALID
  CsvAdd(line,BStr(r.pnlRValid)); // PNL_R_VALID
  CsvAdd(line,BStr(r.volumeClosedComplete)); // VOLUME_CLOSED_COMPLETE
  CsvAdd(line,BStr(r.closed)); // CLOSED
  CsvAdd(line,gDailyEconomicsValid ? CsvD(dailyNetPnL) : ""); // DAILY_NET_PNL
  CsvAdd(line,gDailyPnLRValid ? CsvD(dailyPnLR) : ""); // DAILY_PNL_R
  CsvAdd(line,gDailyEconomicsValid ? IntegerToString(consecutiveLosses) : ""); //
// CONSECUTIVE_LOSSES
  CsvAdd(line,BStr(dailyRiskBlocked)); // DAILY_RISK_BLOCKED
  CsvAdd(line,IntegerToString(requestsAcceptedToday)); //
// REQUESTS_ACCEPTED_TODAY
  CsvAdd(line,IntegerToString(ordersEnteredToday)); // ORDERS_ENTERED_TODAY
  CsvAdd(line,IntegerToString(hardFlipsPerDay)); // HARD_FLIPS_PER_DAY
  CsvAdd(line,IntegerToString(armedCancelHard)); // ARMED_CANCEL_HARD
  CsvAdd(line,IntegerToString(oppositeRearmAfterHard)); //
// OPPOSITE_REARM_AFTER_HARD
}
void WriteAuditHeader()
{
  if(csvAuditHandle == INVALID_HANDLE) return;
  string line = "TIMESTAMP,STATUS";
```

```
  CsvConfigHeader(line);
  CsvMacroHeader(line);
  CsvMatrixHeader(line);
  CsvArmedHeader(line);
  CsvExecutionHeader(line);
  FileWriteString(csvAuditHandle,line + "\r\n");
}

void WriteAuditRow(string status,string dirStr,string mode,
      double price,double posRel,double distD1,double effort,double trigger)
{
  if(Inp_DOM_Enable) DOMObserveAudit(status,dirStr,mode,price,trigger);
  if(csvAuditHandle == INVALID_HANDLE) return;
  string line = "";
  CsvAdd(line,CsvTime(TimeTradeServer()));
  CsvAdd(line,status);
  CsvConfigRow(line);
  CsvMacroRow(line);
CsvMatrixRow(line,gLastTelemetryMatrix,gLastTelemetryMatrixValid,gLastTelemetrySignalB
ar);
  CsvArmedRow(line);
  CsvExecutionRow(line,dirStr,mode,price,posRel,distD1,effort,trigger);
  FileWriteString(csvAuditHandle,line + "\r\n");
}
void WriteShadowHeader()
{
  if(csvShadowHandle == INVALID_HANDLE) return;
  string line = "TIMESTAMP,EVENT";
    CsvConfigHeader(line);
    CsvMacroHeader(line);
    CsvMatrixHeader(line);
    CsvArmedHeader(line);
    FileWriteString(csvShadowHandle,line + "\r\n");
}

void WriteShadowRow(string event,const MatrixSnapshot &s)
{
  if(csvShadowHandle == INVALID_HANDLE) return;
  string line = "";
  CsvAdd(line,CsvTime(TimeTradeServer()));
  CsvAdd(line,event);
  CsvConfigRow(line);
  CsvMacroRow(line);
  CsvMatrixRow(line,s,true,iTime(Symbol(),Inp_SignalTF,1));
    CsvArmedRow(line);
    FileWriteString(csvShadowHandle,line + "\r\n");
}
```

```
void WriteTradeHeader()
{
  if(csvTradeHandle == INVALID_HANDLE) return;
  string line = "";
  CsvConfigHeader(line);
  CsvTradeHeader(line);
  FileWriteString(csvTradeHandle,line + "\r\n");
}

bool WriteTradeRow(const TradeRecord &r)
{
  if(r.positionID==0 || !r.closed) return false;
  if(gTradeCsvIndexLoaded && TradeCsvContainsPID(r.positionID)) return true;
  if(csvTradeHandle == INVALID_HANDLE || !gTradeCsvIndexLoaded ||
!gTradeCsvAppendSafe)
     return false;
  string line = "";
  CsvConfigRow(line);
    CsvTradeRow(line,r);
    string payload=line + "\r\n";
    uchar encoded[];
    int expected=StringToCharArray(payload,encoded,0,WHOLE_ARRAY,CP_ACP)-1;
    if(expected<=0) return false;
      ResetLastError();
    uint written=FileWriteString(csvTradeHandle,payload);
    int writeError=GetLastError();
    if(written!=(uint)expected || writeError!=0)
    { gTradeCsvAppendSafe=false; return false; }
    ResetLastError();
    FileFlush(csvTradeHandle); // Confirmed trade finalization only.
    if(GetLastError()!=0) { gTradeCsvAppendSafe=false; return false; }
    if(!TradeCsvAddPID(r.positionID)) gTradeCsvAppendSafe=false;
    if(Inp_DOM_Enable) DOMObserveTrade(r);
    return true;
}


ENUM_SEND_OUTCOME SendOrder(int direction, double unusedLots, string mode,
     double slPts, double tpPts,
     double posRel, double distD1, double effort, double trigger)
{
  string dStr=DirStr(direction);
  if(HasOpenPosition())
  { WriteAuditRow("LATE_POSITION_BLOCK",dStr,mode,0,posRel,distD1,effort,trigger);
return SEND_FINAL_FAILURE; }
 if(ordersEnteredToday>=Inp_MaxTradesDay)
 { WriteAuditRow("LATE_QUOTA_BLOCK",dStr,mode,0,posRel,distD1,effort,trigger); return
SEND_FINAL_FAILURE; }
```

```
 if(dailyRiskBlocked || gAccountingUnresolved)
 { WriteAuditRow("LATE_DAILY_RISK_BLOCK",dStr,mode,0,posRel,distD1,effort,trigger);
return SEND_FINAL_FAILURE; }
 if(HasExecutionInFlight())
 { WriteAuditRow("LATE_PENDING_BLOCK",dStr,mode,0,posRel,distD1,effort,trigger);
return SEND_FINAL_FAILURE; }

 BookGateDecision bookDecision;
 ArmedSignal bookOriginal=armed;
 PendingExecContext bookIntent;
 ZeroMemory(bookIntent);
  bool bookApiCallStarted=false; // local control-flow witness, never persisted
   // One final snapshot: all risk/stops calculations refer to this executable quote.
   MqlTick finalTick;
   if(!SymbolInfoTick(Symbol(),finalTick) || finalTick.ask<=0 || finalTick.bid<=0 ||
      finalTick.ask<finalTick.bid) return SEND_FINAL_FAILURE;
   double finalExec=(direction==1)?finalTick.ask:finalTick.bid;
   if(finalTick.ask-finalTick.bid>gSpreadMaxPts) return SEND_FINAL_FAILURE;
   bool finalBreakout=(direction==1)?finalExec>=trigger:finalExec<=trigger;
   if(!finalBreakout)
   {
WriteAuditRow("LATE_BREAKOUT_LOST",dStr,mode,finalExec,posRel,distD1,effort,trigger)
; return SEND_FINAL_FAILURE; }
   double over=(direction==1)?finalExec-trigger:trigger-finalExec;
   if(over>gMaxOvershoot)
   {
WriteAuditRow("LATE_OVERSHOOT_BLOCK",dStr,mode,finalExec,posRel,distD1,effort,trig
ger); return
Inp_DOM_TradeGateEnable?SEND_ORIGINAL_CANCEL:SEND_FINAL_FAILURE; }
  double sl=NormEntrySL(direction,finalExec-direction*slPts);
  double tp=NormEntryTP(direction,finalExec+direction*tpPts);
  if(sl<=0 || tp<=0) return SEND_FINAL_FAILURE;
  double minDist=(double)SymbolInfoInteger(Symbol(),SYMBOL_TRADE_STOPS_LEVEL)*
      SymbolInfoDouble(Symbol(),SYMBOL_POINT);
  // Server stops are also measured from the closing quote (bid for buy, ask for sell).
  double closingQuote=(direction==1)?finalTick.bid:finalTick.ask;
  if(direction*(finalExec-sl)<minDist || direction*(tp-finalExec)<minDist ||
     direction*(closingQuote-sl)<minDist || direction*(tp-closingQuote)<minDist)
  {
WriteAuditRow("STOPS_LEVEL_BLOCK",dStr,mode,finalExec,posRel,distD1,effort,trigger);
return SEND_FINAL_FAILURE; }
  double lots=ComputeLotsForEntry(direction,finalExec,sl);
  if(lots<=0) return SEND_FINAL_FAILURE;
  ENUM_ORDER_TYPE type=(direction==1)?ORDER_TYPE_BUY:ORDER_TYPE_SELL;
  double margin=0;
  if(!OrderCalcMargin(type,Symbol(),lots,finalExec,margin) ||
     !MathIsValidNumber(margin) || margin<0 ||
     AccountInfoDouble(ACCOUNT_MARGIN_FREE)<margin)
```

```
  { WriteAuditRow("NO_MARGIN",dStr,mode,finalExec,posRel,distD1,effort,trigger); return
SEND_FINAL_FAILURE; }
   trade.SetExpertMagicNumber(gMagicCore);
   trade.SetTypeFilling(ORDER_FILLING_FOK);
   // Re-read immediately before dispatch. A moved executable quote needs fresh
   // sizing/stops; a changed closing quote always needs a fresh STOPS_LEVEL check.
   MqlTick dispatchTick;
   if(!SymbolInfoTick(Symbol(),dispatchTick)) return SEND_FINAL_FAILURE;
   double dispatchExec=(direction==1)?dispatchTick.ask:dispatchTick.bid;
   if(dispatchExec<=0 || dispatchTick.ask<=0 || dispatchTick.bid<=0 ||
      dispatchTick.ask<dispatchTick.bid || dispatchTick.ask-dispatchTick.bid>gSpreadMaxPts)
return SEND_FINAL_FAILURE;
   if((direction==1 && dispatchExec<trigger) || (direction==-1 && dispatchExec>trigger))
   {
WriteAuditRow("LATE_BREAKOUT_LOST",dStr,mode,dispatchExec,posRel,distD1,effort,tri
gger); return SEND_FINAL_FAILURE; }
   if(direction*(dispatchExec-trigger)>gMaxOvershoot)
   {
WriteAuditRow("LATE_OVERSHOOT_BLOCK",dStr,mode,dispatchExec,posRel,distD1,effor
t,trigger); return
Inp_DOM_TradeGateEnable?SEND_ORIGINAL_CANCEL:SEND_FINAL_FAILURE; }
   if(dispatchExec!=finalExec)
   {
      finalExec=dispatchExec;
      sl=NormEntrySL(direction,finalExec-direction*slPts);
      tp=NormEntryTP(direction,finalExec+direction*tpPts);
      lots=ComputeLotsForEntry(direction,finalExec,sl);
      closingQuote=(direction==1)?dispatchTick.bid:dispatchTick.ask;
      if(sl<=0 || tp<=0 || lots<=0 || direction*(closingQuote-sl)<minDist ||
     direction*(tp-closingQuote)<minDist)
      {
WriteAuditRow("STOPS_LEVEL_BLOCK",dStr,mode,finalExec,posRel,distD1,effort,trigger);
return SEND_FINAL_FAILURE; }
      if(!OrderCalcMargin(type,Symbol(),lots,finalExec,margin) || !MathIsValidNumber(margin)
|| margin<0 || AccountInfoDouble(ACCOUNT_MARGIN_FREE)<margin)
      { WriteAuditRow("NO_MARGIN",dStr,mode,finalExec,posRel,distD1,effort,trigger); return
SEND_FINAL_FAILURE; }
   }
   closingQuote=(direction==1)?dispatchTick.bid:dispatchTick.ask;
   if(direction*(finalExec-sl)<minDist || direction*(tp-finalExec)<minDist ||
      direction*(closingQuote-sl)<minDist || direction*(tp-closingQuote)<minDist)
   {
WriteAuditRow("STOPS_LEVEL_BLOCK",dStr,mode,finalExec,posRel,distD1,effort,trigger);
return SEND_FINAL_FAILURE; }
   if(HasOpenPosition()) return SEND_FINAL_FAILURE;
   if(HasOwnPendingOrder() || HasExecutionInFlight())
   {
WriteAuditRow("LATE_PENDING_BLOCK",dStr,mode,finalExec,posRel,distD1,effort,trigger)
```

```
; return SEND_FINAL_FAILURE; }
   if(Inp_DOM_TradeGateEnable)
   {
      // Original terminal execution guards above take precedence over DOM veto.
      ZeroMemory(bookDecision);
      DOMClearSnapshot(bookDecision.snapshot);
      bookDecision.state=BOOK_UNAVAILABLE;
      bookDecision.beginUS=GetMicrosecondCount();
      bookDecision.signalClock=iTime(Symbol(),Inp_SignalTF,0);
      bookDecision.contextClock=iTime(Symbol(),Inp_ContextTF,0);
      ENUM_APS ready;
      double currentEff,currentTrigger,currentLast;
      if(!DOMGateDecisionCurrent(bookDecision,dispatchTick,bookOriginal,false,bookIntent,
                        ready,currentEff,currentTrigger,currentLast,finalExec,lots,sl,tp))
      {
         if(ready==APS_ENDED_REEVAL_ALLOWED) return SEND_CANCEL_REEVAL;
         if(ready==APS_TRADE_OR_POSITION) return SEND_TRADE_OR_POSITION;
         if(!armed.active || ready==APS_ENDED_FAIL_CLOSED) return
SEND_FINAL_FAILURE;
         return
DOMGateDefer(bookDecision,"CURRENT_CONDITIONS_NOT_READY_BEFORE_INTEN
T");
      }
      effort=currentEff; trigger=currentTrigger; gLastBreakoutPrice=currentLast;
   }
   ulong bookPrepareBegin=Inp_DOM_TradeGateEnable?GetMicrosecondCount():0;
   if(!Inp_DOM_TradeGateEnable) armed.lots=lots; // Original gate-off behavior.
   if(!PreparePendingExec(direction,armed.route,mode,finalExec,lots,sl,tp))
   {
      if(Inp_DOM_TradeGateEnable && (HasExecutionInFlight() ||
         gEntryJournalInvalid || gExitJournalInvalid)) return
SEND_FINANCIAL_UNRESOLVED;
      return SEND_FINAL_FAILURE;
   }
  gPendingExec.breakoutPrice=gLastBreakoutPrice;
  gPendingExec.breakoutPriceAvailable=gLastBreakoutPrice>0;
  if(Inp_DOM_TradeGateEnable)
  {
     ulong elapsed=GetMicrosecondCount()-bookPrepareBegin;
     gBookGatePrepareCalls++; gBookGatePrepareSumUS+=(double)elapsed;
     if(elapsed>gBookGatePrepareMaxUS) gBookGatePrepareMaxUS=elapsed;
     bookIntent=gPendingExec;
     ENUM_APS ready;
     double currentEff,currentTrigger,currentLast;
     if(!DOMGateDecisionCurrent(bookDecision,dispatchTick,bookOriginal,true,bookIntent,
                        ready,currentEff,currentTrigger,currentLast,finalExec,lots,sl,tp))
       return
```

```
DOMGateRejectPrepared(bookDecision,"CURRENT_CONDITIONS_CHANGED_DURING_
PREPARE",
                      bookOriginal,bookIntent,bookApiCallStarted,ready);
   gLastBreakoutPrice=currentLast;
 }
 ulong bookJournalBegin=Inp_DOM_TradeGateEnable?GetMicrosecondCount():0;
 if(!MarkRequestDispatchPossible(false))
 {
   if(Inp_DOM_TradeGateEnable)
   {
     bookIntent.dispatchPhase=gPendingExec.dispatchPhase;
     return
DOMGateRejectPrepared(bookDecision,"DISPATCH_JOURNAL_WRITE_FAILED",
               bookOriginal,bookIntent,bookApiCallStarted,APS_STILL_ARMED);
   }
   ResetPendingExec(); return SEND_FINAL_FAILURE;
 }
 if(Inp_DOM_TradeGateEnable)
 {
   ulong elapsed=GetMicrosecondCount()-bookJournalBegin;
   gBookGateJournalCalls++; gBookGateJournalSumUS+=(double)elapsed;
   if(elapsed>gBookGateJournalMaxUS) gBookGateJournalMaxUS=elapsed;
   // The same call stack has still not called PositionOpen; update ONLY phase witness.
   bookIntent.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
   ENUM_APS ready;
   double currentEff,currentTrigger,currentLast;
   if(!DOMGateDecisionCurrent(bookDecision,dispatchTick,bookOriginal,true,bookIntent,
                ready,currentEff,currentTrigger,currentLast,finalExec,lots,sl,tp))
     return
DOMGateRejectPrepared(bookDecision,"CURRENT_CONDITIONS_CHANGED_DURING_
DISPATCH_JOURNAL",
                      bookOriginal,bookIntent,bookApiCallStarted,ready);
   armed.lots=lots;
   gLastBreakoutPrice=currentLast;
 }
 ulong bookDispatchStartUS=Inp_DOM_TradeGateEnable?GetMicrosecondCount():0;
 bookApiCallStarted=true; // immediately BEFORE the only entry API call
 bool
sent=trade.PositionOpen(Symbol(),type,lots,finalExec,sl,tp,"RC1|"+armed.route+"|"+mode);
 uint ret=trade.ResultRetcode();
 MqlTradeResult executionResult;
 trade.Result(executionResult);

CompletePendingExec(sent,ret,trade.ResultOrder(),trade.ResultDeal(),executionResult.requ
est_id);
 bool accepted=sent && (ret==TRADE_RETCODE_DONE ||
ret==TRADE_RETCODE_PLACED || ret==TRADE_RETCODE_DONE_PARTIAL);
```

```
WriteAuditRow(accepted?"REQUEST_ACCEPTED":"ORDER_FAILED",dStr,mode,finalExec
,posRel,distD1,effort,trigger);
 if(Inp_DOM_TradeGateEnable)
 {
    // Only timing scalars and bounded diagnostics, never a return-value dependency.
    ulong dispatchGap=bookDispatchStartUS-bookDecision.beginUS;
    if(dispatchGap>gBookGateDispatchMaxUS) gBookGateDispatchMaxUS=dispatchGap;
    if(accepted)
    {
    gBookGateReleased++;
    if(DOMGateLogAllowed())
    PrintFormat("[BOOK_GATE][BOOK_ENTRY_RELEASED] symbol=%s dir=%d
armedAt=%I64d waited=%s bidPct=%.8f askPct=%.8f; request accepted, not a fill or profit
guarantee",

gBookGateSymbol,direction,(long)armed.armedAt,BStr(gBookGateWaiting),bookDecision.pc
tBid,bookDecision.pctAsk);
    }
  }
  if(Inp_DOM_TradeGateEnable && !accepted &&
     (HasExecutionInFlight() || gEntryJournalInvalid || gExitJournalInvalid))
     return SEND_FINANCIAL_UNRESOLVED;
  return accepted?SEND_ACCEPTED:SEND_FINAL_FAILURE;
}

//=========================================================================
// STAGE 1 — EVALUATE MATRIX
//=========================================================================
ENUM_MSS SnapshotForSignal(MatrixSnapshot &out)
{
   datetime bar=iTime(Symbol(),Inp_SignalTF,0);
   if(!gCachedMicroReady || gCachedMicroBar!=bar)
   {
    ulong begin=GetMicrosecondCount();
    ComputeMatrixSnapshot(gCachedMicroSnapshot);
    if(iTime(Symbol(),Inp_SignalTF,0)!=bar)
       gCachedMicroSnapshot.status=MSS_STRUCT_INVALID; // discard a torn bar
// snapshot
    gCachedMicroBar=bar;
    gCachedMicroReady=true;
    gLastTelemetryMatrix=gCachedMicroSnapshot;
    gLastTelemetryMatrixValid=true;
    gLastTelemetrySignalBar=iTime(Symbol(),Inp_SignalTF,1);
    RecordPerf(perfMicro,(double)(GetMicrosecondCount()-begin));
  }
  out=gCachedMicroSnapshot;
  return out.status;
}
```

```
void EvaluateMatrix()
{
  gLiveDirectionTelemetryValid=false;
  gLastBreakoutPrice=0;
 MatrixSnapshot s;
 ENUM_MSS status=SnapshotForSignal(s);

// Gates fail-closed
if(status==MSS_VWAP_D1_INVALID)
{ WriteAuditRow("VWAP_D1_INVALID","NONE","NONE",0,0,0,0,0); return; }
if(status==MSS_SIGMA_INVALID)
{ WriteAuditRow("SIGMA_INVALID","NONE","NONE",0,0,0,0,0); return; }
if(status==MSS_MACRO_INVALID)
{
WriteAuditRow("MACRO_INVALID","NONE","NONE",s.price,s.posRel,s.distAbs,s.effort,0);
  WriteShadowRow("MACRO_INVALID",s); return; }
if(status==MSS_STRUCT_INVALID) return;
if(status==MSS_STOP_KAMIKAZE_CONT_ONLY)
{ WriteAuditRow("KAMIKAZE_GLASS","NONE","NONE",s.price,s.posRel,0,0,0); return; }
if(status==MSS_STOP_NO_C5_TRIGGER)
{ WriteAuditRow("NO_C5_TRIGGER","NONE","NONE",s.price,s.posRel,0,0,0); return; }
if(status!=MSS_OK) return;

// Atualizar cache de bandas
UpperBand1=s.upper; LowerBand1=s.lower;

 // Log de FOLLOWING_EXTENSION (telemetria, não bloqueia nada além do que o C1 já
// bloqueia)
  if(Inp_EnableAbsorption&&s.absFollowingExtension)
    PrintFormat("[SCALPER][ABS_FOLLOWING_EXTENSION_BLOCK] Dir=%s
PriceMove=%.4f PocMove=%.4f FLOW=extension",
      DirStr(s.dir),s.priceMove,s.pocMove);
// ROUTE_CONFLICT_BLOCK
if(s.absValid&&s.contValid)
{
    PrintFormat("[SCALPER][ROUTE_CONFLICT_BLOCK] ABS(dir=%s ctx=%s) e
CONT(dir=%s) simultaneamente validas. BLOQUEADO.",
     DirStr(s.dir),PocStateStr(s.pocState),DirStr(s.dirCont));

WriteAuditRow("ROUTE_CONFLICT_BLOCK","NONE","NONE",s.price,s.posRel,s.distAbs,s
.effort,0);
   WriteShadowRow("ROUTE_CONFLICT",s);
   return;
 }

// Sem confluência
if(!s.absValid&&!s.contValid)
```

```
{
    int d2=(s.dir!=0)?s.dir:s.dirCont;

WriteAuditRow("NO_CONFLUENCE",DirStr(d2),"NONE",s.price,s.posRel,s.distAbs,s.effort,0
);
   WriteShadowRow("NO_CONFLUENCE",s);
   return;
 }

// Selecionar rota
bool useCont=(!s.absValid&&s.contValid);
int dirSel =useCont?s.dirCont:s.dir;
bool turboSel=useCont?s.turboCont:s.turboAbs;
int scoreSel=useCont?s.scoreContEff:s.scoreEff;
string mode;
if(useCont) mode=turboSel?"TURBO_CONT":"BASE_CONT";
else mode=turboSel?"TURBO":"BASE";

// Stage 1 arms only a thesis; lots are resolved at the final executable price.
double lots=0.0;

 // Região (N barras M1 fechadas)
 double regHigh=-DBL_MAX, regLow=DBL_MAX;
 MqlRates regionRates[];
 int
regionCopied=CopyRates(Symbol(),Inp_SignalTF,1,Inp_Region_Candles,regionRates);
 bool regionOk=(regionCopied==Inp_Region_Candles);
 if(regionOk)
 {
     for(int i=0;i<regionCopied;i++)
     {
if(regionRates[i].high<=0||regionRates[i].low<=0||regionRates[i].high<regionRates[i].low)
       { regionOk=false; break; }
       if(regionRates[i].high>regHigh) regHigh=regionRates[i].high;
       if(regionRates[i].low <regLow) regLow =regionRates[i].low;
    }
  }
  if(!regionOk||regHigh<=-DBL_MAX||regLow>=DBL_MAX||regHigh<=regLow)
  {

WriteAuditRow("REGION_INVALID",DirStr(dirSel),mode,s.price,s.posRel,s.distAbs,s.effort,0)
;
    return;
  }

 // ARMAR
 if(iTime(Symbol(),Inp_SignalTF,0)!=gCachedMicroBar) return; // closed snapshot/region
// must share the same clock
```

```
 gLastBreakoutPrice=0;
 gLiveDirectionTelemetryValid=false;
 gLateMismatchLogged=false; gSpreadHoldLogged=false;
 armed.active =true;
armed.dir =dirSel;
armed.regionHigh =regHigh;
armed.regionLow =regLow;
armed.activeSL_Pts =gSL_Pts;
armed.activeTP_Pts =gTP_Pts;
armed.regionEndTime =iTime(Symbol(),Inp_SignalTF,1);
armed.mode        =mode;
armed.lots =lots;
armed.route =useCont?"CONT":"ABS";
armed.breakoutEffortMin=useCont?Inp_CONT_BreakoutEffort:Inp_ABS_BreakoutEffort;
armed.armedAt     =TimeTradeServer();
armed.sameSideGlassLogged=false;
armed.posRel=s.posRel; armed.distD1=s.distAbs; armed.effort=s.effort;
armed.vwapVig=s.vwapVig;
armed.c1=(int)(useCont?s.c1Cont:s.c1);
armed.c2=(int)(useCont?s.c2Cont:s.c2);
armed.c3=(int)(useCont?s.c3Cont:s.c3);
armed.c4=(int)(useCont?s.c4Cont:s.c4);
armed.c5=(int)(useCont?s.c5ContOk:s.c5AbsOk);
armed.scoreEff=scoreSel;

double armTrigger=ComputeTrigger(dirSel,regHigh,regLow);
PrintFormat("[SCALPER][ARMED_%s] %s %s %.2f lots SCORE=%d/3 SL=%.1f "
"High=%.4f Low=%.4f Trigger=%.4f OndaViva>=%.2f | "
"Macro: AbsDir=%s ContDir=%s PosRelM3=%.4f",
armed.route,mode,DirStr(dirSel),lots,scoreSel,gSL_Pts,
regHigh,regLow,armTrigger,armed.breakoutEffortMin,
DirStr(gMacro.macroAbsDir),DirStr(gMacro.macroContDir),gMacro.posRel);
WriteAuditRow("ARMED",DirStr(dirSel),mode,s.price,s.posRel,s.distAbs,s.effort,armTrigger);
  WriteShadowRow("ARMED",s);
}

//+------------------------------------------------------------------+
// STAGE 2 — PROCESS ARMED SIGNAL
// Preserva toda a lógica da v2.44.
// Diferenças: timeout por tempo, WW M1, LATE_DIRECTION_BLOCK novo.
//+------------------------------------------------------------------+
ENUM_APS CheckArmedMarketReady(double &curEff,double &trigger,double &last,MqlTick
&validatedStageTick)
{
   curEff=0.0; trigger=0.0; last=0.0;
   datetime now=TimeTradeServer();

// Timeout por tempo (novo vs. v2.44 que usava barras)
```

```
if(armed.armedAt>0&&(now-armed.armedAt)>=(datetime)Inp_ArmedTimeoutSec)
{
   PrintFormat("[SCALPER][TIMEOUT_ARMED] %s %s | Idade=%ds | CANCELADO.",
    DirStr(armed.dir),armed.route,(int)(now-armed.armedAt));
   WriteAuditRow("TIMEOUT_ARMED",DirStr(armed.dir),armed.mode,0,
    armed.posRel,armed.distD1,armed.effort,0);
   ResetArmedSignal();
   return APS_ENDED_REEVAL_ALLOWED;
}

// Fim de sessão
MqlDateTime dt; TimeToStruct(now,dt);
int hhmm=dt.hour*100+dt.min;
if(hhmm>=Inp_StopEntryHHMM)
{
   WriteAuditRow("ABORT_SESSION_END",DirStr(armed.dir),armed.mode,0,
    armed.posRel,armed.distD1,armed.effort,0);
   ResetArmedSignal();
   return APS_ENDED_FAIL_CLOSED;
}

 // Evento de negócio real
 MqlTick tk;
 if(!SymbolInfoTick(Symbol(),tk)) return APS_STILL_ARMED;
 validatedStageTick=tk;
 bool
realTradeEvent=((tk.flags&TICK_FLAG_LAST)!=0)||((tk.flags&TICK_FLAG_VOLUME)!=0)||
    ((tk.flags&TICK_FLAG_BUY)!=0)||((tk.flags&TICK_FLAG_SELL)!=0);
if(!realTradeEvent) return APS_STILL_ARMED;

last=tk.last;
if(last<=0) return APS_STILL_ARMED;
// Trigger
trigger=ComputeTrigger(armed.dir,armed.regionHigh,armed.regionLow);
bool breakout=false;
if(armed.dir==1&&last>=trigger) breakout=true;
if(armed.dir==-1&&last<=trigger) breakout=true;
if(!breakout) return APS_STILL_ARMED;

// Preço executável
double execPrice=(armed.dir==1)?tk.ask:tk.bid;
if(execPrice<=0) return APS_STILL_ARMED;

// Overshoot
double overLast=(armed.dir==1)?(last-trigger):(trigger-last);
double overExec=(armed.dir==1)?(execPrice-trigger):(trigger-execPrice);
double overshoot=MathMax(overLast,overExec);
if(overshoot>gMaxOvershoot)
```

```
{
   PrintFormat("[SCALPER][OVERSHOOT_BLOCK] Last=%.4f Exec=%.4f Trig=%.4f
Over=%.4f Max=%.4f",
    last,execPrice,trigger,overshoot,gMaxOvershoot);
   WriteAuditRow("OVERSHOOT_BLOCK",DirStr(armed.dir),armed.mode,last,
    armed.posRel,armed.distD1,armed.effort,trigger);
   ResetArmedSignal();
   return APS_ENDED_FAIL_CLOSED;
}

// Onda viva M1 (usa cache M1 — idêntico à v2.44 mas em M1)
if(!wwValid) return APS_STILL_ARMED;
double curVol,avgVol; int waveDir;
ComputeLiveWaveO1(curEff,curVol,avgVol,waveDir);
if(waveDir!=armed.dir) return APS_STILL_ARMED;
if(curEff<armed.breakoutEffortMin) return APS_STILL_ARMED;

// VWAP viva
double liveVwap=0;
if(!GetLiveVwapSnapshot((ulong)tk.time_msc,liveVwap))
{
   WriteAuditRow("LATE_VWAP_INVALID",DirStr(armed.dir),armed.mode,last,
   armed.posRel,armed.distD1,curEff,trigger);
   ResetArmedSignal();
  return APS_ENDED_FAIL_CLOSED;
}
double lateUpper=liveVwap+SigmaD1*BandMultiplier;
double lateLower=liveVwap-SigmaD1*BandMultiplier;
double lateRange=lateUpper-lateLower;
if(lateRange<=0)
{
   WriteAuditRow("LATE_VWAP_INVALID",DirStr(armed.dir),armed.mode,last,
   armed.posRel,armed.distD1,curEff,trigger);
  ResetArmedSignal();
  return APS_ENDED_FAIL_CLOSED;
}
double livePosRel=(last-lateLower)/lateRange;

// New hypothesis, OFF in baseline. Neutrality is also a mismatch.
int liveDirAbs=0;
if(livePosRel>=Inp_VWAP_Inner_Pct) liveDirAbs=-1;
else if(livePosRel<=1.0-Inp_VWAP_Inner_Pct) liveDirAbs=1;
int liveDirCont=(liveDirAbs==0)?0:-liveDirAbs;
bool lateDirectionMismatch=(armed.route=="ABS")?
   liveDirAbs!=armed.dir : liveDirCont!=armed.dir;
gLiveDirectionTelemetryValid=true;
gLiveDirAbs=liveDirAbs; gLiveDirCont=liveDirCont;
gLateDirectionMismatch=lateDirectionMismatch;
```

```
if(lateDirectionMismatch)
{
   if(Inp_EnableLateDirectionBlock)
   {
    WriteAuditRow("LATE_DIRECTION_BLOCK",DirStr(armed.dir),armed.mode,last,
     livePosRel,armed.distD1,curEff,trigger);
    ResetArmedSignal();
    return APS_ENDED_FAIL_CLOSED;
  }
  if(!gLateMismatchLogged)
  {
  gLateMismatchLogged=true;
WriteAuditRow("LATE_DIRECTION_MISMATCH_SHADOW",DirStr(armed.dir),armed.mode,
last,
      livePosRel,armed.distD1,curEff,trigger);
    }
  }

    // LATE_GLASS_BLOCK (idêntico à v2.44)
    bool
dynamicGlass=(livePosRel>Inp_VWAP_Outer_Pct)||(livePosRel<(1.0-Inp_VWAP_Outer_Pct
));
    bool isAbsRoute=(armed.route=="ABS");
    bool oppositeGlassABS=
       (armed.dir==-1&&livePosRel<(1.0-Inp_VWAP_Outer_Pct))||
       (armed.dir== 1&&livePosRel>Inp_VWAP_Outer_Pct);
   bool lateGlassBlock=isAbsRoute?oppositeGlassABS:dynamicGlass;
   if(lateGlassBlock)
   {
  PrintFormat("[SCALPER][LATE_GLASS_BLOCK] Rota=%s Dir=%s LivePosRel=%.4f",
    armed.route,DirStr(armed.dir),livePosRel);
 WriteAuditRow("LATE_GLASS_BLOCK",DirStr(armed.dir),armed.mode,last,
   livePosRel,armed.distD1,curEff,trigger);
 ResetArmedSignal();
 return APS_ENDED_FAIL_CLOSED;
}
if(isAbsRoute&&dynamicGlass&&!armed.sameSideGlassLogged)
{
    armed.sameSideGlassLogged=true;
    PrintFormat("[SCALPER][ABS_LATE_GLASS_SAMESIDE_ALLOW] %s ABS
LivePosRel=%.4f",
      DirStr(armed.dir),livePosRel);
}

    return APS_ENTRY_READY;
}

ENUM_APS CheckArmedEntryReady(double &curEff,double &trigger,double &last,MqlTick
```

```
&validatedStageTick)
{
   ENUM_APS market=CheckArmedMarketReady(curEff,trigger,last,validatedStageTick);
   if(market!=APS_ENTRY_READY) return market;
// Position, pending execution and daily quota remain operational guards.
if(HasExecutionInFlight()) { ResetArmedSignal(); return APS_TRADE_OR_POSITION; }
// Posição / cota / risco diário
if(HasOpenPosition())         { ResetArmedSignal(); return APS_TRADE_OR_POSITION; }
if(ordersEnteredToday>=Inp_MaxTradesDay) { ResetArmedSignal(); return
APS_TRADE_OR_POSITION; }
if(dailyRiskBlocked)
{
    WriteAuditRow("DAILY_RISK_BLOCK",DirStr(armed.dir),armed.mode,last,
     armed.posRel,armed.distD1,armed.effort,trigger);
    ResetArmedSignal();
    return APS_ENDED_FAIL_CLOSED;
}
// Spread
if(!SpreadOk())
{
    if(!gSpreadHoldLogged)
    {
       gSpreadHoldLogged=true;
       WriteAuditRow("HOLD_SPREAD",DirStr(armed.dir),armed.mode,last,
         armed.posRel,armed.distD1,armed.effort,trigger);
    }
    return APS_STILL_ARMED;
}

    return APS_ENTRY_READY;
}

ENUM_APS ProcessArmedSignal()
{
   double curEff,trigger,last;
   MqlTick validatedStageTick;
   ENUM_APS readiness=CheckArmedEntryReady(curEff,trigger,last,validatedStageTick);
   if(readiness!=APS_ENTRY_READY) return readiness;
if(!Inp_DOM_TradeGateEnable)
PrintFormat("[SCALPER][DISPARO_%s] %s Trigger=%.4f Last=%.4f WW_EFF=%.2f>=%.2f
Modo=%s",
armed.route,DirStr(armed.dir),trigger,last,curEff,armed.breakoutEffortMin,armed.mode);
   gLastBreakoutPrice=last; // observed real trade, never fabricated
   ENUM_SEND_OUTCOME outcome=SendOrder(armed.dir,armed.lots,armed.mode,
    armed.activeSL_Pts,armed.activeTP_Pts,
     armed.posRel,armed.distD1,curEff,trigger);
   if(outcome==SEND_FINANCIAL_UNRESOLVED)
   {
```

```
      // Ending the original signal never clears its financial obligation.
      ResetArmedSignal(); return APS_TRADE_OR_POSITION;
    }
    if(outcome==SEND_WAIT_ARMED)
    {
      // WAIT is legal only with no financial obligation. Never erase a pending
      // intent here merely to preserve ARMED. Next event uses original guards.
      if(Inp_DOM_TradeGateEnable && (HasExecutionInFlight() ||
         gEntryJournalInvalid || gExitJournalInvalid))
      { ResetArmedSignal(); return APS_TRADE_OR_POSITION; }
      return APS_STILL_ARMED;
    }
    if(outcome==SEND_ORIGINAL_CANCEL)
    { ResetArmedSignal(); return APS_ENDED_FAIL_CLOSED; }
    if(outcome==SEND_CANCEL_REEVAL) return APS_ENDED_REEVAL_ALLOWED;
    if(outcome==SEND_TRADE_OR_POSITION) return APS_TRADE_OR_POSITION;
    ResetArmedSignal();
    return
outcome==SEND_ACCEPTED?APS_TRADE_OR_POSITION:APS_ENDED_FAIL_CLOSE
D;
}

//=========================================================================
// PERSISTENCE (adaptada da v2.44, chaves prefixadas com "SCALP_RC1_")
//=========================================================================
//=========================================================================
// EOD (adaptado da v2.44)
//=========================================================================
void ProcessEOD(int hhmm)
{
  if(hhmm<Inp_EOD_CloseHHMM) return;
  if(armed.active)
  {

WriteAuditRow("ABORT_EOD",DirStr(armed.dir),armed.mode,0,armed.posRel,armed.distD1
,armed.effort,0);
       ResetArmedSignal();
     }
     if(eodDone) return;
     // A live own order can be an entry, a close, or unresolved. Never duplicate it.
     if(gPendingOrderClassificationUnresolved || gPendingExit.active || gPendingExec.active ||
        gOrderObligationScanUnresolved ||
        gRequestJournalUnresolved || gRequestJournalWritePending ||
        gEntryJournalCleanupPending || gExitJournalCleanupPending ||
gIdentityLossUnresolved
||
HasOwnPendingOrder())
     {
```

```
        // Recovery still runs in OnTimer; this diagnostic must precede the guards.
        static datetime nextBlockedDiagnostic=0;
        datetime blockedAt=TimeTradeServer();
        if(blockedAt>=nextBlockedDiagnostic && HasOwnPositionStrict())
        {
          nextBlockedDiagnostic=blockedAt+10;
          PrintFormat("[SCALPER][EOD_CRITICAL_BLOCKED] hhmm=%d entry=%d exit=%d
journal_invalid=%d journal_write=%d entry_cleanup=%d exit_cleanup=%d; reconciliation
required, no close dispatched.",
           hhmm,(int)gPendingExec.active,(int)gPendingExit.active,
         (int)gRequestJournalUnresolved,(int)gRequestJournalWritePending,
         (int)gEntryJournalCleanupPending,(int)gExitJournalCleanupPending);
     }
     return;
    }
    if(!HasOwnPositionStrict())
   { if(!HasExecutionInFlight() && !gLiveOwnershipUnresolved) eodDone=true; return; }
 if(hhmm>=1830)
 {
     if(!eodStuckLogged) { eodStuckLogged=true; Print("[SCALPER][EOD_STUCK] Own
position remains open after 18:30."); }
     return;
 }
 datetime now=TimeTradeServer();
 if(eodNextTry>0 && now<eodNextTry) return;
 eodNextTry=now+10;
 for(int i=PositionsTotal()-1;i>=0;i--)
 {
  if(!posInfo.SelectByIndex(i) || posInfo.Symbol()!=Symbol() ||
posInfo.Magic()!=(long)gMagicCore) continue;
  ulong ticket=posInfo.Ticket(), identifier=posInfo.Identifier();
  if(!OwnPositionSafe(identifier))
  { Print("[SCALPER][EOD_FOREIGN_NETTING_BLOCK] Ownership ambiguous; manual
reconciliation required."); return; }
  BeginExitIntent(identifier,"EOD");
  gPendingExit.dispatchPhase=REQUEST_PREPARED;
  trade.SetExpertMagicNumber(gMagicCore);
  gPendingExit.positionTicket=ticket;
  gPendingExit.requestSnapshot.action=TRADE_ACTION_DEAL;
  gPendingExit.requestSnapshot.position=ticket;
  gPendingExit.requestSnapshot.symbol=Symbol();
  gPendingExit.requestSnapshot.magic=(ulong)gMagicCore;
  gPendingExit.requestSnapshot.type=(posInfo.PositionType()==POSITION_TYPE_BUY?
     ORDER_TYPE_SELL:ORDER_TYPE_BUY);
  gPendingExit.requestSnapshot.volume=posInfo.Volume();
  gPendingExit.requestSnapshot.price=SymbolInfoDouble(Symbol(),

gPendingExit.requestSnapshot.type==ORDER_TYPE_BUY?SYMBOL_ASK:SYMBOL_BID);
```

```
 gPendingExit.requestSnapshot.deviation=(ulong)gMaxDeviationPts;
 gPendingExit.requestSnapshot.type_filling=ORDER_FILLING_FOK;
 if(!PersistRequestJournal(true))
 {
   PrintFormat("[SCALPER][EOD_PERSISTENCE_CRITICAL] pid=%I64u PREPARED write
failed; no close dispatched.",identifier);
   FailExitIntent(identifier,true); return;
 }
 if(!MarkRequestDispatchPossible(true))
 {
    PrintFormat("[SCALPER][EOD_PERSISTENCE_CRITICAL] pid=%I64u dispatch barrier
failed; no close dispatched.",identifier);
    FailExitIntent(identifier,true); return;
  }
  trade.PositionClose(ticket);
   MqlTradeResult closeResult;
   trade.Result(closeResult);
   trade.Request(gPendingExit.requestSnapshot);
   uint ret=closeResult.retcode;
   CompletePendingExit(closeResult);
   PrintFormat("[SCALPER][EOD_REQUEST] ticket=%I64u ret=%u; closure requires
confirmed deals.",ticket,ret);
   break;
   }
}

//+------------------------------------------------------------------+
// DAILY RESET (adaptado da v2.44)
//+------------------------------------------------------------------+
void DailyReset()
{
  UpperBand1=0; LowerBand1=0;
  ResetArmedSignal(); ResetVwapCache(); ResetMacroContext(gMacro);
  wwValid=false; wwCurrentWaveBars=0; wwWaveStartPrice=0;
  VWAP_D1=0; H1=0; L1=0; VWAP_D1_Official=false; SigmaD1=0;
  d1Ready=false; nextD1Retry=0;
  eodDone=false; eodStuckLogged=false; eodNextTry=0;
  gLastMatrixEvalBar=0; gCachedMicroReady=false; gCachedMicroBar=0;
  gLastTelemetryMatrixValid=false; gLastTelemetrySignalBar=0;
  gLiveDirectionTelemetryValid=false;
  hardFlipsPerDay=0; armedCancelHard=0; oppositeRearmAfterHard=0;
  gLastBreakoutPrice=0; gLateMismatchLogged=false; gSpreadHoldLogged=false;
  ResetPerfStat(perfMacro); ResetPerfStat(perfMicro); ResetPerfStat(perfStage2);
  gLastPerfLoggedMicroCount=0;
  ResetAccountingDay(); // preserve open lifecycle and pending requests across midnight
  TryCalcD1();
  BootstrapCachesNoTrade();
}
```

```
//+------------------------------------------------------------------+
// ON INIT
//+------------------------------------------------------------------+
bool Positive(double value)
{ return MathIsValidNumber(value) && value>0; }
bool OnPriceGrid(double value)
{ return Positive(value) && MathAbs(value/gTickSize-MathRound(value/gTickSize))<=1e-6; }
bool ValidHHMM(int hhmm)
{ return hhmm>=0 && hhmm/100<=23 && hhmm%100<=59; }
bool OnVolumeGrid(double value, double step)
{ return Positive(value) && MathAbs(value/step-MathRound(value/step))<=1e-6; }
bool ValidateRC1Inputs()
{
   int ctxSec=PeriodSeconds(Inp_ContextTF), sigSec=PeriodSeconds(Inp_SignalTF);
   if(ctxSec<=0 || sigSec<=0 || ctxSec<=sigSec || ctxSec%sigSec!=0) return false;
   if(!ValidHHMM(Inp_StartHHMM) || !ValidHHMM(Inp_StopEntryHHMM) ||
      !ValidHHMM(Inp_EOD_CloseHHMM) || Inp_StartHHMM>=Inp_StopEntryHHMM ||
      Inp_StopEntryHHMM>=Inp_EOD_CloseHHMM) return false;
   if(Inp_POC_WindowSec<=0 || Inp_ArmedTimeoutSec<=0 || Inp_Region_Candles<1 ||
      Inp_MaxTradesDay<1 || Inp_MaxTradesDay>MAX_TRADE_RECORDS) return false;
   if(!Positive(Inp_ABS_Effort) || !Positive(Inp_CONT_Effort) ||
      !Positive(Inp_ABS_BreakoutEffort) || !Positive(Inp_CONT_BreakoutEffort)) return false;
   if(!Positive(gSL_Pts) || !Positive(gTP_Pts) || !Positive(gSpreadMaxPts) ||
      !Positive(gMaxOvershoot) || !Positive(gABS_MinDisplPts) ||
      !Positive(gABS_MaxBodyPts) || !Positive(gCONT_MinDisplPts)) return false;
   if(!OnPriceGrid(gSL_Pts) || !OnPriceGrid(gTP_Pts) || !OnPriceGrid(gMaxOvershoot) ||
      !OnPriceGrid(gBreakoutMargin) || gBreakoutMargin<gTickSize-1e-9) return false;
   // Analytical thresholds are deliberately not required to be on the tick grid.
   if(!Positive(BandMultiplier) || !MathIsValidNumber(Inp_VWAP_Inner_Pct) ||
      Inp_VWAP_Inner_Pct<=0.5 || Inp_VWAP_Inner_Pct>=1 ||
      !Positive(Inp_VWAP_Outer_Pct) || Inp_VWAP_Outer_Pct<=1 ||
      !MathIsValidNumber(Inp_Rule_40_60) || Inp_Rule_40_60<=0 || Inp_Rule_40_60>=0.5 ||
      !Positive(gC3Min) || !Positive(gC3Max) || !Positive(gC2) ||
      gC3Min>gC3Max || gC3Max>=gC2 || !Positive(Inp_POC_FollowRatio)) return false;
   if(SigmaLookbackDays<1 || SigmaLookbackDays>20 || !Positive(Inp_SigmaFloor_WIN) ||
      !Positive(Inp_SigmaFloor_WDO) || Inp_CooldownSec<0 || Inp_LossCooldownSec<0)
return false;
   if(gMagicCore<=0 || gMaxDeviationPts<0) return false;
   if(Inp_SizingMode!=SIZING_FIXED && Inp_SizingMode!=SIZING_RISK_PERCENT)
return false;
   double minimum=SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_MIN);
   double maximum=SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_MAX);
   double step=SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_STEP);
   if(!Positive(minimum) || !Positive(maximum) || maximum<minimum || !Positive(step)) return
false;
  if(!OnVolumeGrid(Inp_MaxLots,step) || Inp_MaxLots<minimum || Inp_MaxLots>maximum)
return false;
```

```
  if(Inp_SizingMode==SIZING_FIXED && (!OnVolumeGrid(Inp_FixedLots,step) ||
     Inp_FixedLots<minimum || Inp_FixedLots>maximum || Inp_FixedLots>Inp_MaxLots))
return false;
  if(Inp_SizingMode==SIZING_RISK_PERCENT && !Positive(Inp_RiskPerTradePct)) return
false;
  if(Inp_EnableDailyRiskStop && !Positive(Inp_MaxDailyLossR)) return false;
  if(Inp_EnableConsecutiveLossStop && Inp_MaxConsecutiveLosses<1) return false;
  if(!Inp_EnableAbsorption && !Inp_EnableContinuation) return false;
  return true;
}
void BootstrapCachesNoTrade()
{
  datetime contextBar=iTime(Symbol(),Inp_ContextTF,0);
  datetime signalBar=iTime(Symbol(),Inp_SignalTF,0);
   // Set clocks before any fallible data work. Recovery cannot synthesize a close.
   if(contextBar>0) lastContextBarTime=contextBar;
    if(signalBar>0) lastSignalBarTime=signalBar;
    gCachedMicroReady=false; gCachedMicroBar=0;
    gLastTelemetryMatrixValid=false; gLastTelemetrySignalBar=0;
    gLiveDirectionTelemetryValid=false;
    ResetArmedSignal();
    if(contextBar>0)
    {
     gVwapOk=UpdateVwapIncremental(contextBar);
     gVwapValue=(gVwapOk && vwapSumVol>0)?vwapSumPV/vwapSumVol:0;
     if(d1Ready && gVwapOk)
     {
        ulong begin=GetMicrosecondCount();
       ComputeMacroContext();
       RecordPerf(perfMacro,(double)(GetMicrosecondCount()-begin));
     }
    }
    if(signalBar>0)
    {
       gVwapOk=UpdateVwapIncremental(signalBar);
       gVwapValue=(gVwapOk && vwapSumVol>0)?vwapSumPV/vwapSumVol:0;
       RebuildWeisCache();
    }
    // Never EvaluateMatrix here, even if every cache is ready.
}

int OnInit()
{
  string sym=Symbol();
  gIsDollarFamily=(StringFind(sym,"WDO")>=0 || StringFind(sym,"DOL")>=0);
  gIsIndexFamily=!gIsDollarFamily && (StringFind(sym,"WIN")>=0 ||
StringFind(sym,"IND")>=0);
  if(!gIsDollarFamily && !gIsIndexFamily) return INIT_PARAMETERS_INCORRECT;
```

```
  gMagicCore=gIsIndexFamily?Inp_Magic_WIN:Inp_Magic_WDO;
  gC2=gIsIndexFamily?Inp_C2_WIN:Inp_C2_WDO;
  gC3Min=gIsIndexFamily?Inp_C3_Min_WIN:Inp_C3_Min_WDO;
  gC3Max=gIsIndexFamily?Inp_C3_Max_WIN:Inp_C3_Max_WDO;

gMaxDeviationPts=gIsIndexFamily?Inp_MaxDeviationPts_WIN:Inp_MaxDeviationPts_WDO;
gSpreadMaxPts=gIsIndexFamily?Inp_SpreadMaxPts_WIN:Inp_SpreadMaxPts_WDO;
gBreakoutMargin=gIsIndexFamily?Inp_BreakoutMargin_WIN:Inp_BreakoutMargin_WDO;
gMaxOvershoot=gIsIndexFamily?Inp_MaxOvershoot_WIN:Inp_MaxOvershoot_WDO;
gSL_Pts=gIsIndexFamily?Inp_SL_Pts_WIN:Inp_SL_Pts_WDO;
gTP_Pts=gIsIndexFamily?Inp_TP_Pts_WIN:Inp_TP_Pts_WDO;

gABS_MinDisplPts=gIsIndexFamily?Inp_ABS_MinPriceMovePts_WIN:Inp_ABS_MinPriceM
ovePts_WDO;
gABS_MaxBodyPts=gIsIndexFamily?Inp_ABS_MaxBodyPts_WIN:Inp_ABS_MaxBodyPts_
WDO;
gCONT_MinDisplPts=gIsIndexFamily?Inp_CONT_MinPriceMovePts_WIN:Inp_CONT_MinPr
iceMovePts_WDO;
  gTickSize=SymbolInfoDouble(Symbol(),SYMBOL_TRADE_TICK_SIZE);
  if(gTickSize<=0) gTickSize=SymbolInfoDouble(Symbol(),SYMBOL_POINT);
  if(!Positive(gTickSize) || !ValidateRC1Inputs())
  { Print("[SCALPER][FATAL] Invalid RC1 parameters; no silent correction."); return
INIT_PARAMETERS_INCORRECT; }
  if(!DOMGateValidateInputs()) return INIT_PARAMETERS_INCORRECT;
  if(!IsFokSupported())
  { Print("[SCALPER][FATAL] FOK unsupported."); return
INIT_PARAMETERS_INCORRECT; }
  trade.SetExpertMagicNumber(gMagicCore);
  trade.SetDeviationInPoints(gMaxDeviationPts);
  trade.SetTypeFilling(ORDER_FILLING_FOK);
  trade.SetAsyncMode(false);
  ENUM_ACCOUNT_MARGIN_MODE
mm=(ENUM_ACCOUNT_MARGIN_MODE)AccountInfoInteger(ACCOUNT_MARGIN_MOD
E);
  gNetting=mm!=ACCOUNT_MARGIN_MODE_RETAIL_HEDGING;
  if(gNetting)
     Print("[SCALPER][SCALPER_NETTING_SHARED_SYMBOL_RISK] Remora 1.0,
Remora 2.44 and Scalper do not have independent positions on the same symbol/account
merely through Magic numbers.");
  MqlDateTime now; TimeToStruct(TimeTradeServer(),now);
  gDayKey=now.year*1000+now.day_of_year;
  ResetMacroContext(gMacro); ResetArmedSignal(); ResetVwapCache(); ResetOpenPos();
  InitializeAccounting();
  wwValid=false; wwCurrentWaveBars=0; wwWaveStartPrice=0;
 lastContextBarTime=0; lastSignalBarTime=0; gLastMatrixEvalBar=0;
 gCachedMicroReady=false; gCachedMicroBar=0;
 gLastTelemetryMatrixValid=false; gLastTelemetrySignalBar=0;
 gLiveDirectionTelemetryValid=false;
```

```
 gLastBreakoutPrice=0; gLateMismatchLogged=false; gSpreadHoldLogged=false;
 hardFlipsPerDay=0; armedCancelHard=0; oppositeRearmAfterHard=0;
 d1Ready=false; nextD1Retry=0;
 eodDone=false; eodStuckLogged=false; eodNextTry=0;
 ResetPerfStat(perfMacro); ResetPerfStat(perfMicro); ResetPerfStat(perfStage2);
 gLastPerfLoggedMicroCount=0; gLastFlushTime=0;
 RestoreDailyPersistence();
 TryCalcD1();
 BootstrapCachesNoTrade();
// A configurable label cannot remove the explicit schema namespace.
string prefix="Scalper01_RC1_";
if(CSVFileName!="Scalper01_RC1" && CSVFileName!="") prefix+=CSVFileName+"_";
string suffix=Symbol()+"_"+IntegerToString(gMagicCore);
gTradeCsvFileName=prefix+suffix+"_Trades.csv";
csvAuditHandle=FileOpen(prefix+suffix+"_Audit.csv",FILE_READ|FILE_WRITE|FILE_TXT|FI
LE_ANSI);
 if(csvAuditHandle!=INVALID_HANDLE)
 { if(FileSize(csvAuditHandle)==0) WriteAuditHeader();
FileSeek(csvAuditHandle,0,SEEK_END); FileFlush(csvAuditHandle); }

csvShadowHandle=FileOpen(prefix+suffix+"_Shadow.csv",FILE_READ|FILE_WRITE|FILE_
TXT|FILE_ANSI);
 if(csvShadowHandle!=INVALID_HANDLE)
{ if(FileSize(csvShadowHandle)==0) WriteShadowHeader();
FileSeek(csvShadowHandle,0,SEEK_END); FileFlush(csvShadowHandle); }

csvTradeHandle=FileOpen(prefix+suffix+"_Trades.csv",FILE_READ|FILE_WRITE|FILE_TXT
|FILE_ANSI|FILE_SHARE_READ|FILE_SHARE_WRITE);
  if(csvTradeHandle!=INVALID_HANDLE)
  {
     if(FileSize(csvTradeHandle)==0) WriteTradeHeader();
     FileFlush(csvTradeHandle);
     if(!InitializeTradeCsvIndex(prefix+suffix+"_Trades.csv"))
        Print("[SCALPER][TRADES_CSV_INDEX_UNAVAILABLE] Finalization records require
CSV reconciliation.");
     FileSeek(csvTradeHandle,0,SEEK_END);
  }
  int timerSec=(MQLInfoInteger(MQL_TESTER)!=0)?60:1;
  if(!EventSetTimer(timerSec)) return INIT_FAILED;
  PrintFormat("[SCALPER RC1.2] %s Magic=%d Context=%s Signal=%s MacroGate=%s
LateDirection=%s RejectTies=%s; edge empirical not demonstrated.",
     sym,gMagicCore,EnumToString(Inp_ContextTF),EnumToString(Inp_SignalTF),
BStr(Inp_EnableMacroDirectionGate),BStr(Inp_EnableLateDirectionBlock),BStr(Inp_RejectP
OCTies));
  DOMRuntimeInit();
  DOMGateInit();
  return INIT_SUCCEEDED;
}
```

```
//+------------------------------------------------------------------+
// ON DEINIT
//+------------------------------------------------------------------+
void OnDeinit(const int reason)
{
   EventKillTimer();
   if(csvAuditHandle !=INVALID_HANDLE){ FileFlush(csvAuditHandle);
FileClose(csvAuditHandle); csvAuditHandle =INVALID_HANDLE; }
   if(csvShadowHandle!=INVALID_HANDLE){
FileFlush(csvShadowHandle);FileClose(csvShadowHandle);csvShadowHandle=INVALID_H
ANDLE; }
  if(csvTradeHandle !=INVALID_HANDLE){ FileFlush(csvTradeHandle);
FileClose(csvTradeHandle); csvTradeHandle =INVALID_HANDLE; }
   DOMRuntimeDeinit();
   DOMGateDeinit();
}

//+------------------------------------------------------------------+
// ON TICK
//+------------------------------------------------------------------+
// Accounting/execution uncertainty still blocks entries. This observer has no
// access to the arming path and does not populate its shared snapshot cache.
void ObserveAccountingBlockedMatrix(datetime signalBar)
{
  if(!gAccountingUnresolved || armed.active || signalBar<=0 ||
     gLastReadOnlyMatrixBar==signalBar) return;
  if(HasOpenPosition() || ordersEnteredToday>=Inp_MaxTradesDay || dailyRiskBlocked)
return;
  MqlDateTime observationTime; TimeToStruct(TimeTradeServer(),observationTime);
  int observationHHMM=observationTime.hour*100+observationTime.min;
  if(observationHHMM<Inp_StartHHMM || observationHHMM>=Inp_StopEntryHHMM) return;
  if(Inp_CooldownSec>0 && lastCloseConfirmedTime>0 &&
     TimeTradeServer()-lastCloseConfirmedTime<Inp_CooldownSec) return;
  if(Inp_LossCooldownSec>0 && lastTradeWasLoss && lastCloseConfirmedTime>0 &&
     TimeTradeServer()-lastCloseConfirmedTime<Inp_LossCooldownSec) return;
  if(!d1Ready || !gVwapOk || !wwValid) return;
  if(Inp_EnableMacroDirectionGate && gMacro.status!=MACRO_OK) return;
  if(iTime(Symbol(),Inp_SignalTF,0)!=signalBar) return;
  gLastReadOnlyMatrixBar=signalBar; // One bounded attempt, including invalid snapshots.
  MatrixSnapshot observation;
  ComputeMatrixSnapshot(observation);
  if(iTime(Symbol(),Inp_SignalTF,0)!=signalBar)
  {
    Print("[SCALPER][MATRIX_READ_ONLY_DISCARDED] Signal bar changed; entries
remain blocked.");
    return;
  }
```

```
  WriteShadowRow("MATRIX_READ_ONLY_ACCOUNTING_BLOCK",observation);
  PrintFormat("[SCALPER][MATRIX_READ_ONLY_ACCOUNTING_BLOCK] Status=%s
ExecutionInFlight=%s; observation only, entries remain blocked.",
       MssStr(observation.status),BStr(HasExecutionInFlight()));
}

void OnTick()
{
  if(TimeCurrent()<BacktestStart) return;
  MqlDateTime day; TimeToStruct(TimeTradeServer(),day);
  int key=day.year*1000+day.day_of_year;
  if(key!=gDayKey) { gDayKey=key; DailyReset(); }
 SyncAccountingState();
 if(gOpenPos.active) UpdateMAEMFE();
 int hhmm=day.hour*100+day.min;
 ProcessEOD(hhmm);
 ManageBreakEven();
 if(hhmm>=Inp_StopEntryHHMM && armed.active) ResetArmedSignal();
 datetime btCtx=iTime(Symbol(),Inp_ContextTF,0);
 datetime btSig=iTime(Symbol(),Inp_SignalTF,0);
 if(btCtx<=0 || btSig<=0) return;
 if(lastContextBarTime==0 || lastSignalBarTime==0)
 {
  // Late availability of bar clocks initializes state, never synthesizes a signal.
  BootstrapCachesNoTrade();
  return;
 }
 bool newCtx=btCtx!=lastContextBarTime;
 bool newSig=btSig!=lastSignalBarTime;
 if(newCtx)
 {
 lastContextBarTime=btCtx;
 double savedValue=gVwapValue; bool savedOk=gVwapOk;
 gVwapOk=UpdateVwapIncremental(btCtx);
 gVwapValue=(gVwapOk && vwapSumVol>0)?vwapSumPV/vwapSumVol:0;
 ulong begin=GetMicrosecondCount();
 ComputeMacroContext();
 RecordPerf(perfMacro,(double)(GetMicrosecondCount()-begin));
 gVwapValue=savedValue; gVwapOk=savedOk;
}
if(newSig)
{
   lastSignalBarTime=btSig;
   gCachedMicroReady=false;
   gVwapOk=UpdateVwapIncremental(btSig);
   gVwapValue=(gVwapOk && vwapSumVol>0)?vwapSumPV/vwapSumVol:0;
   if(gVwapValue<=0) gVwapOk=false;
   RebuildWeisCache();
```

```
}
bool hardCancelled=false;
int previousDir=0;
if(armed.active)
{
  if(newSig && d1Ready && gVwapOk && wwValid &&
    (!Inp_EnableMacroDirectionGate || gMacro.status==MACRO_OK))
  {
    MatrixSnapshot snapshot;
    SnapshotForSignal(snapshot);
    DOMGateObserveMatrix(snapshot); // Reuses the original snapshot; observation only.
    if(HasHardDirectionalContradiction(snapshot,armed.dir))
    {
      previousDir=armed.dir;
      hardFlipsPerDay++; armedCancelHard++;
      WriteAuditRow("ARMED_CANCEL_HARD",DirStr(armed.dir),armed.mode,
      snapshot.price,armed.posRel,armed.distD1,armed.effort,0);
   WriteShadowRow("ARMED_CANCEL_HARD",snapshot);
   ResetArmedSignal(); hardCancelled=true;
   }
  }
  if(!hardCancelled)
  {
  ulong begin=GetMicrosecondCount();
  ENUM_APS outcome=ProcessArmedSignal();
  RecordPerf(perfStage2,(double)(GetMicrosecondCount()-begin));
  if(outcome!=APS_ENDED_REEVAL_ALLOWED || !newSig) return;
  }
 }
 if(!newSig || hhmm<Inp_StartHHMM || hhmm>=Inp_StopEntryHHMM) return;
 if(HasOpenPosition() || HasExecutionInFlight() ||
ordersEnteredToday>=Inp_MaxTradesDay ||
     dailyRiskBlocked || gAccountingUnresolved)
 {
  if(gAccountingUnresolved) ObserveAccountingBlockedMatrix(btSig);
  return;
 }
  if(Inp_CooldownSec>0 && lastCloseConfirmedTime>0 &&
    TimeTradeServer()-lastCloseConfirmedTime<Inp_CooldownSec) return;
  if(Inp_LossCooldownSec>0 && lastTradeWasLoss && lastCloseConfirmedTime>0 &&
    TimeTradeServer()-lastCloseConfirmedTime<Inp_LossCooldownSec) return;
  if(!d1Ready || !gVwapOk || !wwValid) return;
  if(Inp_EnableMacroDirectionGate && gMacro.status!=MACRO_OK) return;
  if(gLastMatrixEvalBar==btSig) return;
  gLastMatrixEvalBar=btSig;
  EvaluateMatrix(); // reuses exact hard-contradiction snapshot if already computed
  if(hardCancelled && armed.active && armed.dir==-previousDir)
  {
```

```
      oppositeRearmAfterHard++;
      WriteShadowRow("OPPOSITE_REARM_AFTER_HARD",gCachedMicroSnapshot);
   }
}

//+------------------------------------------------------------------+
// ON TIMER
//+------------------------------------------------------------------+
void DOMBaseOnTimer()
{
  if(TimeCurrent()<BacktestStart) return;
  MqlDateTime nowDT; TimeToStruct(TimeTradeServer(),nowDT);
  int key=nowDT.year*1000+nowDT.day_of_year;
 int hhmm=nowDT.hour*100+nowDT.min;
 if(key!=gDayKey) { gDayKey=key; DailyReset(); }
 SyncAccountingState(true);
 ReconcileBreakEvenProofs(); // Proof/persistence only; never dispatches a stop
modification.
 if(!d1Ready && TimeTradeServer()>=nextD1Retry) TryCalcD1();
 if(hhmm>=Inp_StopEntryHHMM && armed.active) ResetArmedSignal();
 ProcessEOD(hhmm);
 datetime now=TimeTradeServer();
 if(now-gLastFlushTime>=30)
{
   gLastFlushTime=now;
   if(csvAuditHandle!=INVALID_HANDLE) FileFlush(csvAuditHandle);
   if(csvShadowHandle!=INVALID_HANDLE) FileFlush(csvShadowHandle);
   if(csvTradeHandle!=INVALID_HANDLE) FileFlush(csvTradeHandle);
 }
 if(MQLInfoInteger(MQL_TESTER)==0 &&
perfMicro.count>=gLastPerfLoggedMicroCount+100)
 {
 PrintFormat("[SCALPER][PERF] Macro(us) count=%I64d mean=%.1f max=%.1f; Micro(us)
count=%I64d mean=%.1f max=%.1f; Stage2(us) count=%I64d mean=%.1f max=%.1f",

perfMacro.count,perfMacro.count>0?perfMacro.sumUs/perfMacro.count:0,perfMacro.maxUs
,

perfMicro.count,perfMicro.count>0?perfMicro.sumUs/perfMicro.count:0,perfMicro.maxUs,

perfStage2.count,perfStage2.count>0?perfStage2.sumUs/perfStage2.count:0,perfStage2.m
axUs);
   gLastPerfLoggedMicroCount=perfMicro.count;
  }
}

//+------------------------------------------------------------------+
// ON TRADE TRANSACTION
```

```
// Trata DEAL_ENTRY_IN (entrada) e DEAL_ENTRY_OUT (saída).
// Agrupa por DEAL_POSITION_ID.
// Exige flat, salvo ciclo próprio fechado com residual INOUT explicitamente estrangeiro.
//==========================================================================


//==========================================================================
// ON TESTER
//==========================================================================
//==========================================================================
// RC1 execution context, confirmed quota and positionID accounting
//==========================================================================
// BE-only lifecycle. Initial TradeRecord.slPrice/risk and all entry rules stay unchanged.
double BEPrice(double entry)
{ return NormPriceDigits(MathRound(entry/gTickSize)*gTickSize); }
bool BEPriceEqual(double a,double b)
{ return MathIsValidNumber(a) && MathIsValidNumber(b) &&
MathAbs(a-b)<=gTickSize*0.000001; }

double BERiskPoints(const TradeRecord &r)
{ return MathAbs(r.weightedEntryPrice-r.slPrice); }

bool BEReferenceValid(const TradeRecord &r)
{
   return r.initialStopsProven && r.riskValid && r.accountingResolved &&
    Positive(r.weightedEntryPrice) && Positive(r.slPrice) &&
    (r.direction==1 || r.direction==-1) &&
    r.direction*(r.weightedEntryPrice-r.slPrice)>0 &&
     r.be.direction==r.direction && BEPriceEqual(r.be.entryPrice,r.weightedEntryPrice) &&
     BEPriceEqual(r.be.initialSL,r.slPrice) && Positive(r.be.price) &&
     BEPriceEqual(r.be.price,BEPrice(r.be.entryPrice));
}

bool BETriggerReached(const TradeRecord &r,const MqlTick &tick)
{
  double trigger=r.weightedEntryPrice+r.direction*BERiskPoints(r);
  return r.direction==1 ? tick.bid>=trigger : tick.ask<=trigger;
}

void BELog(const TradeRecord &r,string event,string detail="")
{
  PrintFormat("[SCALPER][%s] POSITION_ID=%I64u ticket=%I64u dir=%s entry=%.16f
initialSL=%.16f R_points=%.16f trigger=%.16f newSL=%.16f TP_preserved=%.16f %s",
  event,r.positionID,r.be.ticket,DirStr(r.be.direction),r.be.entryPrice,r.be.initialSL,
  MathAbs(r.be.entryPrice-r.be.initialSL),
  r.be.entryPrice+r.be.direction*MathAbs(r.be.entryPrice-r.be.initialSL),
  r.be.price,r.be.tp,detail);
}
```

```
void BEFailure(TradeRecord &r,string detail)
{
  datetime now=TimeTradeServer();
  if(r.be.lastFailureLog==0 || now-r.be.lastFailureLog>=10)
  { BELog(r,"BE_MODIFY_FAIL",detail); r.be.lastFailureLog=now; }
}

string BEJournalPath(ulong pid)
{ return TradeMetaPath(pid)+".be.journal"; }

uint BEChecksum(string text)
{
  uint value=2166136261;
  for(int i=0;i<StringLen(text);i++)
    value=(value^(uint)StringGetCharacter(text,i))*16777619;
   return value;
}

string BEJournalRow(ulong pid,const BreakEvenState &b)
{
  string row="SCALPER_BE1|"+(string)AccountInfoInteger(ACCOUNT_LOGIN)+"|"+
   RJEncode(Symbol())+"|"+(string)gMagicCore+"|"+(string)pid+"|"+
    (string)b.stage+"|"+(string)b.ticket+"|"+(b.direction==1?"BUY":"SELL")+"|"+
    DoubleToString(b.entryPrice,16)+"|"+DoubleToString(b.initialSL,16)+"|"+
    DoubleToString(b.price,16)+"|"+DoubleToString(b.tp,16)+"|"+
    (string)b.triggerMSC+"|"+(string)b.confirmedMSC+"|"+
    (string)b.requestID+"|"+(string)b.retcode;
     return row+"|"+(string)BEChecksum(row)+"|COMPLETE_BE1";
}

bool BEParseRow(string row,ulong pid,BreakEvenState &b)
{
   string f[]; ulong
account,magic,parsedPID,stage,ticket,trigger,confirmed,requestID,retcode,checksum;
   string symbol;
   if(StringSplit(row,'|',f)!=18 || f[0]!="SCALPER_BE1" || f[17]!="COMPLETE_BE1" ||
      !RJUnsigned(f[1],account) || account!=(ulong)AccountInfoInteger(ACCOUNT_LOGIN) ||
      !RJDecode(f[2],symbol) || symbol!=Symbol() ||
      !RJUnsigned(f[3],magic) || magic!=(ulong)gMagicCore ||
      !RJUnsigned(f[4],parsedPID) || parsedPID!=pid || pid==0 ||
      !RJUnsigned(f[5],stage) || stage<BE_TRIGGERED || stage>BE_CONFIRMED ||
      !RJUnsigned(f[6],ticket) || ticket==0 || (f[7]!="BUY" && f[7]!="SELL") ||
      !RJUnsigned(f[12],trigger) || trigger==0 || trigger>9223372036854775807 ||
      !RJUnsigned(f[13],confirmed) || confirmed>9223372036854775807 ||
      !RJUnsigned(f[14],requestID) || requestID>4294967295 ||
      !RJUnsigned(f[15],retcode) || retcode>4294967295 ||
      !RJUnsigned(f[16],checksum) || checksum>4294967295) return false;
```

```
   string prefix=f[0];
   for(int i=1;i<16;i++) prefix+="|"+f[i];
   if(BEChecksum(prefix)!=(uint)checksum) return false;
   ZeroMemory(b);
   if(!RJDouble(f[8],b.entryPrice) || !RJDouble(f[9],b.initialSL) ||
      !RJDouble(f[10],b.price) || !RJDouble(f[11],b.tp) ||
        !Positive(b.entryPrice) || !Positive(b.initialSL) || !Positive(b.price)) return false;
     b.stage=(int)stage; b.ticket=ticket; b.direction=(f[7]=="BUY"?1:-1);
     b.triggerMSC=(long)trigger; b.confirmedMSC=(long)confirmed;
     b.requestID=(uint)requestID; b.retcode=(uint)retcode;
     if(b.direction*(b.entryPrice-b.initialSL)<=0 || !BEPriceEqual(b.price,BEPrice(b.entryPrice))
||
        (b.stage>=BE_ACKNOWLEDGED && (b.retcode!=TRADE_RETCODE_DONE ||
         b.confirmedMSC<b.triggerMSC))) return false;
     b.durable=true;
    return true;
}

bool BEReadJournal(ulong pid,BreakEvenState &b,bool &exists)
{
  exists=FileIsExist(BEJournalPath(pid));
  if(!exists) return false; // Legacy metadata remains valid; BE defaults to NONE.
  int
h=FileOpen(BEJournalPath(pid),FILE_READ|FILE_TXT|FILE_ANSI|FILE_SHARE_READ);
  if(h==INVALID_HANDLE) return false;
  bool found=false,badTail=false;
  while(!FileIsEnding(h))
  {
     string row=FileReadString(h);
     if(row=="") continue;
     BreakEvenState candidate;
     if(BEParseRow(row,pid,candidate)) { b=candidate; found=true; badTail=false; }
     else badTail=true;
    }
    FileClose(h);
    // An immutable confirmed certificate survives an unrelated torn tail.
    // An ambiguous pending tail must never authorize another modification.
    return found && (!badTail || b.stage==BE_CONFIRMED);
}

bool BEPersist(TradeRecord &r)
{
  string row=BEJournalRow(r.positionID,r.be),payload="\r\n"+row+"\r\n";
  int h=FileOpen(BEJournalPath(r.positionID),
     FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_SHARE_READ);
  bool ok=false;
  if(h!=INVALID_HANDLE)
  {
```

```
     if(FileSeek(h,0,SEEK_END))
     {
        ResetLastError(); uint written=FileWriteString(h,payload);
        ok=(written==(uint)StringLen(payload) && GetLastError()==0);
        ResetLastError(); FileFlush(h); if(GetLastError()!=0) ok=false;
     }
     FileClose(h);
    }
    BreakEvenState verified; bool exists=false;
    if(ok) ok=BEReadJournal(r.positionID,verified,exists) &&
      BEJournalRow(r.positionID,verified)==row;
    r.be.durable=ok;
    if(!ok) BEFailure(r,"BE_PERSISTENCE_UNCONFIRMED; no durable activation claimed");
    return ok;
}
void BELoad(TradeRecord &r)
{
  BreakEvenState loaded; bool exists=false;
  if(BEReadJournal(r.positionID,loaded,exists)) r.be=loaded;
  else if(exists)
  {
     r.be.stage=BE_UNRESOLVED;
     BELog(r,"BE_PROOF_UNAVAILABLE","Unreadable/corrupt journal; no inferred BE");
    }
}

bool BESelectOwn(ulong pid,ulong &ticket,int direction)
{
  for(int i=PositionsTotal()-1;i>=0;i--)
  {
    ulong candidate=PositionGetTicket(i);
    if(candidate==0 || !PositionSelectByTicket(candidate) ||
       PositionGetString(POSITION_SYMBOL)!=Symbol() ||
       PositionGetInteger(POSITION_MAGIC)!=(long)gMagicCore ||
       (ulong)PositionGetInteger(POSITION_IDENTIFIER)!=pid ||
       (PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY?1:-1)!=direction)
continue;
    ticket=candidate; return true;
  }
  return false;
}

bool BEEntryComplete(const TradeRecord &r)
{
  if(r.entryOrderTicket==0 || OrderSelect(r.entryOrderTicket) ||
     !HistoryOrderSelect(r.entryOrderTicket) ||
     HistoryOrderGetString(r.entryOrderTicket,ORDER_SYMBOL)!=Symbol() ||
     HistoryOrderGetInteger(r.entryOrderTicket,ORDER_MAGIC)!=(long)gMagicCore ||
```

```
    (ulong)HistoryOrderGetInteger(r.entryOrderTicket,ORDER_POSITION_ID)!=r.positionID)
    return false;
  ENUM_ORDER_STATE
state=(ENUM_ORDER_STATE)HistoryOrderGetInteger(r.entryOrderTicket,ORDER_STATE);
  if(state!=ORDER_STATE_FILLED && state!=ORDER_STATE_CANCELED &&
    state!=ORDER_STATE_REJECTED && state!=ORDER_STATE_EXPIRED) return false;
   double filled=HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_INITIAL)-
     HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_CURRENT);
   return filled>0 && MathAbs(filled-r.totalVolumeIn)<=AccountingVolumeTolerance();
}

bool BEStopsValid(const TradeRecord &r,const MqlTick &tick,double liveSL,double liveTP)
{
  double size=SymbolInfoDouble(Symbol(),SYMBOL_TRADE_TICK_SIZE);
  double point=SymbolInfoDouble(Symbol(),SYMBOL_POINT);
  if(!Positive(size) || !Positive(point) || MathAbs(size-gTickSize)>size*0.000001 ||
    !Positive(tick.bid) || !Positive(tick.ask) || tick.ask<tick.bid ||
    !OnPriceGrid(r.be.price) || !Positive(liveSL) ||
    r.direction*(r.be.price-liveSL)<=size*0.000001) return false;
  long stops=SymbolInfoInteger(Symbol(),SYMBOL_TRADE_STOPS_LEVEL);
  long freeze=SymbolInfoInteger(Symbol(),SYMBOL_TRADE_FREEZE_LEVEL);
  if(stops<0 || freeze<0) return false;
  double quote=(r.direction==1?tick.bid:tick.ask);
   double stopDistance=r.direction*(quote-r.be.price);
   double oldDistance=r.direction*(quote-liveSL);
   if(stopDistance<=0 || stopDistance+size*0.000001<stops*point ||
     (freeze>0 && (stopDistance<=freeze*point || oldDistance<=freeze*point))) return false;
   if(liveTP!=0)
    {
      double tpDistance=r.direction*(liveTP-quote);
      if(!OnPriceGrid(liveTP) || tpDistance<=0 || tpDistance+size*0.000001<stops*point ||
        (freeze>0 && tpDistance<=freeze*point)) return false;
    }
    return true;
}

bool BEExitAtProtectedLevel(const TradeRecord &r,ulong deal)
{
  if(!BEReferenceValid(r) ||
    HistoryDealGetString(deal,DEAL_SYMBOL)!=Symbol() ||
    (ulong)HistoryDealGetInteger(deal,DEAL_POSITION_ID)!=r.positionID ||
    HistoryDealGetInteger(deal,DEAL_ENTRY)!=DEAL_ENTRY_OUT ||
    HistoryDealGetInteger(deal,DEAL_REASON)!=DEAL_REASON_SL ||
    HistoryDealGetInteger(deal,DEAL_TIME_MSC)<r.be.triggerMSC ||
    !BEPriceEqual(HistoryDealGetDouble(deal,DEAL_SL),r.be.price)) return false;
  // DEAL_SL is the historical stop at this exit, not the realized fill price.
  return true;
}
```

```
bool BEProtectedDeal(const TradeRecord &r,ulong deal)
{
  return r.be.stage>=BE_ACKNOWLEDGED && r.be.stage<=BE_CONFIRMED &&
r.be.durable &&
   r.be.retcode==TRADE_RETCODE_DONE && BEExitAtProtectedLevel(r,deal);
}

bool BEConfirm(TradeRecord &r,bool protectedExit=false)
{
  if(r.be.stage==BE_CONFIRMED) return r.be.durable;
  if(r.be.stage!=BE_ACKNOWLEDGED || r.be.retcode!=TRADE_RETCODE_DONE ||
     !BEReferenceValid(r)) return false;
  if(!r.be.durable && !BEPersist(r)) return false;
    ulong ticket=0;
    bool live=BESelectOwn(r.positionID,ticket,r.direction) &&
       BEPriceEqual(PositionGetDouble(POSITION_SL),r.be.price) &&
       PositionGetDouble(POSITION_TP)==r.be.tp;
    if(!live && !protectedExit) return false;
    r.be.stage=BE_CONFIRMED;
    if(!BEPersist(r)) { r.be.stage=BE_ACKNOWLEDGED; return false; }
    if(!r.be.successLogged) { BELog(r,"BE_MODIFY_OK"); r.be.successLogged=true; }
    return true;
}

bool BEManagerLoss(const TradeRecord &r)
{ return r.netProfit<0 && !r.breakEvenExit; }
void BEAcceptResult(TradeRecord &r,const MqlTradeRequest &request,const
MqlTradeResult &result)
{
  if(r.be.stage!=BE_REQUEST_PENDING || request.action!=TRADE_ACTION_SLTP ||
     request.position!=r.be.ticket || request.position_by!=0 ||
     request.symbol!=Symbol() || request.magic!=(ulong)gMagicCore ||
     (r.be.requestID>0 && result.request_id!=r.be.requestID) ||
     !BEPriceEqual(request.sl,r.be.price) || request.tp!=r.be.tp) return;
  if(result.request_id>0) r.be.requestID=result.request_id;
  r.be.retcode=result.retcode;
  if(result.retcode==TRADE_RETCODE_DONE)
  {
     r.be.stage=BE_ACKNOWLEDGED;
     // Second-resolution server clock is a conservative lower bound, not a fill price.
     r.be.confirmedMSC=(long)TimeTradeServer()*1000;
     if(r.be.confirmedMSC<r.be.triggerMSC) r.be.confirmedMSC=r.be.triggerMSC;
     if(BEPersist(r)) BEConfirm(r);
     if(r.volumeClosedComplete || r.breakEvenClassificationPending)
     { r.historyRefreshPending=true; gNeedsAccountingSync=true; }
  }
  else if(IsFinalRejectedRetcode(result.retcode) ||
```

```
result.retcode==TRADE_RETCODE_NO_CHANGES)
  {
     r.be.stage=BE_TRIGGERED; r.be.requestID=0; r.be.confirmedMSC=0;
     r.be.nextTry=TimeTradeServer()+1;
     BEPersist(r); BEFailure(r,"retcode="+(string)result.retcode+"; safe retry allowed");
   if(r.volumeClosedComplete || r.breakEvenClassificationPending)
   { r.historyRefreshPending=true; gNeedsAccountingSync=true; }
  }
  else
  {
    BEPersist(r);
    BEFailure(r,"retcode="+(string)result.retcode+"; awaiting exact request result, no blind
retry");
    }
}

void BEHandleTransaction(const MqlTradeTransaction &trans,
  const MqlTradeRequest &request,const MqlTradeResult &result)
{
  if(trans.type!=TRADE_TRANSACTION_REQUEST ||
request.action!=TRADE_ACTION_SLTP) return;
  for(int i=0;i<gTradeRecordCount;i++)
     if(gTradeRecords[i].be.stage==BE_REQUEST_PENDING &&
     gTradeRecords[i].be.requestID>0 &&
     result.request_id==gTradeRecords[i].be.requestID)
     BEAcceptResult(gTradeRecords[i],request,result);
}
void ReconcileBreakEvenProofs()
{
  for(int i=0;i<gTradeRecordCount;i++)
  {
    TradeRecord r=gTradeRecords[i];
    if(r.be.stage!=BE_ACKNOWLEDGED) continue;
    BEConfirm(r); gTradeRecords[i]=r;
    if(r.volumeClosedComplete && r.breakEvenClassificationPending)
    { gTradeRecords[i].historyRefreshPending=true; gNeedsAccountingSync=true; }
  }
}

void ManageBreakEven()
{
  ReconcileBreakEvenProofs();
  for(int i=0;i<gTradeRecordCount;i++)
  {
    TradeRecord r=gTradeRecords[i];
    if(r.be.stage==BE_ACKNOWLEDGED || r.be.stage==BE_CONFIRMED ||
r.be.stage==BE_UNRESOLVED ||
       r.be.stage==BE_REQUEST_PENDING || !Inp_EnableBreakEven || r.closed ||
```

```
      r.volumeClosedComplete || r.historyRefreshPending || !r.accountingResolved ||
      !r.initialStopsProven || !r.riskValid || !Positive(r.weightedEntryPrice) ||
      !Positive(r.slPrice) || r.direction*(r.weightedEntryPrice-r.slPrice)<=0 ||
      gPendingExec.active || gPendingExit.active || gLiveOwnershipUnresolved ||
     gPendingOrderClassificationUnresolved || gMultipleOwnOrdersActive ||
     gOrderRegistryUnavailable || gOrderObligationScanUnresolved) continue;
   MqlTick tick;
   if(!SymbolInfoTick(Symbol(),tick) || !Positive(tick.bid) || !Positive(tick.ask) ||
     tick.ask<tick.bid || (r.be.stage==BE_NONE && !BETriggerReached(r,tick))) continue;
   if(TimeTradeServer()<r.be.nextTry) continue;
   ulong ticket=0;
   if(!BESelectOwn(r.positionID,ticket,r.direction) || !BEEntryComplete(r) ||
   !OwnPositionSafe(r.positionID) || !PositionSelectByTicket(ticket) ||
   PositionGetString(POSITION_SYMBOL)!=Symbol() ||
   PositionGetInteger(POSITION_MAGIC)!=(long)gMagicCore ||
   (ulong)PositionGetInteger(POSITION_IDENTIFIER)!=r.positionID ||
   (PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY?1:-1)!=r.direction ||
   !BEPriceEqual(PositionGetDouble(POSITION_PRICE_OPEN),r.weightedEntryPrice) ||

MathAbs(PositionGetDouble(POSITION_VOLUME)-(r.totalVolumeIn-r.totalVolumeOut))>
      AccountingVolumeTolerance()) continue;
if(r.be.stage==BE_NONE)
{
    r.be.stage=BE_TRIGGERED; r.be.ticket=ticket; r.be.direction=r.direction;
    r.be.entryPrice=r.weightedEntryPrice; r.be.initialSL=r.slPrice;
    r.be.price=BEPrice(r.weightedEntryPrice);
     r.be.tp=PositionGetDouble(POSITION_TP);
     r.be.triggerMSC=tick.time_msc>0?tick.time_msc:(long)TimeTradeServer()*1000;
     BELog(r,"BE_TRIGGER");
     if(!BEPersist(r)) { r.be.nextTry=TimeTradeServer()+1; gTradeRecords[i]=r; continue; }
    }
    // Each retry reselects the exact live ticket and preserves its current TP.
    r.be.ticket=ticket; r.be.tp=PositionGetDouble(POSITION_TP);
    double liveSL=PositionGetDouble(POSITION_SL);
    r.be.nextTry=TimeTradeServer()+1;
    if(!BEReferenceValid(r) || !SymbolInfoTick(Symbol(),tick) ||
       !BEStopsValid(r,tick,liveSL,r.be.tp))
    { BEFailure(r,"stop/grid/freeze invalid or live SL already at least as protective");
      gTradeRecords[i]=r; continue; }
    r.be.stage=BE_REQUEST_PENDING; r.be.requestID=0; r.be.retcode=0;
    if(!BEPersist(r))
    { r.be.stage=BE_TRIGGERED; gTradeRecords[i]=r; continue; }
    gTradeRecords[i]=r;
    trade.SetExpertMagicNumber(gMagicCore);
    bool dispatched=trade.PositionModify(ticket,r.be.price,r.be.tp);
    MqlTradeRequest request; MqlTradeResult result;
    trade.Request(request); trade.Result(result);
    // A false CTrade return must not adopt a stale DONE result as confirmation.
```

```
    if(!dispatched && result.retcode==TRADE_RETCODE_DONE)
      result.retcode=TRADE_RETCODE_ERROR;
    BEAcceptResult(gTradeRecords[i],request,result);
    if(!dispatched && gTradeRecords[i].be.stage==BE_REQUEST_PENDING)
        BEFailure(gTradeRecords[i],"PositionModify returned false; no success inferred");
     }
}


void ResetTradeRecord(TradeRecord &r)
{
  r.positionID=0; r.direction=0; r.route="UNAVAILABLE"; r.mode="UNAVAILABLE";
 r.quotaCounted=false; r.requestPrice=0; r.breakoutPrice=0;
 r.requestPriceAvailable=false; r.breakoutPriceAvailable=false;
 r.entryOrderTicket=0; r.expectedEntryVolume=0;
 r.entryTime=0; r.entryPrice=0; r.weightedEntryPrice=0; r.totalVolumeIn=0;
 r.entrySlippage=EMPTY_VALUE; r.slPrice=0; r.tpPrice=0;
 r.exitTime=0; r.exitPrice=0; r.weightedExitPrice=0; r.totalVolumeOut=0; r.volume=0;
 r.exitTimeMSC=0; r.lastExitDealTicket=0;
 r.grossProfit=0; r.commission=0; r.fee=0; r.swap=0; r.netProfit=0;
 r.exitReason="UNAVAILABLE"; r.MAE=0; r.MFE=0; r.holdSeconds=0;
 r.initialRiskMoney=0; r.PnL_R=EMPTY_VALUE;
 r.closed=false; r.maeMfePartialAfterRestart=false; r.volumeClosedComplete=false;
 r.finalizationWritten=false; r.dailyApplied=false;
 r.accountingResolved=true; r.riskValid=false; r.pnlRValid=false;
 r.initialStopsProven=false; r.economicMarkerPending=false;
 r.economicMarkerCleanupPending=false; r.economicMarkerDeleteError=0;
 r.historyRefreshPending=false;
 r.residualForeignDetected=false; r.ownCycleEconomicallyClosed=false;
 r.volumeOwnershipResolved=false; r.inoutCostsUnresolved=false;
 ZeroMemory(r.be); r.breakEvenExit=false; r.breakEvenClassificationPending=false;
}
void ResetPendingExec(bool releaseProof=true)
{
  if(releaseProof && gPendingExec.active) ResolveRequestJournal(false);
  gPendingExec.active=false; gPendingExec.direction=0;
  gPendingExec.route="UNAVAILABLE"; gPendingExec.mode="UNAVAILABLE";
  gPendingExec.requestPrice=0; gPendingExec.requestedLots=0;
  gPendingExec.slPrice=0; gPendingExec.tpPrice=0; gPendingExec.sentAt=0;
  gPendingExec.orderTicket=0; gPendingExec.resultDeal=0;
  gPendingExec.requestID=0;
  gPendingExec.accepted=false; gPendingExec.requestPriceAvailable=false;
  gPendingExec.breakoutPrice=0; gPendingExec.breakoutPriceAvailable=false;
  gPendingExec.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
}

void BeginExitIntent(ulong positionID,string intention)
{
```

```
  gPendingExit.active=true; gPendingExit.positionID=positionID;
  gPendingExit.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
  gPendingExit.reason=intention; gPendingExit.sentAt=TimeTradeServer();
  gPendingExit.orderTicket=0; gPendingExit.resultDeal=0; gPendingExit.positionTicket=0;
  gPendingExit.requestID=0; gPendingExit.accepted=false; gPendingExit.uncertain=false;
  ZeroMemory(gPendingExit.requestSnapshot);
}

void ClearPendingExitIntent()
{
  gPendingExit.active=false; gPendingExit.positionID=0; gPendingExit.positionTicket=0;
  gPendingExit.reason=""; gPendingExit.sentAt=0;
  gPendingExit.orderTicket=0; gPendingExit.resultDeal=0; gPendingExit.requestID=0;
  gPendingExit.accepted=false; gPendingExit.uncertain=false;
  gPendingExit.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
  ZeroMemory(gPendingExit.requestSnapshot);
}

void FailExitIntent(ulong positionID,bool requestResolved=false)
{
  if(!gPendingExit.active || gPendingExit.positionID!=positionID) return;
  // A closed position alone is not terminal proof for its exit request.
  if(!requestResolved) return;
 ResolveRequestJournal(true);
 ClearPendingExitIntent();
}
datetime AccountingDayStart()
{
  MqlDateTime dt; TimeToStruct(TimeTradeServer(),dt);
  dt.hour=0; dt.min=0; dt.sec=0;
  return StructToTime(dt);
}

bool HasOwnPendingOrder()
{
  for(int i=OrdersTotal()-1;i>=0;i--)
  {
    ulong ticket=OrderGetTicket(i);
    if(ticket==0) continue;
    if(OrderGetString(ORDER_SYMBOL)==Symbol() &&
       OrderGetInteger(ORDER_MAGIC)==(long)gMagicCore) return true;
  }
  return false;
}

bool HasOwnPendingOrderToday()
{
  datetime start=AccountingDayStart();
```

```
  for(int i=OrdersTotal()-1;i>=0;i--)
  {
    ulong ticket=OrderGetTicket(i);
    if(ticket==0) continue;
  if(OrderGetString(ORDER_SYMBOL)==Symbol() &&
     OrderGetInteger(ORDER_MAGIC)==(long)gMagicCore &&
     (datetime)OrderGetInteger(ORDER_TIME_SETUP)>=start) return true;
 }
 return false;
}
bool HasExecutionInFlight()
{
    if(gOrderObligationScanUnresolved) return true;
    for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
       if(gUnresolvedOrders[i].active && (gUnresolvedOrders[i].classificationPending ||
          gUnresolvedOrders[i].persistencePending)) return true;
    if(gRequestJournalUnresolved || gEntryJournalCleanupPending ||
       gExitJournalCleanupPending || gRequestJournalWritePending) return true;
    if(gPendingExec.active || gPendingExit.active ||
       gPendingOrderClassificationUnresolved || HasOwnPendingOrder()) return true;
    // A confirmed IN can arrive before Positions exposes the position; an OUT
    // can likewise precede flat state. The tracked lifecycle spans both races.
    for(int i=0;i<gTradeRecordCount;i++)
      if(gTradeRecords[i].positionID>0 && !gTradeRecords[i].closed &&
         gTradeRecords[i].totalVolumeIn>0) return true;
    return false;
}

bool PreparePendingExec(int direction,string route,string mode,double requestPrice,
      double lots,double sl,double tp)
{
  // Call only after the final guards; active state blocks another request.
  if(gPendingExec.active || gRequestJournalUnresolved ||
     gEntryJournalCleanupPending || gRequestJournalWritePending) return false;
  ResetPendingExec(false);
  gPendingExec.active=true; gPendingExec.direction=direction;
  gPendingExec.dispatchPhase=REQUEST_PREPARED;
  gPendingExec.route=route; gPendingExec.mode=mode;
  gPendingExec.requestPrice=requestPrice;
gPendingExec.requestPriceAvailable=(requestPrice>0);
  gPendingExec.requestedLots=lots; gPendingExec.slPrice=sl; gPendingExec.tpPrice=tp;
  gPendingExec.sentAt=TimeTradeServer();
  if(!PersistRequestJournal(false)) { ResetPendingExec(); return false; }
  return true;
}

bool IsAcceptedRetcode(uint ret)
{
```

```
  return ret==TRADE_RETCODE_DONE || ret==TRADE_RETCODE_PLACED ||
   ret==TRADE_RETCODE_DONE_PARTIAL;
}
bool IsUncertainRetcode(uint ret)
{ return ret==TRADE_RETCODE_TIMEOUT || ret==TRADE_RETCODE_CONNECTION; }

void CompletePendingExec(bool sent,uint ret,ulong orderTicket,ulong resultDeal,uint
requestID=0)
{
  gPendingExec.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
  if(orderTicket>0) gPendingExec.orderTicket=orderTicket;
    if(resultDeal>0) gPendingExec.resultDeal=resultDeal;
    if(requestID>0) gPendingExec.requestID=requestID;
    if(IsAcceptedRetcode(ret))
    {
       if(!gPendingExec.accepted) requestsAcceptedToday++;
       gPendingExec.accepted=true;
    }
    // FILLED, ORDER_DELETE and HISTORY_ADD are never evidence of no deal.
    // Transport uncertainty keeps the request guard active until reconciliation.
    else if(IsFinalRejectedRetcode(ret) && !gPendingExec.accepted &&
       gPendingExec.orderTicket==0 && gPendingExec.resultDeal==0)
       ResetPendingExec();
    if(gPendingExec.active) PersistRequestJournal(false);
    gNeedsAccountingSync=true;
}

double AccountingVolumeTolerance()
{
  double step=SymbolInfoDouble(Symbol(),SYMBOL_VOLUME_STEP);
  return (step>0 ? step*0.000001 : 0.00000001);
}

double CalcInitialRiskMoney(int direction,double entryPrice,double slPrice,double volume)
{
  if(volume<=0 || entryPrice<=0 || slPrice<=0 ||
     (double)direction*(entryPrice-slPrice)<=0) return 0;
  double profit=0;
  ENUM_ORDER_TYPE type=(direction==1 ? ORDER_TYPE_BUY :
ORDER_TYPE_SELL);
  if(!OrderCalcProfit(type,Symbol(),volume,entryPrice,slPrice,profit) ||
     !MathIsValidNumber(profit) || profit>=0) return 0;
  return MathAbs(profit);
}

int FindTrackedTrade(ulong positionID)
{
  for(int i=0;i<gTradeRecordCount;i++)
```

```
   if(gTradeRecords[i].positionID==positionID) return i;
  return -1;
}
bool HasOwnEntryHistory(ulong positionID)
{
  if(!HistorySelectByPosition(positionID)) return false;
  for(int i=0;i<HistoryDealsTotal();i++)
  {
     ulong deal=HistoryDealGetTicket(i);
     if(HistoryDealGetString(deal,DEAL_SYMBOL)==Symbol() &&
      HistoryDealGetInteger(deal,DEAL_MAGIC)==(long)gMagicCore &&
     HistoryDealGetInteger(deal,DEAL_ENTRY)==DEAL_ENTRY_IN) return true;
    }
    return false;
}

int NewTrackedTrade(ulong positionID)
{
  int known=FindTrackedTrade(positionID); if(known>=0) return known;
  int idx=-1;
  for(int i=0;i<gTradeRecordCount;i++)
    if(gTradeRecords[i].closed && gTradeRecords[i].finalizationWritten &&
       gTradeRecords[i].dailyApplied && gTradeRecords[i].accountingResolved &&
     !gTradeRecords[i].economicMarkerPending &&
!gTradeRecords[i].historyRefreshPending &&
        !gTradeRecords[i].inoutCostsUnresolved &&
        gTradeRecords[i].exitTime<AccountingDayStart()) { idx=i; break; }
  if(idx<0)
  {
     if(ArrayResize(gTradeRecords,gTradeRecordCount+1)!=gTradeRecordCount+1)
     {
        RememberDeferredAccountingPID(positionID);
        return -1;
     }
     idx=gTradeRecordCount++;
  }
  ResetTradeRecord(gTradeRecords[idx]); gTradeRecords[idx].positionID=positionID;
  BELoad(gTradeRecords[idx]);
  ForgetDeferredAccountingPID(positionID);
  return idx;
}

bool PositionStateByID(ulong positionID,double &volume,double &sl,double &tp,int
&direction)
{
  volume=0; sl=0; tp=0; direction=0;
  for(int i=PositionsTotal()-1;i>=0;i--)
  {
```

```
    ulong ticket=PositionGetTicket(i);
    if(ticket==0 || PositionGetString(POSITION_SYMBOL)!=Symbol()) continue;
  if((ulong)PositionGetInteger(POSITION_IDENTIFIER)!=positionID) continue;
  volume=PositionGetDouble(POSITION_VOLUME);
  sl=PositionGetDouble(POSITION_SL); tp=PositionGetDouble(POSITION_TP);
  direction=(PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY ? 1 : -1);
  return true;
 }
 return false;
}
bool ParseScalperComment(string comment,string &route,string &mode)
{
    route="UNAVAILABLE"; mode="UNAVAILABLE";
    if(comment=="SCALPER_BASE" || comment=="RC1|ABS|BASE")
    { route="ABS"; mode="BASE"; return true; }
      if(comment=="SCALPER_TURBO" || comment=="RC1|ABS|TURBO")
      { route="ABS"; mode="TURBO"; return true; }
      if(comment=="SCALPER_BASE_CONT" || comment=="RC1|CONT|BASE_CONT")
      { route="CONT"; mode="BASE_CONT"; return true; }
      if(comment=="SCALPER_TURBO_CONT" || comment=="RC1|CONT|TURBO_CONT")
      { route="CONT"; mode="TURBO_CONT"; return true; }
      return false;
}
string GvDateToday()
{
   MqlDateTime dt; TimeToStruct(TimeTradeServer(),dt);
   return StringFormat("%04d%02d%02d",dt.year,dt.mon,dt.day);
}

string AccountingKey(string field)
{
  return StringFormat("SCALP_RC1_%I64d_%s_%I64d_%s_%s",

AccountInfoInteger(ACCOUNT_LOGIN),Symbol(),(long)gMagicCore,field,GvDateToday());
}
string GvKeyShots() { return AccountingKey("CONF"); }
string GvKeyDailyBlock() { return AccountingKey("BLOCK"); }
string GvKeyConsecLoss() { return AccountingKey("LOSS"); }
string GvKeyDailyPnL() { return AccountingKey("NET"); }
string GvKeyDailyPnLR() { return AccountingKey("R"); }

bool PersistShotCount()
{
  bool ok=(GlobalVariableSet(GvKeyShots(),(double)ordersEnteredToday)!=0);
  if(!ok) Print("[SCALPER][PERSIST_FAIL] CONFIRMED_TRADE_COUNT");
  GlobalVariablesFlush();
  return ok;
}
```

```
void PersistDailyAccounting()
{
  // VALID is a commit marker. Partial state is rebuilt, never silently trusted.
  GlobalVariableSet(AccountingKey("VALID"),0.0);
  bool ok=true;
  if(GlobalVariableSet(AccountingKey("RISK_LATCH"),gDailyRiskLimitLatched?1.0:0.0)==0)
ok=false;
  if(GlobalVariableSet(GvKeyShots(),(double)ordersEnteredToday)==0) ok=false;
  if(GlobalVariableSet(GvKeyDailyPnL(),dailyNetPnL)==0) ok=false;
  if(GlobalVariableSet(GvKeyDailyPnLR(),dailyPnLR)==0) ok=false;
  if(GlobalVariableSet(GvKeyConsecLoss(),(double)consecutiveLosses)==0) ok=false;
  if(GlobalVariableSet(GvKeyDailyBlock(),dailyRiskBlocked?1.0:0.0)==0) ok=false;
  if(GlobalVariableSet(AccountingKey("RVALID"),gDailyPnLRValid?1.0:0.0)==0) ok=false;
  if(GlobalVariableSet(AccountingKey("CLOSE"),(double)lastCloseConfirmedTime)==0)
ok=false;
  if(GlobalVariableSet(AccountingKey("LASTLOSS"),lastTradeWasLoss?1.0:0.0)==0)
ok=false;
  if(ok) GlobalVariableSet(AccountingKey("VALID"),1.0);
  else Print("[SCALPER][PERSIST_FAIL] DAILY_ACCOUNTING_INCOMPLETE");
  GlobalVariablesFlush();
}
void PersistDailyBlock() { PersistDailyAccounting(); }
void PersistConsecLoss(int n) { consecutiveLosses=n; PersistDailyAccounting(); }
void PersistDailyPnL(double pnl) { dailyNetPnL=pnl; PersistDailyAccounting(); }

string TradeMetaPath(ulong positionID)
{
  return StringFormat("Scalper01_RC1_%I64d_%s_%I64d_%I64u.meta.csv",
   AccountInfoInteger(ACCOUNT_LOGIN),Symbol(),(long)gMagicCore,positionID);
}

void SaveTradeMetadata(const TradeRecord &r)
{
  int h=FileOpen(TradeMetaPath(r.positionID),FILE_WRITE|FILE_CSV|FILE_ANSI,';');
  if(h==INVALID_HANDLE) { Print("[SCALPER][PERSIST_FAIL] TRADE_METADATA");
return; }
  FileWrite(h,"RC1",r.route,r.mode,r.requestPriceAvailable?1:0,

DoubleToString(r.requestPrice,16),DoubleToString(r.slPrice,16),DoubleToString(r.tpPrice,16),
 DoubleToString(r.initialRiskMoney,16),r.riskValid?1:0,r.finalizationWritten?1:0,
 r.breakoutPriceAvailable?1:0,DoubleToString(r.breakoutPrice,16),
   (string)r.entryOrderTicket,DoubleToString(r.expectedEntryVolume,16),
   (string)AccountInfoInteger(ACCOUNT_LOGIN),Symbol(),(string)gMagicCore,
   (string)r.positionID,r.initialStopsProven?"COMPLETE_RC12":"COMPLETE");
   FileFlush(h); FileClose(h);
}
```

```
void LoadTradeMetadata(TradeRecord &r)
{
  int h=FileOpen(TradeMetaPath(r.positionID),FILE_READ|FILE_CSV|FILE_ANSI,';');
  if(h==INVALID_HANDLE) return;
  if(FileReadString(h)!="RC1") { FileClose(h); return; }
  TradeRecord parsed=r;
  parsed.route=FileReadString(h); parsed.mode=FileReadString(h);
  string requestFlag=FileReadString(h);
  parsed.requestPriceAvailable=(requestFlag=="1");
  parsed.requestPrice=StringToDouble(FileReadString(h));
  parsed.slPrice=StringToDouble(FileReadString(h));
parsed.tpPrice=StringToDouble(FileReadString(h));
  parsed.initialRiskMoney=StringToDouble(FileReadString(h));
  string riskFlag=FileReadString(h), writtenFlag=FileReadString(h),
breakoutFlag=FileReadString(h);
  parsed.riskValid=(riskFlag=="1"); parsed.finalizationWritten=(writtenFlag=="1");
  parsed.breakoutPriceAvailable=(breakoutFlag=="1");
  parsed.breakoutPrice=StringToDouble(FileReadString(h));
  parsed.entryOrderTicket=(ulong)StringToInteger(FileReadString(h));
  parsed.expectedEntryVolume=StringToDouble(FileReadString(h));
  string account=FileReadString(h), symbol=FileReadString(h), magic=FileReadString(h);
  string pid=FileReadString(h), complete=FileReadString(h);
  FileClose(h);
  bool routeValid=((parsed.route=="ABS" && (parsed.mode=="BASE" ||
parsed.mode=="TURBO")) ||
    (parsed.route=="CONT" && (parsed.mode=="BASE_CONT" ||
parsed.mode=="TURBO_CONT")) ||
    (parsed.route=="UNAVAILABLE" && parsed.mode=="UNAVAILABLE"));
  bool flagsValid=((requestFlag=="0" || requestFlag=="1") &&
    (riskFlag=="0" || riskFlag=="1") && (writtenFlag=="0" || writtenFlag=="1") &&
    (breakoutFlag=="0" || breakoutFlag=="1"));
  bool valuesValid=(MathIsValidNumber(parsed.requestPrice) &&
MathIsValidNumber(parsed.slPrice) &&
    MathIsValidNumber(parsed.tpPrice) && MathIsValidNumber(parsed.initialRiskMoney) &&
    MathIsValidNumber(parsed.breakoutPrice) &&
MathIsValidNumber(parsed.expectedEntryVolume) &&
    parsed.requestPrice>=0 && parsed.slPrice>=0 && parsed.tpPrice>=0 &&
    parsed.initialRiskMoney>=0 && parsed.breakoutPrice>=0 &&
parsed.expectedEntryVolume>=0 &&
    (!parsed.requestPriceAvailable || parsed.requestPrice>0) &&
    (!parsed.riskValid || (parsed.initialRiskMoney>0 && parsed.slPrice>0)) &&
    (!parsed.breakoutPriceAvailable || parsed.breakoutPrice>0));
  bool identityValid=(account==(string)AccountInfoInteger(ACCOUNT_LOGIN) &&
symbol==Symbol() &&
    magic==(string)gMagicCore && pid==(string)r.positionID);
  if((complete!="COMPLETE" && complete!="COMPLETE_RC12") || !routeValid || !flagsValid
|| !valuesValid || !identityValid)
```

```
  { Print("[SCALPER][TRADE_METADATA_INCOMPLETE_OR_INVALID] persisted fields not
adopted"); return; }
  parsed.finalizationWritten=false; // CSV index is the output authority after restart.
    // Legacy RC1.1 metadata could contain a later mutable live SL/TP.
    parsed.initialStopsProven=(complete=="COMPLETE_RC12" && parsed.slPrice>0);
    if(!parsed.initialStopsProven)
    { parsed.slPrice=0; parsed.tpPrice=0; parsed.initialRiskMoney=0; parsed.riskValid=false; }
    r=parsed;
}

bool IsConfirmedToday(ulong positionID)
{
    for(int i=0;i<ArraySize(gConfirmedPositionsToday);i++)
     if(gConfirmedPositionsToday[i]==positionID) return true;
    return false;
}
bool AddConfirmedToday(ulong positionID)
{
  if(positionID==0) return false;
  if(IsConfirmedToday(positionID)) return true;
  int n=ArraySize(gConfirmedPositionsToday);
  if(ArrayResize(gConfirmedPositionsToday,n+1)!=n+1)
  { gQuotaHistoryValid=false; return false; }
    gConfirmedPositionsToday[n]=positionID;
    return true;
}

int CountTodayTrades()
{
  datetime start=AccountingDayStart();
  if(!HistorySelect(start,TimeTradeServer())) return -1;
  ulong candidates[]; int count=0;
  for(int i=0;i<HistoryDealsTotal();i++)
  {
     ulong deal=HistoryDealGetTicket(i);
     if(deal==0 || HistoryDealGetString(deal,DEAL_SYMBOL)!=Symbol() ||
        HistoryDealGetInteger(deal,DEAL_MAGIC)!=(long)gMagicCore ||
        HistoryDealGetInteger(deal,DEAL_ENTRY)!=DEAL_ENTRY_IN) continue;
     ulong pid=(ulong)HistoryDealGetInteger(deal,DEAL_POSITION_ID);
     if(pid==0) return -1;
     bool seen=false;
     for(int j=0;j<count;j++) if(candidates[j]==pid) { seen=true; break; }
     if(!seen)
     {
        if(ArrayResize(candidates,count+1)!=count+1) return -1;
        candidates[count++]=pid;
     }
    }
```

```
   ulong confirmed[]; int confirmedCount=0;
   for(int i=0;i<count;i++)
   {
     if(!HistorySelectByPosition(candidates[i])) return -1;
     datetime first=0;
     for(int j=0;j<HistoryDealsTotal();j++)
     {
       ulong d=HistoryDealGetTicket(j);
       if(HistoryDealGetInteger(d,DEAL_ENTRY)!=DEAL_ENTRY_IN ||
          HistoryDealGetInteger(d,DEAL_MAGIC)!=(long)gMagicCore ||
          HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol()) continue;
    datetime tm=(datetime)HistoryDealGetInteger(d,DEAL_TIME);
    if(first==0 || tm<first) first=tm;
    }
    if(first==0) return -1; // incomplete per-position evidence, retry later
    if(first>=start)
    {
      if(ArrayResize(confirmed,confirmedCount+1)!=confirmedCount+1) return -1;
      confirmed[confirmedCount++]=candidates[i];
    }
   }
   // Publish only a complete authoritative snapshot. A failed scan must not
   // replace the in-memory PID set with a partial one and allow duplicate quota.
   if(ArrayResize(gConfirmedPositionsToday,confirmedCount)!=confirmedCount) return -1;
   for(int i=0;i<confirmedCount;i++) gConfirmedPositionsToday[i]=confirmed[i];
   for(int i=0;i<gTradeRecordCount;i++)
     gTradeRecords[i].quotaCounted=IsConfirmedToday(gTradeRecords[i].positionID);
   return confirmedCount;
}

string DealExitReason(ulong deal,ulong positionID)
{
  ENUM_DEAL_REASON
reason=(ENUM_DEAL_REASON)HistoryDealGetInteger(deal,DEAL_REASON);
  switch(reason)
  {
    case DEAL_REASON_SL: return "SL";
    case DEAL_REASON_TP: return "TP";
    case DEAL_REASON_EXPERT:
     if(gPendingExit.active && gPendingExit.positionID==positionID &&
        HistoryDealGetInteger(deal,DEAL_MAGIC)==(long)gMagicCore &&
        ((gPendingExit.resultDeal>0 && gPendingExit.resultDeal==deal) ||
         (gPendingExit.orderTicket>0 &&
gPendingExit.orderTicket==(ulong)HistoryDealGetInteger(deal,DEAL_ORDER))))
       return gPendingExit.reason;
    return "EXPERT";
   case DEAL_REASON_CLIENT: return "MANUAL_DESKTOP";
   case DEAL_REASON_MOBILE: return "MANUAL_MOBILE";
```

```
   case DEAL_REASON_WEB: return "MANUAL_WEB";
   case DEAL_REASON_SO: return "STOP_OUT";
   case DEAL_REASON_ROLLOVER: return "ROLLOVER";
   case DEAL_REASON_VMARGIN: return "VMARGIN";
   case DEAL_REASON_SPLIT: return "SPLIT";
   case DEAL_REASON_CORPORATE_ACTION: return "CORPORATE_ACTION";
  }
  return EnumToString(reason)+"("+IntegerToString((int)reason)+")";
}

bool PendingMatchesDeal(ulong deal)
{
  if(!gPendingExec.active || HistoryDealGetString(deal,DEAL_SYMBOL)!=Symbol() ||
     HistoryDealGetInteger(deal,DEAL_MAGIC)!=(long)gMagicCore) return false;
  if(gPendingExec.resultDeal>0 && deal==gPendingExec.resultDeal) return true;
  ulong order=(ulong)HistoryDealGetInteger(deal,DEAL_ORDER);
  if(gPendingExec.orderTicket>0) return order==gPendingExec.orderTicket;
  // A request without ticket is first matched uniquely by the controlled
  // reconciler; rebuilding one PID must not silently choose among candidates.
  return false;
}

void AttachPendingToRecord(TradeRecord &r,ulong deal)
{
    if(!PendingMatchesDeal(deal)) return;
      r.route=gPendingExec.route; r.mode=gPendingExec.mode;
       r.requestPrice=gPendingExec.requestPrice;
       r.requestPriceAvailable=gPendingExec.requestPriceAvailable;
       r.slPrice=gPendingExec.slPrice; r.tpPrice=gPendingExec.tpPrice;
       r.initialStopsProven=(gPendingExec.slPrice>0);
       r.entryOrderTicket=(ulong)HistoryDealGetInteger(deal,DEAL_ORDER);
       r.expectedEntryVolume=gPendingExec.requestedLots;
       r.breakoutPrice=gPendingExec.breakoutPrice;
       r.breakoutPriceAvailable=gPendingExec.breakoutPriceAvailable;
       // Save before releasing the request guard. Armed is never an accounting source.
       SaveTradeMetadata(r);
       // A durable PID obligation must replace the request guard before it is
       // released; Positions can still lag this confirmed IN, including at restart.
       if(PersistEconomicMarker(r.positionID)) ResetPendingExec();
       else r.economicMarkerPending=true;
}

bool OwnPositionSafe(ulong positionID)
{
  if(!HistorySelectByPosition(positionID)) return false;
  bool ownEntry=false;
  for(int i=0;i<HistoryDealsTotal();i++)
  {
```

```
    ulong d=HistoryDealGetTicket(i);
    if(d==0 || HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol()) continue;
    long entry=HistoryDealGetInteger(d,DEAL_ENTRY);
    long type=HistoryDealGetInteger(d,DEAL_TYPE);
    if(type!=DEAL_TYPE_BUY && type!=DEAL_TYPE_SELL) continue;
    if(entry==DEAL_ENTRY_IN || entry==DEAL_ENTRY_INOUT)
    {
      if(HistoryDealGetInteger(d,DEAL_MAGIC)!=(long)gMagicCore) return false;
      if(entry==DEAL_ENTRY_IN) ownEntry=true;
      else return false; // A reversal changes ownership of the net position.
    }
   }
   return ownEntry;
}

// Complete history is sorted explicitly. Transaction arrival order is irrelevant;
// economic totals are rebuilt, never incremented twice by replayed notifications.
bool RebuildTrackedHistory(int idx,bool restarting)
{
  ulong positionID=gTradeRecords[idx].positionID;
  if(!HistorySelectByPosition(positionID))
  { gTradeRecords[idx].historyRefreshPending=true;
gTradeRecords[idx].accountingResolved=false;
    gTradeRecords[idx].volumeOwnershipResolved=false;
    gTradeRecords[idx].ownCycleEconomicallyClosed=false;
    MarkTemporaryAccountingHistory(); return false; }
  ulong deals[]; long times[];
  int n=HistoryDealsTotal();
  if(ArrayResize(deals,n)!=n || ArrayResize(times,n)!=n)
  {
    gTradeRecords[idx].historyRefreshPending=true;
    gTradeRecords[idx].accountingResolved=false;
    gTradeRecords[idx].volumeOwnershipResolved=false;
    gTradeRecords[idx].ownCycleEconomicallyClosed=false;
    MarkTemporaryAccountingHistory(); return false;
  }
  for(int i=0;i<n;i++)
  { deals[i]=HistoryDealGetTicket(i);
times[i]=HistoryDealGetInteger(deals[i],DEAL_TIME_MSC); }
 for(int i=1;i<n;i++)
 {
    ulong d=deals[i]; long tm=times[i]; int j=i-1;
    while(j>=0 && (times[j]>tm || (times[j]==tm && deals[j]>d)))
  { deals[j+1]=deals[j]; times[j+1]=times[j]; j--; }
  deals[j+1]=d; times[j+1]=tm;
 }
 TradeRecord r=gTradeRecords[idx];
 r.historyRefreshPending=false;
```

```
  r.entryTime=0; r.exitTime=0; r.totalVolumeIn=0; r.totalVolumeOut=0;
  r.exitTimeMSC=0; r.lastExitDealTicket=0;
  r.weightedEntryPrice=0; r.weightedExitPrice=0;
  r.grossProfit=0; r.commission=0; r.fee=0; r.swap=0;
  r.accountingResolved=true;
  r.residualForeignDetected=false; r.ownCycleEconomicallyClosed=false;
  r.volumeOwnershipResolved=false; r.inoutCostsUnresolved=false;
  bool ownsCycle=false, foreignEntry=false, residualForeign=false;
  bool beHasContext=(r.be.stage>=BE_REQUEST_PENDING &&
r.be.stage<=BE_CONFIRMED);
  ulong beOwnExitDeals[]; int beOwnExitCount=0;
  bool beExitListValid=(!beHasContext || ArrayResize(beOwnExitDeals,n)==n);
  r.breakEvenExit=false; r.breakEvenClassificationPending=false;
  double tol=AccountingVolumeTolerance();
  for(int i=0;i<n;i++)
  {
    // Everything after a proven INOUT rejection belongs to the foreign residual.
    // Do not replay it into the original Scalper ledger, even with the same Magic.
    if(r.ownCycleEconomicallyClosed) break;
    ulong d=deals[i];
    if(d==0 || HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol()) continue;
    long type=HistoryDealGetInteger(d,DEAL_TYPE);
    if(type!=DEAL_TYPE_BUY && type!=DEAL_TYPE_SELL) continue;
   long entry=HistoryDealGetInteger(d,DEAL_ENTRY);
   long magic=HistoryDealGetInteger(d,DEAL_MAGIC);
  int dir=(type==DEAL_TYPE_BUY ? 1 : -1);
  double vol=HistoryDealGetDouble(d,DEAL_VOLUME);
  double price=HistoryDealGetDouble(d,DEAL_PRICE);
  datetime tm=(datetime)HistoryDealGetInteger(d,DEAL_TIME);
  if(vol<=0 || price<=0) { r.accountingResolved=false; continue; }
  if(entry==DEAL_ENTRY_IN)
  {
     if(magic!=(long)gMagicCore)
     { foreignEntry=true; if(ownsCycle) r.accountingResolved=false; continue; }
     if(foreignEntry || residualForeign || (ownsCycle && dir!=r.direction))
     { r.accountingResolved=false; continue; }
     if(!ownsCycle)
     {
        ownsCycle=true; r.direction=dir; r.entryTime=tm;
        AttachPendingToRecord(r,d);
        if(r.entryOrderTicket==0)
r.entryOrderTicket=(ulong)HistoryDealGetInteger(d,DEAL_ORDER);
        if(r.expectedEntryVolume<=0 && r.entryOrderTicket>0 &&
HistoryOrderSelect(r.entryOrderTicket))
     {
   ENUM_ORDER_STATE
orderState=(ENUM_ORDER_STATE)HistoryOrderGetInteger(r.entryOrderTicket,ORDER_S
TATE);
```

```
   if(orderState==ORDER_STATE_FILLED)

r.expectedEntryVolume=HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_INITI
AL);
  else if(orderState==ORDER_STATE_CANCELED ||
orderState==ORDER_STATE_REJECTED ||
orderState==ORDER_STATE_EXPIRED)
r.expectedEntryVolume=HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_INITI
AL)-

HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_CURRENT);
}
if(r.route=="UNAVAILABLE" || r.mode=="UNAVAILABLE")
{
   string route,mode;
   if(ParseScalperComment(HistoryDealGetString(d,DEAL_COMMENT),route,mode))
   { r.route=route; r.mode=mode; }
   else if(HistoryOrderSelect(r.entryOrderTicket) &&

ParseScalperComment(HistoryOrderGetString(r.entryOrderTicket,ORDER_COMMENT),rout
e,mode))
     { r.route=route; r.mode=mode; }
   }
   }
   double total=r.totalVolumeIn+vol;
     r.weightedEntryPrice=(r.weightedEntryPrice*r.totalVolumeIn+price*vol)/total;
     r.totalVolumeIn=total;
     r.grossProfit+=HistoryDealGetDouble(d,DEAL_PROFIT);
     r.commission+=HistoryDealGetDouble(d,DEAL_COMMISSION);
     r.fee+=HistoryDealGetDouble(d,DEAL_FEE);
r.swap+=HistoryDealGetDouble(d,DEAL_SWAP);
     if(r.entryTime>=AccountingDayStart())
     {
        if(!IsConfirmedToday(positionID))
        {
           if(AddConfirmedToday(positionID)) { ordersEnteredToday++; PersistShotCount(); }
        }
        r.quotaCounted=IsConfirmedToday(positionID);
     }
   }
   else if(entry==DEAL_ENTRY_OUT || entry==DEAL_ENTRY_OUT_BY ||
entry==DEAL_ENTRY_INOUT)
   {
    if(!ownsCycle) continue;
    double remaining=MathMax(0.0,r.totalVolumeIn-r.totalVolumeOut);
    if(remaining<=tol) continue;
    double ownVol=MathMin(vol,remaining);
      if(entry!=DEAL_ENTRY_INOUT && vol>remaining+tol) r.accountingResolved=false;
```

```
      double total=r.totalVolumeOut+ownVol;
      r.weightedExitPrice=(r.weightedExitPrice*r.totalVolumeOut+price*ownVol)/total;
      r.totalVolumeOut=total; r.exitTime=tm;
      r.exitTimeMSC=HistoryDealGetInteger(d,DEAL_TIME_MSC); r.lastExitDealTicket=d;
      // DEAL_PROFIT/SWAP of INOUT belong to its closing component.
      r.grossProfit+=HistoryDealGetDouble(d,DEAL_PROFIT);
      r.swap+=HistoryDealGetDouble(d,DEAL_SWAP);
       if(MathAbs(vol-ownVol)<=tol)
       {
          r.commission+=HistoryDealGetDouble(d,DEAL_COMMISSION);
          r.fee+=HistoryDealGetDouble(d,DEAL_FEE);
       }
       else if(HistoryDealGetDouble(d,DEAL_COMMISSION)!=0 ||
HistoryDealGetDouble(d,DEAL_FEE)!=0)
       {
          // A single INOUT cost has no authoritative close/reversal split.
          // Do not invent proportional fees for the unowned residual.
          if(entry==DEAL_ENTRY_INOUT) r.inoutCostsUnresolved=true;
          else r.accountingResolved=false;
          Print("[SCALPER][RESTART_ACCOUNTING_PARTIAL]
INOUT_COST_ALLOCATION_UNAVAILABLE");
       }
       if(beHasContext && beExitListValid) beOwnExitDeals[beOwnExitCount++]=d;
      r.exitReason=DealExitReason(d,positionID);
      if(entry==DEAL_ENTRY_INOUT && vol>ownVol+tol)
     {
        // RC1 never requests a reversal: current-position guards forbid it.
        // Even a matching request cannot turn a mixed netting cycle into
        // a fresh positionID. Preserve the own close; reject appropriation.
        residualForeign=true; r.residualForeignDetected=true;
        r.ownCycleEconomicallyClosed=(gNetting && r.accountingResolved && !foreignEntry
&&
        r.totalVolumeIn>tol && r.totalVolumeOut+tol>=r.totalVolumeIn);
        Print("[SCALPER][INOUT_RESIDUAL_NOT_APPROPRIATED]");
     }
  }
}
if(!ownsCycle)
{ gTradeRecords[idx].historyRefreshPending=true;
gTradeRecords[idx].accountingResolved=false;
  gTradeRecords[idx].volumeOwnershipResolved=false;
  gTradeRecords[idx].ownCycleEconomicallyClosed=false;
   MarkTemporaryAccountingHistory(); return false; }
 if(foreignEntry) r.accountingResolved=false;
 r.entryPrice=r.weightedEntryPrice; r.exitPrice=r.weightedExitPrice;
 r.volume=r.totalVolumeIn;
 r.netProfit=r.grossProfit+r.commission+r.fee+r.swap;
 double liveVol,liveSL,liveTP; int liveDir;
```

```
 bool live=PositionStateByID(positionID,liveVol,liveSL,liveTP,liveDir);
 if(live && !residualForeign)
{
    // Initial SL/TP come from request/metadata/entry order, not mutable live stops.
    double expected=r.totalVolumeIn-r.totalVolumeOut;
    // During transaction delivery, the deal cache and position state can lag
    // independently. A completed OUT waits in TryFinalize until flat; a
    // transient state mismatch does not permanently poison the record.
  if(restarting && (MathAbs(expected-liveVol)>tol || liveDir!=r.direction))
     r.accountingResolved=false;
  }
  if(r.entryOrderTicket>0 && HistoryOrderSelect(r.entryOrderTicket))
  {
     double initialSL=HistoryOrderGetDouble(r.entryOrderTicket,ORDER_SL);
     if(initialSL>0)
     {
        r.slPrice=initialSL; r.tpPrice=HistoryOrderGetDouble(r.entryOrderTicket,ORDER_TP);
        r.initialStopsProven=true;
     }
  }
  if(!r.initialStopsProven) { r.slPrice=0; r.tpPrice=0; }
  double risk=CalcInitialRiskMoney(r.direction,r.weightedEntryPrice,r.slPrice,r.totalVolumeIn);
  if(risk>0) { r.initialRiskMoney=risk; r.riskValid=true; }
  else { r.initialRiskMoney=0; r.riskValid=false; }
  if(r.requestPriceAvailable)
      r.entrySlippage=(double)r.direction*(r.weightedEntryPrice-r.requestPrice);
   else r.entrySlippage=EMPTY_VALUE;
   r.volumeClosedComplete=(r.totalVolumeIn>tol && r.totalVolumeOut+tol>=r.totalVolumeIn);
   r.volumeOwnershipResolved=r.accountingResolved;
   r.ownCycleEconomicallyClosed=(r.ownCycleEconomicallyClosed &&
      r.volumeOwnershipResolved && r.volumeClosedComplete &&
r.residualForeignDetected);
    // Classify only after the complete entry mean and initial order stops are rebuilt.
    if(r.be.stage==BE_ACKNOWLEDGED && !r.be.durable) BEPersist(r);
    bool allOwnExitsBE=(beHasContext && beExitListValid && beOwnExitCount>0);
    bool allExitsAtBELevel=allOwnExitsBE;
    for(int j=0;j<beOwnExitCount;j++)
    {
       if(!BEExitAtProtectedLevel(r,beOwnExitDeals[j])) allExitsAtBELevel=false;
       if(!BEProtectedDeal(r,beOwnExitDeals[j]) || !BEConfirm(r,true)) allOwnExitsBE=false;
    }
    r.breakEvenClassificationPending=(r.volumeClosedComplete && beHasContext &&
       (!beExitListValid || (allExitsAtBELevel &&
         (r.be.stage==BE_REQUEST_PENDING || r.be.stage==BE_ACKNOWLEDGED))));
  if(r.breakEvenClassificationPending)
     BEFailure(r,"BE_EXIT_PROOF_PENDING; finalization/accounting publication deferred");
  r.breakEvenExit=(allOwnExitsBE && r.volumeClosedComplete &&
     r.accountingResolved && r.be.stage==BE_CONFIRMED && r.be.durable &&
```

```
BEReferenceValid(r));
  if(r.breakEvenExit)
  {
     r.exitReason="BE";
     if(!r.be.exitLogged) { BELog(r,"BE_EXIT","BE_NEUTRAL; economic PnL unchanged");
r.be.exitLogged=true; }
  }
  if(r.inoutCostsUnresolved)
  { r.accountingResolved=false; r.netProfit=EMPTY_VALUE; r.PnL_R=EMPTY_VALUE;
r.pnlRValid=false; }
  if(r.volumeClosedComplete && gOpenPos.active && gOpenPos.positionID==positionID &&
     (r.expectedEntryVolume<=0 || r.totalVolumeIn+tol>=r.expectedEntryVolume))
 {
     // Volume already closed stops excursion tracking even if Positions lags
     // or an unowned INOUT residual still occupies the netting identifier.
     r.MAE=gOpenPos.MAE; r.MFE=gOpenPos.MFE; ResetOpenPos();
  }
  if(restarting) r.maeMfePartialAfterRestart=true;
  r.holdSeconds=(r.exitTime>0 ? (long)(r.exitTime-r.entryTime) : 0);
  gTradeRecords[idx]=r;
  SyncEconomicMarker(idx);
  if(!r.accountingResolved)
  {
    if(live && !CanIgnoreForeignResidual(r)) gLiveOwnershipUnresolved=true;
    gDailyEconomicsValid=false;
    gDailyPnLRValid=false;
    gDailyHistoryRetryRequired=true;
    Print("[SCALPER][RESTART_ACCOUNTING_UNRESOLVED] historical/current volume
or ownership mismatch");
  }
  SaveTradeMetadata(gTradeRecords[idx]);
   return true;
}

void SyncOpenPositionFromRecord(int idx)
{
  TradeRecord r=gTradeRecords[idx];
  if(r.volumeClosedComplete) return;
  double volume,sl,tp; int direction;
  if(r.totalVolumeIn<=0 || !r.accountingResolved) return;
  bool positionVisible=PositionStateByID(r.positionID,volume,sl,tp,direction);
  if(positionVisible && direction!=r.direction) return;
  if(gOpenPos.active && gOpenPos.positionID!=r.positionID)
  { gLiveOwnershipUnresolved=true;
Print("[SCALPER][MULTIPLE_OWN_POSITIONS_UNRESOLVED]"); return; }
  if(!gOpenPos.active)
  { ResetOpenPos(); gOpenPos.active=true; gOpenPos.MAE=r.MAE;
gOpenPos.MFE=r.MFE; }
```

```
  gOpenPos.positionID=r.positionID; gOpenPos.direction=r.direction;
  gOpenPos.entryPrice=r.weightedEntryPrice; gOpenPos.lots=r.totalVolumeIn;
  gOpenPos.entryTime=r.entryTime; gOpenPos.route=r.route; gOpenPos.mode=r.mode;
  gOpenPos.initialRiskMoney=r.initialRiskMoney; gOpenPos.tradeRecordIdx=idx;
}

bool CanIgnoreForeignResidual(const TradeRecord &r)
{
  double tol=AccountingVolumeTolerance();
  return gNetting && !r.historyRefreshPending && r.volumeOwnershipResolved &&
   r.residualForeignDetected && r.ownCycleEconomicallyClosed &&
    r.totalVolumeIn>tol && r.totalVolumeOut+tol>=r.totalVolumeIn;
}

bool MayFinalizeTrackedCycle(const TradeRecord &r)
{
  if(r.breakEvenClassificationPending) return false;
  if(r.historyRefreshPending) return false;
  // Unknown costs may keep economics partial, without keeping the own cycle open.
  return r.accountingResolved || (CanIgnoreForeignResidual(r) && r.inoutCostsUnresolved);
}

bool IsTrackedVolumeCycleComplete(const TradeRecord &r)
{
  if(!MayFinalizeTrackedCycle(r) || !r.volumeClosedComplete) return false;
  if(HasOwnPendingOrder() || gPendingOrderClassificationUnresolved ||
     gOrderRegistryUnavailable || gMultipleOwnOrdersActive ||
     gOrderObligationScanUnresolved) return false;
  // Every observed exact-order lifecycle associated with this PID must finish,
  // not only the first entry order. A vanished UNKNOWN still carries its PID.
  for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
     if(gUnresolvedOrders[i].active &&
        (gUnresolvedOrders[i].positionID==r.positionID ||
         gUnresolvedOrders[i].positionByID==r.positionID)) return false;
  double vol,sl,tp; int direction;
  if(PositionStateByID(r.positionID,vol,sl,tp,direction) && !CanIgnoreForeignResidual(r))
return false;
  if(r.entryOrderTicket>0 && OrderSelect(r.entryOrderTicket))
     return false;
  // A notified exit can precede other entry fills. A known requested quantity
  // provides a further guard, without assuming FOK means one deal.
    if(r.expectedEntryVolume>0 &&
r.totalVolumeIn+AccountingVolumeTolerance()<r.expectedEntryVolume)
    {
      bool completed=false;
      if(r.entryOrderTicket>0 && HistoryOrderSelect(r.entryOrderTicket))
      {
        ENUM_ORDER_STATE
```

```
st=(ENUM_ORDER_STATE)HistoryOrderGetInteger(r.entryOrderTicket,ORDER_STATE);
        if(st==ORDER_STATE_CANCELED || st==ORDER_STATE_REJECTED ||
st==ORDER_STATE_EXPIRED)
          // Rejected terminal orders can also have confirmed partial fills.
        {
          double
filled=HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_INITIAL)-
            HistoryOrderGetDouble(r.entryOrderTicket,ORDER_VOLUME_CURRENT);
          completed=(MathAbs(filled-r.totalVolumeIn)<=AccountingVolumeTolerance());
        }
      }
     if(!completed) return false;
    }
    return true;
}

void TryFinalizeTrackedTrade(ulong positionID)
{
  int idx=FindTrackedTrade(positionID);
  if(idx<0 || gTradeRecords[idx].closed || !gTradeRecords[idx].volumeClosedComplete)
return;
  TradeRecord r=gTradeRecords[idx];
  if(!MayFinalizeTrackedCycle(r)) return;
  if(!IsTrackedVolumeCycleComplete(r)) { gNeedsAccountingSync=true; return; }
  if(gOpenPos.active && gOpenPos.positionID==positionID)
 { r.MAE=gOpenPos.MAE; r.MFE=gOpenPos.MFE; }
 r.closed=true; r.holdSeconds=(long)(r.exitTime-r.entryTime);
 r.netProfit=(r.accountingResolved ? r.grossProfit+r.commission+r.fee+r.swap :
EMPTY_VALUE);
 if(r.accountingResolved && r.riskValid && r.initialRiskMoney>0)
 { r.PnL_R=r.netProfit/r.initialRiskMoney; r.pnlRValid=true; }
 else
 { r.PnL_R=EMPTY_VALUE; r.pnlRValid=false;
Print("[SCALPER][ACCOUNTING_RISK_UNAVAILABLE]"); }
 if(!r.accountingResolved)
 {
    // Acknowledge this partial cycle once; no invented net result or loss count.
    r.PnL_R=EMPTY_VALUE; r.pnlRValid=false; r.dailyApplied=true;
    gDailyEconomicsValid=false; gDailyPnLRValid=false;
    gDailyEconomicsState=ECONOMIC_ALLOCATION_IMPOSSIBLE;
    lastCloseConfirmedTime=TimeTradeServer();
    RecomputeDailyRiskBlock();
    PersistDailyAccounting();
    Print("[SCALPER][OWN_CYCLE_CLOSED_ACCOUNTING_PARTIAL]
INOUT_COST_ALLOCATION_UNAVAILABLE");
 }
 if(r.accountingResolved && !r.dailyApplied && r.exitTime>=AccountingDayStart())
 {
```

```
    dailyNetPnL+=r.netProfit;
    if(r.pnlRValid) dailyPnLR+=r.PnL_R;
    else gDailyPnLRValid=false;
    lastTradeWasLoss=BEManagerLoss(r);
    consecutiveLosses=(lastTradeWasLoss ? consecutiveLosses+1 : 0);
    lastCloseConfirmedTime=TimeTradeServer();
    if(Inp_EnableDailyRiskStop && gDailyPnLRValid && dailyPnLR<=-Inp_MaxDailyLossR)
       gDailyRiskLimitLatched=true;
    if(Inp_EnableConsecutiveLossStop && gDailyEconomicsValid &&
consecutiveLosses>=Inp_MaxConsecutiveLosses)
       gDailyRiskLimitLatched=true;
    r.dailyApplied=true;
    gDailyHistoryRetryRequired=true;
    RecomputeDailyRiskBlock();
    PersistDailyAccounting();
   }
   gTradeRecords[idx]=r;
   if(!r.finalizationWritten && csvTradeHandle!=INVALID_HANDLE)
   {
      if(WriteTradeRow(gTradeRecords[idx])) gTradeRecords[idx].finalizationWritten=true;
   }
   SaveTradeMetadata(gTradeRecords[idx]);
   SyncEconomicMarker(idx);
    FailExitIntent(positionID);
    if(gOpenPos.active && gOpenPos.positionID==positionID) ResetOpenPos();
}

int FindUnresolvedOrder(ulong ticket)
{
  for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
   if(gUnresolvedOrders[i].active && gUnresolvedOrders[i].orderTicket==ticket) return i;
  return -1;
}

int RememberUnresolvedOrder(ulong ticket)
{
  if(ticket==0) return -1; // Request context itself protects an unknown ticket.
  // Commit the exact identity before any allocation can lose its RAM slot.
  bool present=FileIsExist(OrderObligationPath(ticket));
  bool durable=EnsureOrderObligationIdentity(ticket);
  if(!durable) gOrderObligationScanUnresolved=true;
  int idx=FindUnresolvedOrder(ticket);
if(idx>=0)
{
   if(!present || !durable) gUnresolvedOrders[idx].persistencePending=true;
   for(int i=0;i<gOrderRegistryFallbackCount;i++)
      if(gOrderRegistryFallback[i]==ticket)
      {
```

```
        for(int j=i+1;j<gOrderRegistryFallbackCount;j++)
          gOrderRegistryFallback[j-1]=gOrderRegistryFallback[j];
        gOrderRegistryFallbackCount--; break;
      }
   return idx;
}
int n=ArraySize(gUnresolvedOrders);
idx=-1;
for(int i=0;i<n;i++) if(!gUnresolvedOrders[i].active) { idx=i; break; }
if(idx<0)
{
    if(ArrayResize(gUnresolvedOrders,n+1)!=n+1)
    {
       bool seen=false;
       for(int i=0;i<gOrderRegistryFallbackCount;i++)
          if(gOrderRegistryFallback[i]==ticket) { seen=true; break; }
       if(!seen)
       {
          if(gOrderRegistryFallbackCount<MAX_TRADE_RECORDS)
             gOrderRegistryFallback[gOrderRegistryFallbackCount++]=ticket;
          else if(!FileIsExist(OrderObligationPath(ticket)))
          {
             // Only a genuinely lost identity is a permanent obligation.
         gOrderRegistryOverflow=true;
         MarkInfrastructureIdentityLoss();
    }
    }
    // A durable filename remains recoverable even if both RAM lists fill.
    gOrderObligationScanUnresolved=true;
    gOrderRegistryUnavailable=true; gPendingOrderClassificationUnresolved=true;
    return -1;
   }
   idx=n;
  }
  UnresolvedOrderContext c;
  UnknownOrderObligationContext(ticket,c);
  c.firstSeen=TimeTradeServer();
  if(!gOrderObligationRestoring) PersistOrderObligationSnapshot(c);
  gUnresolvedOrders[idx]=c;
  gPendingOrderClassificationUnresolved=true;
  for(int i=0;i<gOrderRegistryFallbackCount;i++)
    if(gOrderRegistryFallback[i]==ticket)
     {
       for(int j=i+1;j<gOrderRegistryFallbackCount;j++)
        gOrderRegistryFallback[j-1]=gOrderRegistryFallback[j];
       gOrderRegistryFallbackCount--; break;
     }
    return idx;
```

```
}

bool ReadOwnOrder(UnresolvedOrderContext &c,bool &live)
{
  ulong ticket=c.orderTicket;
  live=OrderSelect(ticket);
  if(!live && !HistoryOrderSelect(ticket)) return false;
  string
symbol=live?OrderGetString(ORDER_SYMBOL):HistoryOrderGetString(ticket,ORDER_SYM
BOL);
  long
magic=live?OrderGetInteger(ORDER_MAGIC):HistoryOrderGetInteger(ticket,ORDER_MAGI
C);
 if(symbol!=Symbol() || magic!=(long)gMagicCore) return false;

c.orderType=live?OrderGetInteger(ORDER_TYPE):HistoryOrderGetInteger(ticket,ORDER_T
YPE);

c.orderReason=live?OrderGetInteger(ORDER_REASON):HistoryOrderGetInteger(ticket,OR
DER_REASON);
 ulong
pid=(ulong)(live?OrderGetInteger(ORDER_POSITION_ID):HistoryOrderGetInteger(ticket,OR
DER_POSITION_ID));
 if(c.positionID>0 && pid>0 && c.positionID!=pid) return false;
 if(pid>0) c.positionID=pid;
c.positionByID=(ulong)(live?OrderGetInteger(ORDER_POSITION_BY_ID):HistoryOrderGetI
nteger(ticket,ORDER_POSITION_BY_ID));

c.setupTime=(datetime)(live?OrderGetInteger(ORDER_TIME_SETUP):HistoryOrderGetInte
ger(ticket,ORDER_TIME_SETUP));
 c.doneTime=live?0:(datetime)HistoryOrderGetInteger(ticket,ORDER_TIME_DONE);
c.lastState=(ENUM_ORDER_STATE)(live?OrderGetInteger(ORDER_STATE):HistoryOrderG
etInteger(ticket,ORDER_STATE));

c.initialVolume=live?OrderGetDouble(ORDER_VOLUME_INITIAL):HistoryOrderGetDouble(ti
cket,ORDER_VOLUME_INITIAL);

c.remainingVolume=live?OrderGetDouble(ORDER_VOLUME_CURRENT):HistoryOrderGet
Double(ticket,ORDER_VOLUME_CURRENT);
c.comment=live?OrderGetString(ORDER_COMMENT):HistoryOrderGetString(ticket,ORDER
_COMMENT);
  return c.setupTime>0 && c.initialVolume>0 && c.remainingVolume>=0;
}

bool PendingOrderFillEvidence(ulong order,bool &entryFill,bool &exitFill,
           bool &conflict,ulong &positionID,double &filledVolume,
           string &route,string &mode)
{
```

```
  entryFill=false; exitFill=false; conflict=false;
  positionID=0; filledVolume=0; route="UNAVAILABLE"; mode="UNAVAILABLE";
  UnresolvedOrderContext c; c.orderTicket=order; c.positionID=0;
  bool live;
  if(!ReadOwnOrder(c,live)) return false;
  // No account-wide scan from zero. Setup is the authoritative lower bound;
  // one second of overlap covers timestamps truncated by ORDER_TIME_SETUP.
  datetime until=TimeTradeServer();
  if(c.doneTime>=until) until=c.doneTime+1;
  if(!HistorySelect(c.setupTime-1,until)) return false;
 for(int i=0;i<HistoryDealsTotal();i++)
 {
    ulong d=HistoryDealGetTicket(i);
    if(d==0 || (ulong)HistoryDealGetInteger(d,DEAL_ORDER)!=order) continue;
    if(HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol()) { conflict=true; continue; }
    long type=HistoryDealGetInteger(d,DEAL_TYPE);
    if(type!=DEAL_TYPE_BUY && type!=DEAL_TYPE_SELL) continue;
    ulong pid=(ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID);
    long entry=HistoryDealGetInteger(d,DEAL_ENTRY);
    // CLOSE_BY has two position IDs. Its paired OUT_BY is not a second
    // fill of the primary position; both ledgers reconcile independently.
    if(c.orderType==ORDER_TYPE_CLOSE_BY && pid==c.positionByID &&
pid!=c.positionID)
   {
   if(entry!=DEAL_ENTRY_OUT_BY) conflict=true;
   continue;
  }
  if(pid==0 || (positionID>0 && positionID!=pid) ||
     (c.positionID>0 && c.positionID!=pid)) conflict=true;
positionID=pid;
if(entry==DEAL_ENTRY_IN)
{
   entryFill=true;
   if(HistoryDealGetInteger(d,DEAL_MAGIC)!=(long)gMagicCore) conflict=true;
   string foundRoute,foundMode;

if(ParseScalperComment(HistoryDealGetString(d,DEAL_COMMENT),foundRoute,foundMod
e))
     {
        if(route!="UNAVAILABLE" && (route!=foundRoute || mode!=foundMode))
conflict=true;
        route=foundRoute; mode=foundMode;
       }
    }
    else if(entry==DEAL_ENTRY_OUT || entry==DEAL_ENTRY_OUT_BY ||
entry==DEAL_ENTRY_INOUT)
       exitFill=true; // Own portion of INOUT must still be proven by the ledger.
    else conflict=true;
```

```
    double volume=HistoryDealGetDouble(d,DEAL_VOLUME);
    if(volume<=0 || !MathIsValidNumber(volume)) conflict=true;
    else filledVolume+=volume;
  }
  if(entryFill && exitFill) conflict=true;
  return true;
}

bool TerminalOrder(ENUM_ORDER_STATE state)
{
  return state==ORDER_STATE_FILLED || state==ORDER_STATE_CANCELED ||
   state==ORDER_STATE_REJECTED || state==ORDER_STATE_EXPIRED;
}

bool ProvenOrderFillComplete(const UnresolvedOrderContext &c,double filled)
{
  double expected=c.initialVolume-c.remainingVolume;
  double tol=AccountingVolumeTolerance();
  return TerminalOrder(c.lastState) && expected>=-tol &&
   MathAbs(expected-filled)<=tol &&
   (c.lastState!=ORDER_STATE_FILLED || filled>tol);
}
bool AdoptLiveEntry(const UnresolvedOrderContext &c)
{
  if(gRequestJournalUnresolved || gRequestJournalWritePending ||
     gEntryJournalCleanupPending || gExitJournalCleanupPending) return false;
  if(gPendingExit.active) return false;
  if(gPendingExec.active)
 {
    // Never overwrite a known request with a newly discovered ambiguous order.
    if(gPendingExec.orderTicket==c.orderTicket) return true;
    return false;
  }
  if(!OrderSelect(c.orderTicket)) return false;
  ResetPendingExec(false); gPendingExec.active=true; gPendingExec.accepted=true;
  gPendingExec.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
  gPendingExec.orderTicket=c.orderTicket;
  gPendingExec.direction=(c.orderType==ORDER_TYPE_BUY?1:-1);
  gPendingExec.sentAt=c.setupTime; gPendingExec.requestedLots=c.initialVolume;
  gPendingExec.slPrice=OrderGetDouble(ORDER_SL);
gPendingExec.tpPrice=OrderGetDouble(ORDER_TP);
  gPendingExec.route=c.route; gPendingExec.mode=c.mode;
  gPendingExec.requestPriceAvailable=false;
  PersistRequestJournal(false);
  return true;
}

bool AdoptLiveExit(const UnresolvedOrderContext &c)
```

```
{
  if(gRequestJournalUnresolved || gRequestJournalWritePending ||
     gEntryJournalCleanupPending || gExitJournalCleanupPending) return false;
  if(c.positionID==0 || gPendingExec.active) return false;
  if(gPendingExit.active && (gPendingExit.positionID!=c.positionID ||
     (gPendingExit.orderTicket>0 && gPendingExit.orderTicket!=c.orderTicket))) return false;
  if(!gPendingExit.active) BeginExitIntent(c.positionID,"RECOVERED_EXIT");
  gPendingExit.orderTicket=c.orderTicket; gPendingExit.sentAt=c.setupTime;
  gPendingExit.accepted=true;
    PersistRequestJournal(true);
    return true;
}

void RestorePendingOrder()
{
  int found=0;
  bool complete=true;
  // Failed allocations retain exact identities independently of OrdersTotal.
  int fallback=gOrderRegistryFallbackCount;
  for(int i=fallback-1;i>=0;i--)
     if(RememberUnresolvedOrder(gOrderRegistryFallback[i])<0) complete=false;
  for(int i=OrdersTotal()-1;i>=0;i--)
 {
     ulong ticket=OrderGetTicket(i);
     if(ticket==0) { complete=false; continue; }
     if(OrderGetString(ORDER_SYMBOL)!=Symbol() ||
OrderGetInteger(ORDER_MAGIC)!=(long)gMagicCore) continue;
     found++;
     if(RememberUnresolvedOrder(ticket)<0) complete=false;
  }
  gMultipleOwnOrdersActive=found>1;
  gOrderRegistryUnavailable=!complete || gOrderRegistryFallbackCount>0 ||
gOrderRegistryOverflow;
  gPendingOrderClassificationUnresolved=gMultipleOwnOrdersActive ||
gOrderRegistryUnavailable ||
     gOrderObligationScanUnresolved;
  for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
     if(gUnresolvedOrders[i].active && (gUnresolvedOrders[i].classificationPending ||
        gUnresolvedOrders[i].persistencePending))
        gPendingOrderClassificationUnresolved=true;
    // Zero live orders does not clear any stored ambiguity.
}

void ReconcileUnresolvedOrders()
{
   RestorePendingOrder();
   for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
   {
```

```
     if(!gUnresolvedOrders[i].active) continue;
     UnresolvedOrderContext c=gUnresolvedOrders[i];
     c.classificationPending=true;
     bool live;
     if(!ReadOwnOrder(c,live)) { StoreOrderObligationContext(i,c,false); continue; }
     bool entryFill,exitFill,conflict; ulong pid; double filled; string route,mode;
     if(!PendingOrderFillEvidence(c.orderTicket,entryFill,exitFill,conflict,pid,filled,route,mode)
|| conflict)
     { StoreOrderObligationContext(i,c,true); continue; }
    if(pid>0) c.positionID=pid;
    bool marker=StringFind(c.comment,"RC1|")==0 &&
ParseScalperComment(c.comment,c.route,c.mode);
    bool market=c.orderType==ORDER_TYPE_BUY || c.orderType==ORDER_TYPE_SELL;
    int idx=FindTrackedTrade(c.positionID);
    if(entryFill || exitFill)
    {
      if(entryFill && (!market || c.positionByID>0 || c.orderReason==ORDER_REASON_SL ||
         c.orderReason==ORDER_REASON_TP ||
         (marker && route!="UNAVAILABLE" && (c.route!=route || c.mode!=mode))))
      { StoreOrderObligationContext(i,c,true); continue; }
      // OUT with another Magic is admitted only through proven own IN history.
      if(idx<0 && (entryFill || HasOwnEntryHistory(c.positionID)))
      {
     idx=NewTrackedTrade(c.positionID);
     if(idx>=0) LoadTradeMetadata(gTradeRecords[idx]);
     }
     if(idx<0 || !RebuildTrackedHistory(idx,false)) { StoreOrderObligationContext(i,c,true);
continue; }
      TradeRecord r=gTradeRecords[idx];
    bool own=r.volumeOwnershipResolved && r.totalVolumeIn>0;
    if(!own || (entryFill && (r.residualForeignDetected || r.entryOrderTicket!=c.orderTicket)))
    { StoreOrderObligationContext(i,c,true); continue; }
    c.nature=entryFill?PENDING_ORDER_ENTRY:PENDING_ORDER_EXIT;
    if(exitFill && gPendingExec.active && gPendingExec.orderTicket==c.orderTicket)
      ResetPendingExec(); // Exact order + own exit ledger, never an inferred entry.
    SyncOpenPositionFromRecord(idx);
    TryFinalizeTrackedTrade(c.positionID);
    if(c.positionByID>0)
 {
     int by=FindTrackedTrade(c.positionByID);
     if(by<0 && HasOwnEntryHistory(c.positionByID))
     { by=NewTrackedTrade(c.positionByID); if(by>=0)
LoadTradeMetadata(gTradeRecords[by]); }
     if(by>=0 && RebuildTrackedHistory(by,false))
TryFinalizeTrackedTrade(c.positionByID);
   }
   if(!live && ProvenOrderFillComplete(c,filled))
   {
```

```
      if(ReleaseOrderObligation(c) && exitFill && gPendingExit.active &&
         gPendingExit.orderTicket==c.orderTicket &&
         gPendingExit.positionID==c.positionID && gTradeRecords[idx].closed)
         FailExitIntent(c.positionID,true);
   }
   else if(live && !gMultipleOwnOrdersActive)
   {
      // A confirmed own IN is already represented by its ledger. Do not
      // recreate/reset entry request metadata after the first fill.
      bool adopted=entryFill?true:AdoptLiveExit(c);
      c.classificationPending=!adopted;
  }
 }
 else if(!live && ProvenOrderFillComplete(c,filled) &&
c.lastState!=ORDER_STATE_FILLED)
 {
   // Terminal negative state plus complete, exact-order zero-fill history.
   if(ReleaseOrderObligation(c))
   {
      if(gPendingExec.active && gPendingExec.orderTicket==c.orderTicket)
ResetPendingExec();
      if(gPendingExit.active && gPendingExit.orderTicket==c.orderTicket)
FailExitIntent(gPendingExit.positionID,true);
   }
  }
   else if(live && !gMultipleOwnOrdersActive)
   {
     if(marker && market && c.orderReason!=ORDER_REASON_SL &&
c.orderReason!=ORDER_REASON_TP &&
        c.positionID==0 && c.positionByID==0 && !HasOpenPosition())
    {
       c.nature=PENDING_ORDER_ENTRY; c.classificationPending=!AdoptLiveEntry(c);
     }
     else if(!marker && idx>=0 && gTradeRecords[idx].accountingResolved &&
OwnPositionSafe(c.positionID))
     {
       TradeRecord r=gTradeRecords[idx]; double volume,sl,tp; int direction;
       double remainder=r.totalVolumeIn-r.totalVolumeOut;
    bool opposite=(c.orderType==ORDER_TYPE_BUY && r.direction==-1) ||
         (c.orderType==ORDER_TYPE_SELL && r.direction==1);
    bool bounded=!r.closed && remainder>0 && c.remainingVolume>0 &&
       c.remainingVolume<=remainder+AccountingVolumeTolerance() &&
       PositionStateByID(c.positionID,volume,sl,tp,direction) && direction==r.direction &&
       MathAbs(volume-remainder)<=AccountingVolumeTolerance();
    if(bounded && ((market && opposite && c.positionByID==0) ||
       (c.orderType==ORDER_TYPE_CLOSE_BY && c.positionByID>0)))
    { c.nature=PENDING_ORDER_EXIT; c.classificationPending=!AdoptLiveExit(c); }
   }
```

```
 }
 StoreOrderObligationContext(i,c,true);
 }
 gPendingOrderClassificationUnresolved=gMultipleOwnOrdersActive ||
gOrderRegistryUnavailable ||
   gOrderObligationScanUnresolved;
 for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
   if(gUnresolvedOrders[i].active && (gUnresolvedOrders[i].classificationPending ||
      gUnresolvedOrders[i].persistencePending))
    gPendingOrderClassificationUnresolved=true;
}

bool MatchesUnidentifiedEntryOrder(ulong ticket)
{
  UnresolvedOrderContext c; ZeroMemory(c);
  c.orderTicket=ticket; bool live;
  if(!ReadOwnOrder(c,live) || gPendingExec.sentAt<=0 ||
     c.setupTime<gPendingExec.sentAt || c.positionByID>0 ||
     c.orderReason==ORDER_REASON_SL || c.orderReason==ORDER_REASON_TP)
return false;
  int direction=(c.orderType==ORDER_TYPE_BUY?1:
     (c.orderType==ORDER_TYPE_SELL?-1:0));
    string route,mode;
     return direction==gPendingExec.direction &&
       ParseScalperComment(c.comment,route,mode) && route==gPendingExec.route &&
       mode==gPendingExec.mode &&

MathAbs(c.initialVolume-gPendingExec.requestedLots)<=AccountingVolumeTolerance();
}

void RecoverUnidentifiedEntryOrder()
{
  if(!gPendingExec.active || gPendingExec.orderTicket>0 ||
      gPendingExec.resultDeal>0 || gPendingExec.sentAt<=0) return;
   datetime start=gPendingExec.sentAt-1;
   if(!HistorySelect(start,TimeTradeServer())) { MarkTemporaryAccountingHistory(); return; }
   // Snapshot exact tickets before HistoryOrderSelect changes the selected list.
   ulong tickets[]; int count=0;
   int total=HistoryOrdersTotal()+OrdersTotal();
   if(ArrayResize(tickets,total)!=total) { MarkTemporaryAccountingHistory(); return; }
   for(int i=0;i<HistoryOrdersTotal();i++)
   {
      ulong ticket=HistoryOrderGetTicket(i);
      if(ticket>0 && HistoryOrderGetString(ticket,ORDER_SYMBOL)==Symbol() &&
         HistoryOrderGetInteger(ticket,ORDER_MAGIC)==(long)gMagicCore)
tickets[count++]=ticket;
   }
   for(int i=0;i<OrdersTotal();i++)
```

```
   {
      ulong ticket=OrderGetTicket(i);
      if(ticket==0 || OrderGetString(ORDER_SYMBOL)!=Symbol() ||
         OrderGetInteger(ORDER_MAGIC)!=(long)gMagicCore) continue;
      bool known=false;
      for(int j=0;j<count;j++) if(tickets[j]==ticket) { known=true; break; }
      if(!known) tickets[count++]=ticket;
   }
   ulong candidate=0; int matches=0;
   for(int i=0;i<count;i++)
       if(MatchesUnidentifiedEntryOrder(tickets[i]))
       {
          matches++; candidate=tickets[i];
          RememberUnresolvedOrder(tickets[i]);
       }
    if(matches==1)
    {
       // Own symbol/Magic, exact route/mode, request volume/type and uniqueness
       // jointly associate the order; time and BUY/SELL alone never do so.
       gPendingExec.orderTicket=candidate;
       gPendingExec.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
       PersistRequestJournal(false);
    }
}
void ReconcilePendingExecution()
{
  if(!gPendingExec.active) return;
  RecoverUnidentifiedEntryOrder();
  ulong order=gPendingExec.orderTicket;
  if(order==0 && gPendingExec.resultDeal>0 &&
HistoryDealSelect(gPendingExec.resultDeal))
  {
     ulong d=gPendingExec.resultDeal;
     if(HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol() ||
       HistoryDealGetInteger(d,DEAL_MAGIC)!=(long)gMagicCore) return;
     order=(ulong)HistoryDealGetInteger(d,DEAL_ORDER);
     gPendingExec.orderTicket=order;
    }
    if(order==0) return; // Transport uncertainty has no negative proof.
    RememberUnresolvedOrder(order);
}

void ReconcilePendingExitIntent()
{
  if(!gPendingExit.active) return;
  ulong pid=gPendingExit.positionID;
  int idx=FindTrackedTrade(pid);
  if(idx>=0 && !gTradeRecords[idx].closed)
```

```
  {
    if(!RebuildTrackedHistory(idx,false)) return;
    TryFinalizeTrackedTrade(pid);
    if(!gPendingExit.active) return;
  }
  ulong order=gPendingExit.orderTicket;
  if(order==0 && gPendingExit.resultDeal>0 && HistoryDealSelect(gPendingExit.resultDeal))
  {
    ulong d=gPendingExit.resultDeal; long entry=HistoryDealGetInteger(d,DEAL_ENTRY);
    if(HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol() ||
       (ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID)!=pid ||
       (entry!=DEAL_ENTRY_OUT && entry!=DEAL_ENTRY_OUT_BY &&
entry!=DEAL_ENTRY_INOUT)) return;
    order=(ulong)HistoryDealGetInteger(d,DEAL_ORDER); gPendingExit.orderTicket=order;
  }
  if(order==0) return;
  int oi=RememberUnresolvedOrder(order);
  if(oi<0) return;
  UnresolvedOrderContext c=gUnresolvedOrders[oi]; bool live;
  if(!ReadOwnOrder(c,live) || live) return;
  bool entryFill,exitFill,conflict; ulong fillPID; double filled; string route,mode;
  if(!PendingOrderFillEvidence(order,entryFill,exitFill,conflict,fillPID,filled,route,mode) ||
     conflict || entryFill || (fillPID>0 && fillPID!=pid) || !ProvenOrderFillComplete(c,filled))
return;
  bool released=ReleaseOrderObligation(c);
  StoreOrderObligationContext(oi,c,false);
  if(!released) return;
  RestorePendingOrder();
  if(idx>=0 && gTradeRecords[idx].closed)
  { FailExitIntent(pid,true); return; }
  if(!exitFill && c.lastState!=ORDER_STATE_FILLED)
  { FailExitIntent(pid,true); return; }
  if(idx<0 || !RebuildTrackedHistory(idx,false)) return;
  TryFinalizeTrackedTrade(pid);
  if(!gPendingExit.active) return;
  if(gTradeRecords[idx].closed) { FailExitIntent(pid,true); return; }
  TradeRecord r=gTradeRecords[idx]; double liveVol,sl,tp; int direction;
  double remainder=r.totalVolumeIn-r.totalVolumeOut;
  if(r.accountingResolved && remainder>AccountingVolumeTolerance() &&
OwnPositionSafe(pid) &&
    PositionStateByID(pid,liveVol,sl,tp,direction) && direction==r.direction &&
    MathAbs(liveVol-remainder)<=AccountingVolumeTolerance()) FailExitIntent(pid,true);
}


void ReconstructOpenPosition(bool restarting=true)
{
  // A fresh authoritative scan owns this flag; absence alone does not clear a
```

```
 // tracked order/deal ambiguity, which has its own independent context.
 gLiveOwnershipUnresolved=false;
 ulong ids[]; int count=0;
 for(int i=PositionsTotal()-1;i>=0;i--)
 {
    ulong ticket=PositionGetTicket(i);
    if(ticket==0 || PositionGetString(POSITION_SYMBOL)!=Symbol()) continue;
    ulong pid=(ulong)PositionGetInteger(POSITION_IDENTIFIER);
    long liveMagic=PositionGetInteger(POSITION_MAGIC);
int tracked=FindTrackedTrade(pid);
if(!HistorySelectByPosition(pid))
{
   if(gNetting || liveMagic==(long)gMagicCore || tracked>=0)
      gLiveOwnershipUnresolved=true;
   continue;
}
bool ownEntry=false;
for(int j=0;j<HistoryDealsTotal();j++)
{
   ulong d=HistoryDealGetTicket(j);
   if(d>0 && HistoryDealGetString(d,DEAL_SYMBOL)==Symbol() &&
      HistoryDealGetInteger(d,DEAL_MAGIC)==(long)gMagicCore &&
      HistoryDealGetInteger(d,DEAL_ENTRY)==DEAL_ENTRY_IN)
 { ownEntry=true; break; }
}
if(!OwnPositionSafe(pid))
{
   if(gNetting && ownEntry)
   {
      int idx=tracked; bool existing=(idx>=0);
      if(idx<0) idx=NewTrackedTrade(pid);
      if(idx>=0)
      {
         if(!existing) LoadTradeMetadata(gTradeRecords[idx]);
         if(RebuildTrackedHistory(idx,restarting) &&
            CanIgnoreForeignResidual(gTradeRecords[idx]))
         { TryFinalizeTrackedTrade(pid); continue; }
      }
   gLiveOwnershipUnresolved=true;
   continue;
}
if(ownEntry || liveMagic==(long)gMagicCore || tracked>=0)
   gLiveOwnershipUnresolved=true;
continue;
}
if(ArrayResize(ids,count+1)!=count+1)
{ gLiveOwnershipUnresolved=true; continue; }
ids[count++]=pid;
```

```
}
if(count>1) gLiveOwnershipUnresolved=true;
for(int i=0;i<count;i++)
{
   int idx=FindTrackedTrade(ids[i]); bool existing=(idx>=0);
   if(idx<0) idx=NewTrackedTrade(ids[i]);
   if(idx<0) { gLiveOwnershipUnresolved=true; continue; }
   if(!existing) LoadTradeMetadata(gTradeRecords[idx]);
   if(!RebuildTrackedHistory(idx,restarting))
       { gLiveOwnershipUnresolved=true; continue; }
       if(!gTradeRecords[idx].accountingResolved)
       { gLiveOwnershipUnresolved=true; continue; }
       // Keep the one-slot excursion tracker from adopting a second position.
       if(count==1) SyncOpenPositionFromRecord(idx);
       if(restarting && !gTradeRecords[idx].requestPriceAvailable)
          Print("[SCALPER][REQUEST_PRICE_UNAVAILABLE_AFTER_RESTART]");
       if(restarting) Print("[SCALPER][MAE_MFE_PARTIAL_AFTER_RESTART] 1");
    }
}

bool RestoreDailyEconomicsFromHistory(bool restarting=true)
{
  datetime start=AccountingDayStart();
  datetime previousCloseConfirmation=lastCloseConfirmedTime;
  if(!HistorySelect(start,TimeTradeServer()))
  {
     gDailyEconomicsValid=false; gDailyPnLRValid=false;
     gDailyEconomicsState=TEMPORARY_HISTORY_UNAVAILABLE;
     gDailyHistoryRetryRequired=true;
     return false;
  }
  ulong ids[]; int n=0;
  for(int i=0;i<HistoryDealsTotal();i++)
  {
     ulong d=HistoryDealGetTicket(i);
     if(d==0 || HistoryDealGetString(d,DEAL_SYMBOL)!=Symbol()) continue;
     long entry=HistoryDealGetInteger(d,DEAL_ENTRY);
     if(entry!=DEAL_ENTRY_OUT && entry!=DEAL_ENTRY_OUT_BY &&
        entry!=DEAL_ENTRY_INOUT) continue;
     ulong pid=(ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID);
 if(pid==0)
 {
    gDailyEconomicsValid=false; gDailyPnLRValid=false;
    gDailyEconomicsState=TEMPORARY_HISTORY_UNAVAILABLE;
 gDailyHistoryRetryRequired=true;
    return false;
 }
 bool seen=false;
```

```
 for(int j=0;j<n;j++) if(ids[j]==pid) { seen=true; break; }
 if(!seen)
 {
   if(ArrayResize(ids,n+1)!=n+1)
   {
     gDailyEconomicsValid=false; gDailyPnLRValid=false;
     gDailyEconomicsState=DAILY_ECONOMICS_UNRESOLVED;
     return false;
   }
  ids[n++]=pid;
 }
}
int closedIdx[]; int closedCount=0;
int finalizeIdx[]; int finalizeCount=0;
double net=0, sumR=0;
bool rValid=true, allResolved=true, historyUnavailable=false;
bool allocationImpossible=false;
for(int i=0;i<n;i++)
{
  int idx=FindTrackedTrade(ids[i]); bool existing=(idx>=0);
  // First establish ownership from complete per-position history. A
  // manual OUT with another Magic is still part of the proven own cycle.
  if(!HistorySelectByPosition(ids[i]))
{ allResolved=false; historyUnavailable=true; continue; }
bool own=false;
for(int j=0;j<HistoryDealsTotal();j++)
{
   ulong d=HistoryDealGetTicket(j);
   if(HistoryDealGetInteger(d,DEAL_ENTRY)==DEAL_ENTRY_IN &&
      HistoryDealGetInteger(d,DEAL_MAGIC)==(long)gMagicCore &&
      HistoryDealGetString(d,DEAL_SYMBOL)==Symbol())
   { own=true; break; }
}
if(!own) continue;
if(idx<0) idx=NewTrackedTrade(ids[i]);
if(idx<0) { allResolved=false; continue; }
if(!existing) LoadTradeMetadata(gTradeRecords[idx]);
if(!RebuildTrackedHistory(idx,restarting))
 { allResolved=false; historyUnavailable=true; continue; }
 TradeRecord r=gTradeRecords[idx];
  if(!MayFinalizeTrackedCycle(r))
  {
     allResolved=false;
     if(r.inoutCostsUnresolved) allocationImpossible=true;
     continue;
  }
  if(!IsTrackedVolumeCycleComplete(r))
  {
```

```
    if(r.volumeClosedComplete)
    { gNeedsAccountingSync=true; allResolved=false; historyUnavailable=true; }
    continue;
  }
  if(r.exitTime<start) continue;
  // The authoritative aggregate applies the day once; TryFinalize remains
  // the closure authority and therefore cannot add this cycle a second time.
  r.dailyApplied=true;
  if(!r.accountingResolved)
{
   r.netProfit=EMPTY_VALUE; r.PnL_R=EMPTY_VALUE; r.pnlRValid=false;
   allResolved=false; rValid=false;
   if(r.inoutCostsUnresolved) allocationImpossible=true;
   gTradeRecords[idx]=r;
  }
  else
  {
    if(r.riskValid && r.initialRiskMoney>0)
    { r.PnL_R=r.netProfit/r.initialRiskMoney; r.pnlRValid=true; sumR+=r.PnL_R; }
    else { r.PnL_R=EMPTY_VALUE; r.pnlRValid=false; rValid=false; }
    net+=r.netProfit;
    gTradeRecords[idx]=r;
    if(ArrayResize(closedIdx,closedCount+1)!=closedCount+1)
   { allResolved=false; continue; }
   closedIdx[closedCount++]=idx;
   }
   if(ArrayResize(finalizeIdx,finalizeCount+1)!=finalizeCount+1)
   { allResolved=false; continue; }
   finalizeIdx[finalizeCount++]=idx;
   }
   for(int i=1;i<closedCount;i++)
   {
     int x=closedIdx[i],j=i-1;
     while(j>=0 &&
(gTradeRecords[closedIdx[j]].exitTimeMSC>gTradeRecords[x].exitTimeMSC ||
       (gTradeRecords[closedIdx[j]].exitTimeMSC==gTradeRecords[x].exitTimeMSC &&
        (gTradeRecords[closedIdx[j]].lastExitDealTicket>gTradeRecords[x].lastExitDealTicket
||
(gTradeRecords[closedIdx[j]].lastExitDealTicket==gTradeRecords[x].lastExitDealTicket &&
       gTradeRecords[closedIdx[j]].positionID>gTradeRecords[x].positionID)))))
   { closedIdx[j+1]=closedIdx[j]; j--; }
   closedIdx[j+1]=x;
 }
 int losses=0; datetime last=0; bool lastLoss=false;
 double runningR=0; bool runningRValid=true;
 for(int i=0;i<closedCount;i++)
 {
   TradeRecord r=gTradeRecords[closedIdx[i]];
```

```
    lastLoss=BEManagerLoss(r); losses=(lastLoss ? losses+1 : 0); last=r.exitTime;
    if(r.pnlRValid) runningR+=r.PnL_R; else runningRValid=false;
    // Recover an already-hit stop from the historical daily path, even if a
    // subsequent close recovered the total. These are the existing limits.
    if(Inp_EnableDailyRiskStop && allResolved && runningRValid &&
       runningR<=-Inp_MaxDailyLossR)
       gDailyRiskLimitLatched=true;
    if(Inp_EnableConsecutiveLossStop && allResolved &&
       losses>=Inp_MaxConsecutiveLosses)
    gDailyRiskLimitLatched=true;
}
dailyNetPnL=net; dailyPnLR=sumR;
consecutiveLosses=losses;
// A retry must not move a known runtime confirmation clock backwards.

lastCloseConfirmedTime=(previousCloseConfirmation>last?previousCloseConfirmation:last);
  lastTradeWasLoss=lastLoss;
  gDailyEconomicsValid=allResolved;
  gDailyHistoryRetryRequired=historyUnavailable;
  gDailyPnLRValid=(rValid && allResolved);
  gDailyEconomicsState=(allocationImpossible ? ECONOMIC_ALLOCATION_IMPOSSIBLE
:
    (historyUnavailable ? TEMPORARY_HISTORY_UNAVAILABLE :
      (allResolved ? DAILY_ECONOMICS_VALID : DAILY_ECONOMICS_UNRESOLVED)));
    RecomputeDailyRiskBlock();
    for(int i=0;i<finalizeCount;i++)
    {
      int idx=finalizeIdx[i];
      if(!gTradeRecords[idx].closed) TryFinalizeTrackedTrade(gTradeRecords[idx].positionID);
      else FailExitIntent(gTradeRecords[idx].positionID);
    }
    return allResolved;
}

void RestoreDailyPersistence(bool restarting=true)
{
  RestoreUnresolvedEconomicMarkers();
  int historyCount=CountTodayTrades();
  bool committed=GlobalVariableCheck(AccountingKey("VALID")) &&
     GlobalVariableGet(AccountingKey("VALID"))==1.0 &&
     GlobalVariableCheck(GvKeyDailyPnL()) && GlobalVariableCheck(GvKeyDailyPnLR())
&&
     GlobalVariableCheck(GvKeyConsecLoss()) &&
GlobalVariableCheck(GvKeyDailyBlock());
  gQuotaHistoryValid=(historyCount>=0);
  if(gQuotaHistoryValid) ordersEnteredToday=historyCount;
  // A failed read does not adopt an old counter or publish a partial PID set.
  // Entries remain blocked until a complete CountTodayTrades succeeds.
```

```
  if(committed && GlobalVariableCheck(AccountingKey("RISK_LATCH")) &&
     GlobalVariableGet(AccountingKey("RISK_LATCH"))==1.0)
     gDailyRiskLimitLatched=true;
  bool reconstructed=RestoreDailyEconomicsFromHistory(restarting);
  if(!reconstructed && committed)
  {
     dailyNetPnL=GlobalVariableGet(GvKeyDailyPnL());
     dailyPnLR=GlobalVariableGet(GvKeyDailyPnLR());
     consecutiveLosses=(int)GlobalVariableGet(GvKeyConsecLoss());
     // A committed snapshot can predate an unprocessed close. It is partial
     // evidence and cannot restore validity while history is unavailable.
     gDailyEconomicsValid=false; gDailyPnLRValid=false;
     if(GlobalVariableCheck(AccountingKey("CLOSE")))
        lastCloseConfirmedTime=(datetime)GlobalVariableGet(AccountingKey("CLOSE"));
     if(GlobalVariableCheck(AccountingKey("LASTLOSS")))
        lastTradeWasLoss=(GlobalVariableGet(AccountingKey("LASTLOSS"))==1.0);
    }
    ReconstructOpenPosition(restarting);
    if(restarting || !gPendingExec.active) RestorePendingOrder();
    RecomputeDailyRiskBlock(); RecomputeAccountingBlock();
    if(gQuotaHistoryValid && reconstructed) PersistDailyAccounting();
}
// RC1.2 infrastructure helpers; final EA lifecycle follows below.
void RememberDeferredAccountingPID(ulong pid)
{
   for(int i=0;i<ArraySize(gDeferredAccountingPIDs);i++)
      if(gDeferredAccountingPIDs[i]==pid) return;
   int n=ArraySize(gDeferredAccountingPIDs);
   if(ArrayResize(gDeferredAccountingPIDs,n+1)!=n+1)
   {
      bool seen=false;
      for(int i=0;i<gDeferredFallbackCount;i++) if(gDeferredFallback[i]==pid) { seen=true;
break; }
      if(!seen)
      {
      if(gDeferredFallbackCount<MAX_TRADE_RECORDS)
gDeferredFallback[gDeferredFallbackCount++]=pid;
      else
      {
         gDeferredRegistryOverflow=true;
         MarkInfrastructureIdentityLoss();
      }
    }
    gDeferredRegistryUnavailable=true;
  }
  else gDeferredAccountingPIDs[n]=pid;
  gAccountingCapacityUnresolved=true;
}
```

```
void ForgetDeferredAccountingPID(ulong pid)
{
  int n=ArraySize(gDeferredAccountingPIDs);
  for(int i=0;i<n;i++)
    if(gDeferredAccountingPIDs[i]==pid)
    {
       for(int j=i+1;j<n;j++) gDeferredAccountingPIDs[j-1]=gDeferredAccountingPIDs[j];
    ArrayResize(gDeferredAccountingPIDs,n-1); break;
   }
  gAccountingCapacityUnresolved=(gDeferredRegistryUnavailable ||
ArraySize(gDeferredAccountingPIDs)>0);
  for(int i=0;i<gDeferredFallbackCount;i++)
   if(gDeferredFallback[i]==pid)
   {
      for(int j=i+1;j<gDeferredFallbackCount;j++)
gDeferredFallback[j-1]=gDeferredFallback[j];
      gDeferredFallbackCount--; break;
   }
  gDeferredRegistryUnavailable=gDeferredRegistryOverflow || gDeferredFallbackCount>0;
  gAccountingCapacityUnresolved=gDeferredRegistryUnavailable ||
ArraySize(gDeferredAccountingPIDs)>0;
}
bool IsFinalRejectedRetcode(uint ret)
{
  switch(ret)
  {
    case TRADE_RETCODE_REQUOTE: case TRADE_RETCODE_REJECT:
    case TRADE_RETCODE_CANCEL: case TRADE_RETCODE_INVALID:
    case TRADE_RETCODE_INVALID_VOLUME: case
TRADE_RETCODE_INVALID_PRICE:
    case TRADE_RETCODE_INVALID_STOPS: case
TRADE_RETCODE_TRADE_DISABLED:
    case TRADE_RETCODE_MARKET_CLOSED: case TRADE_RETCODE_NO_MONEY:
    case TRADE_RETCODE_PRICE_CHANGED: case TRADE_RETCODE_PRICE_OFF:
    case TRADE_RETCODE_INVALID_EXPIRATION: case
TRADE_RETCODE_TOO_MANY_REQUESTS:
    case TRADE_RETCODE_SERVER_DISABLES_AT: case
TRADE_RETCODE_CLIENT_DISABLES_AT:
    case TRADE_RETCODE_INVALID_FILL: case TRADE_RETCODE_ONLY_REAL:
    case TRADE_RETCODE_LIMIT_ORDERS: case TRADE_RETCODE_LIMIT_VOLUME:
    case TRADE_RETCODE_INVALID_ORDER: case
TRADE_RETCODE_POSITION_CLOSED:
    case TRADE_RETCODE_INVALID_CLOSE_VOLUME: case
TRADE_RETCODE_LIMIT_POSITIONS:
    case TRADE_RETCODE_LONG_ONLY: case TRADE_RETCODE_SHORT_ONLY:
    case TRADE_RETCODE_CLOSE_ONLY: case TRADE_RETCODE_FIFO_CLOSE:
    case TRADE_RETCODE_HEDGE_PROHIBITED: return true;
```

```
  }
  return false; // Unknown, LOCKED, ERROR and CLOSE_ORDER_EXIST are uncertain.
}

void CompletePendingExit(const MqlTradeResult &result)
{
  if(!gPendingExit.active) return;
    gPendingExit.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
    if(result.request_id>0) gPendingExit.requestID=result.request_id;
    if(result.order>0) gPendingExit.orderTicket=result.order;
    if(result.deal>0) gPendingExit.resultDeal=result.deal;
    if(IsAcceptedRetcode(result.retcode))
    { gPendingExit.accepted=true; gPendingExit.uncertain=false; }
    else if(IsFinalRejectedRetcode(result.retcode) && !gPendingExit.accepted &&
       gPendingExit.orderTicket==0 && gPendingExit.resultDeal==0)
       FailExitIntent(gPendingExit.positionID,true);
    else gPendingExit.uncertain=true;
    if(gPendingExit.active) PersistRequestJournal(true);
    gNeedsAccountingSync=true;
}
bool MatchesExitRequest(const MqlTradeRequest &request,const MqlTradeResult &result)
{
 if(!gPendingExit.active || gPendingExit.positionTicket==0 ||
    gPendingExit.requestID==0 || result.request_id!=gPendingExit.requestID) return false;
 return request.position==gPendingExit.positionTicket && request.symbol==Symbol() &&
    request.magic==(ulong)gMagicCore && request.action==TRADE_ACTION_DEAL &&
    request.position_by==0 && request.type==gPendingExit.requestSnapshot.type &&

MathAbs(request.volume-gPendingExit.requestSnapshot.volume)<=AccountingVolumeToler
ance();
}
void SyncAccountingState(bool reconcilePending=false)
{
  // Tick path is a cheap projection only. No history selection/finalization.
  if(!reconcilePending) { RecomputeAccountingBlock(); return; }
  datetime now=TimeTradeServer();
  if(now<gNextOrderReconcile) { RecomputeAccountingBlock(); return; }
  gNextOrderReconcile=now+5; // Timer or transaction transition, never each tick.
  RecoverTradeCsvInfrastructure();
  ReconcileDeferredDeals();
  RecoverAccountingInfrastructure();
  ReconcilePendingExecution();
  ReconcileUnresolvedOrders();
  ReconcilePendingExitIntent();
  for(int i=0;i<gTradeRecordCount;i++)
  {
     if(gTradeRecords[i].historyRefreshPending ||
        (!gTradeRecords[i].closed && !gTradeRecords[i].accountingResolved &&
```

```
        !gTradeRecords[i].inoutCostsUnresolved))
     {
       if(RebuildTrackedHistory(i,false)) SyncOpenPositionFromRecord(i);
     }
     if(!gTradeRecords[i].closed) TryFinalizeTrackedTrade(gTradeRecords[i].positionID);
     else if(gPendingExit.active && gPendingExit.positionID==gTradeRecords[i].positionID)
       FailExitIntent(gPendingExit.positionID);
    }
    gNeedsAccountingSync=false;
    if(csvTradeHandle!=INVALID_HANDLE)
      for(int i=0;i<gTradeRecordCount;i++)
        if(gTradeRecords[i].closed && !gTradeRecords[i].finalizationWritten &&
           MayFinalizeTrackedCycle(gTradeRecords[i]))
        {
          if(WriteTradeRow(gTradeRecords[i]))
          {
            gTradeRecords[i].finalizationWritten=true;
            SaveTradeMetadata(gTradeRecords[i]); SyncEconomicMarker(i);
          }
        }
    RecomputeDailyRiskBlock();
    RecomputeAccountingBlock();
}

void RecomputeAccountingBlock()
{
  gAccountingCapacityUnresolved=(ArraySize(gDeferredAccountingPIDs)>0 ||
   gDeferredRegistryUnavailable);
  bool blocked=gPendingOrderClassificationUnresolved || !gQuotaHistoryValid ||
   gLiveOwnershipUnresolved || gAccountingCapacityUnresolved ||
   gMultipleOwnOrdersActive || gOrderRegistryUnavailable ||
   gOrderObligationScanUnresolved ||
    ArraySize(gDeferredDeals)>0 || gDeferredDealFallbackCount>0 ||
gDeferredDealOverflow ||
    !gDailyEconomicsValid || gEconomicMarkerScanUnresolved ||
    gIdentityLossUnresolved || gRequestJournalUnresolved ||
    gRequestJournalWritePending || gEntryJournalCleanupPending ||
gExitJournalCleanupPending;
  for(int i=0;i<gTradeRecordCount;i++)
  {
    TradeRecord r=gTradeRecords[i];
    if(r.positionID==0) continue;
    // An INOUT cost that cannot be allocated is an explicit permanent
    // economic ambiguity; midnight cannot invent the missing allocation.
    bool relevant=!r.closed || r.exitTime>=AccountingDayStart() ||
      r.inoutCostsUnresolved;
    if(r.economicMarkerPending) blocked=true;
    if(r.breakEvenClassificationPending) blocked=true;
```

```
    if(relevant && (r.historyRefreshPending || !r.accountingResolved ||
       r.inoutCostsUnresolved)) blocked=true;
  }
  for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
    if(gUnresolvedOrders[i].active && (gUnresolvedOrders[i].classificationPending ||
       gUnresolvedOrders[i].persistencePending)) blocked=true;
    // The only assignment to the aggregate blocker is here.
    gAccountingUnresolved=blocked;
}

void RecomputeDailyRiskBlock()
{
  // Preserve the existing daily stop latch for an actual economic breach.
  // History validity itself is recomputed and never latched as a breach.
  if(Inp_EnableDailyRiskStop && gDailyPnLRValid &&
     dailyPnLR<=-Inp_MaxDailyLossR) gDailyRiskLimitLatched=true;
  if(Inp_EnableConsecutiveLossStop && gDailyEconomicsValid &&
     consecutiveLosses>=Inp_MaxConsecutiveLosses) gDailyRiskLimitLatched=true;
  // Match RC1.1: a persisted daily stop cannot enable disabled controls.
  dailyRiskBlocked=(gDailyRiskLimitLatched &&
    (Inp_EnableDailyRiskStop || Inp_EnableConsecutiveLossStop)) ||
    (Inp_EnableDailyRiskStop && !gDailyPnLRValid) ||
    (Inp_EnableConsecutiveLossStop && !gDailyEconomicsValid);
}

void RecoverAccountingInfrastructure()
{
  bool needsQuota=!gQuotaHistoryValid;
  bool needsEconomics=gDailyHistoryRetryRequired ||
   (gDailyEconomicsState!=ECONOMIC_ALLOCATION_IMPOSSIBLE &&
    (!gDailyEconomicsValid || !gDailyPnLRValid));
  bool needsRecords=false;
bool needsMarkers=gEMRMaintenanceNeeded || gEconomicMarkerScanUnresolved ||
gIdentityLossPersistPending ||
   gOrderObligationScanUnresolved || gOrderRegistryUnavailable ||
   gRequestJournalUnresolved || gRequestJournalWritePending ||
   gEntryJournalCleanupPending || gExitJournalCleanupPending;
for(int i=0;i<gTradeRecordCount;i++)
   if(gTradeRecords[i].economicMarkerPending ||
      gTradeRecords[i].economicMarkerCleanupPending) needsMarkers=true;
for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
   if(gUnresolvedOrders[i].active && gUnresolvedOrders[i].persistencePending)
needsMarkers=true;
for(int i=0;i<gTradeRecordCount;i++)
{
   TradeRecord r=gTradeRecords[i];
   if(r.positionID>0 && !r.inoutCostsUnresolved &&
      (r.historyRefreshPending || (!r.closed && !r.accountingResolved)))
```

```
   { needsRecords=true; break; }
}
if(!needsQuota && !needsEconomics && !needsRecords && !needsMarkers &&
   !gLiveOwnershipUnresolved && !gAccountingCapacityUnresolved) return;
datetime now=TimeTradeServer();
if(gLastAccountingRecoveryTime>0 && now-gLastAccountingRecoveryTime<10) return;
gLastAccountingRecoveryTime=now;
if(gIdentityLossPersistPending) PersistInfrastructureIdentityLoss();
RecoverRequestJournals();
if(gOrderObligationScanUnresolved || gOrderRegistryUnavailable)
RestoreOrderObligations();
RecoverEconomicMarkers();
// Snapshot deferred identities because NewTrackedTrade removes each PID
// after successful allocation. Failed allocations never forget the PID.
// No snapshot allocation is necessary: iterate backwards and remove only
// the exact PID recovered by NewTrackedTrade/ForgetDeferredAccountingPID.
for(int list=0;list<2;list++)
{
   int
deferredCount=list==0?ArraySize(gDeferredAccountingPIDs):gDeferredFallbackCount;
for(int i=deferredCount-1;i>=0;i--)
{
  ulong pid=list==0?gDeferredAccountingPIDs[i]:gDeferredFallback[i];
  if(!HistorySelectByPosition(pid)) continue;
  bool own=false;
  for(int j=0;j<HistoryDealsTotal();j++)
  {
     ulong deal=HistoryDealGetTicket(j);
     if(deal>0 && HistoryDealGetString(deal,DEAL_SYMBOL)==Symbol() &&
        HistoryDealGetInteger(deal,DEAL_MAGIC)==(long)gMagicCore &&
        HistoryDealGetInteger(deal,DEAL_ENTRY)==DEAL_ENTRY_IN)
     { own=true; break; }
  }
  if(!own) continue;
   int idx=FindTrackedTrade(pid); bool existing=(idx>=0);
   if(idx<0) idx=NewTrackedTrade(pid);
   if(idx<0) continue;
   ForgetDeferredAccountingPID(pid);
   if(!existing) LoadTradeMetadata(gTradeRecords[idx]);
   if(RebuildTrackedHistory(idx,false))
   {
      SyncOpenPositionFromRecord(idx);
      TryFinalizeTrackedTrade(pid); gNeedsAccountingSync=true;
   }
 }
}
if(needsQuota)
{
```

```
   int count=CountTodayTrades();
   if(count>=0) { ordersEnteredToday=count; gQuotaHistoryValid=true; }
}
if(needsRecords)
{
   for(int i=0;i<gTradeRecordCount;i++)
 {
     TradeRecord r=gTradeRecords[i];
     if(r.positionID==0 || r.inoutCostsUnresolved ||
        !(r.historyRefreshPending || (!r.closed && !r.accountingResolved))) continue;
     if(RebuildTrackedHistory(i,false))
     { SyncOpenPositionFromRecord(i); gNeedsAccountingSync=true; }
 }
}
if(gLiveOwnershipUnresolved || gAccountingCapacityUnresolved)
   ReconstructOpenPosition(false);
// Record repair can invalidate or recover today's economics. Rebuild the
// current-day window only; never query the account from time zero here.
if(needsEconomics || needsRecords || !gDailyEconomicsValid)
   RestoreDailyEconomicsFromHistory(false);
   RecomputeDailyRiskBlock(); RecomputeAccountingBlock();
   if(gQuotaHistoryValid && gDailyEconomicsValid) PersistDailyAccounting();
}

void RecoverTradeCsvInfrastructure()
{
  if(gTradeCsvFileName=="" ||
     (csvTradeHandle!=INVALID_HANDLE && gTradeCsvIndexLoaded &&
gTradeCsvAppendSafe)) return;
  datetime now=TimeTradeServer();
  if(now<gNextTradeCsvRecovery) return;
  gNextTradeCsvRecovery=now+30;
  // Reopen the current path after unavailable I/O or external repair. Never
  // truncate/repair an existing file, and never trust metadata as CSV presence.
if(csvTradeHandle!=INVALID_HANDLE)
{ FileClose(csvTradeHandle); csvTradeHandle=INVALID_HANDLE; }
csvTradeHandle=FileOpen(gTradeCsvFileName,

FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_SHARE_READ|FILE_SHARE_WRIT
E);
 if(csvTradeHandle==INVALID_HANDLE) return;
 ResetLastError();
 ulong size=FileSize(csvTradeHandle);
 if(GetLastError()!=0)
 { FileClose(csvTradeHandle); csvTradeHandle=INVALID_HANDLE; return; }
 if(size==0)
 {
    WriteTradeHeader();
```

```
   FileFlush(csvTradeHandle);
 }
 // Invalid schema/tail stays append-disabled. A later full successful scan
 // can recognize an authoritative external repair without destroying rows.
 if(!InitializeTradeCsvIndex(gTradeCsvFileName)) return;
 if(!FileSeek(csvTradeHandle,0,SEEK_END)) gTradeCsvAppendSafe=false;
}

void MarkTemporaryAccountingHistory()
{
  gDailyHistoryRetryRequired=true;
  gDailyEconomicsValid=false; gDailyPnLRValid=false;
  if(gDailyEconomicsState!=ECONOMIC_ALLOCATION_IMPOSSIBLE)
     gDailyEconomicsState=TEMPORARY_HISTORY_UNAVAILABLE;
}

bool ProcessAccountingDeal(ulong ticket)
{
  if(!HistoryDealSelect(ticket)) return false;
  if(HistoryDealGetString(ticket,DEAL_SYMBOL)!=Symbol()) return true;
  long type=HistoryDealGetInteger(ticket,DEAL_TYPE);
  if(type!=DEAL_TYPE_BUY && type!=DEAL_TYPE_SELL) return true;
  long entry=HistoryDealGetInteger(ticket,DEAL_ENTRY);
  ulong pid=(ulong)HistoryDealGetInteger(ticket,DEAL_POSITION_ID);
  long magic=HistoryDealGetInteger(ticket,DEAL_MAGIC);
  if(pid==0) return false;
  int idx=FindTrackedTrade(pid);
  bool ownNew=(entry==DEAL_ENTRY_IN && magic==(long)gMagicCore);
  if(idx<0 && ownNew)
  {
     idx=NewTrackedTrade(pid);
     if(idx>=0) LoadTradeMetadata(gTradeRecords[idx]);
  }
  if(idx<0 && (entry==DEAL_ENTRY_OUT || entry==DEAL_ENTRY_OUT_BY ||
entry==DEAL_ENTRY_INOUT))
  {
     // Unavailable ownership history is not a proven foreign cycle.
     if(!HistorySelectByPosition(pid)) return false;
     bool own=false;
     for(int i=0;i<HistoryDealsTotal();i++)
     {
        ulong d=HistoryDealGetTicket(i);
        if(HistoryDealGetString(d,DEAL_SYMBOL)==Symbol() &&
           HistoryDealGetInteger(d,DEAL_MAGIC)==(long)gMagicCore &&
           HistoryDealGetInteger(d,DEAL_ENTRY)==DEAL_ENTRY_IN)
        { own=true; break; }
     }
     if(!own) return true;
```

```
     idx=NewTrackedTrade(pid);
     if(idx>=0)
     { LoadTradeMetadata(gTradeRecords[idx]);
gTradeRecords[idx].maeMfePartialAfterRestart=true; }
  }
  if(idx<0) return !ownNew; // Capacity failure retains own IN identity for retry.
    if(gTradeRecords[idx].closed) return true;
    if(!RebuildTrackedHistory(idx,false)) return false;
    SyncOpenPositionFromRecord(idx); TryFinalizeTrackedTrade(pid);
    return true;
}

void RememberDeferredDeal(ulong ticket)
{
  MarkTemporaryAccountingHistory();
  // Keep the exact event identity even if no position or own record is visible.
  for(int i=0;i<ArraySize(gDeferredDeals);i++) if(gDeferredDeals[i]==ticket) return;
  for(int i=0;i<gDeferredDealFallbackCount;i++) if(gDeferredDealFallback[i]==ticket) return;
  int n=ArraySize(gDeferredDeals);
  if(ArrayResize(gDeferredDeals,n+1)==n+1) gDeferredDeals[n]=ticket;
    else if(gDeferredDealFallbackCount<MAX_TRADE_RECORDS)
     gDeferredDealFallback[gDeferredDealFallbackCount++]=ticket;
    else
    {
      gDeferredDealOverflow=true;
      MarkInfrastructureIdentityLoss();
    }
}

void ReconcileDeferredDeals()
{
  for(int i=ArraySize(gDeferredDeals)-1;i>=0;i--)
   if(ProcessAccountingDeal(gDeferredDeals[i]))
    {
       int n=ArraySize(gDeferredDeals);
       for(int j=i+1;j<n;j++) gDeferredDeals[j-1]=gDeferredDeals[j];
       ArrayResize(gDeferredDeals,n-1);
     }
    for(int i=gDeferredDealFallbackCount-1;i>=0;i--)
     if(ProcessAccountingDeal(gDeferredDealFallback[i]))
     {
        for(int j=i+1;j<gDeferredDealFallbackCount;j++)
         gDeferredDealFallback[j-1]=gDeferredDealFallback[j];
        gDeferredDealFallbackCount--;
     }
}

string EconomicMarkerPrefix()
```

```
{
  return StringFormat("Scalper01_RC1_%I64d_%s_%I64d_",
   AccountInfoInteger(ACCOUNT_LOGIN),Symbol(),(long)gMagicCore);
}

string EconomicMarkerPath(ulong pid)
{ return EconomicMarkerPrefix()+(string)pid+".econ.partial"; }

string IdentityLossMarkerPath()
{ return EconomicMarkerPrefix()+"identity.loss"; }

void PersistInfrastructureIdentityLoss()
{
  string path=IdentityLossMarkerPath();
  if(FileIsExist(path)) { gIdentityLossPersistPending=false; return; }
  gIdentityLossPersistPending=true;
  int h=FileOpen(path,FILE_WRITE|FILE_TXT|FILE_ANSI);
  if(h==INVALID_HANDLE) return;
  string message="RC1_IDENTITY_EVIDENCE_LOST\r\n";
  ResetLastError();
    uint written=FileWriteString(h,message);
    bool ok=(written==(uint)StringLen(message) && GetLastError()==0);
    ResetLastError(); FileFlush(h);
    if(GetLastError()!=0) ok=false;
    FileClose(h);
    gIdentityLossPersistPending=!ok;
}

void MarkInfrastructureIdentityLoss()
{
  // If even the emergency identity registry is exhausted, restart cannot
  // prove that the forgotten evidence was resolved. Persist that obligation.
  gIdentityLossUnresolved=true;
    PersistInfrastructureIdentityLoss();
}

bool PersistEconomicMarker(ulong pid)
{
  string path=EconomicMarkerPath(pid);
  // Presence is an unresolved obligation, not proof of economic resolution.
  // Even an interrupted marker write must be rebuilt by its exact PID.
  if(FileIsExist(path)) return true;
  int h=FileOpen(path,FILE_WRITE|FILE_TXT|FILE_ANSI);
  if(h==INVALID_HANDLE) return false;
  ResetLastError();
  uint written=FileWriteString(h,"RC1_ECONOMIC_OBLIGATION\r\n");
  bool ok=(written==25 && GetLastError()==0);
  ResetLastError(); FileFlush(h);
```

```
  if(GetLastError()!=0) ok=false;
  FileClose(h);
  return ok;
}

void RestoreUnresolvedEconomicMarkers()
{
  // PID0 is a sentinel: a successful scan must find it. Thus enumeration
  // failure cannot masquerade as an empty registry and release the blocker.
  gEconomicMarkerScanUnresolved=false;
  if(!PersistEconomicMarker(0)) { gEconomicMarkerScanUnresolved=true; return; }
  string name, prefix=EconomicMarkerPrefix(), suffix=".econ.partial";
  long search=FileFindFirst(prefix+"*"+suffix,name);
  if(search==INVALID_HANDLE) { gEconomicMarkerScanUnresolved=true; return; }
  bool sentinel=false;
  ulong markerPIDs[]; int markerCount=0;
  bool more=true;
  while(more)
  {
    if(name==prefix+"0"+suffix) sentinel=true;
       else
       {
        int length=StringLen(name)-StringLen(prefix)-StringLen(suffix);
        ulong pid=0;
        if(length<=0 || StringSubstr(name,0,StringLen(prefix))!=prefix ||
           StringSubstr(name,StringLen(name)-StringLen(suffix))!=suffix ||
           !CsvParsePID(StringSubstr(name,StringLen(prefix),length),pid))
           gEconomicMarkerScanUnresolved=true;
        else if(ArrayResize(markerPIDs,markerCount+1)!=markerCount+1)
           gEconomicMarkerScanUnresolved=true;
        else markerPIDs[markerCount++]=pid;
       }
       ResetLastError();
       more=FileFindNext(search,name);
       // No guessed EOF error code: an explicit enumeration error cannot
       // certify a complete registry. Retry later rather than release it.
       if(!more && GetLastError()!=0) gEconomicMarkerScanUnresolved=true;
      }
      FileFindClose(search);
      if(!sentinel) gEconomicMarkerScanUnresolved=true;
      // Snapshot filenames before rebuild can remove an authoritatively resolved marker.
      for(int i=0;i<markerCount;i++)
      {
        ulong pid=markerPIDs[i];
        int idx=FindTrackedTrade(pid); bool existing=(idx>=0);
        if(idx<0) idx=NewTrackedTrade(pid);
        if(idx<0) { gEconomicMarkerScanUnresolved=true; continue; }
        if(!existing) LoadTradeMetadata(gTradeRecords[idx]);
```

```
         gTradeRecords[idx].economicMarkerPending=true;
         if(!RebuildTrackedHistory(idx,false))
         { gEconomicMarkerScanUnresolved=true; continue; }
         if(!gTradeRecords[idx].volumeClosedComplete)
         {
           gTradeRecords[idx].historyRefreshPending=true;
         MarkTemporaryAccountingHistory();
        }
        TryFinalizeTrackedTrade(pid);
   }
}

void SyncEconomicMarker(int idx)
{
 TradeRecord r=gTradeRecords[idx];
 // A prior administrative classification is invalidated by any fresh rebuild.
 if(r.economicMarkerCleanupPending) gTradeRecords[idx].economicMarkerPending=true;
 if(r.inoutCostsUnresolved || (r.totalVolumeIn>0 &&
    (!r.closed || !r.finalizationWritten)))
 {
 gTradeRecords[idx].economicMarkerCleanupPending=false;
 bool durableIdentity=PersistEconomicMarker(r.positionID);
 // Persisting an obligation is not proof that its finalization is durable.
 gTradeRecords[idx].economicMarkerPending=!durableIdentity ||
   r.inoutCostsUnresolved || (r.closed && !r.finalizationWritten);
 }
 else if(r.closed && r.finalizationWritten && r.accountingResolved &&
  r.volumeClosedComplete)
 {
  // Never remove the durable PID while its own reconstruction is uncertain.
  if(r.historyRefreshPending || r.breakEvenClassificationPending ||
     r.be.stage==BE_REQUEST_PENDING || r.be.stage==BE_ACKNOWLEDGED ||
     r.be.stage==BE_UNRESOLVED)
  {
     gTradeRecords[idx].economicMarkerPending=true;
     if(!PersistEconomicMarker(r.positionID)) gEconomicMarkerScanUnresolved=true;
     return;
  }
  string path=EconomicMarkerPath(r.positionID);
  ResetLastError(); bool present=FileIsExist(path);
  int probeError=GetLastError();
  if(!present && probeError!=0)
  {
     bool confirmed=false;
     if(EMRProbeMarker(r.positionID,confirmed))
     { present=confirmed; probeError=0; }
  }
  bool removed=false;
```

```
   int error=probeError;
   if(present)
   {
     ResetLastError(); removed=FileDelete(path);
     error=GetLastError();
   }
   if((present && !removed) || (!present && probeError!=0) ||
     (!present && r.economicMarkerCleanupPending))
   {
     gTradeRecords[idx].economicMarkerPending=true;
     gTradeRecords[idx].economicMarkerCleanupPending=true;
     gTradeRecords[idx].economicMarkerDeleteError=error;
     if(!r.economicMarkerCleanupPending || r.economicMarkerDeleteError!=error)
       PrintFormat("[SCALPER][ECONOMIC_MARKER_CLEANUP_FAILED] pid=%I64u
file=%s error=%d; authoritative resolution proof required; evidence retained.",
         r.positionID,path,error);
   }
   else
   {
     gTradeRecords[idx].economicMarkerPending=false;
        gTradeRecords[idx].economicMarkerCleanupPending=false;
        gTradeRecords[idx].economicMarkerDeleteError=0;
      }
   }
}

void RecoverEconomicMarkers()
{
 if(gEconomicMarkerScanUnresolved) RestoreUnresolvedEconomicMarkers();
 for(int i=0;i<gTradeRecordCount;i++)
 {
    if(!gTradeRecords[i].economicMarkerPending &&
      !gTradeRecords[i].economicMarkerCleanupPending) continue;
    if(gTradeRecords[i].closed && gTradeRecords[i].finalizationWritten)
      EMRRecoverRecord(i);
    else SyncEconomicMarker(i);
 }
 EMRMaintainArchivedCleanup();
 EMRReleaseArchivedRecords();
}

// Economic-marker certification only; this does not authorize an order.
// Mixed/reversal/INOUT histories are deliberately outside this administrative
// exception. Their original economic and ownership protections remain active.
#define EMR_MAX_HISTORY_DEALS 4096
#define EMR_MAX_HISTORY_ORDERS 512
#define EMR_MAX_LIVE_ITEMS 4096
```

```
struct EMRDealSnapshot
{
   ulong ticket, order;
   long timeMSC, timeSec, magic, type, entry;
   double volume, price, profit, commission, fee, swap;
};
struct EMROrderSnapshot
{
   ulong ticket, pid, by;
   long state, type, magic, setupMSC, doneMSC;
   double initial, remaining, sl, tp, dealVolume, entryVolume, exitVolume;
};

bool EMRDealI(ulong ticket,ENUM_DEAL_PROPERTY_INTEGER property,long &value)
{
  ResetLastError();
  return HistoryDealGetInteger(ticket,property,value) && GetLastError()==0;
}
bool EMRDealD(ulong ticket,ENUM_DEAL_PROPERTY_DOUBLE property,double &value)
{
 ResetLastError();
 return HistoryDealGetDouble(ticket,property,value) && GetLastError()==0 &&
 MathIsValidNumber(value) && value!=EMPTY_VALUE;
}
bool EMRDealS(ulong ticket,ENUM_DEAL_PROPERTY_STRING property,string &value)
{
   ResetLastError();
   return HistoryDealGetString(ticket,property,value) && GetLastError()==0;
}
bool EMROrderI(ulong ticket,ENUM_ORDER_PROPERTY_INTEGER property,long &value)
{
   ResetLastError();
   return HistoryOrderGetInteger(ticket,property,value) && GetLastError()==0;
}
bool EMROrderD(ulong ticket,ENUM_ORDER_PROPERTY_DOUBLE property,double
&value)
{
   ResetLastError();
   return HistoryOrderGetDouble(ticket,property,value) && GetLastError()==0 &&
    MathIsValidNumber(value) && value!=EMPTY_VALUE;
}
bool EMROrderS(ulong ticket,ENUM_ORDER_PROPERTY_STRING property,string &value)
{
   ResetLastError();
   return HistoryOrderGetString(ticket,property,value) && GetLastError()==0;
}
bool EMRFinite(double value)
{ return MathIsValidNumber(value) && value!=EMPTY_VALUE; }
```

```
bool EMRNear(double first,double second,double tolerance)
{
   return EMRFinite(first) && EMRFinite(second) &&
    MathAbs(first-second)<=MathMax(tolerance,
     MathMax(MathAbs(first),MathAbs(second))*1e-11);
}
string EMRD(double value) { return DoubleToString(value,16); }

bool EMRReadDeal(ulong ticket,ulong positionID,EMRDealSnapshot &d)
{
  string symbol; long pid, order;
  d.ticket=ticket;
  if(ticket==0 || !EMRDealS(ticket,DEAL_SYMBOL,symbol) || symbol!=Symbol() ||
     !EMRDealI(ticket,DEAL_POSITION_ID,pid) || (ulong)pid!=positionID ||
     !EMRDealI(ticket,DEAL_ORDER,order) || order<=0 ||
     !EMRDealI(ticket,DEAL_TIME_MSC,d.timeMSC) || d.timeMSC<=0 ||
     !EMRDealI(ticket,DEAL_TIME,d.timeSec) || d.timeSec<=0 ||
     d.timeMSC/1000!=d.timeSec ||
     !EMRDealI(ticket,DEAL_MAGIC,d.magic) ||
     !EMRDealI(ticket,DEAL_TYPE,d.type) ||
      (d.type!=DEAL_TYPE_BUY && d.type!=DEAL_TYPE_SELL) ||
      !EMRDealI(ticket,DEAL_ENTRY,d.entry) ||
     (d.entry!=DEAL_ENTRY_IN && d.entry!=DEAL_ENTRY_OUT &&
      d.entry!=DEAL_ENTRY_OUT_BY) ||
     !EMRDealD(ticket,DEAL_VOLUME,d.volume) || d.volume<=0 ||
     !EMRDealD(ticket,DEAL_PRICE,d.price) || d.price<=0 ||
     !EMRDealD(ticket,DEAL_PROFIT,d.profit) ||
     !EMRDealD(ticket,DEAL_COMMISSION,d.commission) ||
     !EMRDealD(ticket,DEAL_FEE,d.fee) ||
     !EMRDealD(ticket,DEAL_SWAP,d.swap)) return false;
    d.order=(ulong)order;
    return d.entry!=DEAL_ENTRY_IN || d.magic==(long)gMagicCore;
}

string EMRDealEvidence(const EMRDealSnapshot &d)
{
  return (string)d.ticket+"|"+(string)d.order+"|"+(string)d.timeMSC+"|"+
   (string)d.timeSec+"|"+(string)d.magic+"|"+(string)d.type+"|"+
   (string)d.entry+"|"+EMRD(d.volume)+"|"+EMRD(d.price)+"|"+
   EMRD(d.profit)+"|"+EMRD(d.commission)+"|"+EMRD(d.fee)+"|"+EMRD(d.swap);
}

bool EMRReadOrder(ulong ticket,ulong positionID,EMROrderSnapshot &o)
{
  string symbol; long pid, by;
  ZeroMemory(o); o.ticket=ticket;
  if(ticket==0 || !EMROrderS(ticket,ORDER_SYMBOL,symbol) || symbol!=Symbol() ||
     !EMROrderI(ticket,ORDER_POSITION_ID,pid) || pid<0 ||
```

```
     !EMROrderI(ticket,ORDER_POSITION_BY_ID,by) || by<0 ||
     ((ulong)pid!=positionID && (ulong)by!=positionID) ||
     !EMROrderI(ticket,ORDER_STATE,o.state) ||
     (o.state!=ORDER_STATE_FILLED && o.state!=ORDER_STATE_CANCELED &&
      o.state!=ORDER_STATE_REJECTED && o.state!=ORDER_STATE_EXPIRED) ||
     !EMROrderI(ticket,ORDER_TYPE,o.type) ||
      (o.type!=ORDER_TYPE_BUY && o.type!=ORDER_TYPE_SELL &&
       o.type!=ORDER_TYPE_CLOSE_BY) ||
      !EMROrderI(ticket,ORDER_MAGIC,o.magic) ||
      !EMROrderI(ticket,ORDER_TIME_SETUP_MSC,o.setupMSC) || o.setupMSC<=0 ||
      !EMROrderI(ticket,ORDER_TIME_DONE_MSC,o.doneMSC) || o.doneMSC<=0 ||
      o.doneMSC<o.setupMSC ||
      !EMROrderD(ticket,ORDER_VOLUME_INITIAL,o.initial) || o.initial<=0 ||
      !EMROrderD(ticket,ORDER_VOLUME_CURRENT,o.remaining) || o.remaining<0 ||
      !EMROrderD(ticket,ORDER_SL,o.sl) || o.sl<0 ||
      !EMROrderD(ticket,ORDER_TP,o.tp) || o.tp<0) return false;
    o.pid=(ulong)pid; o.by=(ulong)by;
    return true;
}
string EMROrderEvidence(const EMROrderSnapshot &o)
{
  return (string)o.ticket+"|"+(string)o.pid+"|"+(string)o.by+"|"+
    (string)o.state+"|"+(string)o.type+"|"+(string)o.magic+"|"+
    (string)o.setupMSC+"|"+(string)o.doneMSC+"|"+EMRD(o.initial)+"|"+
    EMRD(o.remaining)+"|"+EMRD(o.sl)+"|"+EMRD(o.tp);
}

bool EMRConnected()
{
  ResetLastError();
  long connected=TerminalInfoInteger(TERMINAL_CONNECTED);
  return connected!=0 && GetLastError()==0;
}

// All selections must succeed, including foreign items. Foreign Hedging
// positions/orders with another PID and Magic are independent and permitted.
bool EMRStrictLiveSnapshot(ulong positionID)
{
   ResetLastError(); int positions=PositionsTotal();
   if(GetLastError()!=0 || positions<0 || positions>EMR_MAX_LIVE_ITEMS) return false;
   ResetLastError(); int orders=OrdersTotal();
   if(GetLastError()!=0 || orders<0 || orders>EMR_MAX_LIVE_ITEMS) return false;
   ulong positionTickets[],orderTickets[];
   if(ArrayResize(positionTickets,positions)!=positions ||
      ArrayResize(orderTickets,orders)!=orders) return false;
   for(int i=0;i<positions;i++)
   {
      ResetLastError(); ulong ticket=PositionGetTicket(i);
```

```
  if(ticket==0 || GetLastError()!=0) return false;
  positionTickets[i]=ticket;
  string symbol; long pid,magic; double volume;
  ResetLastError();
  if(!PositionGetString(POSITION_SYMBOL,symbol) || GetLastError()!=0) return false;
 ResetLastError();
 if(!PositionGetInteger(POSITION_IDENTIFIER,pid) || GetLastError()!=0 || pid<=0)
    return false;
 ResetLastError();
 if(!PositionGetInteger(POSITION_MAGIC,magic) || GetLastError()!=0) return false;
 ResetLastError();
 if(!PositionGetDouble(POSITION_VOLUME,volume) || GetLastError()!=0 ||
    !EMRFinite(volume) || volume<=0) return false;
 if(symbol==Symbol() && (ulong)pid==positionID) return false;
}
for(int i=0;i<orders;i++)
{
   ResetLastError(); ulong ticket=OrderGetTicket(i);
   if(ticket==0 || GetLastError()!=0) return false;
  orderTickets[i]=ticket;
  string symbol; long magic,pid,by;
  ResetLastError();
 if(!OrderGetString(ORDER_SYMBOL,symbol) || GetLastError()!=0) return false;
 ResetLastError();
 if(!OrderGetInteger(ORDER_MAGIC,magic) || GetLastError()!=0) return false;
 ResetLastError();
 if(!OrderGetInteger(ORDER_POSITION_ID,pid) || GetLastError()!=0 || pid<0)
    return false;
 ResetLastError();
 if(!OrderGetInteger(ORDER_POSITION_BY_ID,by) || GetLastError()!=0 || by<0)
    return false;
 if(symbol==Symbol() && (magic==(long)gMagicCore ||
    (ulong)pid==positionID || (ulong)by==positionID)) return false;
}
ResetLastError(); int positionsAfter=PositionsTotal();
if(GetLastError()!=0 || positionsAfter!=positions) return false;
ResetLastError(); int ordersAfter=OrdersTotal();
if(GetLastError()!=0 || ordersAfter!=orders) return false;
for(int i=0;i<positions;i++)
{
   ResetLastError(); ulong ticket=PositionGetTicket(i);
   if(GetLastError()!=0 || ticket!=positionTickets[i]) return false;
}
for(int i=0;i<orders;i++)
{
   ResetLastError(); ulong ticket=OrderGetTicket(i);
   if(GetLastError()!=0 || ticket!=orderTickets[i]) return false;
}
```

```
   return true;
}

bool EMRStrictSnapshot(const TradeRecord &r,string &evidence)
{
evidence="";
if(r.positionID==0 || !EMRConnected()) return false;
ResetLastError(); long account=AccountInfoInteger(ACCOUNT_LOGIN);
if(GetLastError()!=0 || account<=0) return false;
ResetLastError(); string server=AccountInfoString(ACCOUNT_SERVER);
if(GetLastError()!=0 || server=="") return false;
ResetLastError();
if(!HistorySelectByPosition(r.positionID) || GetLastError()!=0) return false;
ResetLastError(); int dealCount=HistoryDealsTotal();
if(GetLastError()!=0 || dealCount<=0 || dealCount>EMR_MAX_HISTORY_DEALS)
  return false;
ResetLastError(); int orderCount=HistoryOrdersTotal();
if(GetLastError()!=0 || orderCount<=0 || orderCount>EMR_MAX_HISTORY_ORDERS)
  return false;
EMRDealSnapshot deals[]; EMROrderSnapshot historyOrders[];
ulong dealTickets[],historyOrderTickets[];
if(ArrayResize(deals,dealCount)!=dealCount ||
  ArrayResize(historyOrders,orderCount)!=orderCount ||
  ArrayResize(dealTickets,dealCount)!=dealCount ||
  ArrayResize(historyOrderTickets,orderCount)!=orderCount) return false;
for(int i=0;i<dealCount;i++)
{
  ResetLastError(); ulong ticket=HistoryDealGetTicket(i);
  if(ticket==0 || GetLastError()!=0) return false;
  dealTickets[i]=ticket;
  if(!EMRReadDeal(ticket,r.positionID,deals[i])) return false;
}
for(int i=0;i<orderCount;i++)
{
  ResetLastError(); ulong ticket=HistoryOrderGetTicket(i);
  if(ticket==0 || GetLastError()!=0) return false;
  historyOrderTickets[i]=ticket;
  if(!EMRReadOrder(ticket,r.positionID,historyOrders[i])) return false;
}
// Independent ordering makes the receipt deterministic across terminal restarts.
for(int i=1;i<dealCount;i++)
{
  EMRDealSnapshot value=deals[i]; int j=i-1;
  while(j>=0 && (deals[j].timeMSC>value.timeMSC ||
    (deals[j].timeMSC==value.timeMSC && deals[j].ticket>value.ticket)))
  { deals[j+1]=deals[j]; j--; }
  deals[j+1]=value;
}
```

```
for(int i=1;i<orderCount;i++)
{
   EMROrderSnapshot value=historyOrders[i]; int j=i-1;
   while(j>=0 && historyOrders[j].ticket>value.ticket)
   { historyOrders[j+1]=historyOrders[j]; j--; }
 historyOrders[j+1]=value;
}
for(int i=1;i<dealCount;i++)
   if(deals[i-1].ticket==deals[i].ticket) return false;
for(int i=1;i<orderCount;i++)
   if(historyOrders[i-1].ticket==historyOrders[i].ticket) return false;
double tolerance=AccountingVolumeTolerance();
if(!EMRFinite(tolerance) || tolerance<=0) return false;
double volumeIn=0,volumeOut=0,entryMean=0,exitMean=0;
double gross=0,commission=0,fee=0,swap=0;
long firstEntryTime=0,lastExitTime=0,lastExitMSC=0;
ulong firstEntryOrder=0,lastExitDeal=0;
int direction=0;
bool ownStarted=false,ownFinished=false;
string dealDigest="EMR_DEALS_V1",orderDigest="EMR_ORDERS_V1";
for(int i=0;i<dealCount;i++)
{
 EMRDealSnapshot d=deals[i]; int index=-1;
 for(int j=0;j<orderCount;j++)
    if(historyOrders[j].ticket==d.order) { index=j; break; }
 if(index<0 || historyOrders[index].magic!=d.magic ||
    d.timeMSC<historyOrders[index].setupMSC ||
    d.timeMSC>historyOrders[index].doneMSC) return false;
 if((historyOrders[index].type==ORDER_TYPE_BUY && d.type!=DEAL_TYPE_BUY) ||
    (historyOrders[index].type==ORDER_TYPE_SELL && d.type!=DEAL_TYPE_SELL))
    return false;
 int dealDirection=(d.type==DEAL_TYPE_BUY?1:-1);
 historyOrders[index].dealVolume+=d.volume;
 if(d.entry==DEAL_ENTRY_IN)
 {
    if(ownFinished || historyOrders[index].type==ORDER_TYPE_CLOSE_BY ||
       historyOrders[index].magic!=(long)gMagicCore) return false;
    if(!ownStarted)
    {
       ownStarted=true; direction=dealDirection;
       firstEntryTime=d.timeSec; firstEntryOrder=d.order;
    }
    if(dealDirection!=direction) return false;
    entryMean=(entryMean*volumeIn+d.price*d.volume)/(volumeIn+d.volume);
    volumeIn+=d.volume;
    historyOrders[index].entryVolume+=d.volume;
 }
 else
```

```
 {
    if(!ownStarted || ownFinished || dealDirection==direction ||
       d.volume>volumeIn-volumeOut+tolerance) return false;
    exitMean=(exitMean*volumeOut+d.price*d.volume)/(volumeOut+d.volume);
    volumeOut+=d.volume;
 historyOrders[index].exitVolume+=d.volume;
 lastExitTime=d.timeSec; lastExitMSC=d.timeMSC; lastExitDeal=d.ticket;
 if(volumeIn-volumeOut<=tolerance) ownFinished=true;
 }
 gross+=d.profit; commission+=d.commission; fee+=d.fee; swap+=d.swap;
 if(!EMRFinite(volumeIn) || !EMRFinite(volumeOut) || !EMRFinite(entryMean) ||
    !EMRFinite(exitMean) || !EMRFinite(gross) || !EMRFinite(commission) ||
    !EMRFinite(fee) || !EMRFinite(swap)) return false;
 dealDigest=RJChecksum(dealDigest+"|"+EMRDealEvidence(d));
}
if(!ownStarted || !ownFinished || volumeIn<=tolerance ||
   !EMRNear(volumeIn,volumeOut,tolerance)) return false;
bool entryOrderProven=false;
for(int i=0;i<orderCount;i++)
{
    EMROrderSnapshot o=historyOrders[i];
    double filled=o.initial-o.remaining;
  if(filled < -tolerance || !EMRNear(filled,o.dealVolume,tolerance) ||
     (o.state==ORDER_STATE_FILLED && o.remaining>tolerance) ||
     (o.entryVolume>tolerance && o.exitVolume>tolerance)) return false;
  if(o.entryVolume>tolerance &&
     (!EMRNear(o.sl,r.slPrice,1e-8) || !EMRNear(o.tp,r.tpPrice,1e-8)))
     return false;
  if(o.ticket==firstEntryOrder && o.ticket==r.entryOrderTicket)
  {
     if(r.expectedEntryVolume<=0 ||
        (!EMRNear(r.expectedEntryVolume,o.initial,tolerance) &&
         !EMRNear(r.expectedEntryVolume,filled,tolerance))) return false;
     entryOrderProven=true;
  }
  orderDigest=RJChecksum(orderDigest+"|"+EMROrderEvidence(o));
}
double net=gross+commission+fee+swap;
if(!entryOrderProven || direction!=r.direction ||
   firstEntryTime!=(long)r.entryTime || lastExitTime!=(long)r.exitTime ||
   lastExitMSC!=r.exitTimeMSC || lastExitDeal!=r.lastExitDealTicket ||
   !EMRNear(volumeIn,r.totalVolumeIn,tolerance) ||
   !EMRNear(volumeOut,r.totalVolumeOut,tolerance) ||
   !EMRNear(volumeIn,r.volume,tolerance) ||
   !EMRNear(entryMean,r.weightedEntryPrice,1e-8) ||
   !EMRNear(exitMean,r.weightedExitPrice,1e-8) ||
   !EMRNear(entryMean,r.entryPrice,1e-8) ||
   !EMRNear(exitMean,r.exitPrice,1e-8) ||
```

```
   !EMRNear(gross,r.grossProfit,1e-8) ||
   !EMRNear(commission,r.commission,1e-8) ||
   !EMRNear(fee,r.fee,1e-8) || !EMRNear(swap,r.swap,1e-8) ||
   !EMRNear(net,r.netProfit,1e-8)) return false;
if(r.riskValid)
{
   if(!r.initialStopsProven || r.slPrice<=0 || r.initialRiskMoney<=0 ||
      (double)direction*(entryMean-r.slPrice)<=0) return false;
   double initialProfit=0;
   ResetLastError();
   if(!OrderCalcProfit(direction==1?ORDER_TYPE_BUY:ORDER_TYPE_SELL,Symbol(),
      volumeIn,entryMean,r.slPrice,initialProfit) || GetLastError()!=0 ||
      !EMRFinite(initialProfit) || initialProfit>=0 ||
      !EMRNear(-initialProfit,r.initialRiskMoney,1e-8)) return false;
   if(!r.pnlRValid || !EMRNear(net/r.initialRiskMoney,r.PnL_R,1e-8)) return false;
}
else if(r.pnlRValid || r.initialRiskMoney!=0) return false;
// The selected history must remain identical across a second fresh selection.
// No HistoryOrderSelect/HistoryDealSelect is used: those truncate selection lists.
ResetLastError();
if(!HistorySelectByPosition(r.positionID) || GetLastError()!=0) return false;
ResetLastError(); int dealsAfter=HistoryDealsTotal();
if(GetLastError()!=0 || dealsAfter!=dealCount) return false;
ResetLastError(); int historyOrdersAfter=HistoryOrdersTotal();
if(GetLastError()!=0 || historyOrdersAfter!=orderCount) return false;
for(int i=0;i<dealCount;i++)
{
   ResetLastError(); ulong ticket=HistoryDealGetTicket(i);
   if(GetLastError()!=0 || ticket!=dealTickets[i]) return false;
}
for(int i=0;i<orderCount;i++)
{
   ResetLastError(); ulong ticket=HistoryOrderGetTicket(i);
   if(GetLastError()!=0 || ticket!=historyOrderTickets[i]) return false;
}
for(int i=0;i<dealCount;i++)
{
   EMRDealSnapshot again;
   if(!EMRReadDeal(deals[i].ticket,r.positionID,again) ||
      EMRDealEvidence(again)!=EMRDealEvidence(deals[i])) return false;
}
for(int i=0;i<orderCount;i++)
{
   EMROrderSnapshot again;
   if(!EMRReadOrder(historyOrders[i].ticket,r.positionID,again) ||
      EMROrderEvidence(again)!=EMROrderEvidence(historyOrders[i])) return false;
}
if(!EMRStrictLiveSnapshot(r.positionID) || !EMRConnected()) return false;
```

```
ResetLastError(); long accountAfter=AccountInfoInteger(ACCOUNT_LOGIN);
if(GetLastError()!=0 || accountAfter!=account) return false;
ResetLastError(); string serverAfter=AccountInfoString(ACCOUNT_SERVER);
if(GetLastError()!=0 || serverAfter!=server) return false;
evidence="EMR_STRICT_SNAPSHOT_V1|"+(string)account+"|"+RJEncode(server)+"|"+
   RJEncode(Symbol())+"|"+(string)(ulong)gMagicCore+"|"+(string)r.positionID+"|"+
   (string)direction+"|"+(string)firstEntryOrder+"|"+(string)firstEntryTime+"|"+
   (string)lastExitTime+"|"+(string)lastExitMSC+"|"+(string)lastExitDeal+"|"+

EMRD(volumeIn)+"|"+EMRD(volumeOut)+"|"+EMRD(entryMean)+"|"+EMRD(exitMean)+"|"+

EMRD(gross)+"|"+EMRD(commission)+"|"+EMRD(fee)+"|"+EMRD(swap)+"|"+EMRD(net)+"|"
+
  EMRD(r.slPrice)+"|"+EMRD(r.tpPrice)+"|"+EMRD(r.expectedEntryVolume)+"|"+
  (string)(int)r.riskValid+"|"+EMRD(r.initialRiskMoney)+"|"+
  (string)(int)r.pnlRValid+"|"+(r.pnlRValid?EMRD(r.PnL_R):"NA")+"|"+
  (string)(int)r.breakEvenExit+"|"+(string)(int)r.breakEvenClassificationPending+"|"+
  (string)r.be.stage+"|"+(string)(int)r.be.durable+"|"+
  (string)r.be.confirmedMSC+"|"+EMRD(r.be.price)+"|"+
    (string)dealCount+"|"+(string)orderCount+"|"+dealDigest+"|"+orderDigest;
   return true;
}
// EMR storage only. A receipt never replaces fresh authoritative economic proof.
string EMRResolutionPath(ulong pid)
{ return EconomicMarkerPrefix()+(string)pid+".econ.resolved"; }

bool EMRHasSuffix(string value,string suffix)
{
  int offset=StringLen(value)-StringLen(suffix);
  return offset>=0 && StringSubstr(value,offset)==suffix;
}

bool EMRNamePID(string tail,string suffix,ulong &pid)
{
  pid=0;
  int count=StringLen(tail)-StringLen(suffix);
  if(count<=0 || !EMRHasSuffix(tail,suffix)) return false;
  string digits=StringSubstr(tail,0,count);
  return CsvParsePID(digits,pid) && digits==(string)pid;
}

// FileIsExist(false) with an error is not proof of absence. Confirm the exact
// namespace through a complete enumeration containing the positive sentinel.
bool EMRProbeMarker(ulong pid,bool &present)
{
   present=false;
   if(pid==0 || !PersistEconomicMarker(0)) return false;
   string prefix=EconomicMarkerPrefix(),suffix=".econ.partial",name;
```

```
   ResetLastError(); long search=FileFindFirst(prefix+"*"+suffix,name);
   if(search==INVALID_HANDLE || GetLastError()!=0)
   {
      if(search!=INVALID_HANDLE) FileFindClose(search);
     return false;
    }
    bool more=true,sentinel=false,ok=true;
    int visited=0;
    while(more)
    {
       if(++visited>100000) { ok=false; break; }
       if(name==EconomicMarkerPath(0)) sentinel=true;
       else
       {
          ulong found=0;
          if(StringSubstr(name,0,StringLen(prefix))!=prefix ||
             !EMRNamePID(StringSubstr(name,StringLen(prefix)),suffix,found))
          { ok=false; break; }
       if(found==pid) present=true;
     }
     ResetLastError(); more=FileFindNext(search,name);
     if(!more && GetLastError()!=0) ok=false;
    }
    ResetLastError(); FileFindClose(search);
    if(GetLastError()!=0) ok=false;
    return ok && sentinel && !more;
}

bool EMRInspectFiles(ulong pid,bool &markerPresent,bool &receiptPresent,bool
&bePresent)
{
  markerPresent=false; receiptPresent=false; bePresent=false;
  if(pid==0 || !PersistEconomicMarker(0)) return false;
  string prefix=EconomicMarkerPrefix(),name;
  ResetLastError();
  long search=FileFindFirst(prefix+"*",name);
  if(search==INVALID_HANDLE || GetLastError()!=0)
  {
     if(search!=INVALID_HANDLE) FileFindClose(search);
     return false;
  }
  bool sentinel=false,ok=true,more=true;
  int visited=0;
  while(more)
  {
     if(++visited>100000 || StringSubstr(name,0,StringLen(prefix))!=prefix)
     { ok=false; break; }
     string tail=StringSubstr(name,StringLen(prefix));
```

```
    ulong foundPID=0;
    if(name==EconomicMarkerPath(0)) sentinel=true;
    else if(tail=="identity.loss" || tail=="request.entry.intent" ||
         tail=="request.exit.intent")
    { ok=false; break; }
    else if(tail=="0.order.unresolved") { /* Existing registry sentinel only. */ }
    else if(EMRNamePID(tail,".order.unresolved",foundPID))
    { ok=false; break; } // Includes other PIDs: request/order cleanup is unchanged.
    else if(EMRNamePID(tail,".econ.partial",foundPID))
    { if(foundPID==pid) markerPresent=true; }
    else if(EMRNamePID(tail,".econ.resolved",foundPID))
    { if(foundPID==pid) receiptPresent=true; }
    else if(EMRNamePID(tail,".meta.csv.be.journal",foundPID))
    { if(foundPID==pid) bePresent=true; }
    else if(!EMRNamePID(tail,".meta.csv",foundPID))
    { ok=false; break; } // Malformed/truncated same-identity evidence is ambiguous.
    ResetLastError();
    more=FileFindNext(search,name);
    if(!more && GetLastError()!=0) ok=false;
   }
   ResetLastError(); FileFindClose(search);
   if(GetLastError()!=0) ok=false;
   return ok && sentinel && !more;
}

bool EMRPayloadBytes(string payload,uchar &bytes[])
{
  int length=StringLen(payload);
  if(length<=0 || length>16384) return false;
  for(int i=0;i<length;i++)
  {
    ushort c=StringGetCharacter(payload,i);
    if(c>126 || (c<32 && c!=13 && c!=10)) return false;
  }
  int encoded=StringToCharArray(payload,bytes,0,WHOLE_ARRAY,CP_UTF8);
  if(encoded!=length+1 || ArraySize(bytes)!=encoded || bytes[encoded-1]!=0)
    return false;
  return ArrayResize(bytes,encoded-1)==length;
}

bool EMRReadExact(string path,string payload)
{
  uchar expected[];
  if(!EMRPayloadBytes(payload,expected)) return false;
  ResetLastError();
  int h=FileOpen(path,FILE_READ|FILE_BIN);
  if(h==INVALID_HANDLE) return false;
  bool ok=(GetLastError()==0);
```

```
  ResetLastError(); ulong size=FileSize(h);
  if(GetLastError()!=0 || size==0 || size>16384 || size!=(ulong)ArraySize(expected))
     ok=false;
  uchar actual[];
    int count=ArraySize(expected);
    if(ok && ArrayResize(actual,count)!=count) ok=false;
    if(ok)
    {
       ResetLastError(); uint got=FileReadArray(h,actual,0,count);
       if(got!=(uint)count || GetLastError()!=0) ok=false;
    }
    if(ok)
       for(int i=0;i<count;i++)
          if(actual[i]!=expected[i]) { ok=false; break; }
    ResetLastError(); ulong finalSize=FileSize(h);
    if(GetLastError()!=0 || finalSize!=size) ok=false;
    ResetLastError(); FileClose(h);
    if(GetLastError()!=0) ok=false;
    return ok;
}
bool EMRConfirmExisting(string path,string payload)
{
  if(!EMRReadExact(path,payload)) return false;
  ResetLastError();
  int h=FileOpen(path,FILE_READ|FILE_WRITE|FILE_BIN);
  if(h==INVALID_HANDLE) return false;
  bool ok=(GetLastError()==0);
  ResetLastError(); FileFlush(h);
  if(GetLastError()!=0) ok=false;
  ResetLastError(); FileClose(h);
  if(GetLastError()!=0) ok=false;
  return ok && EMRReadExact(path,payload);
}

bool EMRCommitResolution(ulong pid,string evidence)
{
  if(pid==0 || StringLen(evidence)==0 || StringLen(evidence)>16300) return false;
  // Canonical evidence is printable ASCII; symbol/route text must be RJEncode'd.
  for(int i=0;i<StringLen(evidence);i++)
  {
     ushort c=StringGetCharacter(evidence,i);
     if(c<32 || c>126) return false;
  }
  string prefix="SCALPER_EMR1|"+evidence;
  string payload=prefix+"|"+(string)BEChecksum(prefix)+"|COMPLETE_EMR1\r\n";
  uchar expected[];
  if(!EMRPayloadBytes(payload,expected)) return false;
  bool markerPresent=false,receiptPresent=false,bePresent=false;
```

```
  if(!EMRInspectFiles(pid,markerPresent,receiptPresent,bePresent)) return false;
  string path=EMRResolutionPath(pid);
  // Existing invalid, partial or different evidence is never overwritten.
    if(receiptPresent) return EMRConfirmExisting(path,payload);
    // READ|WRITE preserves an existing file if another writer appeared after
    // enumeration. Never use WRITE-only truncation or delete/rename evidence.
    ResetLastError();
    int h=FileOpen(path,FILE_READ|FILE_WRITE|FILE_BIN);
    if(h==INVALID_HANDLE) return false;
    bool ok=(GetLastError()==0);
    ResetLastError(); ulong size=FileSize(h);
    if(GetLastError()!=0) ok=false;
    if(ok && size>0)
    {
       ResetLastError(); FileClose(h);
       if(GetLastError()!=0) return false;
       return EMRConfirmExisting(path,payload);
    }
    if(ok)
    {
     ResetLastError(); uint written=FileWriteArray(h,expected,0,ArraySize(expected));
     if(written!=(uint)ArraySize(expected) || GetLastError()!=0) ok=false;
     ResetLastError(); FileFlush(h);
     if(GetLastError()!=0) ok=false;
     ResetLastError(); ulong writtenSize=FileSize(h);
     if(GetLastError()!=0 || writtenSize!=(ulong)ArraySize(expected)) ok=false;
    }
    ResetLastError(); FileClose(h);
    if(GetLastError()!=0) ok=false;
    return ok && EMRReadExact(path,payload);
}

// Only failed economic-marker cleanup uses this proof. No strategy gate is added.
bool EMRReady(const TradeRecord &r)
{
   if(r.positionID==0 || !r.closed || !r.finalizationWritten || !r.accountingResolved ||
      !r.volumeClosedComplete || !r.volumeOwnershipResolved || r.historyRefreshPending ||
      r.inoutCostsUnresolved || r.residualForeignDetected || r.breakEvenClassificationPending
||
      !r.riskValid || !r.pnlRValid || !r.initialStopsProven ||
      r.be.stage==BE_REQUEST_PENDING || r.be.stage==BE_ACKNOWLEDGED ||
      r.be.stage==BE_UNRESOLVED) return false;
   if(r.exitTime>=AccountingDayStart() && !r.dailyApplied) return false;
   if(gPendingExec.active || gPendingExit.active || gPendingOrderClassificationUnresolved ||
      !gQuotaHistoryValid || gLiveOwnershipUnresolved || gAccountingCapacityUnresolved ||
      gMultipleOwnOrdersActive || gOrderRegistryUnavailable ||
gOrderObligationScanUnresolved ||
      !gDailyEconomicsValid || !gDailyPnLRValid || gDailyHistoryRetryRequired ||
```

```
      gEconomicMarkerScanUnresolved || gIdentityLossUnresolved ||
gIdentityLossPersistPending ||
      gRequestJournalUnresolved || gRequestJournalWritePending ||
      gEntryJournalCleanupPending || gExitJournalCleanupPending ||
      gEntryJournalInvalid || gExitJournalInvalid || gEntryJournalWritePending ||
      gExitJournalWritePending || gDeferredRegistryUnavailable ||
      ArraySize(gDeferredAccountingPIDs)>0 || ArraySize(gDeferredDeals)>0 ||
      gDeferredDealFallbackCount>0 || gDeferredDealOverflow) return false;
    for(int i=0;i<ArraySize(gUnresolvedOrders);i++)
      if(gUnresolvedOrders[i].active) return false;
    return true;
}

bool EMRBreakEvenEvidence(const TradeRecord &r,bool journalPresent)
{
  if(!journalPresent) return r.be.stage==BE_NONE && !r.breakEvenExit;
  if(r.be.stage!=BE_TRIGGERED && r.be.stage!=BE_CONFIRMED) return false;
    // Bound both bytes and iterations. A failed read can never become EOF.
    ResetLastError();
    int h=FileOpen(BEJournalPath(r.positionID),FILE_READ|FILE_BIN|FILE_SHARE_READ);
    if(h==INVALID_HANDLE) return false;
    ResetLastError(); ulong size=FileSize(h);
    bool ok=(GetLastError()==0 && size>0 && size<=1048576);
    uchar bytes[4096];
    ulong offset=0;
    string row="";
    bool cr=false,found=false;
    BreakEvenState verified;
    while(ok && offset<size)
    {
      int want=(size-offset>4096)?4096:(int)(size-offset);
      ResetLastError(); uint got=FileReadArray(h,bytes,0,want);
      if(got!=(uint)want || GetLastError()!=0) { ok=false; break; }
      offset+=got;
      for(int i=0;i<want;i++)
      {
        uchar b=bytes[i];
        if(b==10 && cr)
        {
          if(row!="")
          {
            BreakEvenState candidate;
            if(!BEParseRow(row,r.positionID,candidate)) { ok=false; break; }
            verified=candidate; found=true;
          }
          row=""; cr=false; continue;
        }
        if(cr) { ok=false; break; }
```

```
         if(b==13) { cr=true; continue; }
         if(b<32 || b>126 || StringLen(row)>=32768) { ok=false; break; }
         row+=ShortToString((ushort)b);
       }
      }
      if(row!="" || cr || !found) ok=false;
      ResetLastError(); ulong finalSize=FileSize(h);
      if(GetLastError()!=0 || finalSize!=size) ok=false;
      ResetLastError(); FileClose(h);
      if(GetLastError()!=0 || !ok) return false;
      return r.be.durable && verified.durable &&
        BEJournalRow(r.positionID,verified)==BEJournalRow(r.positionID,r.be);
}

void EMRRecoverRecord(int idx)
{
  bool wasAdministrative=(gTradeRecords[idx].economicMarkerCleanupPending &&
           !gTradeRecords[idx].economicMarkerPending);
   gTradeRecords[idx].economicMarkerPending=true;
   if(!RebuildTrackedHistory(idx,false)) return;
      // Rebuild calls SyncEconomicMarker. Successful physical cleanup keeps the
      // original behavior; only a residual/error needs the additional proof.
      if(!gTradeRecords[idx].economicMarkerPending) return;
      TradeRecord r=gTradeRecords[idx];
      bool markerPresent=false,receiptPresent=false,bePresent=false;
      if(!EMRReady(r) ||
!EMRInspectFiles(r.positionID,markerPresent,receiptPresent,bePresent)
||
      !EMRBreakEvenEvidence(r,bePresent)) return;
   string evidence="";
   if(!EMRStrictSnapshot(r,evidence)) return;
   if(r.be.stage!=BE_NONE) evidence+="|BE_JOURNAL|"+BEJournalRow(r.positionID,r.be);
   if(!EMRCommitResolution(r.positionID,evidence)) return;
   // Recheck authoritative data after persistence, before publishing RAM state.
   // The receipt is a durable finalization record, never a substitute for history.
   string verified="";
   if(!EMRReady(gTradeRecords[idx]) || !EMRStrictSnapshot(gTradeRecords[idx],verified))
return;
   if(gTradeRecords[idx].be.stage!=BE_NONE)
      verified+="|BE_JOURNAL|"+BEJournalRow(r.positionID,gTradeRecords[idx].be);
   if(verified!=evidence ||
      !EMRInspectFiles(r.positionID,markerPresent,receiptPresent,bePresent) ||
      !receiptPresent || !EMRBreakEvenEvidence(gTradeRecords[idx],bePresent)) return;
   gTradeRecords[idx].economicMarkerPending=false;
   gTradeRecords[idx].economicMarkerCleanupPending=markerPresent;
   if(!wasAdministrative)
      PrintFormat("[SCALPER][ECONOMIC_RESOLUTION_VERIFIED] pid=%I64u
cleanup_only=%d; exact history, orders, positions and durable receipt revalidated.",
```

```
        r.positionID,(int)markerPresent);
}

// A constant-size cursor replaces retention of old resolved PIDs in RAM.
// One archived PID at most is rebuilt per existing ten-second recovery pass.
void EMRMaintainArchivedCleanup()
{
  if(!gEMRMaintenanceNeeded) return;
  string prefix=EconomicMarkerPrefix(),suffix=".econ.partial",name,chosen="",first="";
  ResetLastError(); long search=FileFindFirst(prefix+"*"+suffix,name);
  if(search==INVALID_HANDLE || GetLastError()!=0)
  {
    if(search!=INVALID_HANDLE) FileFindClose(search);
    gEconomicMarkerScanUnresolved=true; return;
  }
  bool more=true,sentinel=false,ok=true;
  int visited=0;
  while(more)
   {
     if(++visited>100000) { ok=false; break; }
    if(name==EconomicMarkerPath(0)) sentinel=true;
    else
    {
      ulong pid=0;
      if(StringSubstr(name,0,StringLen(prefix))!=prefix ||
        !EMRNamePID(StringSubstr(name,StringLen(prefix)),suffix,pid))
      { ok=false; break; }
      if(FindTrackedTrade(pid)<0)
      {
        if(first=="" || StringCompare(name,first)<0) first=name;
        if(StringCompare(name,gEMRMaintenanceCursor)>0 &&
          (chosen=="" || StringCompare(name,chosen)<0)) chosen=name;
      }
    }
    ResetLastError(); more=FileFindNext(search,name);
    if(!more && GetLastError()!=0) ok=false;
   }
  ResetLastError(); FileFindClose(search);
  if(GetLastError()!=0 || !ok || !sentinel || more)
  { gEconomicMarkerScanUnresolved=true; return; }
  if(first=="")
  { gEMRMaintenanceNeeded=false; gEMRMaintenanceCursor=""; return; }
  if(chosen=="") chosen=first;
  gEMRMaintenanceCursor=chosen;
  ulong pid=0;
  if(!EMRNamePID(StringSubstr(chosen,StringLen(prefix)),suffix,pid))
  { gEconomicMarkerScanUnresolved=true; return; }
  int idx=NewTrackedTrade(pid);
```

```
   if(idx<0) { gEconomicMarkerScanUnresolved=true; return; }
   LoadTradeMetadata(gTradeRecords[idx]);
   gTradeRecords[idx].economicMarkerPending=true;
   if(!RebuildTrackedHistory(idx,false)) return;
    TryFinalizeTrackedTrade(pid);
    if(gTradeRecords[idx].closed && gTradeRecords[idx].finalizationWritten &&
       gTradeRecords[idx].economicMarkerPending) EMRRecoverRecord(idx);
    // This slot was allocated solely for one archived maintenance revisit.
    // Successful eventual deletion must not retain an old, dailyApplied=false PID.
    EMRReleaseArchivedRecord(idx);
}

bool EMRReleaseArchivedRecord(int idx)
{
  if(idx<0 || idx>=gTradeRecordCount) return false;
  TradeRecord r=gTradeRecords[idx];
  if(!r.closed || !r.finalizationWritten || !r.accountingResolved ||
     !r.volumeClosedComplete || !r.volumeOwnershipResolved ||
       r.economicMarkerPending || r.historyRefreshPending || r.inoutCostsUnresolved ||
       r.residualForeignDetected || r.breakEvenClassificationPending ||
       r.be.stage==BE_REQUEST_PENDING || r.be.stage==BE_ACKNOWLEDGED ||
       r.be.stage==BE_UNRESOLVED || r.exitTime>=AccountingDayStart()) return false;
   // No financial aggregate is modified. Exact residual evidence stays on disk.
   int tail=gTradeRecordCount-1;
   TradeRecord moved=gTradeRecords[tail];
   if(gOpenPos.active && gOpenPos.tradeRecordIdx==idx) return false;
   if(ArrayResize(gTradeRecords,tail)!=tail) return false;
   if(idx<tail) gTradeRecords[idx]=moved;
   if(gOpenPos.active && gOpenPos.tradeRecordIdx==tail) gOpenPos.tradeRecordIdx=idx;
   gTradeRecordCount=tail;
   if(r.economicMarkerCleanupPending) gEMRMaintenanceNeeded=true;
   return true;
}

void EMRReleaseArchivedRecords()
{
  for(int i=gTradeRecordCount-1;i>=0;i--)
   if(gTradeRecords[i].economicMarkerCleanupPending) EMRReleaseArchivedRecord(i);
}

void EMRInvalidateTransaction(const MqlTradeTransaction &trans)
{
  if(trans.type!=TRADE_TRANSACTION_DEAL_ADD &&
     trans.type!=TRADE_TRANSACTION_DEAL_UPDATE &&
     trans.type!=TRADE_TRANSACTION_DEAL_DELETE) return;
  if(trans.symbol!="" && trans.symbol!=Symbol()) return;
  ulong pid=0; long magic=0; bool identified=false;
  if(trans.type!=TRADE_TRANSACTION_DEAL_DELETE && trans.deal>0)
```

```
  {
    ResetLastError();
    long id=HistoryDealGetInteger(trans.deal,DEAL_POSITION_ID);
    magic=HistoryDealGetInteger(trans.deal,DEAL_MAGIC);
    string symbol=HistoryDealGetString(trans.deal,DEAL_SYMBOL);
    identified=(GetLastError()==0 && id>0 && symbol==Symbol());
    if(identified) pid=(ulong)id;
   }
   bool matched=false;
   for(int i=0;i<gTradeRecordCount;i++)
   {
     if(!gTradeRecords[i].economicMarkerCleanupPending ||
        (identified && gTradeRecords[i].positionID!=pid)) continue;
     gTradeRecords[i].economicMarkerPending=true;
     gTradeRecords[i].historyRefreshPending=true;
     matched=true;
   }
   if(matched)
   {
      // A changed closed cycle requires a fresh daily reconstruction too.
     // Never increment the financial result here.
     MarkTemporaryAccountingHistory();
   }
   if(gEMRMaintenanceNeeded && (!identified ||
     (!matched && (trans.type!=TRADE_TRANSACTION_DEAL_ADD ||
        (magic==(long)gMagicCore && !gPendingExec.active)))))
     gEconomicMarkerScanUnresolved=true;
}


// Durable request obligations, RC1.2.
void RecomputeRequestJournalFlags()
{
  gRequestJournalUnresolved=gEntryJournalInvalid || gExitJournalInvalid;
  gRequestJournalWritePending=gEntryJournalWritePending || gExitJournalWritePending;
}

string RequestJournalPath(bool exitRequest)
{ return EconomicMarkerPrefix()+(exitRequest?"request.exit.intent":"request.entry.intent"); }

bool RJUnsigned(string text,ulong &value)
{
 if(text=="0") { value=0; return true; }
 return CsvParsePID(text,value); // Exact ulong; rejects overflow and non-digits.
}

bool RJBoolean(string text,bool &value)
{
```

```
  if(text!="0" && text!="1") return false;
  value=(text=="1"); return true;
}
bool RJDouble(string text,double &value)
{
  bool digit=false,point=false;
  for(int i=0;i<StringLen(text);i++)
  {
     ushort c=StringGetCharacter(text,i);
     if(c>=48 && c<=57) digit=true;
     else if(c==46 && !point) point=true;
     else return false;
  }
  if(!digit) return false;
  value=StringToDouble(text);
  return MathIsValidNumber(value) && value>=0;
}
string RJEncode(string value)
{
  string out="";
 for(int i=0;i<StringLen(value);i++)
  out+=StringFormat("%04X",(uint)StringGetCharacter(value,i));
 return out;
}
bool RJDecode(string text,string &value)
{
  value="";
  if(StringLen(text)%4!=0 || StringLen(text)>8192) return false;
  for(int i=0;i<StringLen(text);i+=4)
  {
     uint code=0;
     for(int j=0;j<4;j++)
     {
        ushort c=StringGetCharacter(text,i+j); uint digit=0;
        if(c>=48 && c<=57) digit=(uint)(c-48);
        else if(c>=65 && c<=70) digit=(uint)(c-55);
        else return false;
        code=code*16+digit;
     }
     if(code==0) return false;
     value+=ShortToString((ushort)code);
  }
  return true;
}

bool RJRoute(string route,string mode)
{
  return (route=="ABS" && (mode=="BASE" || mode=="TURBO")) ||
```

```
   (route=="CONT" && (mode=="BASE_CONT" || mode=="TURBO_CONT")) ||
   (route=="UNAVAILABLE" && mode=="UNAVAILABLE");
}

string RJChecksum(string text)
{
  // ASCII journal fields only. Integrity check, not authentication.
  uint hash=2166136261;
  for(int i=0;i<StringLen(text);i++)
     hash=(hash^(uint)StringGetCharacter(text,i))*(uint)16777619;
  return StringFormat("%08X",hash);
}

bool RJParse(string row,bool exitRequest,bool &resolved,
      PendingExecContext &entry,PendingExitIntent &exitIntent)
{
  string f[];
  int fields=StringSplit(row,'|',f);
  bool legacy=(fields==38 && f[0]=="RC12_REQUEST_JOURNAL_V1");
  bool current=(fields==40 && f[0]=="RC12_REQUEST_JOURNAL_V2");
  if((!legacy && !current) ||
     f[37]!="COMPLETE" || f[4]!=(exitRequest?"EXIT":"ENTRY") ||
     (f[5]!="ACTIVE" && f[5]!="RESOLVED")) return false;
  ENUM_REQUEST_DISPATCH_PHASE phase=REQUEST_MAY_HAVE_DISPATCHED;
  if(current)
  {
 if(f[38]!="0" && f[38]!="1") return false;
 string checked=f[0];
 for(int i=1;i<39;i++) checked+="|"+f[i];
 if(f[39]!=RJChecksum(checked)) return false;
 phase=(f[38]=="1"?REQUEST_PREPARED:REQUEST_MAY_HAVE_DISPATCHED);
}
ulong account,magic;
string symbol;
if(!RJUnsigned(f[1],account) || account!=(ulong)AccountInfoInteger(ACCOUNT_LOGIN) ||
   !RJDecode(f[2],symbol) || symbol!=Symbol() ||
   !RJUnsigned(f[3],magic) || magic!=(ulong)gMagicCore) return false;
resolved=(f[5]=="RESOLVED");
ZeroMemory(entry); ZeroMemory(exitIntent);
entry.dispatchPhase=phase; exitIntent.dispatchPhase=phase;
bool active;
if(!RJBoolean(f[6],active) || (!resolved && !active)) return false;
entry.active=active; exitIntent.active=active;
if(f[7]!="-1" && f[7]!="0" && f[7]!="1") return false;
entry.direction=(int)StringToInteger(f[7]);
if(!RJDecode(f[8],entry.route) || !RJDecode(f[9],entry.mode) ||
   !RJRoute(entry.route,entry.mode) ||
   !RJDouble(f[10],entry.requestPrice) || !RJDouble(f[11],entry.requestedLots) ||
```

```
   !RJDouble(f[12],entry.slPrice) || !RJDouble(f[13],entry.tpPrice)) return false;
ulong sentAt,order,deal,requestID,pid,ticket,action,type,position,byPosition,requestMagic;
if(!RJUnsigned(f[14],sentAt) || (long)sentAt<0 ||
   !RJUnsigned(f[15],order) || !RJUnsigned(f[16],deal) ||
   !RJUnsigned(f[17],requestID) || requestID>(ulong)4294967295 ||
   !RJBoolean(f[18],entry.accepted) || !RJBoolean(f[19],entry.requestPriceAvailable) ||
   !RJDouble(f[20],entry.breakoutPrice) || !RJBoolean(f[21],entry.breakoutPriceAvailable) ||
   !RJUnsigned(f[22],pid) || !RJUnsigned(f[23],ticket) ||
   !RJDecode(f[24],exitIntent.reason) || !RJBoolean(f[25],exitIntent.uncertain) ||
   !RJUnsigned(f[26],action) || action>100 ||
   !RJUnsigned(f[27],type) || type>100 ||
   !RJDouble(f[28],exitIntent.requestSnapshot.volume) ||
   !RJDouble(f[29],exitIntent.requestSnapshot.price) ||
   !RJDouble(f[30],exitIntent.requestSnapshot.sl) ||
   !RJDouble(f[31],exitIntent.requestSnapshot.tp) ||
   !RJUnsigned(f[32],position) || !RJUnsigned(f[33],byPosition) ||
   !RJUnsigned(f[34],requestMagic) ||
   !RJDecode(f[35],exitIntent.requestSnapshot.symbol) ||
   !RJDecode(f[36],exitIntent.requestSnapshot.comment)) return false;
entry.sentAt=(datetime)sentAt; entry.orderTicket=order; entry.resultDeal=deal;
entry.requestID=(uint)requestID;
exitIntent.sentAt=entry.sentAt; exitIntent.orderTicket=order;
exitIntent.resultDeal=deal; exitIntent.requestID=entry.requestID;
exitIntent.accepted=entry.accepted; exitIntent.positionID=pid;
exitIntent.positionTicket=ticket;
exitIntent.requestSnapshot.action=(ENUM_TRADE_REQUEST_ACTIONS)action;
  exitIntent.requestSnapshot.type=(ENUM_ORDER_TYPE)type;
  exitIntent.requestSnapshot.position=position;
  exitIntent.requestSnapshot.position_by=byPosition;
  exitIntent.requestSnapshot.magic=requestMagic;
  if(phase==REQUEST_PREPARED &&
     (order>0 || deal>0 || requestID>0 || entry.accepted || exitIntent.uncertain))
     return false;
  if(resolved) return true; // Tombstone itself is the caller's durable proof.
  if(sentAt==0) return false;
  if(!exitRequest)
     return (entry.direction==1 || entry.direction==-1) && entry.requestedLots>0 &&
        (!entry.requestPriceAvailable || entry.requestPrice>0) &&
        (!entry.breakoutPriceAvailable || entry.breakoutPrice>0);
  if(pid==0 || (exitIntent.reason!="EOD" && exitIntent.reason!="RECOVERED_EXIT")) return
false;
  // A pre-dispatch/recovered close can lack the CTrade request snapshot.
  // Its PID/ticket/intent still protects it; no success/failure is inferred.
  if(exitIntent.requestSnapshot.symbol=="")
     return action==0 && type==0 && exitIntent.requestSnapshot.volume==0 &&
        requestMagic==0 && position==0 && byPosition==0;
  return exitIntent.requestSnapshot.symbol==Symbol() &&
requestMagic==(ulong)gMagicCore &&
```

```
   (action==(ulong)TRADE_ACTION_DEAL ||
action==(ulong)TRADE_ACTION_CLOSE_BY) &&
  (type==(ulong)ORDER_TYPE_BUY || type==(ulong)ORDER_TYPE_SELL ||
type==(ulong)ORDER_TYPE_CLOSE_BY) &&
  exitIntent.requestSnapshot.volume>0 && (ticket==0 || position==ticket);
}

string RJRow(bool exitRequest,bool resolved)
{
 PendingExecContext e=gPendingExec;
 PendingExitIntent x=gPendingExit;
 if(exitRequest)
 {
   ZeroMemory(e); e.route="UNAVAILABLE"; e.mode="UNAVAILABLE";
   e.active=x.active; e.sentAt=x.sentAt; e.orderTicket=x.orderTicket;
   e.resultDeal=x.resultDeal; e.requestID=x.requestID; e.accepted=x.accepted;
   e.dispatchPhase=x.dispatchPhase;
 }
 else ZeroMemory(x);
 string f[];
 if(ArrayResize(f,40)!=40) return "";
 f[0]="RC12_REQUEST_JOURNAL_V2";
f[1]=(string)AccountInfoInteger(ACCOUNT_LOGIN);
 f[2]=RJEncode(Symbol()); f[3]=(string)(ulong)gMagicCore;
 f[4]=exitRequest?"EXIT":"ENTRY"; f[5]=resolved?"RESOLVED":"ACTIVE";
 f[6]=e.active?"1":"0"; f[7]=(string)e.direction;
 f[8]=RJEncode(e.route); f[9]=RJEncode(e.mode);
 f[10]=DoubleToString(e.requestPrice,16); f[11]=DoubleToString(e.requestedLots,16);
 f[12]=DoubleToString(e.slPrice,16); f[13]=DoubleToString(e.tpPrice,16);
 f[14]=(string)(ulong)e.sentAt; f[15]=(string)e.orderTicket; f[16]=(string)e.resultDeal;
 f[17]=(string)e.requestID; f[18]=e.accepted?"1":"0";
 f[19]=e.requestPriceAvailable?"1":"0"; f[20]=DoubleToString(e.breakoutPrice,16);
 f[21]=e.breakoutPriceAvailable?"1":"0"; f[22]=(string)x.positionID;
 f[23]=(string)x.positionTicket; f[24]=RJEncode(x.reason); f[25]=x.uncertain?"1":"0";
 f[26]=(string)(int)x.requestSnapshot.action; f[27]=(string)(int)x.requestSnapshot.type;
 f[28]=DoubleToString(x.requestSnapshot.volume,16);
f[29]=DoubleToString(x.requestSnapshot.price,16);
 f[30]=DoubleToString(x.requestSnapshot.sl,16);
f[31]=DoubleToString(x.requestSnapshot.tp,16);
 f[32]=(string)x.requestSnapshot.position; f[33]=(string)x.requestSnapshot.position_by;
 f[34]=(string)x.requestSnapshot.magic; f[35]=RJEncode(x.requestSnapshot.symbol);
 f[36]=RJEncode(x.requestSnapshot.comment); f[37]="COMPLETE";
 f[38]=(string)(int)e.dispatchPhase;
 string row=f[0];
 for(int i=1;i<39;i++) row+="|"+f[i];
 f[39]=RJChecksum(row); row+="|"+f[39];
 bool checkResolved; PendingExecContext checkEntry; PendingExitIntent checkExit;
 if(!RJParse(row,exitRequest,checkResolved,checkEntry,checkExit)) return "";
```

```
  return row;
}
bool PersistRequestJournal(bool exitRequest,bool resolved=false)
{
  // Never turn an older PREPARED snapshot plus torn MAY tail into negative proof.
  if(!resolved &&
     ((exitRequest && gExitJournalInvalid &&
gPendingExit.dispatchPhase==REQUEST_PREPARED) ||
      (!exitRequest && gEntryJournalInvalid &&
gPendingExec.dispatchPhase==REQUEST_PREPARED)))
     return false;
  string row=RJRow(exitRequest,resolved);
  bool ok=false;
  if(row!="")
  {
     int h=FileOpen(RequestJournalPath(exitRequest),
        FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_SHARE_READ);
     if(h!=INVALID_HANDLE)
     {
        if(FileSeek(h,0,SEEK_END))
      {
         // Leading CRLF separates a torn previous tail from this snapshot.
         string payload="\r\n"+row+"\r\n";
         ResetLastError(); uint written=FileWriteString(h,payload);
         ok=(written==(uint)StringLen(payload) && GetLastError()==0);
         ResetLastError(); FileFlush(h);
        if(GetLastError()!=0) ok=false;
       }
       FileClose(h);
     }
    }
    if(exitRequest)
    { gExitJournalWritePending=!ok; if(ok) gExitJournalInvalid=false; }
    else
    { gEntryJournalWritePending=!ok; if(ok) gEntryJournalInvalid=false; }
    RecomputeRequestJournalFlags();
    return ok;
}

bool MarkRequestDispatchPossible(bool exitRequest)
{
  if(exitRequest)
  {
     if(!gPendingExit.active || gPendingExit.dispatchPhase!=REQUEST_PREPARED ||
        gExitJournalInvalid || gExitJournalCleanupPending) return false;
     gPendingExit.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
  }
  else
```

```
  {
    if(!gPendingExec.active || gPendingExec.dispatchPhase!=REQUEST_PREPARED ||
     gEntryJournalInvalid || gEntryJournalCleanupPending) return false;
    gPendingExec.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
   }
   // Once this flush succeeds, a crash is ambiguous even before the API call.
   return PersistRequestJournal(exitRequest);
}

bool ResolveRequestJournal(bool exitRequest)
{
 if(exitRequest) gExitJournalCleanupPending=true;
 else gEntryJournalCleanupPending=true;
 PersistRequestJournal(exitRequest,true);
 string path=RequestJournalPath(exitRequest);
 // The caller supplies terminal proof. Successful whole-file deletion is
 // also a durable resolution even if appending the tombstone failed.
  bool gone=!FileIsExist(path);
  if(!gone && FileDelete(path)) gone=!FileIsExist(path);
  if(exitRequest)
  {
     gExitJournalCleanupPending=!gone;
     if(gone) { gExitJournalWritePending=false; gExitJournalInvalid=false; }
  }
  else
  {
    gEntryJournalCleanupPending=!gone;
    if(gone) { gEntryJournalWritePending=false; gEntryJournalInvalid=false; }
   }
   RecomputeRequestJournalFlags();
   return gone;
}

bool ReleasePreparedRequest(bool exitRequest)
{
 // Caller must supply a pristine restored snapshot, or retained local cleanup proof.
 if(exitRequest)
 {
    if(!gPendingExit.active || gPendingExit.dispatchPhase!=REQUEST_PREPARED ||
       gPendingExit.orderTicket>0 || gPendingExit.resultDeal>0 ||
       gPendingExit.requestID>0 || gPendingExit.accepted || gPendingExit.uncertain ||
       (gExitJournalInvalid && !gExitJournalCleanupPending)) return false;
 }
 else
 {
    if(!gPendingExec.active || gPendingExec.dispatchPhase!=REQUEST_PREPARED ||
       gPendingExec.orderTicket>0 || gPendingExec.resultDeal>0 ||
       gPendingExec.requestID>0 || gPendingExec.accepted ||
```

```
       (gEntryJournalInvalid && !gEntryJournalCleanupPending)) return false;
  }
    if(!ResolveRequestJournal(exitRequest)) return false;
    if(exitRequest) ClearPendingExitIntent();
    else ResetPendingExec(false);
    return true;
}

bool RJRead(bool exitRequest,bool &exists,bool &resolved,
       PendingExecContext &entry,PendingExitIntent &exitIntent,bool &hasSnapshot)
{
  exists=FileIsExist(RequestJournalPath(exitRequest)); resolved=false; hasSnapshot=false;
  if(!exists) return true;
  int
h=FileOpen(RequestJournalPath(exitRequest),FILE_READ|FILE_BIN|FILE_SHARE_READ)
;
  if(h==INVALID_HANDLE) return false;
 ResetLastError(); ulong size=FileSize(h);
 if(GetLastError()!=0) { FileClose(h); return false; }
 uchar bytes[4096]; string row=""; bool cr=false,malformed=false,tailInvalid=false;
 ulong offset=0;
 while(offset<size)
 {
    int want=(size-offset>4096)?4096:(int)(size-offset);
    ResetLastError(); uint got=FileReadArray(h,bytes,0,want);
    if(got!=(uint)want || GetLastError()!=0) { FileClose(h); return false; }
  offset+=got;
  for(int i=0;i<want;i++)
  {
    uchar b=bytes[i];
    if(b==10 && cr)
    {
      if(row!="" || malformed)
      {
        bool terminal; PendingExecContext candidateEntry; PendingExitIntent
candidateExit;
        if(!malformed && RJParse(row,exitRequest,terminal,candidateEntry,candidateExit))
        {
          // An earlier ambiguous dispatch cannot become negative proof.
          if(!terminal &&

(exitRequest?candidateExit.dispatchPhase:candidateEntry.dispatchPhase)==REQUEST_PR
EPARED &&
   (tailInvalid || (hasSnapshot && !resolved &&

(exitRequest?exitIntent.dispatchPhase:entry.dispatchPhase)==REQUEST_MAY_HAVE_DIS
PATCHED)))
      { FileClose(h); return false; }
```

```
      if(hasSnapshot && !resolved && !terminal &&
         ((exitRequest && exitIntent.positionID!=candidateExit.positionID) ||
          ((exitRequest?exitIntent.orderTicket:entry.orderTicket)>0 &&
           (exitRequest?exitIntent.orderTicket:entry.orderTicket)!=
           (exitRequest?candidateExit.orderTicket:candidateEntry.orderTicket))))
       { FileClose(h); return false; }
       entry=candidateEntry; exitIntent=candidateExit;
       resolved=terminal; hasSnapshot=true; tailInvalid=false;
      }
      else tailInvalid=true;
    }
    row=""; malformed=false; cr=false; continue;
  }
  if(cr) { malformed=true; cr=false; }
  if(b==13) { cr=true; continue; }
  if(b<32 || b>126) malformed=true;
  if(StringLen(row)<32768) row+=ShortToString((ushort)b);
   else malformed=true;
  }
 }
 FileClose(h);
 if(row!="" || malformed || cr) tailInvalid=true;
 // A later torn row never turns an older ACTIVE snapshot into absence.
 // With no usable snapshot, or after a tombstone, ambiguity stays explicit.
 return hasSnapshot && !tailInvalid;
}
bool RJCompatibleEntry(const PendingExecContext &a,const PendingExecContext &b)
{
  if(a.orderTicket>0 && b.orderTicket>0) return a.orderTicket==b.orderTicket;
  if(a.requestID>0 && b.requestID>0) return a.requestID==b.requestID;
  if(a.resultDeal>0 && b.resultDeal>0) return a.resultDeal==b.resultDeal;
  return a.sentAt==b.sentAt && a.direction==b.direction && a.route==b.route &&
     a.mode==b.mode &&
MathAbs(a.requestedLots-b.requestedLots)<=AccountingVolumeTolerance() &&
     MathAbs(a.requestPrice-b.requestPrice)<=gTickSize*0.000001 &&
     MathAbs(a.slPrice-b.slPrice)<=gTickSize*0.000001 &&
     MathAbs(a.tpPrice-b.tpPrice)<=gTickSize*0.000001;
}

bool RJCompatibleExit(const PendingExitIntent &a,const PendingExitIntent &b)
{
  if(a.positionID!=b.positionID) return false;
  if(a.orderTicket>0 && b.orderTicket>0) return a.orderTicket==b.orderTicket;
  if(a.requestID>0 && b.requestID>0) return a.requestID==b.requestID;
  if(a.resultDeal>0 && b.resultDeal>0) return a.resultDeal==b.resultDeal;
  return a.sentAt==b.sentAt && a.reason==b.reason &&
     (a.positionTicket==0 || b.positionTicket==0 || a.positionTicket==b.positionTicket);
}
```

```
bool RestoreRequestJournals()
{
 gRequestJournalRestoring=true;
 for(int side=0;side<2;side++)
 {
  bool isExit=(side==1),exists,resolved,snapshot;
  PendingExecContext entry; PendingExitIntent exitIntent;
  bool valid=RJRead(isExit,exists,resolved,entry,exitIntent,snapshot);
  if(isExit) gExitJournalInvalid=!valid;
  else gEntryJournalInvalid=!valid;
  if(!exists) continue;
  if(snapshot && !resolved)
  {
    if(isExit)
    {
      if(!gPendingExit.active) gPendingExit=exitIntent;
     else if(!RJCompatibleExit(gPendingExit,exitIntent)) gExitJournalInvalid=true;
     else if(gPendingExit.dispatchPhase==REQUEST_MAY_HAVE_DISPATCHED &&
         exitIntent.dispatchPhase==REQUEST_PREPARED) gExitJournalInvalid=true;
     else if(exitIntent.dispatchPhase==REQUEST_MAY_HAVE_DISPATCHED)
       gPendingExit.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
     if(valid && !gExitJournalInvalid &&
exitIntent.dispatchPhase==REQUEST_PREPARED)
       ReleasePreparedRequest(true);
  }
  else
  {
   if(!gPendingExec.active) gPendingExec=entry;
   else if(!RJCompatibleEntry(gPendingExec,entry)) gEntryJournalInvalid=true;
   else if(gPendingExec.dispatchPhase==REQUEST_MAY_HAVE_DISPATCHED &&
        entry.dispatchPhase==REQUEST_PREPARED) gEntryJournalInvalid=true;
   else if(entry.dispatchPhase==REQUEST_MAY_HAVE_DISPATCHED)
     gPendingExec.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
   if(valid && !gEntryJournalInvalid && entry.dispatchPhase==REQUEST_PREPARED)
     ReleasePreparedRequest(false);
  }
  }
  else if(valid && resolved)
  {
   // A committed tombstone is proof for deletion retry, never ACTIVE.
   // Do not remove a conflicting live request's obligation on recovery.
   if((isExit && gPendingExit.active) || (!isExit && gPendingExec.active))
   {
     if(isExit) gExitJournalInvalid=true;
     else gEntryJournalInvalid=true;
   }
   else
```

```
    {
      if(isExit) gExitJournalCleanupPending=true;
      else gEntryJournalCleanupPending=true;
      ResolveRequestJournal(isExit);
   }
   }
  }
  gRequestJournalRestoring=false;
  RecomputeRequestJournalFlags();
  return !gRequestJournalUnresolved && !gRequestJournalWritePending &&
    !gEntryJournalCleanupPending && !gExitJournalCleanupPending;
}

void RecoverRequestJournals()
{
  datetime now=TimeTradeServer();
  if(now<gNextRequestJournalRecovery) return;
  gNextRequestJournalRecovery=now+10;
  if(gEntryJournalCleanupPending)
  {
   if(gPendingExec.active && gPendingExec.dispatchPhase==REQUEST_PREPARED)
      ReleasePreparedRequest(false);
   else ResolveRequestJournal(false);
  }
  if(gExitJournalCleanupPending)
 {
     if(gPendingExit.active && gPendingExit.dispatchPhase==REQUEST_PREPARED)
        ReleasePreparedRequest(true);
     else ResolveRequestJournal(true);
  }
  if(gEntryJournalInvalid || gExitJournalInvalid) RestoreRequestJournals();
  if(gEntryJournalWritePending && !gEntryJournalCleanupPending &&
gPendingExec.active)
     PersistRequestJournal(false);
  if(gExitJournalWritePending && !gExitJournalCleanupPending && gPendingExit.active)
     PersistRequestJournal(true);
  RecomputeRequestJournalFlags();
}

// Durable exact-order obligations; metadata is never terminal proof.
string OrderObligationPath(ulong ticket)
{
  return EconomicMarkerPrefix()+(string)ticket+".order.unresolved";
}

void UnknownOrderObligationContext(ulong ticket,UnresolvedOrderContext &c)
{
  ZeroMemory(c);
```

```
  c.active=true; c.classificationPending=true; c.orderTicket=ticket;
  c.nature=PENDING_ORDER_UNKNOWN; c.lastState=ORDER_STATE_STARTED;
   c.orderType=-1; c.orderReason=-1;
   c.route="UNAVAILABLE"; c.mode="UNAVAILABLE";
   c.persistencePending=true; c.persistenceSnapshot="";
   // Unknown/corrupt metadata cannot invent a setup lower bound or firstSeen.
   c.firstSeen=0; c.setupTime=0; c.doneTime=0;
}

bool EnsureOrderObligationIdentity(ulong ticket)
{
  string path=OrderObligationPath(ticket);
  // Filename presence alone is an unresolved identity, even if body is torn.
  // Ticket zero is the enumeration sentinel and never an operational order.
  if(FileIsExist(path)) return true;
  int h=FileOpen(path,FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_SHARE_READ);
  if(h==INVALID_HANDLE) return false;
 string payload="RC12_ORDER_IDENTITY_V1\r\n";
 ResetLastError(); uint written=FileWriteString(h,payload);
 bool ok=(written==(uint)StringLen(payload) && GetLastError()==0);
 ResetLastError(); FileFlush(h);
 if(GetLastError()!=0) ok=false;
 FileClose(h);
 return ok && FileIsExist(path);
}
bool OrderObligationOptionalEnum(string text,long maximum,long &value)
{
  if(text=="-1") { value=-1; return true; }
  ulong parsed;
  if(!RJUnsigned(text,parsed) || parsed>(ulong)maximum) return false;
  value=(long)parsed;
  return true;
}

bool ParseOrderObligationSnapshot(string row,ulong filenameTicket,
         UnresolvedOrderContext &c)
{
  string f[];
  if(filenameTicket==0 || StringSplit(row,'|',f)!=20 ||
     f[0]!="RC12_ORDER_OBLIGATION_V1" || f[19]!="COMPLETE") return false;
  ulong account,magic,ticket,firstSeen,setup,done,pid,by,nature,state;
  string symbol;
  if(!RJUnsigned(f[1],account) || account!=(ulong)AccountInfoInteger(ACCOUNT_LOGIN) ||
     !RJDecode(f[2],symbol) || symbol!=Symbol() ||
     !RJUnsigned(f[3],magic) || magic!=(ulong)gMagicCore ||
     !RJUnsigned(f[4],ticket) || ticket!=filenameTicket ||
     !RJUnsigned(f[5],firstSeen) || (long)firstSeen<0 ||
     !RJUnsigned(f[6],setup) || (long)setup<0 ||
```

```
     !RJUnsigned(f[7],done) || (long)done<0 ||
     !RJUnsigned(f[8],pid) || !RJUnsigned(f[9],by) ||
     !RJUnsigned(f[10],nature) || nature>(ulong)PENDING_ORDER_EXIT ||
     !RJUnsigned(f[11],state) || state>(ulong)ORDER_STATE_REQUEST_CANCEL)
     return false;
  UnknownOrderObligationContext(ticket,c);
  if(!OrderObligationOptionalEnum(f[12],(long)ORDER_TYPE_CLOSE_BY,c.orderType) ||
     !OrderObligationOptionalEnum(f[13],(long)ORDER_REASON_SO,c.orderReason) ||
     !RJDouble(f[14],c.initialVolume) || !RJDouble(f[15],c.remainingVolume) ||
     !RJDecode(f[16],c.comment) || !RJDecode(f[17],c.route) ||
     !RJDecode(f[18],c.mode) || !RJRoute(c.route,c.mode)) return false;
  c.firstSeen=(datetime)firstSeen; c.setupTime=(datetime)setup;
c.doneTime=(datetime)done;
  c.positionID=pid; c.positionByID=by;
  c.nature=(PendingOrderNature)nature; c.lastState=(ENUM_ORDER_STATE)state;
  // Persisted classification/state/volumes are metadata, never terminal proof.
  c.active=true; c.classificationPending=true;
    c.persistencePending=false; c.persistenceSnapshot=row;
    return true;
}
string OrderObligationSnapshot(const UnresolvedOrderContext &c)
{
  if(c.orderTicket==0) return "";
  string f[];
  if(ArrayResize(f,20)!=20) return "";
  f[0]="RC12_ORDER_OBLIGATION_V1";
f[1]=(string)AccountInfoInteger(ACCOUNT_LOGIN);
  f[2]=RJEncode(Symbol()); f[3]=(string)(ulong)gMagicCore;
  f[4]=(string)c.orderTicket; f[5]=(string)(ulong)c.firstSeen;
  f[6]=(string)(ulong)c.setupTime; f[7]=(string)(ulong)c.doneTime;
  f[8]=(string)c.positionID; f[9]=(string)c.positionByID;
  f[10]=(string)(int)c.nature; f[11]=(string)(int)c.lastState;
  f[12]=(string)c.orderType; f[13]=(string)c.orderReason;
  f[14]=DoubleToString(c.initialVolume,16); f[15]=DoubleToString(c.remainingVolume,16);
  f[16]=RJEncode(c.comment); f[17]=RJEncode(c.route); f[18]=RJEncode(c.mode);
  f[19]="COMPLETE";
  string row=f[0];
  for(int i=1;i<20;i++) row+="|"+f[i];
    UnresolvedOrderContext check;
    if(!ParseOrderObligationSnapshot(row,c.orderTicket,check)) return "";
    return row;
}

bool PersistOrderObligationSnapshot(UnresolvedOrderContext &c)
{
  bool alreadyPresent=FileIsExist(OrderObligationPath(c.orderTicket));
  if(c.orderTicket==0 || !EnsureOrderObligationIdentity(c.orderTicket))
  { c.persistencePending=true; return false; }
```

```
 string row=OrderObligationSnapshot(c);
if(row=="") { c.persistencePending=true; return false; }
// An unchanged observed context never grows the file on every scan/tick.
if(alreadyPresent && !c.persistencePending && row==c.persistenceSnapshot) return true;
int h=FileOpen(OrderObligationPath(c.orderTicket),
   FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_SHARE_READ);
bool ok=false;
if(h!=INVALID_HANDLE)
{
   if(FileSeek(h,0,SEEK_END))
   {
      string payload="\r\n"+row+"\r\n";
      ResetLastError(); uint written=FileWriteString(h,payload);
      ok=(written==(uint)StringLen(payload) && GetLastError()==0);
      ResetLastError(); FileFlush(h);
      if(GetLastError()!=0) ok=false;
 }
 FileClose(h);
}
c.persistencePending=!ok;
if(ok) c.persistenceSnapshot=row;
return ok;
}
bool ReadOrderObligationSnapshot(ulong ticket,UnresolvedOrderContext &c)
{
 UnknownOrderObligationContext(ticket,c);
 int h=FileOpen(OrderObligationPath(ticket),FILE_READ|FILE_BIN|FILE_SHARE_READ);
 if(h==INVALID_HANDLE) return false;
 ResetLastError(); ulong size=FileSize(h);
 if(GetLastError()!=0) { FileClose(h); return false; }
 uchar bytes[4096]; string row="";
 bool cr=false,malformed=false,tailInvalid=false,hasSnapshot=false;
 UnresolvedOrderContext latest;
 ulong offset=0;
 while(offset<size)
 {
    int want=(size-offset>4096)?4096:(int)(size-offset);
    ResetLastError(); uint got=FileReadArray(h,bytes,0,want);
    if(got!=(uint)want || GetLastError()!=0) { FileClose(h); return false; }
    offset+=got;
    for(int i=0;i<want;i++)
    {
       uchar b=bytes[i];
       if(b==10 && cr)
       {
          if(row!="" || malformed)
          {
             UnresolvedOrderContext candidate;
```

```
            if(!malformed && ParseOrderObligationSnapshot(row,ticket,candidate))
       { latest=candidate; hasSnapshot=true; tailInvalid=false; }
       else tailInvalid=true;
      }
      row=""; malformed=false; cr=false; continue;
    }
    if(cr) { malformed=true; cr=false; }
    if(b==13) { cr=true; continue; }
    if(b<32 || b>126) malformed=true;
    if(StringLen(row)<32768) row+=ShortToString((ushort)b);
    else malformed=true;
  }
 }
 FileClose(h);
 if(row!="" || malformed || cr) tailInvalid=true;
 // Empty/identity-only/invalid-tail files retain an UNKNOWN exact ticket.
// No prior metadata snapshot can authorize classification/terminal cleanup.
if(!hasSnapshot || tailInvalid) return false;
c=latest;
return true;
}
bool ReleaseOrderObligation(UnresolvedOrderContext &c)
{
 // CALL ONLY AFTER fresh ReadOwnOrder + exact complete history/ledger proof.
 // A persisted lastState, missing live order or flat Positions is insufficient.
 if(c.orderTicket==0) return false;
 string path=OrderObligationPath(c.orderTicket);
 bool gone=!FileIsExist(path);
 if(!gone && FileDelete(path)) gone=!FileIsExist(path);
 if(!gone)
 {
    c.active=true; c.classificationPending=true; c.persistencePending=true;
    gOrderObligationScanUnresolved=true;
    return false;
 }
 c.active=false; c.classificationPending=false;
   c.persistencePending=false; c.persistenceSnapshot="";
   return true;
}

void StoreOrderObligationContext(int idx,UnresolvedOrderContext &c,bool
authoritativeOrder)
{
 if(c.active)
 {
    if(authoritativeOrder) PersistOrderObligationSnapshot(c);
    else if(!EnsureOrderObligationIdentity(c.orderTicket)) c.persistencePending=true;
 }
```

```
  // A deleted terminal obligation must not be recreated by this assignment.
    gUnresolvedOrders[idx]=c;
}

void RestoreOrderObligations()
{
  if(gOrderObligationRestoring) return;
  gOrderObligationRestoring=true;
  gOrderObligationScanUnresolved=false;
  if(!EnsureOrderObligationIdentity(0))
  {
     gOrderObligationScanUnresolved=true;
     gOrderObligationRestoring=false; return;
  }
  string prefix=EconomicMarkerPrefix(),suffix=".order.unresolved",name;
  long search=FileFindFirst(prefix+"*"+suffix,name);
if(search==INVALID_HANDLE)
{
   gOrderObligationScanUnresolved=true;
    gOrderObligationRestoring=false; return;
}
    ulong tickets[]; int count=0; bool sentinel=false,more=true;
    while(more)
    {
       if(name==prefix+"0"+suffix) sentinel=true;
       else
       {
          int length=StringLen(name)-StringLen(prefix)-StringLen(suffix);
          ulong ticket=0;
          if(length<=0 || StringSubstr(name,0,StringLen(prefix))!=prefix ||
             StringSubstr(name,StringLen(name)-StringLen(suffix))!=suffix ||
             !CsvParsePID(StringSubstr(name,StringLen(prefix),length),ticket))
             gOrderObligationScanUnresolved=true;
          else if(ArrayResize(tickets,count+1)!=count+1)
             gOrderObligationScanUnresolved=true; // Exact filename remains durable.
          else tickets[count++]=ticket;
       }
       ResetLastError(); more=FileFindNext(search,name);
       // Only an explicit clean EOF (zero after ResetLastError) is accepted.
       // No guessed nonzero EOF code can turn an incomplete scan into emptiness.
       if(!more && GetLastError()!=0) gOrderObligationScanUnresolved=true;
      }
      FileFindClose(search);
      if(!sentinel) gOrderObligationScanUnresolved=true;
      // Enumeration is closed before allocation/rebuild/persistence can mutate files.
      for(int i=0;i<count;i++)
      {
         ulong ticket=tickets[i];
```

```
      int known=FindUnresolvedOrder(ticket);
    int idx=RememberUnresolvedOrder(ticket);
    if(idx<0) { gOrderObligationScanUnresolved=true; continue; }
    if(known>=0)
    {
      // A retry never overwrites a newer in-memory authoritative context.
      if(gUnresolvedOrders[idx].persistencePending)
        gOrderObligationScanUnresolved=true;
      continue;
    }
    UnresolvedOrderContext restored;
    bool valid=ReadOrderObligationSnapshot(ticket,restored);
    // Remember may have written an UNKNOWN initial snapshot; therefore the
    // integration must suppress snapshot writes while gOrderObligationRestoring.
    gUnresolvedOrders[idx]=restored;
    if(!valid) gOrderObligationScanUnresolved=true;
     }
     gOrderObligationRestoring=false;
}
void InitializeAccounting()
{
  ArrayResize(gTradeRecords,0); ArrayResize(gDeferredAccountingPIDs,0);
  gDeferredRegistryUnavailable=false;
  gTradeRecordCount=0; ResetOpenPos(); ResetPendingExec(false);
  gPendingExit.active=false; gPendingExit.positionID=0; gPendingExit.reason="";
  gPendingExit.dispatchPhase=REQUEST_MAY_HAVE_DISPATCHED;
gPendingExit.sentAt=0;
  gPendingExit.orderTicket=0; gPendingExit.resultDeal=0;
  gPendingExit.positionTicket=0; gPendingExit.requestID=0;
  gPendingExit.accepted=false; gPendingExit.uncertain=false;
  ZeroMemory(gPendingExit.requestSnapshot);
  ArrayResize(gUnresolvedOrders,0);
  gMultipleOwnOrdersActive=false; gOrderRegistryUnavailable=false;
  gOrderRegistryFallbackCount=0; gOrderRegistryOverflow=false;
  gOrderObligationScanUnresolved=false; gOrderObligationRestoring=false;
  gDeferredFallbackCount=0; gDeferredRegistryOverflow=false;
  gEconomicMarkerScanUnresolved=false;
  gEMRMaintenanceNeeded=false; gEMRMaintenanceCursor="";
  gIdentityLossUnresolved=FileIsExist(IdentityLossMarkerPath());
  gIdentityLossPersistPending=false;
  gRequestJournalUnresolved=false; gRequestJournalWritePending=false;
  gEntryJournalCleanupPending=false; gExitJournalCleanupPending=false;
  gEntryJournalInvalid=false; gExitJournalInvalid=false;
  gEntryJournalWritePending=false; gExitJournalWritePending=false;
  gRequestJournalRestoring=false; gNextRequestJournalRecovery=0;
  ArrayResize(gDeferredDeals,0); gDeferredDealFallbackCount=0;
gDeferredDealOverflow=false;
  gNextOrderReconcile=0;
```

```
  gLiveOwnershipUnresolved=false; gAccountingCapacityUnresolved=false;
  gDailyRiskLimitLatched=false; gDailyHistoryRetryRequired=false;
  gDailyEconomicsState=DAILY_ECONOMICS_VALID; gLastAccountingRecoveryTime=0;
  gPendingOrderClassificationUnresolved=false;
  requestsAcceptedToday=0; ordersEnteredToday=0; dailyNetPnL=0; dailyPnLR=0;
  consecutiveLosses=0; dailyRiskBlocked=false; lastTradeWasLoss=false;
  lastCloseConfirmedTime=0; gDailyPnLRValid=true;
  gDailyEconomicsValid=true; gQuotaHistoryValid=true;
  gNeedsAccountingSync=false; gLastPendingReconcileTime=0;
  ArrayResize(gConfirmedPositionsToday,0);
  RestoreRequestJournals();
  RestoreOrderObligations();
}

void ResetAccountingDay()
{
  // An open cycle, its exit intent and pending request survive midnight.
  requestsAcceptedToday=0; ordersEnteredToday=0; dailyNetPnL=0; dailyPnLR=0;
  consecutiveLosses=0; dailyRiskBlocked=false; lastTradeWasLoss=false;
  lastCloseConfirmedTime=0; gDailyPnLRValid=true;
  gDailyRiskLimitLatched=false; gDailyHistoryRetryRequired=true;
gLastAccountingRecoveryTime=0;
  gDailyEconomicsValid=true; gQuotaHistoryValid=true;
  ArrayResize(gConfirmedPositionsToday,0);
  for(int i=0;i<gTradeRecordCount;i++)
  {
    // Prior finalized records remain reusable; an open cycle is accounted
    // against the date on which its closure will be confirmed.
    if(!gTradeRecords[i].closed) gTradeRecords[i].dailyApplied=false;
    gTradeRecords[i].quotaCounted=false;
  }
  RestoreDailyPersistence(false);
  RecomputeAccountingBlock();
}

void OnTradeTransaction(const MqlTradeTransaction &trans,
     const MqlTradeRequest &request,const MqlTradeResult &result)
{
  EMRInvalidateTransaction(trans);
  BEHandleTransaction(trans,request,result);
  bool ownEntryRequest=(gPendingExec.active && request.position==0 &&
request.position_by==0 &&
  request.symbol==Symbol() && request.magic==gMagicCore &&
request.action==TRADE_ACTION_DEAL &&
   ((gPendingExec.direction==1 && request.type==ORDER_TYPE_BUY) ||
    (gPendingExec.direction==-1 && request.type==ORDER_TYPE_SELL)) &&
   MathAbs(request.volume-gPendingExec.requestedLots)<=AccountingVolumeTolerance()
&&
```

```
      request.comment=="RC1|"+gPendingExec.route+"|"+gPendingExec.mode &&
      MathAbs(request.price-gPendingExec.requestPrice)<=gTickSize*0.000001 &&
      MathAbs(request.sl-gPendingExec.slPrice)<=gTickSize*0.000001 &&
      MathAbs(request.tp-gPendingExec.tpPrice)<=gTickSize*0.000001 &&
      gPendingExec.requestID>0 && result.request_id==gPendingExec.requestID);
   if(trans.type==TRADE_TRANSACTION_REQUEST && ownEntryRequest)
   {
      if(result.order>0) gPendingExec.orderTicket=result.order;
      if(result.deal>0) gPendingExec.resultDeal=result.deal;
      if(IsAcceptedRetcode(result.retcode))
      { if(!gPendingExec.accepted) requestsAcceptedToday++; gPendingExec.accepted=true;
}
    else if(IsFinalRejectedRetcode(result.retcode) && !gPendingExec.accepted &&
      gPendingExec.orderTicket==0 && gPendingExec.resultDeal==0)
     ResetPendingExec();
   if(gPendingExec.active) PersistRequestJournal(false);
  }
  if(trans.type==TRADE_TRANSACTION_REQUEST &&
MatchesExitRequest(request,result))
     CompletePendingExit(result);
  if(trans.type==TRADE_TRANSACTION_DEAL_ADD && trans.deal>0)
  {
     if(!ProcessAccountingDeal(trans.deal)) RememberDeferredDeal(trans.deal);
    }
    // Every transaction may reconcile a position state that lagged DEAL_ADD.
    gNeedsAccountingSync=true;
    SyncAccountingState(true);
}


double OnTester() { return 0.0; }
```